On Monte Carlo Tree Search with Multiple Objectives

Michael Painter

Pembroke College

University of Oxford

A thesis submitted for the degree of

Doctor of Philosophy

Hilary 2026

Acknowledgements

TODO: acknowledgements here

Abstract

TODO: abstract here

Contents

List of Figures **xi**

List of Tables **xiii**

List of Abbreviations **xix**

List of Notation **xxi**

1 Introduction **1**

1.1 Overview 1

1.2 Contributions 2

1.3 Structure of Thesis 4

1.4 Publications 4

2 Background **5**

2.1 Multi-Armed Bandits 6

2.1.1 K-Armed Bandits and the Upper Confidence Bound Algorithm 7

2.1.2 Exploring Bandits 8

2.1.3 Contextual Bandits 10

2.2 Markov Decision Processes 11

2.3 Reinforcement Learning 14

2.3.1 Maximum Entropy Reinforcement Learning 17

2.4 Trial-Based Heuristic Tree Search and Monte Carlo Tree Search . . 19

2.4.1 Notation 21

2.4.2 Trial Based Heuristic Tree Search 22

2.4.3 Upper Confidence Bound Applied to Trees (UCT) 28

2.4.4 Predictor-UCT (PUCT) 31

2.4.5 Maximum Entropy Tree Search 32

2.5 Convergence of Random Variables 33

2.6 Multi-Objective Reinforcement Learning 35

2.6.1 Convex Hull Value Iteration 41

3 Literature Review **43**
3.1 Multi-Armed Bandits . . . 43
3.2 Reinforcement Learning . . . 44
3.3 Trial-Based Heuristic Tree Search and Monte-Carlo Tree Search . . 46
3.3.1 Trial Based Heuristic Tree Search . . . 46
3.3.2 Monte-Carlo Tree Search . . . 46
3.3.3 Maximum Entropy Tree Search . . . 47
3.4 Multi-Objective Reinforcement Learning . . . 47
3.5 Multi-Objective Monte Carlo Tree Search . . . 49

4 Monte Carlo Tree Search With Boltzmann Exploration **51**
4.1 Introduction and Motivation . . . 53
4.1.1 UCT . . . 54
4.1.2 MENTS . . . 55
4.1.3 Simple Regret and Consistency . . . 57
4.2 Boltzmann Search . . . 58
4.2.1 Boltzmann Tree Search . . . 59
4.2.2 Decaying ENtropy Tree Search . . . 62
4.2.3 Stochastic Search Policies: Prior Knowledge and Computational Efficiency . . . 64
4.3 Theoretical MDPs . . . 69
4.3.1 The D-Chain Problem . . . 71
4.3.2 The Entropy Trap . . . 73
4.4 Empirical Results . . . 75
4.4.1 Environments . . . 76
4.4.2 Evaluation Proceedure . . . 79
4.4.3 Results and Discussion . . . 81
4.5 Theoretical Results . . . 86
4.5.1 Notation . . . 87
4.5.2 Preliminaries . . . 89
4.5.3 Consistency Results . . . 91
4.5.4 The Entropy Trap . . . 99
4.6 Summary . . . 103

5 Convex Hull Monte Carlo Tree Search **105**
5.1 Introduction and Motivation . . . 106
5.1.1 Consistent Action Selection . . . 107
5.1.2 Contextual Tree Search . . . 108
5.2 Contextual Zooming for Trees . . . 109
5.2.1 Algorithm Definition . . . 110

5.2.2 Differences from Contextual Zooming . . . 113
5.3 Convex Hull Monte Carlo Tree Search . . . 114
5.3.1 Common Framework . . . 114
5.3.2 Convex Hull UCT . . . 115
5.3.3 Convex Hull BTS . . . 115
5.3.4 Convex Hull CZT . . . 116
5.3.5 Prior Search Policies . . . 117
5.3.6 Optimisations . . . 119
5.3.7 Entropy and Convex Hulls . . . 122
5.4 Empirical Results . . . 123
5.4.1 Evaluation Proceedure . . . 124
5.4.2 Environments . . . 126
5.4.3 Results and Discussion . . . 126
5.5 Theoretical Discussion . . . 126
5.5.1 Context Independent Policies . . . 126
5.5.2 Convergence of CHMCTS . . . 127
5.5.3 The Fundamental Assumption of MOMCTS . . . 127
5.6 Summary . . . 134

6 Simplex Maps for Multi-Objective Monte Carlo Tree Search 135
6.1 Introduction and Motivation . . . 136
6.1.1 Limitations of CHMCTS and CZT . . . 136
6.1.2 Motivation for Working in Weight Space . . . 137
6.1.3 Adaptive Subdivisions of the Weight Space . . . 138
6.2 Simplex Maps . . . 139
6.2.1 Interface . . . 139
6.2.2 Implementation . . . 141
6.2.3 Complexity Discussion . . . 145
6.3 Simplex Maps for Tree Search . . . 147
6.4 Empirical Results . . . 149
6.5 Summary . . . 150

7 Conclusion 151
7.1 Summary of Contributions . . . 151
7.2 Future Work . . . 151

Appendices

A Additional Algorithm Details For Chapter 4 155
A.1 AR-BTS . . . 155
A.2 AR-DENTS . . . 157

B Additional Experimental Details for Chapter 4 **159**
B.1 Additional DeterministicGridWorld, FrozenLake details 159
B.2 Hyperparameter Details for DeterministicGridWorld, FrozenLake and Sailing Experiments 160
B.2.1 Common Hyperparameters Reference 160
B.2.2 Integration of Common Hyperparameters into Prior Algorithms 162
B.2.3 Hyperparameter Optimisation 164
B.3 Additional Details for Go Experiments 166
B.3.1 AR-MENTS, AR-RENTS and AR-TENTS 168
B.3.2 Go Round Robin Hyperparameter Selection 168
B.3.3 Average Returns vs Bellman Updates for Go 173

C Additional Theoretical Results For Chapter 4 **175**
C.1 The DENTS Stochastic Process 175
C.1.1 The AR-DENTS Process 177
C.1.2 The (AR-)BTS process. 177
C.2 Exponential Convergence Results 178
C.2.1 Preliminaries 179
C.2.2 Maximum Entropy Reinforcement Learning Results 185
C.2.3 General Q-value Convergence Result 188
C.2.4 MENTS Results 191
C.2.5 DENTS (and BTS) Results 196

Bibliography **201**

List of Figures

2.1 The procedure of a multi-armed bandit problem, following a strategy σ. 7
2.2 The procedure of an exploring multi-armed bandit problem, following an exploration strategy σ, and recommendation strategy ψ. 9
2.3 The procedure of a contextual multi-armed bandit problem, following a strategy σ. 11
2.4 An example MDP $\mathcal{M}$. 12
2.5 An overview of reinforcement learning. 14
2.6 Overview of one trial of MCTS-1. 19
2.7 Tree diagrams notation. 21
2.8 Overview of one trial of `THTS++`. 23
2.9 A complication arising from the use of a transposition table. 25
2.10 The decision-support scenario for multi-objective reinforcement learning [25]. 35
2.11 An example MOMDP $\boldsymbol{\mathcal{M}}$. 36
2.12 The geometry of convex coverage sets. 39
2.13 An example Pareto front. 40
2.14 An example of arithmetic over vector sets. 42

4.1 Exploration setting in reinforcement learning. 52
4.2 An example grid world shortest path problem. 54
4.3 Results on the example grid world problem for a variety of bias and temperature parameters. 55
4.4 The procedure of an exploration planning problem for MDPs 57
4.5 Example of building and sampling from an alias table. 66
4.6 An illustration of the *(modified) D-chain problem.* 70
4.7 Results on the (modified) D-chain problem. 72
4.8 An illustration of the *Entropy Trap problem.* 73
4.9 Results on the Entropy Trap problem. 75
4.10 Maps used for the Frozen Lake environment. 77
4.11 The map and wind transition probabilities used for the 6x6 Sailing Problem. 78

4.12 MENTS with a variety of temperatures on an 8x8 Frozen Lake environment. BTS and DENTS are included for reference. TODO: Update fig? 82
4.13 RENTS with a variety of temperatures on an 8x8 Frozen Lake environment. BTS and DENTS are included for reference. TODO: Update fig? 83
4.14 TENTS with a variety of temperatures on an 8x8 Frozen Lake environment. BTS and DENTS are included for reference. TODO: Update fig? 83
4.15 Results for gridworld environments. TODO: update fig? TODO: add more caption? 85
4.16 Bounding the empirical transition probabilities to the true transition probabilities. 91
4.17 (Repeated). An illustration of the *Entropy Trap problem*. 100

5.1 A motivating example MOMDP for contextual tree search. 107
5.2 A motivating example MOMDP for contextual tree search. 108
5.3 Design paradigm for CZT. 110
5.4 Entropy and convex hull value sets. 122

6.1 Convex hull value set and corresponding value curve. 137
6.2 Illustration of a simplex map. 140
6.3 An example of a conforming and non-conforming triangulation. . . . 142
6.4 Closest vertex lookup examples. 144
6.5 Example of the `maybe_split` operation of a simplex map. 144
6.6 Illustration of the `push` operation of a simplex map. 146

B.1 Maps used for the DeterministicGridWorld and FrozenLake environment. 160

C.1 Bounding the empirical transition probabilities to the true transition probabilities. 189

List of Tables

4.1 Summary of complexities for different MCTS algorithms. 70
4.2 Results for the Go round-robin tournament. The first column gives the agent playing as black. The final column gives the average trials run per move across the entire round-robin. In the top row, we abbreviate the algorithm names for space. 86

B.1 The hyperparameter search space for gridworld experiments (Section TODO: ref) 165
B.2 Hyperparameters for DeterministicGridWorld experiments. 166
B.3 Hyperparameters for FrozenLake experiments. 167
B.4 Hyperparameters for Sailing(North) experiments. 167
B.5 Hyperparameters for Sailing(South-East) experiments. 168
B.6 Round robin hyperparameter selection for α_{init} for BTS. 169
B.7 Round robin hyperparameter selection for α_{init} for BTS. 169
B.8 Round robin hyperparameter selection for $\epsilon_{\tilde{\lambda}}$ for BTS. 170
B.9 Round robin hyperparameter selection for $\epsilon_{\tilde{\lambda}}$ for AR-BTS. 170
B.10 Round robin hyperparameter selection for ϵ for BTS. 170
B.11 Round robin hyperparameter selection for ϵ for AR-BTS. 171
B.12 Round robin hyperparameter selection for α for MENTS. 171
B.13 Round robin hyperparameter selection for α for AR-MENTS. 171
B.14 Round robin hyperparameter selection for α for RENTS. 172
B.15 Round robin hyperparameter selection for α for AR-RENTS. 172
B.16 Round robin hyperparameter selection for α for TENTS. 172
B.17 Round robin hyperparameter selection for α for AR-TENTS. 172
B.18 Round robin hyperparameter selection for β_{init} for DENTS. 173
B.19 Round robin hyperparameter selection for β_{init} for AR-DENTS. . . 173
B.20 Results for matches between the average return and Bellman versions of BTS, DENTS, MENTS, RENTS and TENTS. 174

List of Code Listings

2.1.1 Definition 7
2.1.2 Definition 9
2.1.3 Definition 10
2.1.4 Definition 11
2.2.1 Definition 12
2.2.2 Definition 12
2.2.3 Definition 12
2.2.4 Definition 13
2.2.5 Definition 13
2.2.6 Definition 13
2.3.1 Definition 15
2.3.2 Definition 16
2.3.3 Definition 16
2.3.4 Definition 16
2.3.5 Definition 17
2.3.6 Definition 18
2.3.7 Definition 18
2.3.8 Definition 18
2.4.1 Definition 22
2.4.2 Definition 23
2.4.3 Definition 24
2.1 Pseudocode for running a trial in `THTS++`. 29
2.5.1 Definition 33
2.5.2 Definition 33
2.5.1 Theorem 33
2.5.2 Theorem 34
2.6.1 Definition 35
2.6.2 Definition 35
2.6.3 Definition 36
2.6.4 Definition 37
2.6.5 Definition 37
2.6.6 Definition 37

2.6.7 Definition 38
2.6.8 Definition 38
2.6.9 Definition 38
2.6.10 Definition 39
2.6.11 Definition 40
2.6.12 Definition 41

4.1.1 Definition 56
4.1.2 Definition 57
4.1.3 Definition 58
4.2.1 Theorem 61
4.2.2 Theorem 61
4.2.3 Theorem 61
4.2.4 Theorem 62
4.2.5 Theorem 64
4.2.6 Theorem 64
4.2.7 Theorem 64
4.2.8 Theorem 64
4.3.1 Theorem 74
4.5.1 Definition 89
4.5.1 Lemma 89
4.5.1 Corollary 91
4.2.8 Theorem 92
4.2.5 Theorem 93
4.2.6 Theorem 93
4.5.2 Lemma 94
4.5.2 Corollary 95
4.5.3 Lemma 96
4.5.4 Lemma 97
4.5.5 Lemma 97
4.5.6 Lemma 98
4.5.7 Lemma 99
4.3.1 Theorem 100

5.1.1 Definition 108
5.1.2 Definition 109
5.1.3 Definition 109
5.3.1 Definition 117
5.3.2 Definition 118
5.5.1 Theorem (Per-step action selection requires linear utility) 127

5.5.1 Proposition 132
5.5.1 Remark 133

C.2.1Lemma 179
C.2.2Lemma 180
C.2.3Lemma 181
C.2.1Theorem 182
C.2.4Lemma 183
C.2.5Lemma 184
C.2.6Lemma 185
C.2.7Lemma 186
C.2.2Theorem 188
C.2.1Corollary 188
C.2.8Lemma 189
C.2.9Lemma 192
C.2.3Theorem 193
C.2.10Lemma 194
C.2.4Theorem 196
C.2.11Lemma 196
C.2.5Theorem 198
C.2.12Lemma 198
C.2.6Theorem 199

List of Abbreviations

BTS . . . . . Boltzmann Tree Search.

CCS . . . . . Convex Coverage Set.

CHVI . . . . . Convex Hull Value Iteration.

CHVS . . . . . Convex Hull Value Set.

CMAB . . . . Contextual Multi-Armed Bandit (problem).

CZ . . . . . Contextual Zooming.

DENTS . . . . Decaying ENtropy Tree Search.

EMAB . . . . Exploring Multi-Armed Bandit (problem).

MAB . . . . . Multi-Armed Bandit (problem).

MCCS . . . . . Minimal Convex Coverage Set.

MCTS . . . . . Monte Carlo Tree Search.

MCTS-1 . . . . A specific and common presentation of Monte Carlo Tree Search, presented in Section 2.4.

MDP . . . . . Markov Decision Process.

MENTS . . . . Maximum ENtropy Tree Search.

MOMDP . . . Multi-Objective Markov Decision Process.

MORL . . . . Multi-Objective Reinforcement Learning.

PF . . . . . Pareto Front.

PUCT . . . . . Predictor-UCT.

THTS . . . . . Trial-Based Heuristic Tree Search.

`THTS++` . . . . . An extension of THTS used in this thesis.

UCB . . . . . Upper Confidence Bound (algorithm).

UCT . . . . . Upper Confidence Bound Applied to Trees.

List of Notation

General Notation

$\mathbb{1}$ . . . . . The indicator function, where $\mathbb{1}(A) = 1$ when A is true, and $\mathbb{1}(A) = 0$ when A is false.

$\hat{A}$ . . . . . The hat notation is used to denote an estimate, i.e. $\hat{A}$ is an estimate for some value A or some optimal value A^*

$\bar{A}$ . . . . . The bar notation is used to denote sample averages, $\bar{A}$ is a sample average of a number of samples $A_1, ..., A_k$. i.e. $\bar{A} = \frac{1}{k}\sum_{i=1}^{k} A_i$.

$\tilde{A}$ . . . . . The tilde notation is used to denote a (function) approximation, such as a neural network.

$\mathcal{A}$ . . . . . Calligraphic font is used to denote variables that are sets or tuples (ordered sets).

$\mathbf{A}$ . . . . . Bold font is used to denote vectors, and A_i is used to denote the ith component of the vector $\mathbf{A}$.

$\texttt{A}$ . . . . . Typewriter text is used to refer to variables relating to an algorithm (or the analysis of an algorithm).

$\boldsymbol{\mathcal{A}}$ . . . . . The notations above will also be combined at times, so $\boldsymbol{\mathcal{A}}$ is used to denote a set that contains vectors.

$\mathbb{E}$ . . . . . The expectation operator. $\mathbb{E}[f]$ denotes the expectation of a distribution f, and $\mathbb{E}_{x\sim f}[g(x)]$ denotes the expected value of $g(x)$ under the distribution f.

$x \sim f$ . . . . . If $f : X \to \mathbb{R}$ specifies a probability distribution in some way (e.g. a probability density function or cumulative density function), then $x \sim f$ denotes that the variable x is sampled from the distribution f.

$O(f(x))$ . . . . Big-O notation: $g(x) = O(f(x))$ if there exist constants $c > 0$ and x_0 such that $|g(x)| \leq c \cdot f(x)$ for all $x \geq x_0$. Used to describe an asymptotic upper bound on the growth rate of a function.

$X_i \xrightarrow{p} X$ . . . . The sequence of random variables X_i converges in probability to the random variable X. $\Pr(|X_i - X| > \varepsilon) \to 0$ as $i \to \infty$ for all $\varepsilon > 0$.

$X_i \xrightarrow{as} X$ . . . . The sequence of random variables X_i converges almost surely to the random variable X. $\Pr(\lim_{i\to\infty} X_i = X) = 1$.

(Multi-Objective) Markov Decision Processes (Defined in Sections 2.2 and 2.6)

$\mathcal{A}$ . . . . . A (finite) set of actions.

a_t . . . . . The action at the tth timestep of a trajectory.

D . . . . . The dimension of rewards in a MOMDP.

H . . . . . The finite-horizon time bound of an MDP.

$\mathcal{M}$ . . . . . A Markov Decision Process, which is a tuple $\mathcal{M} = (\mathcal{S}, s_0, \mathcal{A}, R, p, H)$.

$\boldsymbol{\mathcal{M}}$ . . . . . A Multi-Objective Markov Decision Process, which is a tuple $\boldsymbol{\mathcal{M}} = (\mathcal{S}, s_0, \mathcal{A}, \mathbf{R}, p, H)$.

p . . . . . The next-state transition distribution of an MDP. $p(s'|s,a)$ is the probability of transitioning to state s' from state s by taking action a.

π . . . . . A policy, mapping a state $s \in \mathcal{S}$ to a probability distribution over actions $\mathcal{A}$. That is, $\pi(a|s)$ is the probability of taking action a at state s. $\pi(s) = a'$ is used a shorthand for $\pi(a|s) = \mathbb{1}[a = a']$.

R . . . . . The reward function of an MDP: $R(s,a)$ is the reward received from taking action a at state s.

$\mathbf{R}$ . . . . . The D dimensional reward function of a MOMDP: $\mathbf{R}(s,a) \in \mathbb{R}^D$ is the vector of rewards received from taking action a at state s.

r_t . . . . . The reward at the tth timestep of a trajectory.

$\mathcal{S}$ . . . . . A (finite) set of states.

$\mathcal{S}_0$ . . . . . The set of reachable states from the starting state s_0 in an MDP.

$\text{Succ}(s,a)$ . . . . The set of successor states of a state-action pair (s,a), with respect to an MDP: $\text{Succ}(s,a) = \{s' \in \mathcal{S} | p(s'|s,a) > 0\}$.

s_0 . . . . . $s_0 \in \mathcal{S}$ is the initial starting state of an MDP.

s_t . . . . . The state at the tth timestep of a trajectory.

`terminal` . . . . `terminal` $\subseteq \mathcal{S}$ is the set of terminal states. A state is terminal if the transition distribution and reward function of the MDP are such that once reached, the state will never be left and no more reward will be received.

τ . . . . . A trajectory, or sequence, of states, actions and rewards that are sampled according to a policy π and an MDP $\mathcal{M}$: $\tau = (s_0, a_0, r_0, s_1, ..., s_{H-1}, a_{H-1}, r_{H-1}, s_H)$.

τ . . . . . A multi-objective trajectory, of states, actions and rewards that are sampled according to a policy π and an MDP $\mathcal{M}$: $\tau = (s_0, a_0, \mathbf{r}_0, s_1, ..., s_{H-1}, a_{H-1}, \mathbf{r}_{H-1}, s_H)$.

$\tau_{i:j}$ . . . . . A truncated trajectory, starting at timestep i, and ending at timestep j: $\tau_{i:j} = (s_i, a_i, r_i, s_{i+1}, ..., s_{j-1}, a_{j-1}, r_{j-1}, s_j)$.

$\tau_{i:j}$ . . . . . A multi-objective truncated trajectory, starting at timestep i and ending at timestep j: $\tau_{i:j} = (s_i, a_i, \mathbf{r}_i, s_{i+1}, ..., s_{j-1}, a_{j-1}, \mathbf{r}_{j-1}, s_j)$.

Reinforcement Learning (Section 2.3)

$J(\pi)$ . . . . . The objective function for (standard) reinforcement learning: $J(\pi) = V^\pi(s_0; 0)$.

π^* . . . . . The optimal standard policy, that maximises the objective function $J(\pi)$.

Q^* . . . . . The optimal Q-value function. $Q^*(s, a; t)$ denotes the maximal expected value that can be achieved by any policy, starting from state $s_t = s$, with action $a_t = a$.

Q^π . . . . . The Q-value function of a policy π. $Q^\pi(s, a; t)$ denotes the expected cumulative reward that policy π will obtain, starting from state $s_t = s$, starting by taking action $a_t = a$.

V^* . . . . . The optimal value function. $V^*(s; t)$ denotes the maximal expected value that can be achieved by any policy, starting from state $s_t = s$.

V^π . . . . . The value of a policy π. $V^\pi(s; t)$ denotes the expected cumulative reward that policy π will obtain, starting from state $s_t = s$.

Maximum Entropy Reinforcement Learning (Section 2.3.1)

α . . . . . The temperature parameter, or, the coefficient of the entropy term in the maximum entropy (soft) objective.

$\mathcal{H}$ . . . . . The Shannon entropy, of a probability distribution or policy.

$J_{\text{sft}}(\pi)$ . . . . . The objective function for maximum entropy (soft) reinforcement learning: $J_{\text{sft}}(\pi) = V^\pi_{\text{sft}}(s_0; 0)$.

π^*_{sft} . . . . . The optimal soft policy, that maximises the soft objective function $J_{\text{sft}}(\pi)$.

Q^*_{sft} . . . . . The optimal soft Q-value function. $Q^*_{\text{sft}}(s, a; t)$ denotes the maximal expected soft value that can be achieved by any policy, from state $s_t = s$ and taking action $a_t = a$.

Q^π_{sft} . . . . . The soft Q-value function of a policy π. $Q^\pi_{\text{sft}}(s, a; t)$ is the expected value of the policy from $s_t = s$, starting with action $a_t = a$, with an addition of the entropy of the policy weighted by the temperature α.

V^*_{sft} . . . . . The optimal soft value function. $V^*_{\text{sft}}(s; t)$ denotes the maximal expected soft value that can be achieved by any policy, from state $s_t = s$.

V^π_{sft} . . . . . The soft value of a policy π. $V^\pi_{\text{sft}}(s; t)$ is the expected value of the policy from $s_t = s$, with an addition of the entropy of the policy weighted by the temperature α.

Trial Based Heuristic Tree Search (Section 2.4)

`backup_v` . . . . Updates the values at a decision node in the backup phase of a trial `THTS++`, using the decision node's children, the trajectory sampled for the trial and the heuristic value function.

`backup_q` . . . . Updates the values at a chance node in the backup phase of a trial `THTS++`, using the chance node's children, the trajectory sampled for the trial and the heuristic value function.

$H_{\texttt{THTS++}}$ . . . . . The planning horizon used in `THTS++`, with $H_{\texttt{THTS++}} \leq H$.

$N(s)$ . . . . . The number of visits at the decision node corresponding to state s.

$N(s, a)$ . . . . . The number of visits at the chance node corresponding to state-action pair (s, a).

`node`(s) . . . . . The decision node corresponding to the state s.

`node`(s)`.chldrn` The set of chance nodes that are children of `node`(s).

`node`(s)`.data` . The set of variables stored at decision node `node`(s), typically used for estimating values.

`node`(s, a) . . . The chance node corresponding to the state-action pair (s, a).

`node`(s, a)`.chldrn` The set of decision nodes that are children of `node`(s, a).

`node`(s, a)`.data` The set of variables stored at chance node `node`(s, a), typically used for estimating Q-values.

$\pi_{\text{heuristic}}$ . . . . . A heuristic policy, which can be used by the search policy to utilise prior knowledge of the environment.

π_{search} . . . . . The search policy used in `THTS++` to sample a trajectory in the selection phase.

$\hat{Q}_{\text{heuristic}}$ . . . . The heuristic action function used in `THTS++`, used to provide a Q-value estimate for any state-action pairs that aren't in the search tree.

`sample_context` A function used in `THTS++` that creates a context, or key-value store, and samples any initial values to be stored in the context.

`sample_outcome` A function used in `THTS++` to sample outcomes (successor states) from the environment (MDP).

`select_action` A function used in `THTS++` to select an action at a decision node.

`single_node_expansion` Specifies if `THTS++` samples a trajectory such that only one decision node is added to the search tree per trial. If not running with `single_node_expansion` set, `THTS++` samples a trajectory until a terminal state or the planning horizon $H_{\texttt{THTS++}}$ is reached.

$\mathcal{T}$ . . . . . The `THTS++` search tree. $\mathcal{T} \subseteq \mathcal{S} \cup \mathcal{S} \times \mathcal{A}$. Initially it consists of the root node, and new nodes are added from the trajectories/trials sampled in the selection phase.

`transposition_table` Specifies if `THTS++` uses a transposition table to store and re-use the decision nodes of the search tree when they are reached again via another path.

$\hat{V}_{\text{heuristic}}$ . . . . . The heuristic value function used in `THTS++`, used to initialise the value of a new decision node.

Upper Confidence Bound applied to Trees (Section 2.4.3)

b_{UCT} . . . . . The bias parameter used in UCT to control exploration.

`normalised_Q` . Specifies if the Q-values are linearly normalised (between 0 and 1) by the UCT search policy to select actions.

π_{rollout} . . . . . A policy used to sample a trajectory that is used to estimate the value of a state being added to a search tree.

π_{UCT} . . . . . The search policy used in UCT.

$\bar{Q}_{\text{UCT}}(s, a)$ . . . The sample-average value estimate used to estimate the Q-value at a chance node (s, a) in UCT.

$\bar{Q}^{\text{norm}}_{\text{UCT}}(s, a)$ . . . The optionally linearly normalised (between 0 and 1) Q-value. Used by the UCT search policy to select actions. If `normalised_Q` is False, then $\bar{Q}^{\text{norm}}_{\text{UCT}}(s, a) = \bar{Q}_{\text{UCT}}(s, a)$.

$\hat{V}^{\pi_{\text{rollout}}}(s)$ . . . An estimate of $V^{\pi_{\text{rollout}}}(s)$ computed using the return from a single trajectory sampled using policy π_{rollout}. It is used to initialise the value of a state being added to the search tree.

$\bar{V}_{\text{UCT}}(s)$ . . . . . The sample-average value estimate used to estimate the value at a decision node s in UCT.

Maximum ENtropy Tree Search (Section 2.4.5)

α_{MENTS} . . . . . The temperature parameter used in MENTS, which is the coefficient of the entropy terms in the maximum entropy objective.

ϵ_{MENTS} . . . . . An exploration parameter used in MENTS.

$\lambda(s, \epsilon)$ . . . . . A decaying function used to control and anneal exploration in MENTS. $\lambda(s, \epsilon) = \min\left(1, \frac{\epsilon}{\log(e+N(s))}\right)$.

$\tilde{\pi}$ . . . . . A policy neural network, used to initialise soft Q-value estimates in MENTS, if available.

π_{MENTS} . . . . . The search policy used in MENTS.

$\hat{Q}_{\text{MENTS}}(s, a)$ . . A soft Q-value estimate at the chance node (s, a) in MENTS.

$\tilde{V}$ . . . . . A function approximation to V^*.

$\hat{V}_{\text{MENTS}}(s)$ . . . A soft value estimate at the decision node s in MENTS.

Multi-Objective Reinforcement Learning (Section 2.6)

$CCS(\Pi)$ . . . . A convex coverage set of policies, which contains at least one policy that maximises the linear utility $u_{\text{lin}}(\cdot; \mathbf{w})$ for each weight vector $\mathbf{w}$.

$MCCS(\Pi)$ . . . The minimal convex coverage set of policies, which has no redundant policies and for each policy in $MCCS(\Pi)$ there is some weight that it uniquely (with respect to $MCCS(\Pi)$) obtains the optimal linear utility.

$CH(\Pi)$ . . . . . The undominated set of policies in Π (the set of all possible policies), with respect to the linear utility function u_{lin}. $CH(\Pi) = U(\Pi; u_{\text{lin}})$.

$CS(\Pi; u)$ . . . . A coverage set of policies, which contains at least one policy that maximises the utility $u(\cdot; \mathbf{w})$ for each weight vector $\mathbf{w}$.

`cvx_prune` . . . An operation that takes an arbitrary set of vectors, and returns the subset that lie at the vertices of the geometric (partial) convex hull.

Δ^D . . . . . The $D-1$ simplex simplex (with D dimensions), or, the set of possible weightings over the D objectives of a MOMDP. $\Delta^D = \{\mathbf{w} \in \mathbb{R}^D | w_i \geq 0, \sum_i w_i = 1\}$.

Π . . . . . The set of all possible policies in a MOMDP.

$PF(\Pi)$ . . . . . The Pareto front of the set of policies Π.

$\mathbf{Q}^\pi$ . . . . . The multi-objective Q-value function of a policy π. $\mathbf{Q}^\pi(s, a; t)$ denotes the expected cumulative vector reward that policy π will obtain, starting from state $s_t = s$, starting by taking action $a_t = a$.

$U(\Pi; u)$ . . . . . The undominated set of policies in Π (the set of all possible policies), with respect to the utility function u. Each policy in $U(\Pi; u)$ achieves a maximal utility for some weighting over the objectives.

u . . . . . A utility function, mapping multi-objective values to scalar values, that depends on a weighting over objectives. The multi-objective value $\mathbf{V}^\pi(s; t)$ is mapped to $u(\mathbf{V}^\pi(s; t); \mathbf{w})$, where $\mathbf{w}$ is a weighting over the objectives.

u_{lin} . . . . . The linear utility function: $u_{\text{lin}}(\mathbf{v}; \mathbf{w}) = \mathbf{w}^\top \mathbf{v}$.

$\mathbf{V}^\pi$ . . . . . The multi-objective value of a policy π. $\mathbf{V}^\pi(s; t)$ denotes the expected cumulative reward that policy π will obtain, starting from state $s_t = s$.

$\boldsymbol{\mathcal{V}}\mathbf{als}(\Pi')$ . . . . The set of multi-objective values obtained by a set of policies Π'.

$\mathbf{w}$ . . . . . A weighting over objectives that quantifies preferences over the multiple objectives, to be used in a utility function.

1 Introduction

Contents

1.1 Overview 1
1.2 Contributions 2
1.3 Structure of Thesis 4
1.4 Publications 4

TODO: chapter structure (i.e. in the introduction section I give some background in the field(s), cover the main contributions of this thesis, etc, etc).

1.1 Overview

TODO: list

- Give some context around MCTS (and talk about exploration and exploitation), and why we might use it
  - Larger scale than tabular methods
  - Can do probability and theory stuff (and some explainability, by looking at stats in the tree the agent used)
  - Can use tree search with neural networks to get some of the above (and use for neural network training as in alpha zero)

- Argument from DENTS paper for exploration > exploitation (in context of planning in a simulator)
- Give high level overview of Multi-Objective RL, and why it can be useful
- Give an idea of how my work fits into MCTS and MORL as a whole
- Discuss research questions/issues with current literature (i.e. introduce some of the ideas from contributions section below)

1.2 Contributions

TODO: Extra pre-amble before questions?

The following questions, related to Monte Carlo Tree Search and multi-objective reinforcement learning, are considered throughout this thesis:

Q1 - Exploration: When planning in a simulator with limited time, how can Monte Carlo Tree Search algorithms prioritise exploration to make effective decisions?

Q1.1 - Entropy: Entropy is used as an exploration objective in reinforcement learning — under what conditions is it compatible with optimal decision-making?

Q1.2 - Multi-Objective Exploration: Can the exploration insights from (maximum-entropy) single-objective Monte Carlo Tree Search algorithms be applied to multi-objective domains?

Q2 - Stochastic Multi-Objective Domains: Multi-objective Monte Carlo Tree Search has been studied in deterministic domains — can these methods be generalised to stochastic settings?

Q3 - Scalability: How can the scalability of Monte Carlo Tree Search methods be improved, in both single- and multi-objective settings?

Q3.1 - Complexity: Monte Carlo Tree Search algorithms typically run linearly in the number of trials, actions and decision horizon — can this computational complexity be reduced?

Q3.2 - Multi-Objective Scalability: With respect to the size of environments, how scalable are multi-objective Monte Carlo Tree Search methods?

Q3.3 - Curse of Dimensionality: With respect to the number of objectives, to what extent do multi-objective Monte Carlo Tree Search methods suffer from the curse of dimensionality?

TODO: paragraphs pointing at the different chapters and how they answer the research questions.

TODO: some words about how below is the contributions we're making in this thesis and expand these bullets a bit more

- Max Entropy can be misaligned with reward maximisation (**Q1.1 - Entropy**)
- Boltzmann Search Policies - BTS and DENTS (**Q1.1 - Entropy**)
- Use the alias method to make faster algorithms (**Q2.1 - Complexity**)
- Ch5 will answer **Q2 - Stochastic Multi-Objective Domains**
- Simple regret (**Q1 - Exploration**)
- Use of contexts in THTS to make consistent decisions in each trial (**Q1.2 - Multi-Objective Exploration**)
- Contextual regret introduced in CHMCTS (**Q2.2 - Multi-Objective Scalability**)
- Contextual Zooming and CHMCTS (designed for **Q1.2 - Multi-Objective Exploration**, runtimes cover **Q2.3 - Curse of Dimensionality**)
- Simplex maps (**Q1.2 - Multi-Objective Exploration**, **Q2.2 - Multi-Objective Scalability**, **Q2.3 - Curse of Dimensionality**)

1.3 Structure of Thesis

TODO: a paragraph with a couple lines to a paragraph about each chapter. This is the high level overview/intro to the thesis paragraph. I.e. this section is "this is the story of my thesis in a page or two"

1.4 Publications

TODO: update final publication when submit

The work covered in this thesis also appears in the following publications:

- Painter, M; Lacerda, B; and Hawes, N. "Convex Hull Monte-Carlo Tree-Search." In *Proceedings of the international conference on automated planning and scheduling. Vol. 30. 2020*, ICAPS, 2020.
- Painter, M; Baioumy, M; Hawes, N; and Lacerda, B. "Monte Carlo Tree Search With Boltzmann Exploration." In *Advances in Neural Information Processing Systems, 36, 2023*, NeurIPS, 2023.
- Painter, M; Hawes, N; and Lacerda, B. "Simplex Maps for Multi-Objective Monte Carlo Tree Search." In *TODO*, **Under Review at conf_name**.

2
Background

Contents

2.1 Multi-Armed Bandits 6
2.1.1 K-Armed Bandits and the Upper Confidence Bound Algorithm 7
2.1.2 Exploring Bandits 8
2.1.3 Contextual Bandits 10
2.2 Markov Decision Processes 11
2.3 Reinforcement Learning 14
2.3.1 Maximum Entropy Reinforcement Learning 17
2.4 Trial-Based Heuristic Tree Search and Monte Carlo Tree Search 19
2.4.1 Notation 21
2.4.2 Trial Based Heuristic Tree Search 22
2.4.3 Upper Confidence Bound Applied to Trees (UCT) . . . 28
2.4.4 Predictor-UCT (PUCT) 31
2.4.5 Maximum Entropy Tree Search 32
2.5 Convergence of Random Variables 33
2.6 Multi-Objective Reinforcement Learning 35
2.6.1 Convex Hull Value Iteration 41

This chapter introduces the fundamental concepts which are used throughout this thesis, in particular Section 2.4.2 introduces the `THTS++` schema, which will be used throughout this thesis to define Monte Carlo Tree Search algorithms modularly.

Section 2.1 briefly covers Multi-Armed Bandit problems, which are commonly used to study decision making algorithms and as a building block in Monte Carlo

Tree Search methods. In particular, *Contextual* Multi-Armed Bandit problems (Section 2.1.3) are covered, which will be relevant in the design and analysis of the multi-objective algorithms discussed in Chapters 5 and 6.

Sections 2.2 and 2.3 outline the sequential decision-making framework used in this thesis. Additionally, the Maximum Entropy Reinforcement Learning framework is covered (Section 2.3.1) which is a major topic discussed in Chapter 4.

Section 2.4 covers the Monte Carlo Tree Search framework which will be used for solving sequential decision-making problems.

Finally, Section 2.6 covers the Multi-Objective Reinforcement Learning framework, which is used to study sequential decision-making algorithms with multiple objectives.

2.1 Multi-Armed Bandits

The Multi-Armed Bandit (MAB) problem [33] is a foundational problem considered in decision theory. In the K-armed bandit problem, an agent must choose which of the K arms to pull over multiple rounds. On each round, after pulling an arm, the agent receives a reward which is sampled from a probability distribution associated with the arm.

The MAB problem was first considered by Robbins [46] in 1952, in the context of clinical trials. In a clinical trial, each patient is treated with one of two possible treatments, where the effectiveness of each treatment is unknown. In this setting, it is essential that the best treatment is given to as many patients as possible, while each treatment needs to be considered to determine its effectiveness. The balance between *exploring* to find the best treatment (arm) and *exploiting* the best treatment (arm) is known as the *exploration-exploitation trade-off*.

The remainder of this section will formalise the MAB problem, give details of the widely used Upper Confidence Bound (UCB) algorithm [3], and consider two variations on the MAB problem (Sections 2.1.2 and 2.1.3) that are relevant in this thesis.

Parameters: K probability distributions for the rewards of each arm $f(1), \ldots, f(K)$.

- For each round $m = 1, 2, \ldots$:
  - the agent selects an arm $x^m \sim \sigma^m$ to pull;
  - the agent receives a reward $y^m \sim f(x^m)$ from the environment;
  - if the environment sends a stop signal, then the game ends, otherwise the next round starts.

Figure 2.1: The procedure of a multi-armed bandit problem, following a strategy σ.

2.1.1 K-Armed Bandits and the Upper Confidence Bound Algorithm

Figure 2.1 shows how the operation of a K-armed bandit problem proceeds. Let $f(1), \ldots, f(K)$ be the probability distributions for the rewards of each of the K arms, with expected values of $\mu(1), \ldots, \mu(K)$ respectively. On the mth round, if the arm $x^m \in \{1, \ldots, K\}$ is pulled, then a reward $y^m \sim f(x^m)$ is received. To specify a *strategy* σ, on each round m, a probability distribution σ^m over $\{1, \ldots, K\}$ is given, which is sampled to decide which arm to pull, i.e. $x^m \sim \sigma^m$.

Regret is a common metric used to analyse the performance of algorithms for MAB problems, which is the difference between the cumulative reward obtained and the maximal expected reward that could be obtained with full knowledge of $\{f(i)\}$.

Definition 2.1.1. *The* (cumulative) regret *of strategy σ after n rounds in the MAB process is:*

$$\texttt{cum_regr}_{\text{MAB}}(n, \sigma) = n\mu^* - \sum_{m=1}^{n} y^m, \tag{2.1}$$

where $\mu^* = \max_i \mu(i)$.

The expected regret, $\mathbb{E}[\texttt{cum_regr}_{\text{MAB}}(n, \sigma)]$, is often used to analyse the performance of algorithms for MAB problems. Lai and Robbins [32] show a general lower bound of $\Omega(\log n)$ on the expected regret that an agent can achieve. Auer [3] introduces the Upper Confidence Bound (UCB) algorithm, which achieves a matching upper bound of $O(\log n)$ on the expected regret.

To define the UCB strategy, let $N^m(x)$ be the number of times that arm x has been pulled after m rounds, and let $\bar{y}^m(x)$ be the average reward obtained by pulling arm x after m rounds:

$$N^m(x) = \sum_{i=1}^{m} \mathbb{1}[x^i = x], \tag{2.2}$$

$$\bar{y}^m(x) = \frac{1}{N^m(x)} \sum_{i=1}^{m} y^i \mathbb{1}[x^i = x]. \tag{2.3}$$

The strategy followed by the UCB algorithm on the mth round is given by:

$$x^m_{\text{UCB}} = \arg\max_{i \in \{1,..,K\}} \bar{y}^{m-1}(i) + b_{\text{UCB}} \sqrt{\frac{\log(m)}{N^{m-1}(i)}}, \tag{2.4}$$

$$\sigma^m_{\text{UCB}}(x) = \mathbb{1}[x = x^m_{\text{UCB}}], \tag{2.5}$$

where b_{UCB} is a bias parameter used to control how much the strategy explores. In the first K rounds, UCB pulls each arm once to obtain an initial sample for each arm. Alternatively, the division by zero in Equation 2.4 can be considered to give a value of ∞, to give the same effect.

2.1.2 Exploring Bandits

In the pure exploration problem for K-armed bandits [12], the format of each round is changed slightly. The agent still pulls an arm each round and receives a reward from the environment, but after, the agent is given an opportunity to recommend which arms are the best. This better reflects scenarios where time is spent planning before any action is taken in the real world. Because the agent is only assessed on the quality of the recommendations, the agent is free to purely explore using its arm pulls and only needs to exploit when providing a recommendation.

Figure 2.2 gives an overview of the Exploring Multi-Armed Bandit (EMAB) problem. The agent follows an *exploration strategy* σ to pull an arm each round, and receives a reward from the environment. Then the agent provides a recommendation in the form of a probability distribution, or *recommendation strategy* ψ, over the K arms.

Algorithms for EMAB problems can be analysed by considering the expected regret of following the recommendation policy for one round, known as *simple regret*.

Parameters: K probability distributions for the rewards of each arm $f(1), \ldots, f(K)$.

- For each round $m = 1, 2, \ldots$:
  - the agent selects an arm $x^m \sim \sigma^m$ to pull;
  - the agent receives a reward $y^m \sim f(x^m)$ from the environment;
  - the agent outputs a recommendation distribution ψ^m, over the arms $\{1, \ldots, K\}$;
  - if the environment sends a stop signal, then the game ends, otherwise the next round starts.

Figure 2.2: The procedure of an exploring multi-armed bandit problem, following an exploration strategy σ, and recommendation strategy ψ.

Definition 2.1.2. *The* simple regret *of the recommendation strategy ψ is:*

$$\texttt{sim_regr}_{\text{EMAB}}(\psi) = \mathbb{E}_{i \sim \psi}[\mu^* - \mu(i)]. \tag{2.6}$$

Bubeck et al. [12] analyse a range of exploration and recommendation strategies in their work. Two of the recommendation strategies considered are the *empirical best arm* (EBA) and *most played arm* (MPA):

$$x^m_{\text{EBA}} = \arg\max_{i \in \{1,..,K\}} \bar{y}^m(i), \tag{2.7}$$

$$\psi^m_{\text{EBA}}(x) = \mathbb{1}[x = x^m_{\text{EBA}}], \tag{2.8}$$

$$x^m_{\text{MPA}} = \arg\max_{i \in \{1,..,K\}} N^m(i), \tag{2.9}$$

$$\psi^m_{\text{MPA}}(x) = \mathbb{1}[x = x^m_{\text{MPA}}]. \tag{2.10}$$

In addition to the UCB exploration strategy (Equation 2.5), a *uniform exploration strategy* is considered:

$$x^m_{\text{uniform}} = (m \mod K) + 1, \tag{2.11}$$

$$\sigma^m_{\text{uniform}}(x) = \mathbb{1}[x = x^m_{\text{uniform}}]. \tag{2.12}$$

It is shown that exploring with UCB leads to a polynomial upper bound on the simple regret for both recommendation strategies. That is, when ψ^m_{EBA}

and ψ^m_{MPA} are computed using visit counts and sample averages from following σ_{UCB}, then $\texttt{sim_regr}_{\text{EMAB}}(\psi^m_{\text{EBA}}) = O(m^{-k_{\text{UCB,EBA}}})$ and $\texttt{sim_regr}_{\text{EMAB}}(\psi^m_{\text{MPA}}) = O(m^{-k_{\text{UCB,MPA}}})$ for some appropriate constants $k_{\text{UCB,EBA}}, k_{\text{UCB,MPA}} > 0$.

When the uniform exploration strategy is used instead, and ψ^m_{EBA} is computed similarly using σ_{uniform}, then $\texttt{sim_regr}_{\text{EMAB}}(\psi^m_{\text{EBA}}) = O(e^{-k_{\text{uniform}}m})$ for an appropriate constant $k_{\text{uniform}} > 0$.

2.1.3 Contextual Bandits

The Contextual Multi-Armed Bandit (CMAB) problem [29, 34, 59] is another extension of the MAB problem, where the agent is provided with a context $w \in \mathcal{W}$ at the start of each round. After the arm $x \in \mathcal{X}$ is selected, the reward is sampled from the distribution $f(w, x)$, with mean $\mu(w, x)$, which also depends on the context. For each round m, a strategy maps contexts to arms via $\sigma^m(w)$. Figure 2.3 gives an overview of the CMAB problem.

It is assumed that the *similarity space* $\mathcal{W} \times \mathcal{X}$ is *compact*, and the mean reward function μ is Lipschitz continuous. This requires a *metric* d over the similarity space.

Definition 2.1.3. *d is a* metric *over the similarity space $\mathcal{W} \times \mathcal{X}$ if it satisfies the following conditions for any $z, z', z'' \in \mathcal{W} \times \mathcal{X}$ with $z \neq z' \neq z''$:*

$$d(z, z) = 0, \tag{2.13}$$

$$d(z, z') > 0, \tag{2.14}$$

$$d(z, z') = d(z', z), \tag{2.15}$$

$$d(z, z'') \leq d(z, z') + d(z', z''). \tag{2.16}$$

μ is then Lipschitz continuous if the following condition holds:

$$|\mu(w, x) - \mu(w, x')| \leq d((w, x), (w, x')). \tag{2.17}$$

The performance of CMAB algorithms is analysed with *contextual regret*, a notion of regret that accounts for the contexts drawn.

Parameters: the set of arms $\mathcal{X}$, the set of contexts $\mathcal{W}$ and a mapping from $\mathcal{W} \times \mathcal{X}$ to probability distributions: $f(w, x)$.

- For each round $m = 1, 2, \ldots$:
  - the agent receives a context $w^m \in \mathcal{W}$ from the environment;
  - the agent selects an arm $x^m \sim \sigma^m(w^m)$ to pull;
  - the agent receives a reward $y^m \sim f(w^m, x^m)$ from the environment;
  - if the environment sends a stop signal, then the game ends, otherwise the next round starts.

Figure 2.3: The procedure of a contextual multi-armed bandit problem, following a strategy σ.

Definition 2.1.4. *The* (cumulative) contextual regret *of the strategy σ over n rounds in the process given in Figure 2.3 is defined as:*

$$\texttt{ctx_regr}_{\text{CMAB}}(n, \sigma) = \sum_{m=1}^{n} \mu^*(w^m) - y^m, \tag{2.18}$$

where $\mu^*(w) = \sup_i \mu(w, i)$.

The *contextual zooming* algorithm [29] is an algorithm for CMAB problems which is presented where relevant in TODO: ref to where it got updated and placed.

2.2 Markov Decision Processes

Markov decision processes (MDPs) provide a mathematical description of sequential decision making problems [38]. This section provides definitions and notation related to MDPs used throughout this thesis.

MDPs are used throughout this thesis to describe the environments in which agents act. An MDP contains, among other things, a set of states and actions. States are sampled according to a transition distribution that depends on the current state and current action being taken (the Markov assumption). Whenever an action is taken, the agent receives a reward according to a reward function that depends on the current state and action.

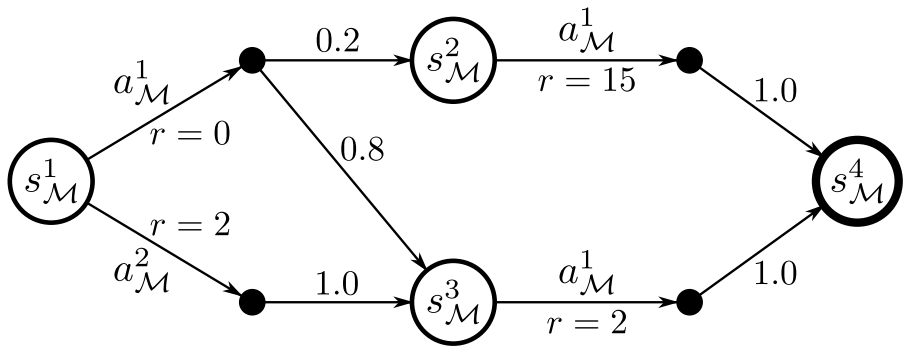


Figure 2.4: An example MDP $\mathcal{M}$, where $s_0 = s^1_{\mathcal{M}}$, the finite horizon is $H = 2$, and terminal states are marked with a thicker border. Edges are appropriately marked with actions and rewards, or marked with transition probabilities.

In this thesis, unless otherwise stated, MDPs are assumed to have finite state and action spaces, and *trajectories* (sequences of states, actions and rewards) are of finite length.

Definition 2.2.1. *A* (Finite-Horizon) Markov decision process *(MDP) is a tuple* $\mathcal{M} = (\mathcal{S}, s_0, \mathcal{A}, R, p, H)$*, where* $\mathcal{S}$ *is a set of states,* $s_0 \in \mathcal{S}$ *is an initial state,* $\mathcal{A}$ *is a set of actions,* $R(s, a)$ *is the reward function* $\mathcal{S} \times \mathcal{A} \to \mathbb{R}$*,* $p(s'|s, a)$ *is a next-state transition distribution* $\mathcal{S} \times \mathcal{A} \to (\mathcal{S} \to [0, 1])$ *and* $H \in \mathbb{N}$ *is the time horizon.*

An example MDP is shown in Figure 2.4. The set of *successor states* is the set of states that can be reached after taking an action from the current state of the MDP:

Definition 2.2.2. *The set of* successor states $\mathrm{Succ}(s, a)$ *of a state-action pair* (s, a)*, with respect to an MDP, is defined as:*

$$\mathrm{Succ}(s, a) := \{s' \in \mathcal{S} | p(s'|s, a) > 0\}. \tag{2.19}$$

The set of reachable states from the starting state s_0 can then be defined as the smallest set closed under the successor relation:

Definition 2.2.3. *The set of* reachable states $\mathcal{S}_0$ *from the starting state* s_0 *is the smallest set satisfying:*

$$s_0 \in \mathcal{S}_0, \qquad s \in \mathcal{S}_0 \Rightarrow \bigcup_{a \in \mathcal{A}} \mathrm{Succ}(s, a) \subseteq \mathcal{S}_0. \tag{2.20}$$

In this thesis, MDPs may contain *trap states*, which are states that, once reached, can never be left and from which no further reward is obtained.

Definition 2.2.4. *A state $s \in \mathcal{S}$ is a* terminal *state (equivalently, a* trap *state), if the transition distribution always returns the same state (i.e. $p(s|s,a) = 1$ for all actions a), and if no more reward is obtained from that state ($R(s,a) = 0$ for all a). The set of terminal states is denoted* `terminal` $\subseteq \mathcal{S}$.

A *policy* maps each state to a probability distribution over the action space, and defines a strategy that an agent will follow in an MDP. Often it is desirable to define deterministic policies, which always produce the same action when given the same state, and can be represented as *one-hot* distributions.

Definition 2.2.5. *A* (stochastic) policy $\pi : \mathcal{S} \to (\mathcal{A} \to [0,1])$ *is a mapping from states to a probability distribution over actions. $\pi(a|s)$ is the probability of sampling action a at state s. The policy π must satisfy: $\sum_{a \in \mathcal{A}} \pi(a|s) = 1$ and $\pi(a|s) \geq 0$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}$.*

Additionally, a deterministic policy *is defined as a one-hot policy, that is, the policy π is deterministic if it can be written as $\pi(a|s) = \mathbb{1}[a = a']$ for some $a' \in \mathcal{A}$.*

Moreover, the following notations are used for policies:

- *$a \sim \pi(\cdot|s)$ denotes sampling an action a from the distribution $\pi(\cdot|s)$;*
- *$\pi(s) = a'$ is a shorthand to define the deterministic policy $\pi(a|s) = \mathbb{1}[a = a']$;*
- *$\pi(s)$ denotes the action a' of the deterministic policy $\pi(a|s) = \mathbb{1}[a = a']$.*

Given an MDP $\mathcal{M}$ and a policy π, a *trajectory* is a sampled sequence of states, actions and rewards, where the policy is used to sample actions and the MDP is used to sample the next state. A trajectory simulates one possible sequence that could occur if an agent follows policy π in $\mathcal{M}$.

Definition 2.2.6. *A* trajectory *τ is a sequence of states, actions and rewards that is induced by a policy π and MDP $\mathcal{M}$. Let the trajectory be $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_{H-1}, a_{H-1}, r_{H-1}, s_H)$, where $a_t \sim \pi(\cdot|s_t)$, $r_t = R(s_t, a_t)$ and $s_{t+1} \sim p(\cdot|s_t, a_t)$.*

The following notation is also used for trajectories:

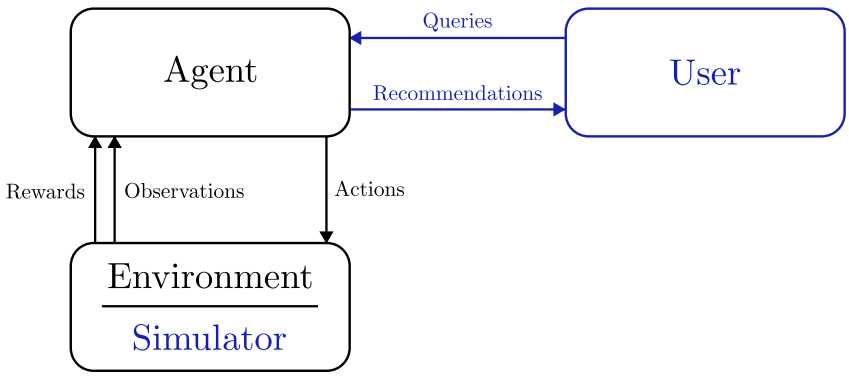


Figure 2.5: An overview of reinforcement learning. Depicted in black is the standard agent-environment interface for reinforcement learning [61], where an agent can perform actions in an environment and is given feedback in the form of observations and rewards. In blue the environment is replaced by a simulated environment which the agent can use for planning, and is queried by a user for recommendations on how to act in the real environment.

- *$\tau \sim \pi$ denotes a trajectory that is sampled using the policy π, where the MDP $\mathcal{M}$ is implicit;*
- *$\tau_{i:j}$ denotes the* truncated trajectory *$\tau_{i:j} := (s_i, a_i, r_i, s_{i+1}, \ldots, s_{j-1}, a_{j-1}, r_{j-1}, s_j)$, between the timesteps $0 \leq i < j \leq H$ inclusive;*
- *$\tau_{:j} := \tau_{0:j}$ denotes a trajectory that is truncated on only one end;*
- *$s \in \tau$ is used as a shorthand for $s \in \{s_i | i = 0, \ldots, H\}$;*
- *$(s, a) \in \tau$ is used as a shorthand for $(s, a) \in \{(s_i, a_i) | i = 0, \ldots, H-1\}$;*
- *and finally, when states, actions and rewards are timestep-indexed (i.e. s_t, a_t, r_t), they always correspond to a trajectory.*

2.3 Reinforcement Learning

This section provides a brief introduction to fundamental concepts and definitions in reinforcement learning and motivates an alternative lens on reinforcement learning considered in this thesis. Typically reinforcement learning is presented as a sequence of agent-environment interactions, where the agent takes actions in an environment

and receives *observations* and *rewards* for those actions [61]. The objective of the agent is to maximise the rewards that it receives.

However, in the modern era of reinforcement learning, the agent is often equipped with a simulator in which it can plan, learn and explore. The "target" environment that the user cares about may be the same simulated environment, or a real-world environment where actions carry genuine costs, unlike interactions with the simulator. After the learning phase, one typically cares only about the policy that the agent recommends at the end, rather than the rewards accrued during planning.

Figure 2.5 depicts the classical reinforcement learning setting (in black) where the agent interacts with the environment directly, and this alternative setting (in blue) where the agent is equipped with a simulator to plan and explore, until a user queries the agent for a recommendation. This perspective shift changes the nature of the exploration-exploitation trade off, almost separating the two issues. During the learning phase there is a greater emphasis on exploring, while recommendations are concerned with pure exploitation. This perspective on reinforcement learning motivates the research questions around exploration: **Q1 - Exploration**.

Following on from Section 2.2, the remainder of this section defines *value functions* and the objectives of reinforcement learning and covers *Value Iteration*, a tabular dynamic programming approach to reinforcement learning. Finally, Section 2.3.1 covers *maximum-entropy reinforcement learning.*

The value of a policy π is the expected cumulative reward that is obtained by following the policy:

Definition 2.3.1. *The* value *of a policy π from state s at time t is:*

$$V^{\pi}(s;t) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{i=t}^{H-1} r_i \middle| s_t = s\right]. \tag{2.21}$$

The Q-value *of a policy π, from state s, with action a, at time t is:*

$$Q^{\pi}(s,a;t) = R(s,a) + \mathbb{E}_{s'\sim p(\cdot|s,a)}[V^{\pi}(s';t+1)]. \tag{2.22}$$

From the definition of the value functions, the optimal value functions can be defined by taking the maximum value over all policies:

Definition 2.3.2. *The* Optimal (Q-)Value *of a state(-action pair) is defined as:*

$$V^*(s;t) = \max_\pi V^\pi(s;t) \tag{2.23}$$

$$Q^*(s,a;t) = \max_\pi Q^\pi(s,a;t). \tag{2.24}$$

Value functions can also be used to define an objective function:

Definition 2.3.3. *The* (standard) reinforcement learning objective function $J(\pi)$ *is defined as:*

$$J(\pi) = V^\pi(s_0;0). \tag{2.25}$$

The objective of (standard) reinforcement learning can then be stated as finding $\max_\pi J(\pi)$.

The *optimal policy* is the policy that maximises the objective function J, and can be shown to be deterministic [38]:

Definition 2.3.4. *The* optimal (standard) policy π^* *is the policy maximising the standard objective function:*

$$\pi^* = \arg\max_\pi J(\pi) \tag{2.26}$$

or equivalently, the optimal standard policy can be expressed in terms of the optimal Q-value function:

$$\pi^*(s) = \arg\max_a Q^*(s,a). \tag{2.27}$$

It can be shown that the optimal (Q-)value functions satisfy the *Bellman equations* [7, 38]:

$$V^*(s;t) = \max_{a\in\mathcal{A}} Q^*(s,a;t), \tag{2.28}$$

$$Q^*(s,a;t) = R(s,a) + \mathbb{E}_{s'\sim p(\cdot|s,a)}[V^*(s';t+1)]. \tag{2.29}$$

The Bellman equations admit a *dynamic programming* approach which can be used to compute the optimal value functions, known as *Value Iteration* [7, 38]. In

Value Iteration, a table of value estimates $\hat{V}(s;t)$ is kept for each s, t. Given any initial estimate of the value function $\hat{V}^0$, the *Bellman backup* operations are:

$$\hat{V}^{k+1}(s;t) = \max_{a \in \mathcal{A}} \hat{Q}^{k+1}(s,a;t), \tag{2.30}$$

$$\hat{Q}^{k+1}(s,a;t) = \mathbb{E}_{s' \sim p(\cdot|s,a)}[R(s,a) + \hat{V}^k(s';t+1)]. \tag{2.31}$$

In each iteration of Value Iteration, these values are computed for all $s \in \mathcal{S}$, $a \in \mathcal{A}$ and $t \in \{0, 1, \ldots, H\}$. The value estimates computed from the Bellman backups will converge to the optimal values in a finite number of iterations: $V^k \to V^*$.[1]

2.3.1 Maximum Entropy Reinforcement Learning

In *maximum-entropy reinforcement learning*, the objective function is altered by the addition of an entropy term. This is motivated by the desire to learn stochastic behaviours that better explore large state spaces and are more robust under uncertainty by learning multiple solutions rather than a single deterministic solution [23].

Let $\mathcal{H}$ denote the (Shannon) entropy function [54]:

$$\mathcal{H}(\pi(\cdot|s)) = -\mathbb{E}_{a \sim \pi(\cdot|s)}[\log \pi(a|s)] = -\sum_{a \in \mathcal{A}} \pi(a|s) \log \pi(a|s). \tag{2.32}$$

In the maximum entropy objective, the relative weighting of entropy terms is included using a coefficient α, called the *temperature*. In the maximum entropy objective, analogues of the value functions can be defined, which are typically referred to as *soft (Q-)values*, and similarly the maximum entropy objective is often referred to as the *soft objective*. Where disambiguation is necessary, the *standard (Q-)value* (Definition 2.3.1) and *standard objective* (Definition 2.3.3) will be used to refer to the standard reinforcement learning counterparts.

Definition 2.3.5. *The* soft value *of a policy π from state s at time t is:*

$$V^{\pi}_{\text{sft}}(s;t) = \mathbb{E}_{\tau \sim \pi}\left[\sum_{i=t}^{H-1} r_i + \alpha \mathcal{H}(\pi(\cdot|s_i)) \middle| s_t = s\right]. \tag{2.33}$$

The soft Q-value *of a policy π, from state s, with action a, at time t is:*

$$Q^{\pi}_{\text{sft}}(s,a;t) = R(s,a) + \mathbb{E}_{s' \sim p(\cdot|s,a)}[V^{\pi}_{\text{sft}}(s';t+1)]. \tag{2.34}$$

[1] Strictly speaking, if backups are performed in reverse order from $t = H$ to $t = 0$, convergence is achieved in a single iteration. The iterative presentation here is adopted to more closely resemble the backup procedures used later in the thesis, where the transition distribution p is unknown.

The optimal soft (Q-)values can be defined by taking the maximum over policies, similar to the standard setting:

Definition 2.3.6. *The* optimal soft (Q-)Value *of a state(-action pair) is defined as:*

$$V^*_{\text{sft}}(s;t) = \max_{\pi} V^{\pi}_{\text{sft}}(s;t), \tag{2.35}$$

$$Q^*_{\text{sft}}(s,a;t) = \max_{\pi} Q^{\pi}_{\text{sft}}(s,a;t). \tag{2.36}$$

In maximum entropy reinforcement learning, the objective is to find a policy with maximal soft value.

Definition 2.3.7. *The* maximum entropy (or soft) reinforcement learning objective function $J_{\text{sft}}(\pi)$ *is defined as:*

$$J_{\text{sft}}(\pi) = V^{\pi}_{\text{sft}}(s_0;0). \tag{2.37}$$

The objective of maximum entropy (or soft) reinforcement learning can then be stated as finding $\max_{\pi} J_{\text{sft}}(\pi)$.

The optimal soft policy is defined as the policy that maximises the soft objective function J_{sft}, and it can be computed uniquely from the optimal soft (Q-)values.

Definition 2.3.8. *The* optimal soft policy π^*_{sft} *is the policy maximising the soft objective function:*

$$\pi^*_{\text{sft}} = \arg\max_{\pi} J_{\text{sft}}(\pi). \tag{2.38}$$

Given V^*_{sft} *and* Q^*_{sft} *the optimal soft policy is known to take the form [23]:*

$$\pi^*_{\text{sft}}(a|s;t) = \exp\left(\left(Q^*_{\text{sft}}(s,a;t) - V^*_{\text{sft}}(s;t)\right)/\alpha\right). \tag{2.39}$$

The *soft Bellman equations* are the maximum entropy analogues of the Bellman equations. These equations differ from equations (2.28) and (2.29) by the replacement of the max operation with a *softmax* or *log-sum-exp* operation, which

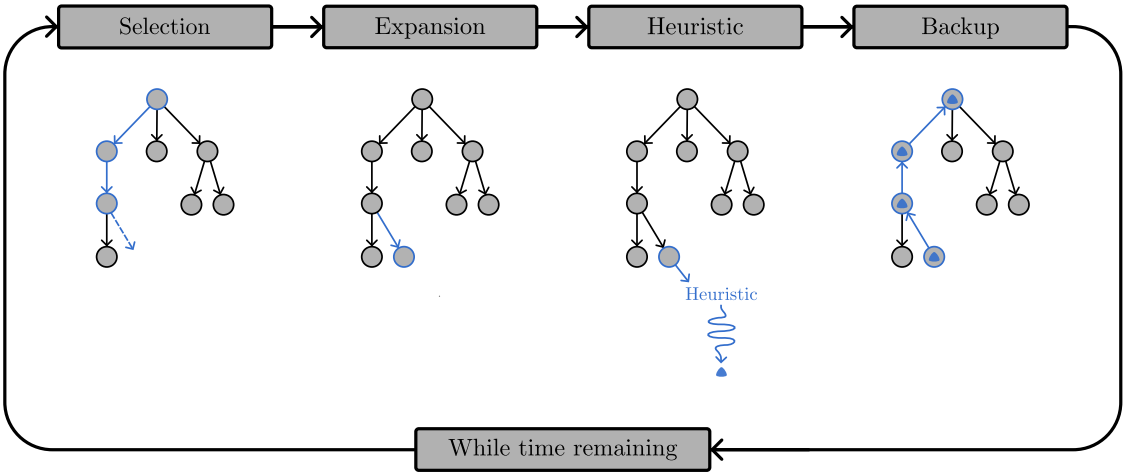


Figure 2.6: Overview of one trial of MCTS-1, depicting the four commonly presented stages [11]. First, in the selection phase a search policy is used to sample a path down the tree. Next a new node is added to the search tree. Then the new node is initialised using a heuristic, often using a *simulation* using a *rollout policy* as depicted. Finally, values (triangles) are updated through *backups* along the path sampled in the selection phase.

is why the maximum entropy analogues are referred to as the *soft* versions of their standard reinforcement learning counterparts. The *soft Bellman equations* are [23]:

$$V^*_{\text{sft}}(s;t) = \alpha \log \sum_{a \in \mathcal{A}} \exp\left(Q^*_{\text{sft}}(s,a;t)/\alpha\right), \tag{2.40}$$

$$Q^*_{\text{sft}}(s,a;t) = R(s,a) + \mathbb{E}_{s' \sim p(\cdot|s,a)}[V^*_{\text{sft}}(s';t+1)]. \tag{2.41}$$

Analogous to standard reinforcement learning, the soft Bellman equations can be used in *soft Bellman backups* for a *Soft Value Iteration* algorithm [23]:

$$\hat{V}^{k+1}_{\text{sft}}(s;t) = \alpha \log \sum_{a \in \mathcal{A}} \exp\left(\frac{1}{\alpha}\hat{Q}^{k+1}_{\text{sft}}(s,a;t)\right), \tag{2.42}$$

$$\hat{Q}^{k+1}_{\text{sft}}(s,a;t) = R(s,a) + \mathbb{E}_{s' \sim p(\cdot|s,a)}[\hat{V}^{k}_{\text{sft}}(s';t+1)]. \tag{2.43}$$

2.4 Trial-Based Heuristic Tree Search and Monte Carlo Tree Search

When considering MDPs with large state-action spaces, using tabular dynamic programming methods becomes infeasibly slow. One approach for handling large state-action spaces is to use *heuristic methods* that consider a subset of the state-action space and utilise heuristic estimates for the value of states. Often these heuristic methods build a *search tree* from the initial state of the MDP.

Monte Carlo Tree Search (MCTS) refers to heuristic algorithms that build a search tree using *Monte Carlo Trials*, where nodes are added to the tree from randomly sampled trajectories. Two advantages of using MCTS methods in the modern day are: that they allow for statistical analysis, which may provide *probabilistic guarantees* on the performance of the algorithm; and that they offer interpretability, as the search tree can be inspected post-hoc to identify why the algorithm made certain decisions.

MCTS trials are commonly presented using four stages, selection, expansion, simulation/heuristic and backup phases [11], which are depicted in Figure 2.6. To avoid confusion with the broader definition of MCTS above, this thesis refers to this specific form of MCTS as MCTS-1. Each of the four phases of MCTS-1 are described in more detail below:

1. *Selection phase* - a *search policy* is used to traverse the search tree from the root node until it reaches a state not contained in the search tree;

2. *Expansion phase* - a new child node is added to the search tree;

3. *Simulation/Heuristic phase* - the new child node's value is initialised using a *heuristic*, often taking the form of a Monte Carlo return, obtained using a *simulation* with a *rollout policy*;

4. *Backup phase* - the value from the heuristic or simulation is used to back up values through the path traversed in the selection phase.

Trial-Based Heuristic Tree Search (THTS) [28] generalises heuristic tree search methods used for planning, such as Trial-Based Real Time Dynamic Programming [6] and LAO* [24]. However, THTS differs slightly from how MCTS-1 is presented above. In a THTS trial, after the expansion/heuristic phases are run, the trial "switches back to the selection phase and alternates between those phases", until the planning horizon is reached.

`THTS++` [40] is introduced in Section 2.4.2, which is an open-source, parallelised extension of the THTS schema — which also generalises MCTS-1 algorithms —

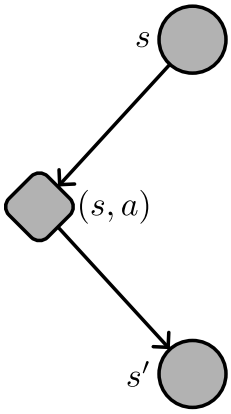


Figure 2.7: Tree diagrams notation, circles denote *decision nodes* that are associated with states, diamonds denote *chance nodes* that are associated with state-action pairs and arrows denote parent/child relationships in the tree.

and was built to facilitate the work in this thesis. Section 2.4.3 covers the Upper Confidence Bound applied to Trees (UCT) algorithm [30, 31], which is a commonly used MCTS algorithm, and Section 2.4.5 presents the Maximum ENtropy Tree Search (MENTS) algorithm [69], which is related to the algorithms introduced in Chapter 4 of this thesis.

2.4.1 Notation

To simplify notation in the presentation and analysis of `THTS++` algorithms, this thesis assumes that states and state-action pairs have a one-to-one correspondence with nodes in the search tree. This *unique state assumption* is purely to simplify notation for a clean presentation, and any results discussed in this thesis generalise to settings where this assumption does not hold.

Specifically, this allows the notation for value functions to avoid explicitly writing the timestep parameter, so that $V^\pi(s)$ can be written instead of $V^\pi(s; t)$. With the exception of Section 2.6, the timestep parameter is dropped for the remainder of this thesis.

`THTS++` builds a *search tree* $\mathcal{T}$ incrementally by running *trials* from the root node and adding them to the tree until a stopping criterion is met. Trees consist of *decision nodes* that correspond to states $s \in \mathcal{T}$, and *chance nodes* that correspond to state-action pairs $(s, a) \in \mathcal{T}$. For brevity, s refers to both states and decision nodes,

and (s, a) refers to both state-action pairs and chance nodes. Where necessary for clarity, the notation $\texttt{node}(s)$ and $\texttt{node}(s, a)$ is used to explicitly refer to nodes rather than states or state-action pairs. At the start of search, the search tree is initialised to contain only the root node, $\mathcal{T} \leftarrow \{s_0\}$. Figure 2.7 gives an example of how decision and chance nodes are depicted as circles and diamonds in diagrams throughout this thesis.

Definition 2.4.1. *A* search tree $\mathcal{T}$ *is a subset of the state and state-action spaces,* $\mathcal{T} \subseteq \mathcal{S} \cup \mathcal{S} \times \mathcal{A}$*, where for each* $s \in \mathcal{T}$*, there exists some truncated trajectory* $\tau_{:h}$ *such that* $s_h = s$*, each* $s' \in \tau_{:h}$ *is also in the tree,* $s' \in \mathcal{T}$*, and each* $s', a' \in \tau_{:h}$ *is also in the tree,* $(s', a') \in \mathcal{T}$*.*

Each decision and chance node stores some algorithm-dependent data, typically including but not limited to a *value estimate* and a *visit count*. Let $N(s)$ and $N(s, a)$ denote the number of times $\texttt{node}(s)$ and $\texttt{node}(s, a)$ have been visited, or equivalently, the number of times s and (s, a) appear in trajectories sampled in `THTS++` trials. $\texttt{node}(s)\texttt{.data}$ denotes the set of variables stored at the decision node $\texttt{node}(s)$, and $\texttt{node}(s, a)\texttt{.data}$ denotes the set of variables stored at the chance node $\texttt{node}(s, a)$. For example $N(s) \in \texttt{node}(s)\texttt{.data}$ and $N(s, a) \in \texttt{node}(s, a)\texttt{.data}$.

Additionally, let $\texttt{node}(s)\texttt{.chldrn}$ be a mapping from actions to the chance nodes that are children of $\texttt{node}(s)$. Likewise, $\texttt{node}(s, a)\texttt{.chldrn}$ is a mapping from outcomes (successor states) to the decision nodes that are children of $\texttt{node}(s, a)$. For example, $\texttt{node}(s)\texttt{.chldrn}[a] = \texttt{node}(s, a)$.

2.4.2 Trial Based Heuristic Tree Search

`THTS++` [40] provides a modular framework for defining and implementing MCTS algorithms. It extends the Trial-Based Heuristic Tree Search (THTS) schema [28] in a way that incorporates MCTS-1 algorithms and more modern MCTS algorithms that use a neural network or heuristic in place of a rollout simulation [56, 57]. It is largely presented in a single-threaded manner, which is sufficient for the purposes of this thesis.

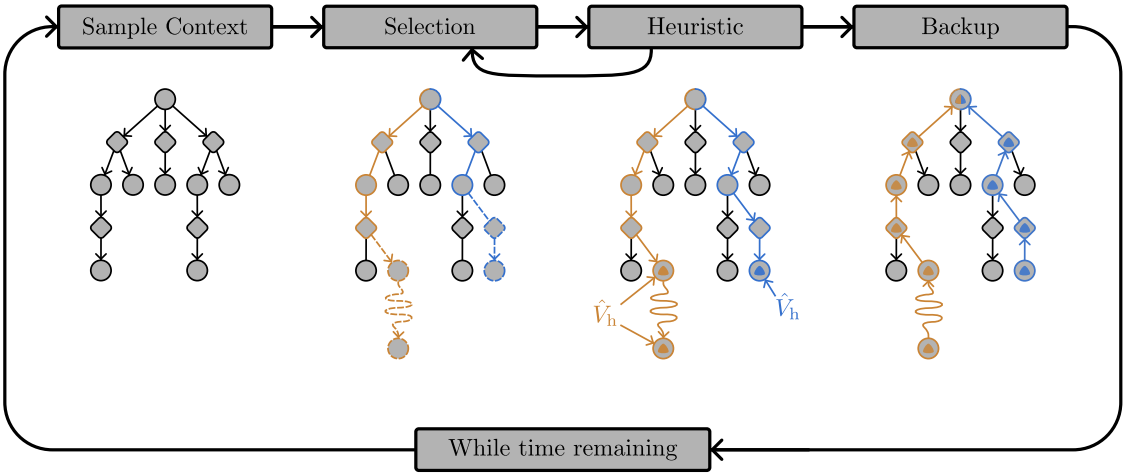


Figure 2.8: Overview of one trial of `THTS++`, blue shows an example when `single_node_expansion` is True, and orange shows an example when `single_node_expansion` is False. From left to right: first a context is sampled, which stores any necessary per-trial state (not depicted) and the search tree at the beginning of the trial is shown; second shows the selection phase, where a trajectory is sampled and where dashed lines indicate any new nodes to be added; third shows that new nodes are initialised using the heuristic function $\hat{V}_{\text{heuristic}}$ (written as $\hat{V}_h$ for short); and finally on the right, shows the backup phase, where the arrow directions are changed to show that information is being propagated back up the tree to update the values in the nodes (triangles).

Each trial has an associated *context*, which is a *per-trial* key-value store that can be used to store any algorithm-dependent data. The algorithms defined in Sections 2.4.3 and 2.4.5 will not make use of this context, but it will be used in Chapters 5 and 6 for multi-objective MCTS algorithms where *contextual tree search* is discussed. Contexts allow algorithms to store additional information within a trial without altering the interface. Beyond the scope of this thesis, contexts enable algorithms that require additional per-trial state to be implemented in `THTS++`, such as Partially Observable Monte Carlo Planning [55], where a state is sampled from a *belief state* at the beginning of each trial.

Definition 2.4.2. *A* context *is an arbitrary key-value store used to store any per-trial data.*

The main four steps performed during a `THTS++` trial are (depicted in Figure 2.8):

1. *Context Sampling* - a *context* is sampled for the trial;

2. *Selection Step* - starting from a decision node s, an action is selected to reach a chance node (s, a), and an outcome is sampled to reach the next decision node s'. (s, a) and s' are added to the search tree $\mathcal{T}$ if they were not already.

3. *Heuristic Step* - any new nodes added from the selection step are initialised using a *heuristic function*;

4. *Backup Step* - value estimates in the search tree are updated, along the path sampled in the selection step(s).

Collectively, the selection and heuristic steps are referred to as the *selection phase* of the trial, and the backup steps are collectively referred to as the *backup phase*.

In `THTS++` there are two options for when to progress from the selection phase to the backup phase, which is indicated by the boolean flag `single_node_expansion`. The selection phase will always end if a terminal state is reached, which includes reaching the planning horizon. When `single_node_expansion` is True, the selection phase will end earlier, once a new decision node is added to the search tree. Figure 2.8 shows an example of a trial when `single_node_expansion` is True (blue), and when it is False (orange).

Definition 2.4.3. *When* `single_node_expansion` *is True, the selection phase will end once a new decision node is added to the search tree.*

If the search is equipped with an informative heuristic function, then running with `single_node_expansion` set to True can allow the algorithm to avoid wasting resources (time and memory) on parts of the search tree that are not promising. Moreover, if the search is using a large planning horizon or the search will run for a long time, then it may be necessary to run with `single_node_expansion` set to True to avoid running out of memory.

In contrast, a random *rollout* is often used in MCTS-1 algorithms when no informative heuristic is available (for example Section 2.4.3). In such cases, where memory allows, running with `single_node_expansion` set to False can be beneficial,

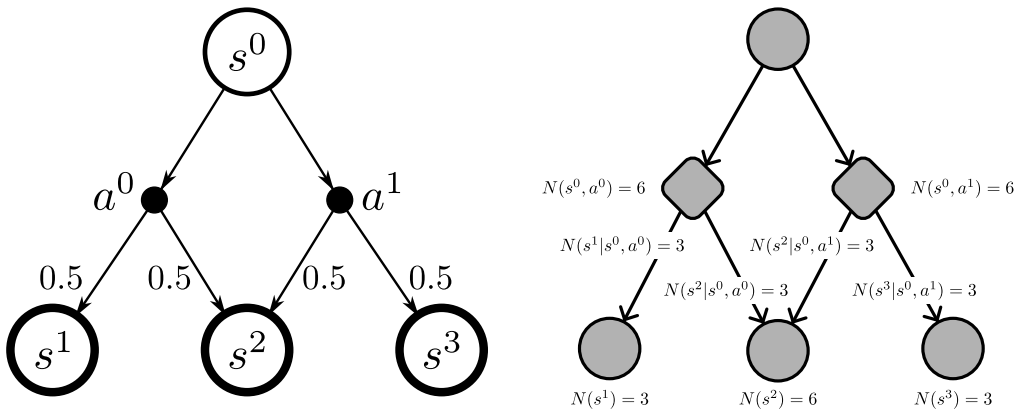


Figure 2.9: A complication arising from the use of a transposition table. Many MCTS algorithms use the ratio $\frac{N(s')}{N(s,a)}$ as an empirical estimate of transition probability $p(s'|s,a)$. Left: an example MDP with horizon $H = 1$ where using this ratio to estimate transition probabilities gives the incorrect ratios. Right: the corresponding *acyclic graph* created by running `THTS++` on the MDP with `transposition_table` set to True and sampling actions uniformly. After 12 trials, the resulting visit counts at the leaf nodes are $N(s^1) = 3, N(s^2) = 6, N(s^3) = 3$. The ratio of visit counts $\frac{N(s^1)}{N(s^0,a^1)} = \frac{1}{2}$ and $\frac{N(s^2)}{N(s^0,a^1)} = 1$, which do not form an (empirical) probability distribution, and even if normalised do not give the correct uniform distribution over successor states. To fix this issue, the *conditional visit counts* $N(s'|s,a)$ for $s' \in \text{Succ}((,s),a)$ are needed instead, that keep track of the number of times s' was sampled from the chance node (s,a). In this example, the conditional visit counts at node (s^0,a^0) would be $N(s^1|s^0,a^0) = 3$ and $N(s^2|s^0,a^0) = 3$, and give the correct ratios of $\frac{N(s^1|s^0,a^0)}{N(s^0,a^0)} = \frac{N(s^2|s^0,a^0)}{N(s^0,a^0)} = \frac{1}{2}$. That is, the ratios $\frac{N(s'|s,a)}{N(s,a)}$ correctly maintain an empirical estimate for the probability $p(s'|s,a)$, even when using a transposition table.

because it avoids throwing away potentially useful information from states visited during the rollout [60, 63].

If the unique state assumption (Section 2.4.1) is set aside, environments may contain *transpositions*, where the same state can be reached from multiple different paths. When the `transposition_table` flag is set, a *transposition table* is used to re-use the `node`(s) when the state s is reached again via another path [14, 51, 68].

Many MCTS algorithms, whether explicitly or implicitly, use the ratio of visit counts, $\frac{N(s')}{N(s,a)}$, as an empirical estimate of the transition distribution $p(s'|s,a)$. Figure 2.9 illustrates the issue that arises with this when using a transposition table and how it can be resolved, following an argument similar to that of Wu [68].

To run an algorithm in the `THTS++` schema, the following *modules* and *parameters* need to be specified. Algorithms specify the operation of each module:

Value Estimates: What data will be stored at each decision and chance node. That is, `node`(s)`.data` and `node`(s, a)`.data` need to be defined. It is assumed that `node`(s)`.data` and `node`(s, a)`.data` always contain the visit counts $N(s)$, and $N(s, a)$ respectively;

Initialisation: Two functions `init_v` and `init_q` which specify how to initialise `node`(s)`.data` and `node`(s, a)`.data` respectively;

Context Sampler: A function `sample_context` which creates a context key-value store, and samples any initial values to be stored in the context;

Action Selection (Search Policy): A function `select_action` which selects an action at a decision node. In this thesis, this will be specified as a function π_{search}, and `select_action` will return an action sampled from the search policy: $a \sim \pi_{\text{search}}(s)$;

(Optional) Outcome Sampler: A function `sample_outcome` that samples outcomes from chance nodes. By default, this will sample a state from the transition distribution of the MDP: $s' \sim p(\cdot|s, a)$ and this default will always be used in this thesis[2];

Backup functions: Two functions `backup_v` and `backup_q` which specify how to update `node`(s)`.data` and `node`(s, a)`.data` respectively.

Parameters are set on a per-environment basis:

(Optional) Heuristic functions: The functions $\hat{V}_{\text{heuristic}}$, $\hat{Q}_{\text{heuristic}}$ and $\pi_{\text{heuristic}}$ are the heuristic functions. $\hat{V}_{\text{heuristic}}$ is used to initialise value estimates in `node`(s)`.data`. $\hat{Q}_{\text{heuristic}}$ and $\pi_{\text{heuristic}}$ can be used by the search policy to incorporate prior knowledge about unexplored actions. If not specified then by default $\hat{V}_{\text{heuristic}}$ and $\hat{Q}_{\text{heuristic}}$ are set to zero, and $\pi_{\text{heuristic}}$ is set to a uniform distribution;

[2] One may want to use a different `sample_outcome` function to implement *double progressive widening* [16] for example.

Single Node Expansion: A boolean flag, `single_node_expansion`, specifying whether `THTS++` should only expand one decision node per trial;

Transposition Table: A boolean flag, `transposition_table`, specifying whether `THTS++` should use a transposition table to re-use nodes when the same state is reached via another path;

Planning horizon: A (problem dependent) planning horizon for the tree search $H_{\texttt{THTS++}}$, which is no longer than the horizon of the MDP, $H_{\texttt{THTS++}} \leq H$[3].

Given all of the modules and parameters are specified, the full operation of a trial in `THTS++` is described below. Additionally, pseudocode is given in Listing 2.1.

Sample Context: At the start of the trial, a context is sampled using the `sample_context` function, and the context is passed to all subsequent steps of the trial.

Selection Phase: A trajectory is sampled using the `select_action` and `sample_outcome` functions, and new nodes are initialised using the heuristic functions and added to the search tree. More specifically, starting with $i = 0$:

Selection Step: Action a_i is selected using `select_action`(s_i), and an outcome s_{i+1} is sampled using `sample_outcome`(s_i, a_i).

Heuristic Step: If $(s_i, a_i) \notin \mathcal{T}$ initialise `node`(s_i, a_i)`.data` using `init_q`, and if $s_{i+1} \notin \mathcal{T}$ initialise `node`(s_{i+1})`.data` using `init_v`. New nodes are added to the tree, $\mathcal{T} \leftarrow \mathcal{T} \cup \{(s_i, a_i), s_{i+1}\}$.

If $s_{i+1} \in$ `terminal` or $i + 1 = H_{\texttt{THTS++}}$, then proceed to the backup phase.

If `single_node_expansion` and s_{i+1} was just added to the tree, then proceed to the backup phase.

Otherwise, increment i and return to the selection step.

[3] In this thesis, MDPs have a finite horizon H, so $H_{\texttt{THTS++}}$ is somewhat redundant. However, it is useful in practice for environments with a long or infinite horizon, where a finite planning horizon is still required for the tree search.

Backup Phase: For $i = h-1, \ldots, 0$, where h is the final value of i in the selection phase, the visit counts $N(s_i)$ and $N(s_i, a_i)$ are incremented, and the `backup_q` and `backup_v` functions are used to update the values of `node`(s_i, a_i)`.data` and `node`(s_i)`.data` respectively.

2.4.3 Upper Confidence Bound Applied to Trees (UCT)

Upper Confidence Bound Applied to Trees (UCT) [30, 31] is a commonly used tree search algorithm, which is based on the Upper Confidence Bound (UCB) algorithm covered in Section 2.1. UCT is a good example of a common paradigm in tree search algorithms for sequential decision making, where each node in the tree is tasked with making a single decision in the sequence, and so adapts methods from the MAB literature. More specifically, UCT can be viewed as running UCB on a non-stationary MAB at every decision node, where it is non-stationary because the rewards obtained depend on the decisions that children make. The remainder of this subsection will outline how UCT can be defined using the `THTS++` schema given in Section 2.4.2.

The variables used in UCT consist of sample average value estimates $\bar{V}_{\text{UCT}}(s)$ and $\bar{Q}_{\text{UCT}}(s, a)$, visit counts $N(s)$ and $N(s, a)$. The parameters consist of: b_{UCT}, a bias parameter which controls the amount of exploration UCT will perform and `normalised_Q`, a flag indicating whether the Q-value estimates should be normalised when used in the search policy, which is useful if the Q-values can vary in magnitude throughout the search tree. Typically `single_node_expansion` is set to True for UCT, and this section assumes that this is the case.

The search policy that UCT follows is given by:

$$\bar{Q}^{\text{norm}}_{\text{UCT}}(s, a) = \begin{cases} \frac{\bar{Q}_{\text{UCT}}(s,a) - \min_{a \in \mathcal{A}} \bar{Q}_{\text{UCT}}(s,a)}{\max_{a \in \mathcal{A}} \bar{Q}_{\text{UCT}}(s,a) - \min_{a \in \mathcal{A}} \bar{Q}_{\text{UCT}}(s,a)} & \text{if } \texttt{normalised_Q}, \\ \bar{Q}_{\text{UCT}}(s, a) & \text{otherwise}, \end{cases} \tag{2.44}$$

$$\pi_{\text{UCT}}(s) = \arg\max_{a \in \mathcal{A}} \bar{Q}^{\text{norm}}_{\text{UCT}}(s, a) + b_{\text{UCT}} \sqrt{\frac{\log(N(s))}{N(s, a)}}, \tag{2.45}$$

In Equation (2.45), when $N(s, a) = 0$ there is a division by zero, which is taken as ∞, and ties are broken uniformly randomly. Note that like UCB (Section 2.1), this

```
def run_trial(search_tree: 𝒯,
              search_policy: π_search,
              heuristic_fn: V̂_heuristic,
              heuristic_action_fn: Q̂_heuristic,
              heuristic_policy: π_heuristic,
              single_node_expansion,
              planning_horizon: H_THTS++):
    ctx = sample_context()
    τ_:h = selection_phase(𝒯, π_search, V̂_heuristic, Q̂_heuristic, π_heuristic, ↩
        single_node_expansion, H_THTS++, ctx)
    backup_phase(τ_:h, ctx)

def selection_phase(search_tree: 𝒯,
                    search_policy: π_search,
                    heuristic_fn: V̂_heuristic,
                    heuristic_action_fn: Q̂_heuristic,
                    heuristic_policy: π_heuristic,
                    single_node_expansion,
                    planning_horizon: H_THTS++,
                    ctx):
    i = 0
    while (not single_node_expansion or s_i ∈ 𝒯)
            and (i < H_THTS++)
            and (s_i ∉ terminal):
        # selection step
        a_i ← select_action(s_i, Q̂_heuristic, π_heuristic, ctx)
        r_i ← R(s_i, a_i)
        s_i+1 ← sample_outcome(s_i, a_i, ctx)
        # heuristic step
        if (s_i, a_i) ∉ 𝒯:
            init_q((s_i, a_i), Q̂_heuristic, π_heuristic, ctx)
        if s_i+1 ∉ 𝒯:
            init_v(s_i+1, V̂_heuristic, π_heuristic, ctx)
        𝒯 ← 𝒯 ∪ {(s_i, a_i), s_i+1}
        i += 1
    return (s_0, a_0, r_0, s_1, ..., s_i-1, a_i-1, r_i-1, s_i)

def select_action(s, Q̂_heuristic, π_heuristic, ctx):
    a ~ π_search(·|s, Q̂_heuristic, π_heuristic, ctx)
    return a

def sample_outcome(s, a, ctx):
    s' ~ p(·|s, a)
    return s'

def backup_phase(τ_:h, ctx)
    for i in {h−1, h−2, ..., 1, 0}:
        N(s_i, a_i) ← N(s_i, a_i) + 1
        N(s_i) ← N(s_i) + 1
        node(s_i, a_i).backup_q(node(s_i, a_i).chldrn, τ_i:h, ctx)
        node(s_i).backup_v(node(s_i).chldrn, τ_i:h, V̂_heuristic(s_h), ctx)
```

Listing 2.1: Pseudocode for running a trial in THTS++.

results in every action being taken once before any action is selected for a second time. The heuristic functions $\hat{Q}_{\text{heuristic}}$ and $\pi_{\text{heuristic}}$ are not used by UCT.

Let $\tau_{:h} \sim \pi_{\text{UCT}}$ be the trajectory sampled in the selection phase of UCT. For any chance nodes (s, a) added to the tree, the `init_q` function initialises the visit count $N(s, a) = 0$ and the Q-value estimate $\bar{Q}_{\text{UCT}}(s, a) = \hat{Q}_{\text{heuristic}}(s, a)$, which will be overwritten by `backup_q` on the first visit (Equation 2.48). The `init_v` function initialises the leaf node s_h using the heuristic function and with a single visit:

$$N(s_h) = 1, \tag{2.46}$$

$$\bar{V}_{\text{UCT}}(s_h) = \hat{V}_{\text{heuristic}}(s_h). \tag{2.47}$$

In UCT the backups `backup_v` and `backup_q` update the (sample average) value estimates incrementally. For $t = h - 1, \ldots, 0$:

$$\bar{Q}_{\text{UCT}}(s_t, a_t) \leftarrow \frac{N(s_t, a_t) - 1}{N(s_t, a_t)} \bar{Q}_{\text{UCT}}(s_t, a_t) + \frac{\hat{V}_{\text{heuristic}}(s_h) + \sum_{i=t}^{h-1} r_i}{N(s_t, a_t)}, \tag{2.48}$$

$$\bar{V}_{\text{UCT}}(s_t) \leftarrow \frac{N(s_t) - 1}{N(s_t)} \bar{V}_{\text{UCT}}(s_t) + \frac{\hat{V}_{\text{heuristic}}(s_h) + \sum_{i=t}^{h-1} r_i}{N(s_t)}. \tag{2.49}$$

There are two common approaches for implementing $\hat{V}_{\text{heuristic}}$ in UCT. The first approach uses a function approximation $\hat{V}_{\text{heuristic}} = \tilde{V}$ [57, 58, 37], where $\tilde{V}$ is often a neural network that aims to approximate the optimal value function V^*. The second approach uses a *rollout*, which completes the trajectory from the selection phase using a *rollout policy* and uses the rewards from the rollout to give a heuristic value estimate [11, 21, 27, 13]. If no informative policy is available for π_{rollout}, then a uniformly random policy is often used [11, 1].

To use a rollout policy π_{rollout} in `THTS++`, a Monte Carlo estimate $\hat{V}^{\pi_{\text{rollout}}}$ of the value function $V^{\pi_{\text{rollout}}}$ is used for $\hat{V}_{\text{heuristic}}$. Let $\tau_{h:h'} \sim \pi_{\text{rollout}}$ be the rollout trajectory, where $h' = H_{\texttt{THTS++}}$ or $s_{h'} \in \texttt{terminal}$. The Monte Carlo estimate is then the sum of the rewards from the rollout:

$$\hat{V}^{\pi_{\text{rollout}}}(s_h) = \sum_{i=h}^{h'-1} r_i. \tag{2.50}$$

2.4.4 Predictor-UCT (PUCT)

TODO: Maybe move to related work Ch3, Sec 2.4 either is THTS with all the algorithms, or THTS with a couple examples. PUCT, RENTS, TENTS can go in the related work section. Should also define MaxUCT.

AlphaGo and AlphaGo Zero [56, 57] popularised a variant of UCT which is given in this section and referred to as *Predictor-UCT* (PUCT). In [56], this variant is ambiguously defined as "PUCT", citing *predictor UCB* [50] as the source, while more closely resembling *polynomial UCB* [4].

PUCT uses similar variables to UCT (Section 2.4.3), the sample average value estimates $\bar{V}_{\text{PUCT}}(s)$ and $\bar{Q}_{\text{PUCT}}(s,a)$, visit counts $N(s)$ and $N(s,a)$, the bias parameter b_{PUCT}, and the flag `normalised_Q`.

The PUCT search policy is similar to the UCT search policy (Equation 2.45), with the following changes:

- The logarithmic confidence interval term in Equation (2.45) is replaced by a polynomial term [4];
- The confidence interval term is weighted by the *predictor values* $\pi_{\text{heuristic}}(a|s)$, inspired by predictor UCB [50];
- The $N(s,a)$ denominator is modified to $N(s,a)+1$, which allows actions to be selected more than once before all actions have been tried, making the policy better suited to large action spaces such as in Go.

The operation of PUCT is otherwise identical to UCT, except for the changes to the search policy. The PUCT selection policy is given by:

$$\pi_{\text{PUCT}}(s) = \arg\max_{a \in \mathcal{A}} \bar{Q}^{\text{norm}}_{\text{PUCT}}(s,a) + b_{\text{PUCT}} \pi_{\text{heuristic}}(a|s) \frac{\sqrt{N(s)}}{N(s,a)+1}. \tag{2.51}$$

In `THTS++`, the values of $\bar{Q}_{\text{PUCT}}(s,a)$ are initialised using the heuristic action value $\hat{Q}_{\text{heuristic}}(s,a)$. When $\bar{Q}_{\text{PUCT}}(s,a)$ is updated via the backup (Equation (2.48)), the heuristic initialisation is replaced by the sample average estimate.

2.4.5 Maximum Entropy Tree Search

Maximum ENtropy Tree Search (MENTS) [69], in contrast to UCT, focuses on the maximum-entropy objective, and uses soft Bellman backups to update its value estimates. In its original presentation, `single_node_expansion` is set to True, and it uses the soft value estimates $\hat{V}_{\text{MENTS}}$ and $\hat{Q}_{\text{MENTS}}$. The MENTS search policy is given by:

$$\pi_{\text{MENTS}}(a|s) = (1 - \lambda(s, \epsilon_{\text{MENTS}}))\rho_{\text{MENTS}}(a|s) + \frac{\lambda(s, \epsilon_{\text{MENTS}})}{|\mathcal{A}|} \tag{2.52}$$

$$\rho_{\text{MENTS}}(a|s) = \exp\left(\frac{1}{\alpha_{\text{MENTS}}}\left(\hat{Q}_{\text{MENTS}}(s,a) - \hat{V}_{\text{MENTS}}(s)\right)\right), \tag{2.53}$$

$$\lambda(s, \epsilon) = \min\left(1, \frac{\epsilon}{\log(e + N(s))}\right). \tag{2.54}$$

where $\epsilon_{\text{MENTS}} \in (0, \infty)$ is an exploration parameter, α_{MENTS} is the temperature parameter used in MENTS for the maximum entropy objective (the coefficient of the entropy term in Equation (2.33)), and λ is a decay function that is also used in later chapters with different arguments.

Let $\tau_{:h} \sim \pi_{\text{MENTS}}$ be the trajectory sampled in the selection phase of MENTS. For any chance nodes (s, a) added to the tree, the `init_q` function initialises the visit count $N(s, a) = 0$ and the Q-value estimate $\hat{Q}_{\text{MENTS}}(s, a) = \hat{Q}_{\text{heuristic}}(s, a)$, which will be overwritten by `backup_q` on the first visit (Equation 2.57). The `init_v` function initialises the leaf node s_h using the heuristic function and with a single visit:

$$N(s_h) = 1, \tag{2.55}$$

$$\hat{V}_{\text{MENTS}}(s_h) = \hat{V}_{\text{heuristic}}(s_h). \tag{2.56}$$

The backups `backup_v` and `backup_q` then update the soft value estimates using soft Bellman backups (Equations (2.42) and (2.43)) as follows for $t = h - 1, \ldots, 0$:

$$\hat{Q}_{\text{MENTS}}(s_t, a_t) \leftarrow R(s_t, a_t) + \sum_{s' \in \text{Succ}(s_t, a_t)} \left(\frac{N(s')}{N(s_t, a_t)} \hat{V}_{\text{MENTS}}(s')\right), \tag{2.57}$$

$$\hat{V}_{\text{MENTS}}(s_t) \leftarrow \alpha_{\text{MENTS}} \log \sum_{a \in \mathcal{A}} \exp\left(\frac{1}{\alpha_{\text{MENTS}}} \hat{Q}_{\text{MENTS}}(s_t, a)\right). \tag{2.58}$$

In [69], the heuristic value function is left as an arbitrary evaluation function, but is set using a function approximation $\hat{V}_{\text{heuristic}} = \tilde{V}$ in experiments. The

heuristic action function is set to zero, $\hat{Q}_{\text{heuristic}}(s, a) = 0$, however the authors also suggest that if *policy network* $\tilde{\pi}$ is available, then the heuristic action function can alternatively be set to $\hat{Q}_{\text{heuristic}}(s, a) = \log \tilde{\pi}(a|s)$.

2.5 Convergence of Random Variables

This section introduces the two modes of convergence used in the proofs in this thesis and states existing results that will be used.

Definition 2.5.1. *A sequence of random variables* X_i converges in probability *to a random variable* X*, written as* $X_i \xrightarrow{p} X$*, if for all* $\varepsilon > 0$ *the following holds:*

$$\Pr\left(|X_i - X| > \varepsilon\right) \to 0 \text{ as } i \to \infty. \tag{2.59}$$

Definition 2.5.2. *A sequence of random variables* X_i converges almost surely *to a random variable* X*, written as* $X_i \xrightarrow{as} X$*, if the following holds:*

$$\Pr\left(\lim_{i\to\infty} X_i = X\right) = 1. \tag{2.60}$$

Almost sure convergence is a stronger condition than convergence in probability: almost sure convergence implies convergence in probability, but not vice versa [10].

The Dvoretzky-Kiefer-Wolfowitz (DKW) inequality [18] bounds the maximum difference between the empirical distribution of samples and the true distribution:

Theorem 2.5.1. *Let* $\{X_i\}_{i=1}^m$ *be random variables drawn from a probability distribution with a cumulative distribution function of* F*. Let the empirical cumulative distribution function be* $F_m(x) = \frac{1}{m}\sum_{i=1}^m \mathbb{1}[X_i < x]$*. Then the Dvoretzky-Kiefer-Wolfowitz inequality is:*

$$\Pr\left(\sup_x |F_m(x) - F(x)| > \varepsilon\right) \leq 2\exp\left(-2m\varepsilon^2\right). \tag{2.61}$$

The DKW inequality is used in Chapters 4 and 5 to bound the error when using the empirical visit count ratio $\frac{N(s')}{N(s,a)}$ as an approximation to the transition probability $p(s'|s, a)$.

The *Good-Turing estimate* TODO: cite addresses a related but distinct problem: rather than bounding the error in the empirical distribution, it estimates the total probability mass of outcomes that have not yet been observed.

Let f be a probability mass function over the categories $\mathcal{C} = \{C_1, \ldots, C_k\}$, where the number of categories k is unknown or potentially infinite. Let $X_1, \ldots, X_m$ be m independent samples drawn from f, and let $M(C)$ be the number of times category C was observed:

$$M(C) = \sum_{i=1}^{m} \mathbb{1}[X_i = C]. \tag{2.62}$$

Let $\mathcal{C}_{\text{obs}} = \{C \in \mathcal{C} \mid M(C) > 0\}$ be the set of observed categories. The *missing mass* is the total probability of unobserved categories:

$$\texttt{missing_mass} = \sum_{C \in \mathcal{C} - \mathcal{C}_{\text{obs}}} f(C). \tag{2.63}$$

The Good-Turing estimate of the missing mass is:

$$\texttt{missing_mass}_{\text{GT}} = \frac{M_1}{m}, \tag{2.64}$$

where $M_1 = |\{C \in \mathcal{C}_{\text{obs}} \mid M(C) = 1\}|$ is the number of categories observed exactly once (the number of *singletons*).

TODO: citet mcallester_schapire show that the Good-Turing estimate satisfies the following bound.

Theorem 2.5.2. *For $0 < \delta \leq 1$, the Good-Turing estimate (Equation 2.64) satisfies the following bound:*

$$\Pr\left(\texttt{missing_mass} \leq \texttt{missing_mass}_{\text{GT}} + \sqrt{\frac{\log(1/\delta)}{2m}}\right) \geq 1 - \delta. \tag{2.65}$$

The Good-Turing estimate is used in Chapter 5 to bound the probability mass of unobserved successor states when computing the solved value of a chance node (s, a) (Section **??**).

Figure 2.10: The decision-support scenario for multi-objective reinforcement learning [25].

2.6 Multi-Objective Reinforcement Learning

This thesis follows a utility-based approach to multi-objective reinforcement learning similar to the review of Hayes et al. [25]. This work will specifically consider *linear utility* functions and the *decision support scenario* which is depicted in Figure 2.10. In the decision support scenario, an algorithm computes a *solution set* consisting of multiple *possibly optimal* solutions, from which a user then picks their most preferred option to be used. This scenario is useful when a user's preferences over the multiple objectives are unknown or difficult to specify in advance.

This section defines the multi-objective counterparts to various definitions given in Section 2.2 and 2.3. Outside of this section, the prefix "multi-objective" may be dropped where it should be clear from context; however, bold typeface will consistently be used to denote any vector variables or functions. In Section 2.6.1, a multi-objective extension of Value Iteration is given.

To specify problems with multiple objectives, the reward function of an MDP is changed to give a vector of rewards, rather than a scalar reward:

Definition 2.6.1. *A* Multi-Objective Markov Decision Process *(MOMDP) is a tuple* $\boldsymbol{\mathcal{M}} = (\mathcal{S}, s_0, \mathcal{A}, \mathbf{R}, p, H)$*, where* $\mathcal{S}$ *is a set of states,* $s_0 \in \mathcal{S}$ *is an initial state,* $\mathcal{A}$ *is a set of actions,* $\mathbf{R}(s, a)$ *is a vector reward function* $\mathcal{S} \times \mathcal{A} \to \mathbb{R}^D$*, where* D *is the dimension of the rewards and the MOMDP,* $p(s'|s, a)$ *is a next state transition distribution* $\mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ *and* $H \in \mathbb{N}$ *is a finite-horizon time bound.*

Multi-objective trajectories are defined analogously to single-objective trajectories (Definition 2.2.6):

Definition 2.6.2. *A* multi-objective trajectory τ*, is a sequence of state, action and vector rewards, that is induced by a policy* π *and MOMDP* $\boldsymbol{\mathcal{M}}$ *pair. Let the*

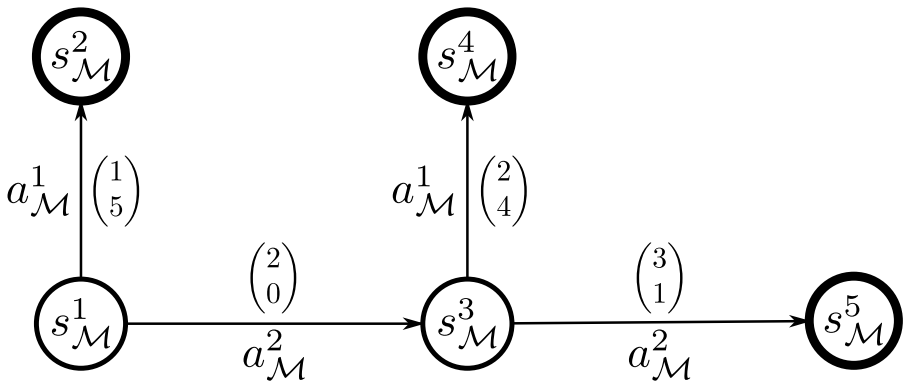


Figure 2.11: An example MOMDP $\boldsymbol{\mathcal{M}}$, where $s_0 = s^1_{\mathcal{M}}$, the finite horizon is $H = 2$ and terminal states are marked with thicker borders. This particular MOMDP is deterministic, so that action nodes are omitted and all transition probabilities are one.

trajectory be $\tau = (s_0, a_0, \mathbf{r}_0, s_1, a_1, \mathbf{r}_1, \ldots, s_{H-1}, a_{H-1}, \mathbf{r}_{H-1}, s_H)$*, where* $a_t \sim \pi(\cdot|s_t)$*,* $\mathbf{r}_t = \mathbf{R}(s_t, a_t)$ *and* $s_{t+1} \sim p(\cdot|s_t, a_t)$*.*

The notations used for single-objective trajectories (Definition 2.2.6) will also be used for multi-objective trajectories. Such as, $\tau \sim \pi$ *for sampling trajectories using policies, and* $\tau_{i:j}$ *for truncated trajectories.*

Similarly, multi-objective variants of the (Q-)value of a policy are defined:

Definition 2.6.3. *The* multi-objective value *of a policy* π *from state* s *at time* t *is:*

$$\mathbf{V}^\pi(s;t) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{i=t}^{H-1} \mathbf{r}_i \middle| s_t = s\right]. \tag{2.66}$$

The multi-objective Q-value *of a policy* π*, from state* s*, with action* a*, at time* t *is:*

$$\mathbf{Q}^\pi(s,a;t) = \mathbf{R}(s,a) + \mathbb{E}_{s'\sim p(\cdot|s,a)}[\mathbf{V}^\pi(s';t+1)]. \tag{2.67}$$

In the corresponding point of the single-objective reinforcement learning section (Section 2.3), the optimal (Q-)value functions and the objective of single-objective reinforcement learning were defined. However, in a multi-objective setting there is no longer a *total ordering* over values, and so there may be multiple vectors that could be "optimal". For example, consider two vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^2$ which are such that $u_1 > v_1$ and $u_2 < v_2$. To resolve this issue, a *utility function* or *scalarisation function* is used to map multi-objective values to scalar values.

Definition 2.6.4. *The* D-dimensional Unit Simplex *consists of the set of* D-*dimensional vectors, whose entries are non-negative and sum to one*[4]*. More formally, the* D-*dimensional simplex is*

$$\Delta^D = \{\mathbf{w} \in \mathbb{R}^D | w_i \geq 0, \sum_i w_i = 1\}. \tag{2.68}$$

The elements of the D-*dimensional simplex will be referred to as* weight vectors *(or just* weights*) in this thesis, as they will be used to specify preferences over the* D *dimensions of the reward function.*

Definition 2.6.5. *A* utility function *(or* scalarisation function*)* $u : \mathbb{R}^D \times \Delta^D \to \mathbb{R}$ *is used to map from a multi-objective value* $\mathbf{v} \in \mathbb{R}^D$ *and a weighting over the objectives* $\mathbf{w} \in \Delta^D$ *to a scalar value. That is, according to the utility function* $u(\cdot; \mathbf{w})$ *the multi-objective value* $\mathbf{v}$ *is mapped to the scalar value* $u(\mathbf{v}; \mathbf{w})$*.*

Of particular interest in this thesis is the *linear utility function* where the scalar value takes the form of a dot-product:

Definition 2.6.6. *The* linear utility function u_{lin} *is the utility function defined by:*

$$u_{\text{lin}}(\mathbf{v}; \mathbf{w}) = \mathbf{w}^\top \mathbf{v}. \tag{2.69}$$

Equipped with a utility function and a weight vector, any set of multi-objective values can be mapped to scalars and ordered. A policy is *possibly optimal*, or *undominated*, with respect to utility function u if it achieves a maximal utility for some weight $w \in \Delta^D$. In contrast, a policy is *dominated* if for every weight there is another policy that achieves a better utility.

Let Π be the set of all possible policies for a given MOMDP. When constructing a solution set, a subset of Π, it makes sense that it should not contain any dominated policies. This leads to the first notion of a solution set, called an *undominated set*, consisting of all undominated policies:

[4]Note that the D-dimensional simplex is actually the $(D-1)$ simplex. The notation Δ^D is adopted here to emphasise that weight vectors have D components.

Definition 2.6.7. *The* undominated set *of policies $U(\Pi; u) \subseteq \Pi$, with respect to a utility function u, is the set of policies for which there is a weight vector $\mathbf{w} \in \Delta^D$ where the utility (scalarised value) is maximised:*

$$U(\Pi; u) = \left\{ \pi \in \Pi \;\middle|\; \exists \mathbf{w} \in \Delta^D . \forall \pi' \in \Pi : u(\mathbf{V}^{\pi}(s_0; 0); \mathbf{w}) \geq u(\mathbf{V}^{\pi'}(s_0; 0); \mathbf{w}) \right\}. \tag{2.70}$$

In particular, the convex hull *of policies $CH(\Pi)$ is the undominated set with respect to the linear utility function u_{lin}. That is $CH(\Pi) = U(\Pi; u_{\text{lin}})$.*

Undominated sets often have an infinite cardinality, and as such are infeasible to compute. However, in undominated sets there are often many redundant policies that obtain the same scalarised values. Instead of computing an undominated set, it is more feasible to compute a *coverage set* that contains at least one policy that maximises the scalarised value given any weight vector $\mathbf{w}$:

Definition 2.6.8. *A set $CS(\Pi; u) \subseteq U(\Pi)$, is a* coverage set *with respect to a utility function u, if for every weight vector $\mathbf{w} \in \Delta^D$, there is a policy $\pi \in CS(\Pi; u)$ that maximises the value of $u(\cdot; \mathbf{w})$. That is, for $CS(\Pi; u)$ to be a coverage set, the following statement must be true:*

$$\begin{aligned} &\forall \mathbf{w} \in \Delta^D .\ \exists \pi \in CS(\Pi; u).\ \forall \pi' \in \Pi : \\ &\quad u(\mathbf{V}^{\pi}(s_0; 0); \mathbf{w}) \geq u(\mathbf{V}^{\pi'}(s_0; 0); \mathbf{w}). \end{aligned} \tag{2.71}$$

In particular, any set $CCS(\Pi)$ is a convex coverage set *if it is a coverage set with respect to the linear utility function u_{lin}.*

A convex coverage set may contain redundant policies and is not necessarily minimal. A minimal convex coverage set is a convex coverage set that contains no redundant policies. That is, for every policy in the minimal convex coverage set, there is some weight for which it uniquely maximises the utility within the set.

Definition 2.6.9. *A set $MCCS(\Pi) \subseteq \Pi$ is a* minimal convex coverage set *if it is a convex coverage set and if the following holds*

$$\begin{aligned} &\forall \pi \in MCCS(\Pi).\ \exists \mathbf{w} \in \Delta^D .\ \forall \pi' \in MCCS(\Pi) - \{\pi\} : \\ &\quad u_{\text{lin}}(\mathbf{V}^{\pi}(s_0; 0); \mathbf{w}) > u_{\text{lin}}(\mathbf{V}^{\pi'}(s_0; 0); \mathbf{w}). \end{aligned} \tag{2.72}$$

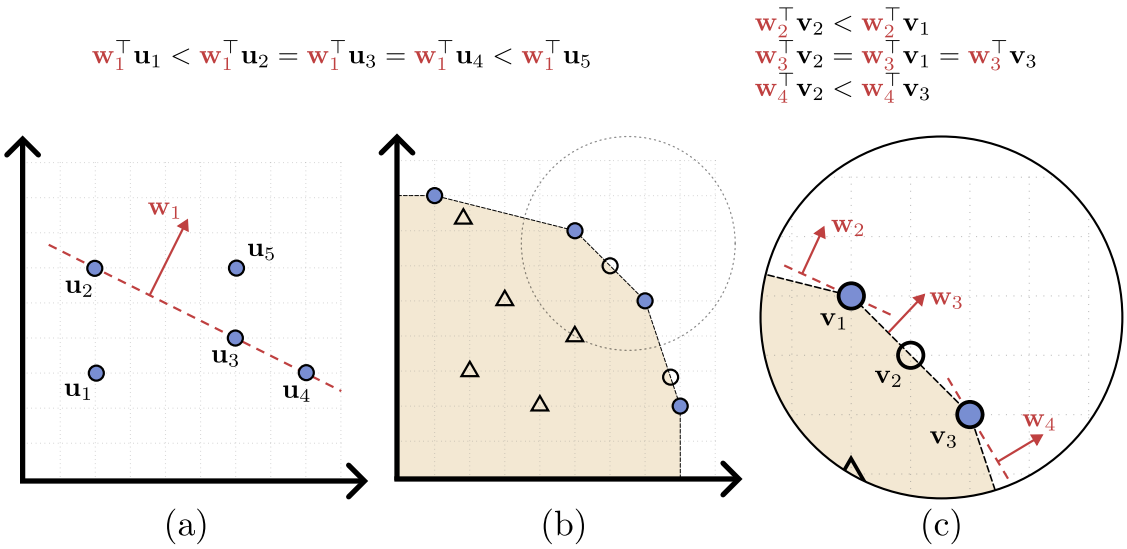


Figure 2.12: The geometry of convex coverage sets, shown with $D = 2$. In each image the points depicted correspond to a value set $\boldsymbol{\mathcal{V}}\mathbf{als}(\Pi')$, for some set of policies Π'. (a) demonstrates that values ($\mathbf{u}_2$,$\mathbf{u}_3$,$\mathbf{u}_4$) obtaining the same utility lie on a hyperplane. Moving in the direction of the weight vector increases the utility ($\mathbf{u}_5$) and likewise moving in the opposite direction decreases utility ($\mathbf{u}_1$); (b) circles correspond to values that are on the convex hull, so form the set $\boldsymbol{\mathcal{V}}\mathbf{als}(CH(\Pi'))$, the blue circles form the minimal convex coverage set, $\boldsymbol{\mathcal{V}}\mathbf{als}(MCCS(\Pi'))$, and triangles are not part of the convex hull. All of the unfilled shapes would be pruned by `cvx_prune` (Equation (2.73)); (c) shows a magnification of (b) with additional labels, it shows that the value $\mathbf{v}_2$ is redundant because one of $\mathbf{v}_1$ or $\mathbf{v}_3$ will achieve an equal or greater utility. Additionally, it shows $\mathbf{w}_2$ and $\mathbf{w}_4$ which are weights that the values $\mathbf{v}_1$ and $\mathbf{v}_3$ uniquely maximise the utility within $\boldsymbol{\mathcal{V}}\mathbf{als}(\Pi')$.

When computing solution sets, it is often useful to first compute the multi-objective values that could be obtained, and then later use the data structures used by the algorithm to read out the selected policy or solution set (for example, Convex Hull Value Iteration covered in Section 2.6.1). This thesis will refer to the values obtained by a set of policies as a *value set*:

Definition 2.6.10. *The* (multi-objective) value set *with respect to a set of policies* $\Pi' \subseteq \Pi$ *is defined as* $\boldsymbol{\mathcal{V}}\mathbf{als}(\Pi') = \{\mathbf{V}^\pi(s_0; 0) | \pi \in \Pi'\}$.

Given a value set, the `cvx_prune` operation removes any vectors that are dominated or redundant. That is, for every vector that remains after the pruning operation, there is some weight for which it *uniquely* gives the maximal linear utility.

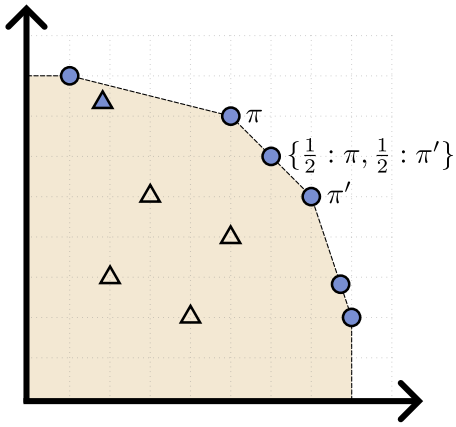


Figure 2.13: An example Pareto front, where the circles correspond to a convex coverage set, and the filled blue points denote the Pareto front. Two points are marked with their corresponding policies π and π', and a mixture policy is shown, denoted $\{\frac{1}{2} : \pi, \frac{1}{2} : \pi'\}$ that follows each of π and π' with probability $\frac{1}{2}$.

Given a set of vectors $\boldsymbol{\mathcal{V}}$, the `cvx_prune` operation is defined as:

$$\texttt{cvx_prune}(\boldsymbol{\mathcal{V}}) = \{\mathbf{v} \in \boldsymbol{\mathcal{V}} | \exists \mathbf{w} \in \Delta^D . \forall \mathbf{v}' \in \boldsymbol{\mathcal{V}} - \{\mathbf{v}\} . \mathbf{w}^\top \mathbf{v} > \mathbf{w}^\top \mathbf{v}'\}. \tag{2.73}$$

As illustrated in Figure 2.12, the `cvx_prune` operation computes the *extreme points* of a *(partial) geometric convex hull*. As such, for some set of policies Π' the `cvx_prune` operation satisfies $\texttt{cvx_prune}(\boldsymbol{\mathcal{V}}\mathbf{als}(\Pi')) = \boldsymbol{\mathcal{V}}\mathbf{als}(MCCS(\Pi'))$, and `cvx_prune` can be implemented using *linear programming* [47].

To distinguish between the convex hull of policies, and the geometric convex hulls of values, this thesis will refer to any value set that forms a geometric convex hull as a *convex hull value set* (CHVS).

Definition 2.6.11. *The set of vectors $\boldsymbol{\mathcal{V}} \subseteq \mathbb{R}^D$ is a* convex hull value set *(CHVS) if its points form a partial geometric convex hull. More specifically, $\boldsymbol{\mathcal{V}}$ is a convex hull value set if it satisfies the equation:*

$$\boldsymbol{\mathcal{V}} = \texttt{cvx_prune}(\boldsymbol{\mathcal{V}}). \tag{2.74}$$

Whilst this thesis focuses on the linear utility setting, another commonly used solution set is the *Pareto front*, which is defined using a *strict partial ordering* over vectors, known as *Pareto domination*, as opposed to using utility. An example is shown in Figure 2.13.

Definition 2.6.12. *The* Pareto front $PF(\Pi) \subseteq \Pi$ *is defined as:*

$$PF(\Pi) = \{\pi \in \Pi | \nexists \pi' \in \Pi . \mathbf{V}^{\pi'}(s_0; 0) \succ_P \mathbf{V}^{\pi}(s_0; 0)\}, \tag{2.75}$$

where the Pareto domination *relation is defined as:*

$$\mathbf{v} \succ_P \mathbf{v}' \iff (\forall i . v_i \geq v'_i) \wedge (\exists j . v_j > v'_j). \tag{2.76}$$

The Pareto front can be related to the previously defined solution sets if *mixture policies* are allowed, where a mixture policy between π and π' will follow each policy with some probability. Specifically, if Π is closed with respect to mixture policies, then the Pareto front is a convex coverage set.

2.6.1 Convex Hull Value Iteration

Convex Hull Value Iteration (CHVI) [5] is a tabular dynamic programming algorithm similar to Value Iteration (Section 2.3) that computes optimal convex hull value sets and provides a method for extracting policies for any given weight vector. In CHVI the value estimates of Value Iteration are replaced by CHVSs, which are estimates of the optimal CHVS. These estimates are denoted $\hat{\boldsymbol{\mathcal{V}}}_{\text{CHVI}}(s; t)$ and $\hat{\boldsymbol{\mathcal{Q}}}_{\text{CHVI}}(s, a; t)$.

Firstly, to define a multi-objective version of Value Iteration, arithmetic over sets of vectors needs to be defined. An example of the following arithmetic is given in Figure 2.14. Given the vector sets $\boldsymbol{\mathcal{U}}$ and $\boldsymbol{\mathcal{V}}$, define multiplication by a scalar s, addition with a vector $\mathbf{x}$ and addition between sets as follows:

$$\mathbf{x} + s\boldsymbol{\mathcal{V}} = \{\mathbf{x} + s\mathbf{v} | \mathbf{v} \in \boldsymbol{\mathcal{V}}\} \tag{2.77}$$

$$\boldsymbol{\mathcal{U}} + \boldsymbol{\mathcal{V}} = \{\mathbf{u} + \mathbf{v} | \mathbf{u} \in \boldsymbol{\mathcal{U}}, \mathbf{v} \in \boldsymbol{\mathcal{V}}\}. \tag{2.78}$$

Now to define the multi-objective Bellman backups used in CHVI, let $\hat{\boldsymbol{\mathcal{V}}}^0_{\text{CHVI}}(s; t) = \{\mathbf{0}\}$, where $\mathbf{0} = (0, \ldots, 0) \in \mathbb{R}^D$. The CHVI backups are then:

$$\hat{\boldsymbol{\mathcal{V}}}^{k+1}_{\text{CHVI}}(s; t) = \texttt{cvx_prune}\left(\bigcup_{a \in \mathcal{A}} \hat{\boldsymbol{\mathcal{Q}}}^{k+1}_{\text{CHVI}}(s, a; t)\right), \tag{2.79}$$

$$\hat{\boldsymbol{\mathcal{Q}}}^{k+1}_{\text{CHVI}}(s, a; t) = \mathbb{E}_{s' \sim p(\cdot|s,a)}[\mathbf{R}(s, a) + \hat{\boldsymbol{\mathcal{V}}}^{k}_{\text{CHVI}}(s'; t + 1)]. \tag{2.80}$$

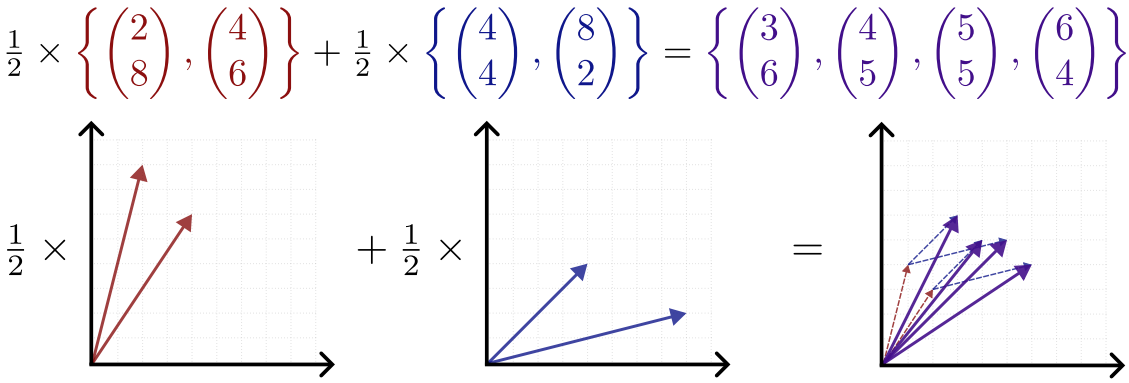


Figure 2.14: An example of arithmetic over vector sets. Two vector sets are shown in red and blue, which are multiplied by $\frac{1}{2}$ using Equation (2.77), and then added using Equation (2.78). On the right the resulting vector set is shown in purple, which are the sum of appropriately scaled red and blue vectors.

This again parallels the Bellman backups used in single-objective Value Iteration, where the max operation is replaced by the `cvx_prune` operation over the set of achievable values from the current state s.

Once the algorithm has terminated and a weight $\mathbf{w}$ is provided, a policy can be extracted by computing scalar Q-values from the CHVSs [5]:

$$Q^{\mathbf{w}}_{\text{CHVI}}(s, a; t) = \max\{\mathbf{w}^\top \mathbf{q} | \mathbf{q} \in \hat{\mathcal{Q}}_{\text{CHVI}}(s, a; t)\}, \tag{2.81}$$

$$\pi_{\text{CHVI}}(s; t, \mathbf{w}) = \arg\max_{a \in \mathcal{A}} Q^{\mathbf{w}}_{\text{CHVI}}(s, a; t). \tag{2.82}$$

Note that if a policy is extracted for every weight vector $\mathbf{w} \in \Delta^D$, the resulting set of policies forms a convex coverage set.

3
Literature Review

Contents

3.1 Multi-Armed Bandits **43**
3.2 Reinforcement Learning **44**
3.3 Trial-Based Heuristic Tree Search and Monte-Carlo Tree Search **46**
3.3.1 Trial Based Heuristic Tree Search 46
3.3.2 Monte-Carlo Tree Search 46
3.3.3 Maximum Entropy Tree Search 47
3.4 Multi-Objective Reinforcement Learning **47**
3.5 Multi-Objective Monte Carlo Tree Search **49**

TODO: should add some boltzmann search stuff here too

TODO: Change chapter title to related work rather than literature review

TODO: currently this is a copy and paste of what I originally wrote for background chapter 2. Deleted parts which are irrelevant for litreview here (and vice versa for the background section).

TODO: I'm also going to use this as a space to paste papers I should write about as they come up while writing later chapters

3.1 Multi-Armed Bandits

TODO: Maybe dont need to cover this in litrev, but should talk about exploring

bandits, UCT and contextual bandits either in background or in litrev

TODO: linUCB for contextual bandits think this is the linucb paper: https://arxiv.org/pdf/1003.0146

Designing multi-objective multi-armed bandits algorithms: a study - Madalina M. Drugan and Ann Nowe

TODO: MAB book

Talk about Exp3?

Contextual Zooming

3.2 Reinforcement Learning

TODO: Intro should say that look at Sutton and Barto and something else for deep RL, for a more complete overview. Here we will just discuss papers that consider entropy in their work, as thats the most relevant part for this thesis.

TODO: list

- Talk about entropy and some of that work (probably a subsection)

TODO: In the entropy bit talk add this, removed from ch2: Note that there are other forms of entropy, such as relative and Tsallis entropy, which can be used in place of Shannon entropy (TODO cite). For the work considered in this thesis, the other forms of entropy can be used by replacing the definition of $\mathcal{H}$ by the relevant definition.

TODO: talk about some deep learning methods here

TODO: Talk about entropy in (deep) RL methods too. Say that it is primarily introduced primarily for an exploration benifit. A3C "prevent converging to deterministic suboptimal behaviour". While there is some additional. Intro in the soft Q learning provides some alternative motivations, "In some cases, we might actually prefer to learn stochastic behaviors. In this paper, we explore two potential reasons for this: exploration in the presence of multimodal objectives, and compositionality attained via pretraining. Otherbenefits include robustness in the face of uncertain dynamics (Ziebart, 2010), imitation learning (Ziebart et al., 2008), and improved convergence and computational properties (Gu et al., 2016a).

Multi-modality also has application in real robot tasks, as demonstrated in (Daniel et al., 2012). However, in order to learn such policies, we must define an objective that promotes stochasticity.". ANd also talks about adversarital pertobations. So basically: good for meta learning/pretraining/foundation models, good for robustness against uncertain dynamics. BASICALLY, HERE we want to talk about the other reasons why we might want to use maximum entropy.

Entropy widely used in RL literature. Most commonly used for regularization when learning a policy [53, 36, 66]. This is significantly different to the maximum entropy setting, where entropy is used in the loss function to prevent the policy from degenerating into a deterministic policy. While it is motivated similarly, promoting the policy to remain stochastic and enable exploration.

The significant difference between *entropy regularization* and *maximum entropy reinforcement learning* is that the former is that from some state s_t, regularization only includes the entropy of the policy at s_t, while the latter includes the entropy of the policy at all states $s_t, s_{t+1}, ...$ in the objective/loss function.

Probably write this a bit better, say that entropy regularization dates back to the REINFORCE/MENT paper, but is consistently used to the modern day.

Old entropy stuff from ch4 before rewrite: Loosely, the maximum entropy principle says that given partial knowledge about a probability distribution, the least biased estimate of the probability distribution is the one that maximises the entropy [26]. This principle can naturally be applied to inverse reinforcement learning [71, 70], where the goal is to learn the transition/reward functions from a set of expert trajectories.

In contrast, in maximum entropy reinforcement learning (Section 2.3.1) entropy is added directly into the objective function to be optimised, rather than using entropy as a constraint to disambiguate between multiple optimal policies. While the maximum entropy objective does yield a unique optimal policy (with respect to the maximum entropy objective), the policy is not guaranteed to align with the standard objective.

3.3 Trial-Based Heuristic Tree Search and Monte-Carlo Tree Search

TODO: Rename this subsection to just Trial Based Search I think makes sense

TODO: talk about UCT, and multiarmed bandits. Specifically that the theoretical lower bound is log T and UCT achieves log T lower bound. Define cumulative regret here.

3.3.1 Trial Based Heuristic Tree Search

TODO: THTS paper, talk about the differences that the paper has to our presentation of `THTS++`

TODO: cut from ch2: Finally we will briefly point out the differences between `THTS++`and the original THTS schema in subsection

TODO: talk about how these methods are still relevant with deep learning because of algorithms that use both, such as alpha zero

3.3.2 Monte-Carlo Tree Search

TODO: list

- Talk about the things that are ambiguous from literature (e.g. people will just say UCT, which originally presented doesn't run in `single_node_expansion`, but often assumed it does)
- Should talk about multi-armed bandits here?

https://inria.hal.science/inria-00164003/document

https://pdf.sciencedirectassets.com/271585/1-s2.0-S0004370211X0005X/1-s2.0-S000437021100052X/main.pdf?X-Amz-Security-Token=IQoJb3JpZ2luX2VjEOP

TODO: removed from ch2, this can be litrev: add polynomial UCT here? and or prioritised UCT from alpha go here?

TODO: removed from ch2: add stuff about regret here, give the O(logn) bound for UCT, and also talk about the papers that

TODO: removed from ch4: H-MCTS algorithm <TODO: cite>which combines UCT and Sequential Halving. BD: H-MCTS is used as an UCT style MCTS algorithm that is also designed using simple regret for comparison. (this was in intro to section 4.4, DENTS results section)

TODO: Should make sure to talk about all of the MCTS algorithms designed around simple regret. Confirm the claim that they all are really just some mix of uniform and UCT

TODO: should talk about MCTS methods that use simple regret, specifically H-MCTS and probably BRUE etc

TODO: Also talk about Trial-Based RTDP here

TODO: introduce RENTS and TENTS outside of the introduction Entropy is widely used in the reinforcement learning literature, commonly introduced to promote exploration and discourage convergence to suboptimal deterministic policies [53, 23, 36, 66]. MENTS (Section 2.4.5), RENTS and TENTS (Section 3.3.3) are all MCTS algoritms that optimise for maximum entropy objectives. While RENTS and TENTS will be considered as baselines in empirical experiments, MENTS will be used to facilitate and provide discussion around the use of entropy and maximum entropy objective.

TODO: make sure H-MCTS, (BRUE), RENTS and TENTS baselines all introduced in related work section here.

3.3.3 Maximum Entropy Tree Search

TODO: MENTS, RENTS and TENTS

3.4 Multi-Objective Reinforcement Learning

TODO: list

- Should talk about multi-objective and/or contextual multi-armed bandits here?
- Bunch of the work covered in recent MORL survey [25]

- Mention some deep MORL stuff, say that this work (given AlphaZero) is adjacent work

TODO: removed from ch2: comment here or in literature review about there being more types of scalariation function that arent necessarily weighted by a weight, and ESR vs SER stuff

TODO: talk about more of the MORL survey, including some of the other motivating scenarios

TODO: Talk about the better way of doing CHVI? https://www.jmlr.org/papers/volume13/lizotte12a and Efficient reinforcement learning with multiple reward functions for randomized controlled trial analysis. Also by Lizotte. But also say that its approximate for $D > 2$ and so we stick with the slower one? But the CHVS operations can be computed more efficiently in $D = 2$ using this stuff. Did it for the python implementation.

TODO: also need to talk about the MO sequential decision survey. Define Pareto Front. Say that the Pareto Front "Definition 3 If the utility function u is any monotonically increasing function, then the Pareto Front (PF) is the undominated set [Roijers et al., 2013]:"

TODO: need to talk about this because some prior/related work uses PF rather than CH

TODO: cover some inner loop and outer loop things? Roijers computing CCS work? See what MORL survey says about it

TODO: Maybe talk about the *witness algorithm* for Partially Observable MDPs as its very similar to CHVI stuff

Ann Nowe papers:

https://www.jmlr.org/papers/volume15/vanmoffaert14a/vanmoffaert14a.pdf

https://arxiv.org/pdf/2402.07182

https://www.ifaamas.org/Proceedings/aamas2024/pdfs/p1611.pdf

TODO: Go through Roijers and Ann Nowe and Mykel Kochenderfer

TODO: Deep MORL things in MO-gymnasium

TODO: Some of the outer and inner loop things - prism, computing convex coverage sets

TODO: Mention the prior work that uses Point Estimates, and make a brief (informal) argument for why this is insufficient, and say that won't be considering this approach in this thesis, or even implementing it as baseline

This paper make similar arguments to the ones I'm making in Ch5, should talk about it here too

Claude suggestion on how to justify use of linear utilities in this thesis: How I'd actually write this in the thesis. TODO: see comment

3.5 Multi-Objective Monte Carlo Tree Search

TODO: I think this whole section can just go in litrev

TODO: list

- Define the old methods (using the CH object methods, so clear that not doing direct arithmetic)
- Mention that old method could be written using the arithmetic of CHMCTS (but they don't)
- Different flavours copy UCT action selection, but with different variants
- Link back to contributions and front load our results showing that all of the old methods don't explore correctly

TODO: There has been some prior work in multi-objective MCTS which we will outline here

TODO: Write out implementations of prior works using THTS

TODO: define pareto front

TODO: perez algorithms

TODO: https://ieeexplore.ieee.org/document/8107102

TODO: https://www.roboticsproceedings.org/rss15/p72.pdf TODO: https://arxiv.org/abs/211

TODO: https://proceedings.mlr.press/v25/wang12b/wang12b.pdf

TODO: https://ifmas.csc.liv.ac.uk/Proceedings/aamas2021/pdfs/p1530.pdf TODO: https://link.springer.com/article/10.1007/s10458-022-09596-0

TODO: Some stuff we wrote that didnt use in ch2. Might want to talk about it with eval:

Moreover, in the case of MCTS algorithms, by having the user select a preferred policy, it implicitly forces the user to chose a preference over the objectives, as the policy corresponds to a weight vector that it is optimal for. As MCTS algorithms are often used in an online fashion, where planning is interleaved with execution, this implicitly selected weight can be used for any online execution needed, effectively reducing the multi-objective problem into a single-objective problem.

4

Monte Carlo Tree Search With Boltzmann Exploration

Contents

4.1 Introduction and Motivation 53
4.1.1 UCT 54
4.1.2 MENTS 55
4.1.3 Simple Regret and Consistency 57
4.2 Boltzmann Search 58
4.2.1 Boltzmann Tree Search 59
4.2.2 Decaying ENtropy Tree Search 62
4.2.3 Stochastic Search Policies: Prior Knowledge and Computational Efficiency 64
4.3 Theoretical MDPs 69
4.3.1 The D-Chain Problem 71
4.3.2 The Entropy Trap 73
4.4 Empirical Results 75
4.4.1 Environments 76
4.4.2 Evaluation Proceedure 79
4.4.3 Results and Discussion 81
4.5 Theoretical Results 86
4.5.1 Notation 87
4.5.2 Preliminaries 89
4.5.3 Consistency Results 91
4.5.4 The Entropy Trap 99
4.6 Summary 103

This chapter considers MCTS algorithms for planning in single-objective environ-

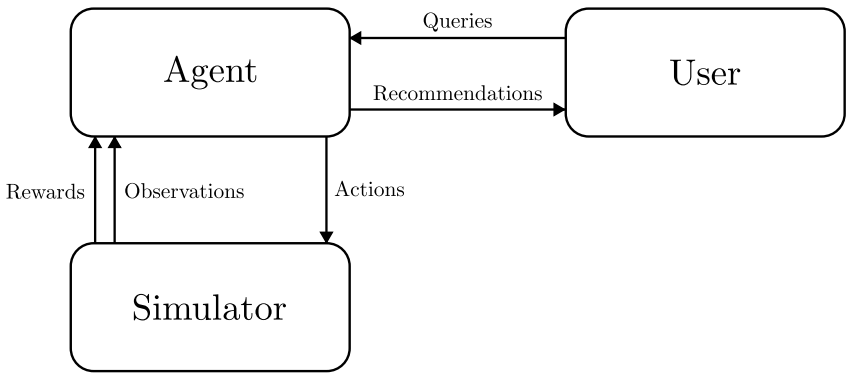


Figure 4.1: Exploration setting in reinforcement learning (edited from Figure 2.5), where the agent is only assessed on the recommendations that it provides, and is not penalised for considering poor actions in the simulator. This setting therefore places greater emphasis on exploration than exploitation.

ments, with the optional use of entropy as a secondary objective for exploration. The discussion focuses on the exploration setting for reinforcement learning (see Section 2.3 and Figure 4.1), where the agent is assessed purely on the recommendations it makes, rather than what actions it explored in simulation.

In maximum entropy reinforcement learning (Section 2.3.1), entropy is added directly to the objective function rather than used as a constraint to remove ambiguity between multiple optimal policies, as in inverse reinforcement learning [71, 70]. Whilst the maximum entropy objective yields a unique optimal policy, it is not guaranteed to align with the standard objective. This chapter demonstrates that the temperature parameter may have to be set to a prohibitively small value for the objectives to align, which in practice can negate the benefit of entropy-based exploration.

This motivates the investigation of **Q1.1 - Entropy**in this chapter, and lays the foundations for **Q1.2 - Multi-Objective Exploration** in Chapter 6.

The remainder of this chapter is organised as follows:

- Section 4.1 discusses limitations of existing MCTS algorithms in the exploration setting of reinforcement learning. *Simple regret* is defined in the context of sequential decision making, to theoretically analyse the performance of MCTS algorithms in the exploration setting.

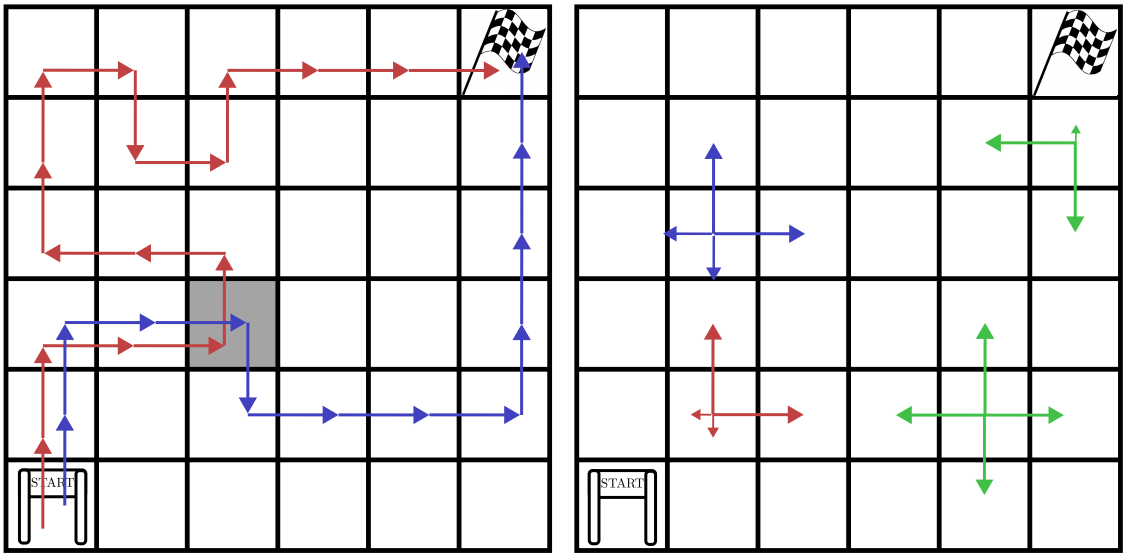
- Section 4.2 defines the *Boltzmann Tree Search* (BTS) and *Decaying ENtropy Tree Search* (DENTS) algorithms.
- Section 4.2.3 discusses some properties of using a stochastic search policy in MCTS, and examines how the *Alias Method* can be used to address **Q2.1 - Complexity**.
- Section 4.3 introduces two theoretical MDPs, which are used to empirically demonstrate the limitations of the existing MCTS algorithms.
- Section 4.4 presents experimental results on grid world environments and the game of Go, and gives an empirical answer to **Q1.1 - Entropy**.
- Section 4.5 presents the main theoretical analysis and proofs. Convergence properties of MENTS, BTS and DENTS are proven to provide a theoretical answer to **Q1.1 - Entropy**.

4.1 Introduction and Motivation

This section considers a grid world shortest path problem, which is used to discuss the behaviour of UCT and MENTS in the exploration setting, and highlight some limitations of these algorithms.

In this grid world, the agent starts at the origin $(0, 0)$, and must reach the goal state at $(6, 6)$. Each timestep, the agent may move deterministically in any cardinal direction, $\{\texttt{UP}, \texttt{RIGHT}, \texttt{DOWN}, \texttt{LEFT}\}$, as long as it remains in the grid. Each action incurs a cost of 1 (or a reward of -1), so the cumulative cost of a path is equal to its length. The time horizon is chosen to be 36 (six times the width of the grid) so that the horizon is long enough for exploration. The optimal policy will always select from the actions `UP` and `RIGHT`. The problem, with example paths, is illustrated in Figure 4.2.

While UCT and MENTS can easily solve this problem given enough trials and appropriately set parameters, it is used with a limited number of trials to illustrate the behaviour of the algorithms in the exploration setting. In Section 4.3, MDPs are



(a) Some possible paths from start to finish.

(b) Optimal policy with the maximum entropy objective for varying temperatures.

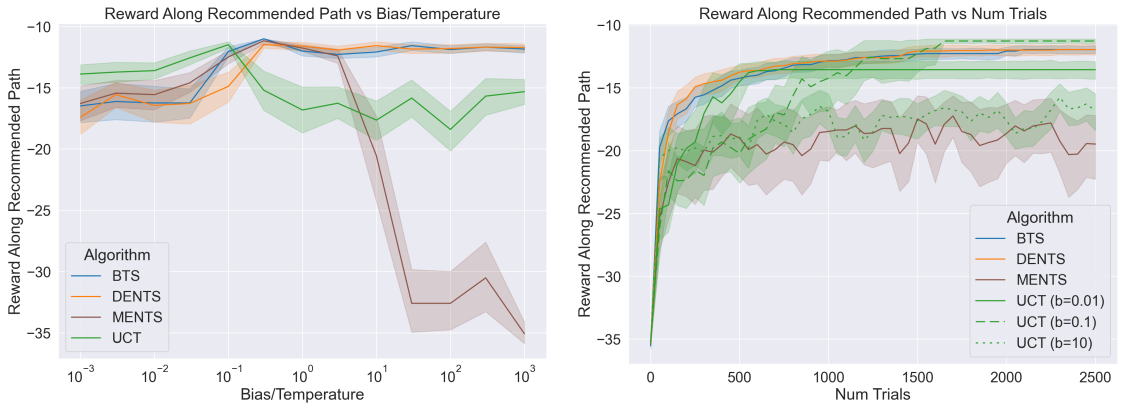
Figure 4.2: An example grid world shortest path problem. (a) Shows two paths that UCT may have explored, where the blue path is shorter, but the red path takes one of the optimal actions at the (shaded) state $(2, 2)$. (b) Illustrates optimal probability distributions at four states in the maximum entropy objective, with varying temperatures. In red, the temperature is low and in blue the temperature is larger. In green, the temperature is large so that gathering entropy "reward" outweighs reaching the goal.

constructed adversarially to expose limitations of UCT and MENTS, and Section 4.4 considers more complex grid worlds — including terminal states, sparse rewards and stochastic transition distributions — that are harder to solve in practice.

4.1.1 UCT

The UCT algorithm (Section 2.4.3) is designed in the traditional reinforcement learning setting described in Section 2.3, where the UCT agent aims to minimise the cumulative regret. Thus UCT makes a trade-off between exploration and exploitation during its trials, and frequently chooses the same action to exploit, which can result in it getting stuck in local optima, particularly when rewards are sparse or not informative.

A *sparse reward* is where the reward signal is infrequent, that is, most actions give a reward of zero. In the shortest path example, the reward is dense but uninformative, as the immediate cost of taking any action is the same.



(a) Reward of best path, after 2500 trials, for a range of bias/temperature parameters.

(b) Reward of best path relative to the number of trials. ($\alpha = 1$ and $b_{\text{PUCT}} = 0.01, 0.1, 1$).

Figure 4.3: Results on the example grid world problem for a variety of bias and temperature parameters. For comparison, the Boltzmann Tree Search (BTS) and Decaying ENtropy Tree Search (DENTS) algorithms defined in Section 4.2 are also shown. (a) Each algorithm is run for 2500 trials for a range of bias and temperature parameters, and (b) shows the evolution of the reward of the recommended best path as a function of the number of trials. In (a) and (b) each data point is averaged over 25 repeated experiments.

Suppose that UCT has been run for many trials, and the best paths found by UCT so far are shown in red and blue in Figure 4.2a. The two paths coincide until the shaded state at $(2, 2)$, where UCT has the Q-value estimates $\bar{Q}_{\text{UCT}}((2, 2), \texttt{DOWN}) \approx -8$ and $\bar{Q}_{\text{UCT}}((2, 2), \texttt{UP}) \approx -12$. When a UCT trial reaches the shaded state at $(2, 2)$, it will frequently select the action `DOWN` over `UP` to exploit the best path it has found so far, rather than exploring the optimal actions `UP` and `RIGHT`.

In theory, UCT selects every action infinitely often (that is, $N(s, a) \to \infty$ for all reachable s, a) and will converge to the optimal values and policy. In practice, UCT shifts its focus from exploration to exploitation throughout the search, which is determined by the confidence intervals and the bias parameter b_{UCT} from Equation (2.45). This is illustrated in Figure 4.3, where for small bias parameters UCT quickly exploits a suboptimal solution and for large bias parameters UCT is still exploring and its value estimates have not yet converged.

4.1.2 MENTS

One can argue that when using the maximum entropy objective, it is possible to recover the standard objective by setting the temperature parameter to zero, or an

infinitesimally small value. However, in practice, setting α to a tiny value will nullify the exploration advantages of using entropy. Considering the other extreme, when a very large temperature is used, entropy becomes the dominant objective in the maximum entropy objective, and the optimal soft policy will be (almost) uniform.

The optimal soft policy for a large temperature may be different from the optimal policy for the standard objective. In cases such as this, it can be said that the maximum entropy objective is *misaligned* with the standard objective.

Definition 4.1.1. *The policy induced by the optimal soft Q-values is* $\psi^*_{\text{sft}}(s) = \arg\max_{a'} Q^*_{\text{sft}}(s, a')$. *The maximum entropy objective is* misaligned *when the policy* $\psi^*_{\text{sft}}(s) \neq \pi^*(s)$, *where* π^* *is the optimal standard policy (Definition 2.3.4). The* misalignment threshold *is the smallest temperature parameter at which the maximum entropy objective is misaligned.*

In Figure 4.2b, the optimal soft policy for the maximum entropy objective is shown for varying temperature parameters. For small temperature values ($\alpha < 1$, illustrated in red), the optimal soft policy is close to the optimal standard policy, and for large temperature values ($\alpha > 100$, illustrated in green) the optimal soft policy is nearly uniform, except at states adjacent to the goal, where the agent will avoid reaching the goal to obtain more entropy "reward".

When using the maximum entropy objective, the temperature parameter often needs to be carefully tuned, large enough to gain an exploration benefit, but small enough to avoid misalignment issues. In this example problem, the misalignment threshold is around $\alpha = 10$, as can be seen in Figure 4.3a. Moreover, because the entropy of a policy depends on the MDP, the temperature typically needs to be tuned for each MDP. In Section 4.3, an MDP is constructed where the misalignment threshold is prohibitively small for the maximum entropy objective to be effective, and Section 4.4 considers more complex grid worlds where the misalignment threshold is low enough to significantly affect performance.

Parameters: An MDP $\mathcal{M}$.

- For each round $m = 1, 2, \ldots$:
  1. the agent produces a search policy π^m to follow;
  2. the environment samples a trajectory $\tau \sim \pi^m$ (including rewards $r_t = R(s_t, a_t)$ for each s_t, a_t pair in τ);
  3. the agent produces a recommendation policy ψ^m;
  4. if the environment sends a stop signal, then the game ends, otherwise the next round starts.

Figure 4.4: The procedure of an exploration planning problem for MDPs, where ψ^m is the recommendation policy the agent produces after m trajectories are sampled using the exploration policy π^m.

4.1.3 Simple Regret and Consistency

This section generalises the concept of simple regret (Definition 2.1.2) to MDPs and introduces the concept of consistency for MCTS algorithms. *Simple regret* is used to motivate and analyse the algorithms introduced in this chapter. *Consistency* refers to the correctness of an MCTS algorithm in the limit — informally, a consistent algorithm will eventually find the optimal policy given enough trials. The term originates from the MCTS literature, to describe whether modifications to UCT preserve its asymptotic behaviour [8, 11, 4].

The *simple regret* of a policy is the difference between the value of the policy and optimal value of the policy (Equation (4.1)). Whilst *cumulative regret* is used to analyse algorithms in the standard reinforcement learning setting, simple regret is more appropriate for the exploration setting, where the agent is assessed only on its recommendations and not penalised for exploring poor actions in simulation.

Definition 4.1.2. *The* simple regret *of a policy ψ at state $s \in \mathcal{S}$ is the difference between the value of the policy and the optimal value at that state:*

$$\texttt{sim_regr}(s, \psi) = V^*(s) - V^{\psi}(s). \tag{4.1}$$

TODO: make sure simple regret is in the table of notation

By definition, the optimal policy achieves a simple regret of zero. In this thesis an MCTS algorithm is considered *consistent* if its expected simple regret tends to zero. Informally, a consistent algorithm will eventually output an optimal policy if run indefinitely, and is expected to improve its recommendation with more trials.

Definition 4.1.3. *An agent, that produces recommendation policies* $\psi^1, \psi^2, \ldots$ *is said to be* consistent *if* $\mathbb{E}[\texttt{sim_regr}(s, \psi^m)] \to 0$ *as* $m \to \infty$.

Note that the simple regret (Equation (4.1)) is a random variable, as it depends on the recommendation policy ψ, which itself depends on the random trajectories that are sampled. Hence the expected simple regret is used in Definition 4.1.3 to define consistency, and is the main value of interest in the theoretical analysis in Section 4.5.

?] show that upper bounds on cumulative regret lead to both upper and lower bounds on simple regret. Returning to the discussion around UCT and MENTS, using this result, UCT is consistent with respect to Definition 4.1.3 because its upper bound on cumulative regret [31] leads to an upper bound on simple regret that tends to zero. However, MENTS is only consistent for a sufficiently small temperature parameter that depends on the MDP (TODO: ref section and/or theorem later in chapter).

4.2 Boltzmann Search

This section defines the new algorithms Boltzmann Tree Search (BTS) and Decaying ENtropy Tree Search (DENTS). Following the discussion from Section 4.1, the design aims of these algorithms are as follows:

- to be as simple to define and implement as UCT and MENTS[1];
- to be able to explore effectively when rewards are sparse or uninformative;
- to be able to exploit when necessary, in large or stochastic domains;
- to be consistent using parameters that do not depend on the MDP.

[1] i.e. each decision node operates on a multi-armed bandit problem

In Section 4.2.1 BTS is defined, which uses a Boltzmann search policy, similar to MENTS, but using the standard objective and Bellman backups rather than the soft Bellman backups used in MENTS. Section 4.2.2 defines DENTS which re-introduces entropy as an exploration bonus while maintaining consistency. Section 4.2.3 discusses the use of stochastic search policies in MCTS further, highlighting that mixture policies can be used to incorporate prior knowledge from $\pi_{\text{heuristic}}$ (Section 4.2.3.1) and how the *Alias Method* can be used to arrive at a more efficient MCTS algorithm addressing **Q2.1 - Complexity** (Section 4.2.3.3).

4.2.1 Boltzmann Tree Search

Algorithm Definition The *Boltzmann Tree Search* (BTS) algorithm uses the variables $\hat{V}_{\text{BTS}}$ and $\hat{Q}_{\text{BTS}}$ for value estimates, $N(s)$ and $N(s,a)$ for visit counts, and the function $\alpha_{\text{BTS}}(x) > 0$ for the search temperature. The temperature used at each decision node s is computed with respect to the visit count $N(s)$ as $\alpha_{\text{BTS}}(N(s))$.

The BTS search policy uses the flag `normalised_Q` to optionally normalise Q-values, and incorporates prior knowledge from $\hat{Q}_{\text{heuristic}}$ weighted by $N_{\text{BTS}}^{\hat{Q}_{\text{heuristic}}}$ pseudo-visits:

$$\hat{Q}_{\text{BTS}}^{\text{bias}}(s,a) = \frac{N(s,a)\hat{Q}_{\text{BTS}}(s,a) + N_{\text{BTS}}^{\hat{Q}_{\text{heuristic}}}\hat{Q}_{\text{heuristic}}(s,a)}{N(s,a) + N_{\text{BTS}}^{\hat{Q}_{\text{heuristic}}}}, \tag{4.2}$$

$$\hat{Q}_{\text{BTS}}^{\text{norm}}(s,a) = \begin{cases} \frac{\hat{Q}_{\text{BTS}}^{\text{bias}}(s,a) - \min_{a\in\mathcal{A}} \hat{Q}_{\text{BTS}}^{\text{bias}}(s,a)}{\max_{a\in\mathcal{A}} \hat{Q}_{\text{BTS}}^{\text{bias}}(s,a) - \min_{a\in\mathcal{A}} \hat{Q}_{\text{BTS}}^{\text{bias}}(s,a)} & \text{if } \texttt{normalised_Q}, \\ \hat{Q}_{\text{BTS}}^{\text{bias}}(s,a) & \text{otherwise.} \end{cases} \tag{4.3}$$

The search policy of BTS is similar to MENTS (Section 2.4.5), but using a variable temperature and (the normalised and biased) Bellman value estimates rather than soft value estimates.

$$\pi_{\text{BTS}}(a|s) = (1 - \lambda(s, \epsilon_{\text{BTS}}))\rho_{\text{BTS}}(a|s) + \frac{\lambda(s, \epsilon_{\text{BTS}})}{|\mathcal{A}|}, \tag{4.4}$$

$$\rho_{\text{BTS}}(a|s) \propto \exp\left(\frac{\hat{Q}_{\text{BTS}}^{\text{norm}}(s,a)}{\alpha_{\text{BTS}}(N(s))}\right), \tag{4.5}$$

$$\lambda(s,\epsilon) = \min\left(1, \frac{\epsilon}{\log(e + N(s))}\right), \tag{4.6}$$

where $\epsilon_{\text{BTS}} \in [0, \infty)$ is an exploration parameter.

For any new decision nodes s added to the tree, the `init_v` function initialises the visit count to $N(s) = 0$ and the value estimate to $\hat{V}_{\text{BTS}}(s) = \hat{V}_{\text{heuristic}}(s)$. For any new chance nodes (s, a) added to the tree, the `init_q` function initialises the visit count to $N(s, a) = 0$ and the Q-value estimate to $\hat{Q}_{\text{BTS}}(s, a) = \hat{Q}_{\text{heuristic}}(s, a)$.

BTS also allows the value estimates to be biased by $\hat{V}_{\text{heuristic}}$ with a weighting of $N_{\text{BTS}}^{\hat{V}_{\text{heuristic}}}$ pseudo-visits. When `single_node_expansion` is True, $N_{\text{BTS}}^{\hat{V}_{\text{heuristic}}}$ is typically set to one, weighting the heuristic value estimate as a single pseudo-trial. When `single_node_expansion` is False, $N_{\text{BTS}}^{\hat{V}_{\text{heuristic}}}$ is set to zero, as the trajectory already captures the rollout information.

Let $\tau_{:h} = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h) \sim \pi_{\text{BTS}}$ be the trajectory sampled during the selection phase. The `backup_v` and `backup_q` functions then update the value estimates for $t = h - 1, \ldots, 0$:

$$\hat{Q}_{\text{BTS}}(s_t, a_t) \leftarrow R(s_t, a_t) + \sum_{s' \in \text{Succ}(s_t, a_t)} \left(\frac{N(s')}{N(s_t, a_t)} \hat{V}_{\text{BTS}}(s') \right), \tag{4.7}$$

$$\hat{V}_{\text{BTS}}(s_t) \leftarrow \frac{N_{\text{BTS}}^{\hat{V}_{\text{heuristic}}} \hat{V}_{\text{heuristic}}(s_t) + N(s_t) \max_{a \in \mathcal{A}} \hat{Q}_{\text{BTS}}(s_t, a)}{N(s_t) + N_{\text{BTS}}^{\hat{V}_{\text{heuristic}}}}. \tag{4.8}$$

When BTS needs to recommend a policy, it can use its Q-value estimates, or the visit counts:

$$\psi_{\text{BTS}}(s) = \arg\max_{a \in \mathcal{A}} \hat{Q}_{\text{BTS}}(s, a), \tag{4.9}$$

$$\texttt{mv}_{\text{BTS}}(s) = \arg\max_{a \in \mathcal{A}} N(s, a). \tag{4.10}$$

Average Returns The BTS search policy can still be used with average returns. In *Boltzmann Tree Search with Average Returns* (AR-BTS) the Bellman value estimates are replaced with average returns $\bar{V}_{\text{AR-BTS}}$ and $\bar{Q}_{\text{AR-BTS}}$. Given a trajectory $\tau = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h)$ the average returns are updated for $t = h - 1, \ldots, 0$:

$$\bar{Q}_{\text{AR-BTS}}(s_t, a_t) \leftarrow \frac{N(s_t, a_t) - 1}{N(s_t, a_t)} \left(\bar{Q}_{\text{AR-BTS}}(s_t, a_t) + \frac{\hat{V}_{\text{h}}(s_h) + \sum_{i=t}^{h-1} r_i}{N(s_t, a_t) - 1} \right), \tag{4.11}$$

$$\bar{V}_{\text{AR-BTS}}(s_t) \leftarrow \frac{N(s_t) - 1}{N(s_t)} \left(\bar{V}_{\text{AR-BTS}}(s_t) + \frac{\hat{V}_{\text{h}}(s_h) + \sum_{i=t}^{h-1} r_i}{N(s_t) - 1} \right), \tag{4.12}$$

where $\hat{V}_{\text{h}}$ is a shorthand for $\hat{V}_{\text{heuristic}}$.

The corresponding definitions of $\bar{Q}^{\text{norm}}_{\text{AR-BTS}}$, $\pi_{\text{AR-BTS}}$, $\psi_{\text{AR-BTS}}$, $\texttt{mv}_{\text{BTS}}$ and so on are similar to the definitions for BTS and are omitted for brevity. The complete equations for all algorithms can be found in Appendix **??** TODO: double check this ref.

Summary of Theoretical Results TODO: Work out how to have the theorem descriptions using restatable environment, and add theorem titles for theory section too

In Section 4.5 it is shown that the BTS recommendation policy will eventually converge to the optimal policy.

Theorem 4.2.1. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{BTS}} > 0$, the expected simple regret of ψ_{BTS} tends to zero: $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{BTS}})] \to 0$.*

Additionally, if the temperature function is allowed to tend to zero, then the search policy will also converge to the optimal policy, and the most visited recommendation policy will also converge to the optimal policy as a result.

Theorem 4.2.2. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{BTS}} > 0$ and $\alpha_{\text{BTS}}(x) \to 0$, then $\pi_{\text{BTS}} \xrightarrow{p} \pi^*$ and $\mathbb{E}[\texttt{sim_regr}(s_0, \texttt{mv}_{\text{BTS}})] \to 0$.*

Conversely, if the temperature has a strictly positive lower bound (and does not converge to zero), then the BTS recommendation policy will converge to the optimal policy exponentially fast. However, note that neither the search policy nor the most-visited recommendation policy is guaranteed to converge to the optimal policy in this case.

Theorem 4.2.3. *For any MDP $\mathcal{M}$, if there exists $\alpha_{\text{BTS}}(x) \geq L > 0$, then there exist constants $C, k > 0$ such that $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{BTS}})] \leq C \exp(-kn)$ after running n trials of BTS.*

For AR-BTS, the temperature function always needs to be restricted to functions that tend to zero. Intuitively, the average return estimates converge only if the search policy converges, and the search policy converges only if the temperature tends to zero.

Theorem 4.2.4. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{AR-BTS}} > 0$ and $\alpha_{\text{AR-BTS}}(x) \to 0$, then $\pi_{\text{AR-BTS}} \xrightarrow{p} \pi^*$, $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{AR-BTS}})] \to 0$ and $\mathbb{E}[\texttt{sim_regr}(s_0, \texttt{mv}_{\text{AR-BTS}})] \to 0$.*

4.2.2 Decaying ENtropy Tree Search

Algorithm Definition *Decaying ENtropy Tree Search* (DENTS) extends the BTS algorithm by adding *entropy estimates* as an exploration bonus. DENTS uses Bellman value estimates $\hat{V}_{\text{DNTS}}$ and $\hat{Q}_{\text{DNTS}}$ similarly to BTS, but also uses entropy value estimates, $\bar{\mathcal{H}}_{V,\text{DNTS}}$ and $\bar{\mathcal{H}}_{Q,\text{DNTS}}$, which are Monte Carlo estimates of the entropy of the search policy rooted from the relevant node. By keeping a separate estimate for entropy, DENTS is able to use entropy in its search policy, and discard it for recommendations.

DENTS uses parameters that correspond to the BTS parameters: `normalised_Q`, $N_{\text{DNTS}}^{\hat{Q}_{\text{heuristic}}}$, $N_{\text{DNTS}}^{\hat{V}_{\text{heuristic}}}$, $\alpha_{\text{DNTS}}(x) > 0$ and ϵ_{DNTS}. Additionally, the entropy terms are weighted by a non-negative function $\beta_{\text{DNTS}}(x) \geq 0$ during the selection phase. Typically, β_{DNTS} is chosen such that $\beta_{\text{DNTS}}(x) \to 0$ as $x \to \infty$, so that the entropy weighting diminishes over time, allowing deeper parts of the tree to be explored[2].

The Q-values used during search are computed as follows:

$$\hat{Q}_{\text{DNTS}}^{\text{sft}}(s,a) = \hat{Q}_{\text{DNTS}}(s,a) + \beta_{\text{DNTS}}(N(s))\bar{\mathcal{H}}_{Q,\text{DNTS}}(s,a), \tag{4.13}$$

$$\hat{Q}_{\text{DNTS}}^{\text{sft,bias}}(s,a) = \frac{N(s,a)\hat{Q}_{\text{DNTS}}^{\text{sft}}(s,a) + N_{\text{DNTS}}^{\hat{Q}_{\text{heuristic}}}\hat{Q}_{\text{heuristic}}(s,a)}{N(s,a) + N_{\text{DNTS}}^{\hat{Q}_{\text{heuristic}}}}, \tag{4.14}$$

$$\hat{Q}_{\text{DNTS}}^{\text{sft,norm}}(s,a) = \begin{cases} \frac{\hat{Q}_{\text{DNTS}}^{\text{sft,bias}}(s,a) - \min_{a\in\mathcal{A}} \hat{Q}_{\text{DNTS}}^{\text{sft,bias}}(s,a)}{\max_{a\in\mathcal{A}} \hat{Q}_{\text{DNTS}}^{\text{sft,bias}}(s,a) - \min_{a\in\mathcal{A}} \hat{Q}_{\text{DNTS}}^{\text{sft,bias}}(s,a)} & \text{if } \texttt{normalised_Q}, \\ \hat{Q}_{\text{DNTS}}^{\text{sft,bias}}(s,a) & \text{otherwise.} \end{cases} \tag{4.15}$$

[2]In practice, it is also advisable to choose β_{DNTS} such that $\frac{\beta_{\text{DNTS}}(x)}{\alpha_{\text{DNTS}}(x)} \to 0$, so that the entropy weighting decays faster than the temperature. Otherwise, even as the temperature vanishes, the entropy bonus may still dominate the Q-values in the Boltzmann exponent, skewing the search policy away from the optimal action.

The DENTS search policy is similar to BTS:

$$\pi_{\text{DNTS}}(a|s) = (1 - \lambda(s, \epsilon_{\text{DNTS}}))\rho_{\text{DNTS}}(a|s) + \frac{\lambda(s, \epsilon_{\text{DNTS}})}{|\mathcal{A}|}, \tag{4.16}$$

$$\rho_{\text{DNTS}}(a|s) \propto \exp\left(\frac{\hat{Q}^{\text{sft,norm}}_{\text{DNTS}}(s,a)}{\alpha_{\text{DNTS}}(N(s))}\right), \tag{4.17}$$

$$\lambda(s, \epsilon) = \min\left(1, \frac{\epsilon}{\log(e + N(s))}\right). \tag{4.18}$$

The `init_v` and `init_q` functions are also similar to BTS, setting: $N(s) = 0$, $N(s,a) = 0$, $\hat{V}_{\text{DNTS}}(s) = \hat{V}_{\text{heuristic}}(s)$ and $\hat{Q}_{\text{DNTS}}(s,a) = \hat{Q}_{\text{heuristic}}(s,a)$. Additionally, the entropy value estimates are initialised to zero: $\bar{\mathcal{H}}_{V,\text{DNTS}}(s) = 0$ and $\bar{\mathcal{H}}_{Q,\text{DNTS}}(s,a) = 0$.

Given a trajectory $\tau = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h) \sim \pi_{\text{DNTS}}$, the `backup_q` and `backup_v` functions update the entropy value estimates for $t = h-1, \ldots, 0$:

$$\bar{\mathcal{H}}_{Q,\text{DNTS}}(s_t, a_t) \leftarrow \sum_{s' \in \text{Succ}(s_t, a_t)} \frac{N(s')}{N(s_t, a_t)} \bar{\mathcal{H}}_{V,\text{DNTS}}(s'), \tag{4.19}$$

$$\bar{\mathcal{H}}_{V,\text{DNTS}}(s_t) \leftarrow \mathcal{H}(\pi_{\text{DNTS}}(\cdot|s_t)) + \sum_{a \in \mathcal{A}} \pi_{\text{DNTS}}(a|s_t) \bar{\mathcal{H}}_{Q,\text{DNTS}}(s_t, a), \tag{4.20}$$

where $\mathcal{H}$ is the Shannon entropy function. The Bellman value estimates are updated similarly to BTS, for $t = h-1, \ldots, 0$:

$$\hat{Q}_{\text{DNTS}}(s_t, a_t) \leftarrow R(s_t, a_t) + \sum_{s' \in \text{Succ}(s_t, a_t)} \left(\frac{N(s')}{N(s_t, a_t)} \hat{V}_{\text{DNTS}}(s')\right), \tag{4.21}$$

$$\hat{V}_{\text{DNTS}}(s_t) \leftarrow \frac{N^{\hat{V}_{\text{heuristic}}}_{\text{DNTS}} \hat{V}_{\text{heuristic}}(s_t) + N(s_t) \max_{a \in \mathcal{A}} \hat{Q}_{\text{DNTS}}(s_t, a)}{N(s_t) + N^{\hat{V}_{\text{heuristic}}}_{\text{DNTS}}}. \tag{4.22}$$

The recommendation policies for DENTS are also defined identically to BTS:

$$\psi_{\text{DNTS}}(s) = \arg\max_{a \in \mathcal{A}} \hat{Q}_{\text{DNTS}}(s, a), \tag{4.23}$$

$$\texttt{mv}_{\text{DNTS}}(s) = \arg\max_{a \in \mathcal{A}} N(s, a), \tag{4.24}$$

Average Returns In *Decaying ENtropy Tree Search with Average Returns* (AR-DENTS) the Bellman value estimates are replaced with average returns $\bar{V}_{\text{AR-DNTS}}$ and $\bar{Q}_{\text{AR-DNTS}}$. For brevity and to avoid repetition, the equations are omitted but can be found in Appendix **??** TODO: double check this ref.

Summary of Theoretical Results Corresponding theoretical results hold for DENTS and AR-DENTS to those stated for BTS. It is additionally required that $\beta_{\text{DNTS}} \to 0$, so that the entropy bonus vanishes and the search policy is not skewed away from the optimal action. Proofs of the theorems below are given in Section 4.5.

Theorem 4.2.5. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{DNTS}} > 0$, the expected simple regret of ψ_{DNTS} tends to zero: $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{DNTS}})] \to 0$.*

Theorem 4.2.6. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{DNTS}} > 0$, $\alpha_{\text{DNTS}}(x) \to 0$ and $\beta_{\text{DNTS}}(x) \to 0$, then $\pi_{\text{DNTS}} \xrightarrow{p} \pi^*$ and $\mathbb{E}[\texttt{sim_regr}(s_0, \texttt{mv}_{\text{DNTS}})] \to 0$.*

Theorem 4.2.7. *For any MDP $\mathcal{M}$, if there exists $\alpha_{\text{DNTS}}(x) \geq L > 0$ and $U \geq \beta_{\text{DNTS}}(x)$, then there exist constants $C, k > 0$ such that $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{DNTS}})] \leq C \exp(-kn)$ after running n trials of DENTS.*

Theorem 4.2.8. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{AR-DNTS}} > 0$, $\alpha_{\text{AR-DNTS}}(x) \to 0$ and $\beta_{\text{AR-DNTS}}(x) \to 0$, then $\pi_{\text{AR-DNTS}} \xrightarrow{p} \pi^*$, $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{AR-DNTS}})] \to 0$ and $\mathbb{E}[\texttt{sim_regr}(s_0, \texttt{mv}_{\text{AR-DNTS}})] \to 0$.*

4.2.3 Stochastic Search Policies: Prior Knowledge and Computational Efficiency

This subsection discusses two properties of using a stochastic search policy in MCTS. First, the prior policy $\pi_{\text{heuristic}}$ can be incorporated into the search policy as a mixture policy, allowing prior knowledge to guide the selection phase. Second, the *Alias Method* [65, 64] is used to derive a more efficient MCTS algorithm by trading off policy accuracy for computational speed, providing an answer to **Q2.1 - Complexity**.

4.2.3.1 Incorporating Prior Knowledge Through Mixture Policies

In MENTS, BTS and DENTS, a uniform distribution is mixed with the Boltzmann distribution to form the search policy, using a logarithmically decaying weight that gradually transitions towards the Boltzmann distribution as the number of visits increases. The prior policy $\pi_{\text{heuristic}}$ can be incorporated in the same way: by mixing it into the algorithm's search policy π_{ALG} with a decaying weight, prior knowledge guides exploration early in the search while leaving the asymptotic behaviour unchanged.

The mixture policy π_{mix} is defined by:

$$\pi_{\text{mix}}(a|s) = (1 - \lambda(s, \epsilon_{\text{mix}}))\pi_{\text{ALG}}(a|s) + \lambda(s, \epsilon_{\text{mix}})\pi_{\text{heuristic}}(a|s), \tag{4.25}$$

$$\lambda(s, \epsilon) = \min\left(1, \frac{\epsilon}{\log(e + N(s))}\right). \tag{4.26}$$

where $\epsilon_{\text{mix}} \in [0, \infty)$ controls the weight given to the prior policy in the mixture. The weight given to $\pi_{\text{heuristic}}$ decays with the visit count, eventually reaching zero, so that the behaviour of the algorithm is unchanged in the limit.

4.2.3.2 The Alias Method

This subsection briefly covers the *Alias Method* [65, 64], which is used to sample from a categorical distribution in constant time, with a linear cost for preprocessing.

Let $f : \{C_1, \ldots, C_m\} \to [0, 1]$ be the probability mass function of a categorical distribution with m categories. Suppose a sample $c \sim f$ is required.

The naive method to sample from f takes $O(m)$ time, where a continuous uniform random value is sampled, $u \sim \text{Uniform}(0, 1)$, and f is linearly scanned until a probability mass of u has been passed.

The Alias Method, however, uses an $O(m)$ pre-processing step to construct an *alias table*, which is a table consisting of m entries, each accounting for $\frac{1}{m}$ of the probability mass of f. Each entry takes the form $(c_0, c_1, \texttt{thrsh})$, where $c_0, c_1 \in \{C_1, \ldots, C_m\}$ are categories and $\texttt{thrsh} \in [0, 1]$ gives the ratio of the $\frac{1}{m}$ probability mass to assign to c_0 and c_1. That is, a mass of $\frac{\texttt{thrsh}}{m}$ is assigned to c_0 and $\frac{1-\texttt{thrsh}}{m}$ is assigned to c_1.

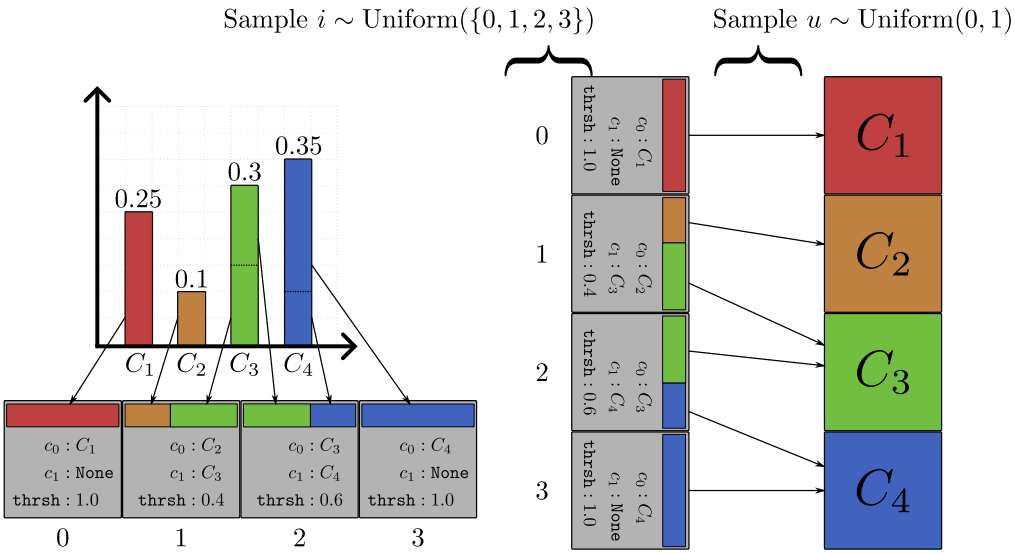


Figure 4.5: Example of building and sampling from an alias table, on a distribution with four categories. Left: shows how the probability mass is split into the four entries of the table. Right: shows how two random variables can be used to sample from the table in $O(1)$ time, where i is used to index into the table, and u is used to compare with `thrsh` and decide which of the two arrows from the entry to follow.

Sampling from the alias table can be performed in constant time by sampling two random numbers. The first number is a random integer index into the table, and the second number is a random continuous uniform value between 0 and 1. If the entry $(c_0, c_1, \texttt{thrsh})$ is sampled and the second number is less than or equal to `thrsh`, then c_0 is returned, otherwise c_1 is returned. Figure 4.5 illustrates an example of constructing an alias table and sampling from it.

Vose [64] proves that an alias table can always be constructed from an arbitrary categorical distribution, such that the probability of sampling any category from the alias table is identical to the probability of sampling it from the original probability mass function.

4.2.3.3 A More Efficient MCTS

The discussion in this section assumes that `single_node_expansion` is true, and that `transposition_table` is false. Additionally, the runtime complexities of `init_q` and `sample_outcome` are assumed to be $O(1)$[3]. In this case, the runtime

[3]This is typically the case when using a simulator. If given an arbitrary MDP, the Alias Method can be used with some preprocessing cost to sample outcomes in $O(1)$ time.

of one MCTS trial consists of the following time costs:

Selection Phase: The `select_action` function is called at up to $H_{\texttt{THTS++}}$ many decision nodes.

Heuristic Step: The `init_v` function is called once to initialise the value of the new decision node.

Backup Phase: The `backup_v` and `backup_q` functions are called at up to $H_{\texttt{THTS++}}$ many decision nodes and chance nodes.

Hence, the worst case complexity of an MCTS trial in `THTS++` is:

$$O(H_{\texttt{THTS++}} \cdot (O(\texttt{select_action}) + O(\texttt{backup_v}) + O(\texttt{backup_q})) + O(\texttt{init_v})). \tag{4.27}$$

When a rollout is used in `init_v`, its complexity is $O(H_{\texttt{THTS++}} \cdot A)$, where $A = |\mathcal{A}|$ is the number of actions. Unfortunately this is the dominating term in the runtime complexity of an MCTS trial. However, when using function approximation in `init_v`, i.e. $\hat{V}_{\text{heuristic}} = \tilde{V}$, and $\pi_{\text{heuristic}} = \tilde{\pi}$, the runtime complexity of `init_v` is $O(A)$, as $\pi_{\text{heuristic}}$ is evaluated for each action in $\mathcal{A}$ and the results cached. The rest of this subsection will assume that `init_v` runs in $O(A)$ time using function approximation.

Otherwise, `select_action` is typically the dominating factor in Equation (4.27), taking $O(A)$ time. For example in UCT (Section 2.4.3), the `select_action` function must compute a UCB score for each action in $\mathcal{A}$, taking $O(A)$ time.

However, if one is using a stochastic search policy and willing to use a slightly outdated search policy during selection, the Alias Method can be used as follows:

- In `select_action`, actions are sampled from an alias table stored in the decision node;
- In `init_v`, an initial alias table is constructed, with an additional $O(A)$ preprocessing cost, maintaining the $O(A)$ time complexity;

- In `backup_v`, the alias table is updated every A visits to the decision node, with an additional $O(A)$ time cost every A visits, giving only an amortised $O(1)$ time complexity on top of any backup equations used in `backup_v`.

The only remaining significant time costs are in `backup_v` and `backup_q`.

Average Return Backups. It is well known that the incremental updates for computing the average returns used in UCT (Equations (2.48) and (2.49)), AR-BTS and AR-DENTS have a constant $O(1)$ complexity.

Bellman `backup_v`. In BTS and DENTS, the `backup_v` function (Equations (4.8) and (4.22)) computes a maximum over actions in $\mathcal{A}$. Because the Q-value of only one action a_t is changed between the selection and backup phases, the maximum can be computed more efficiently using a max heap. `backup_v` updates the Q-value[4] corresponding to a_t in the max heap, in $O(\log(A))$ time, and then the maximum value can be read from the top of the heap.

Bellman `backup_q`. In the algorithms considered in this thesis so far, the Q-value backups are all either Bellman or average return backups. Bellman Q-value backups can be computed incrementally in $O(1)$ time. Consider the general form of the Bellman update at chance node (s_t, a_t), using the value estimates $\hat{V}$ and $\hat{Q}$:

$$\hat{Q}(s_t, a_t) \leftarrow R(s_t, a_t) + \sum_{s' \in \mathrm{Succ}(s_t, a_t)} \left(\frac{N(s')}{N(s_t, a_t)} \hat{V}(s') \right). \tag{4.28}$$

Because the value of only one child node, s_{t+1}, is changed between the selection and backup phases, Equation (4.28) can be computed incrementally in $O(1)$ time as:

$$\hat{Q}(s_t, a_t) \leftarrow \begin{cases} R(s_t, a_t) + \hat{V}(s_{t+1}) & \text{if } N(s_t, a_t) = 1, \\ R(s_t, a_t) + S(s_t, a_t) + \frac{N(s_{t+1})}{N(s_t, a_t)} \hat{V}(s_{t+1}) & \text{otherwise,} \end{cases} \tag{4.29}$$

$$S(s_t, a_t) = \frac{N^{\text{prev}}(s_t, a_t)}{N(s_t, a_t)} \left(Q(s_t, a_t) - R(s_t, a_t) - \frac{N^{\text{prev}}(s_{t+1})}{N^{\text{prev}}(s_t, a_t)} \hat{V}^{\text{prev}}(s_{t+1}) \right), \tag{4.30}$$

[4]Note that in an arbitrary MDP, the Q-value may decrease due to stochasticity in the MDP. If the MDP is deterministic, computing the Q-values will only increase, and the maximum can be computed in $O(1)$ time.

where S computes the corrected average of $\hat{V}(s')$ for $s' \in \text{Succ}(s_t, a_t) - \{s_{t+1}\}$, $\hat{V}^{\text{prev}}(s_{t+1})$ is the previous value of $\hat{V}(s_{t+1})$, and $N^{\text{prev}}(s_{t+1}) = N(s_{t+1}) - 1$, $N^{\text{prev}}(s_t, a_t) = N(s_t, a_t) - 1$ are the previous values of $N(s_{t+1})$, $N(s_t, a_t)$.

MENTS `backup_v`. Using the same observation as in the previous paragraph, the `backup_v` function in MENTS (Equation (2.58)) can be computed in $O(1)$ time. However, this is difficult to implement in a numerically stable way, because the *correction term*[5] involves products of exponentials.

DENTS Entropy `backup_v`. When the Alias Method is used with DENTS, the search policy π_{DNTS} is updated every A visits. On trials where the search policy is not updated, the same observation can be used again, correcting the value estimates using only the updated value of the child node whose value changed.

When the Alias Method is not used, the search policy π_{DNTS} may change on each trial, so Equation (4.20) needs to be recomputed, taking $O(A)$ time.

Summary of Complexities. The complexities of the different components of an MCTS trial are summarised in Table 4.1. To answer **Q2.1 - Complexity**in the affirmative, when the Alias Method is used with the most efficient implementations of `backup_v` and `backup_q`, the complexity of running a trial of MENTS can be improved to $O(H_{\texttt{THTS++}} + A)$. The complexity of running a trial of BTS and DENTS can be improved to $O(H_{\texttt{THTS++}} \log(A) + A)$, and the complexity of running a trial of AR-BTS and AR-DENTS can be improved to $O(H_{\texttt{THTS++}} + A)$.

4.3 Theoretical MDPs

This section builds on the discussion from Section 4.1 to construct adversarial MDPs that expose the limitations of UCT and the maximum entropy objective.

Coquelin and Munos [15] introduce the *D-chain problem*, which is a theoretical MDP for which UCT requires exponentially many trials to find the optimal policy.

[5] e.g. $S(s_t, a_t)$ in Equation (4.29)

Algorithm	`select_action`	`backup_v`	`init_v`	Trial Complexity
UCT	$O(A)$	$O(1)$	$O(1)$	$O(H_{\texttt{THTS++}} \cdot A)$
PUCT	$O(A)$	$O(1)$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot A)$
MENTS	$O(A)$	$O(1)$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot A)$
MENTS+AM	$O(1)$	$O(1)$	$O(A)$	$O(H_{\texttt{THTS++}} + A)$
BTS	$O(A)$	$O(\log(A))$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot A)$
BTS+AM	$O(1)$	$O(\log(A))$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot \log(A) + A)$
AR-BTS	$O(A)$	$O(1)$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot A)$
AR-BTS+AM	$O(1)$	$O(1)$	$O(A)$	$O(H_{\texttt{THTS++}} + A)$
DENTS	$O(A)$	$O(A)$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot A)$
DENTS+AM	$O(1)$	$O(\log(A))$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot \log(A) + A)$
AR-DENTS	$O(A)$	$O(1)$	$O(A)$	$O(H_{\texttt{THTS++}} \cdot A)$
AR-DENTS+AM	$O(1)$	$O(1)$	$O(A)$	$O(H_{\texttt{THTS++}} + A)$

Table 4.1: Summary of complexities for different MCTS algorithms. Complexities for each of the significant components are shown for each algorithm. AM is short for Alias Method. `backup_q` can be implemented in $O(1)$ time for all algorithms in this table so is omitted. Finally, UCT does not use $\pi_{\text{heuristic}}$, so it can implement `init_v` in $O(1)$ by only computing the function approximation $\hat{V}_{\text{heuristic}} = \tilde{V}$.

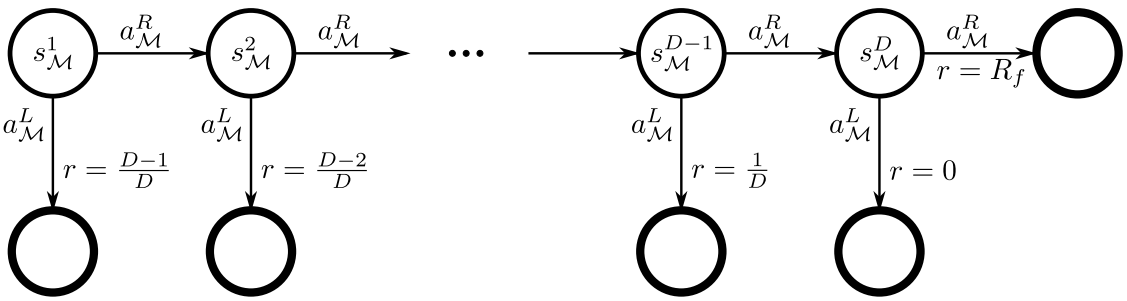


Figure 4.6: An illustration of the *(modified) D-chain problem*, where $s^1_{\mathcal{M}}$ is the starting state and transitions are deterministic. The action $a^L_{\mathcal{M}}$ from state $s^i_{\mathcal{M}}$ results in an immediate reward of $\frac{D-i}{D}$ and the trial ends. Taking action $a^R_{\mathcal{M}}$ to the end of the chain results in a reward of R_f. Where $R_f = 1$ in the original D-chain, and $R_f = \frac{1}{2}$ in the modified D-chain.

MENTS is able to solve the problem in a small number of trials due to its maximum entropy objective. However, Section 4.3.1 demonstrates that changing a single reward in the MDP can shift the misalignment threshold to a prohibitively small value, rendering the maximum entropy objective ineffective.

In Section 4.3.2, the D-chain problem is extended to include an *entropy trap*, to create an MDP for which both UCT and MENTS cannot solve in any feasible time.

4.3.1 The D-Chain Problem

The *D-chain problem* consists of a chain of D states with two possible actions and is depicted in Figure 4.6. If the action $a^L_{\mathcal{M}}$ is taken from state $s^i_{\mathcal{M}}$, then an immediate reward of $R(s^i_{\mathcal{M}}, a^L_{\mathcal{M}}) = \frac{D-i}{D}$ is collected and the trial ends. If the action $a^R_{\mathcal{M}}$ is taken the agent continues along the chain without any reward, until the end of the chain is reached where the agent receives a reward of $R(s^D_{\mathcal{M}}, a^R_{\mathcal{M}}) = 1$. By inspection, the optimal policy is $\pi^*(s) = a^R_{\mathcal{M}}$, to follow the chain to the end and collect the maximum reward of one.

To intuitively understand why UCT will struggle on this MDP, the reward function can be decomposed into a dense and sparse component:

$$R_{\text{dense}}(s^i_{\mathcal{M}}, a^L_{\mathcal{M}}) = \frac{D-i}{D}, \tag{4.31}$$

$$R_{\text{dense}}(s^i_{\mathcal{M}}, a^R_{\mathcal{M}}) = 0, \tag{4.32}$$

$$R_{\text{sparse}}(s^i_{\mathcal{M}}, a) = \mathbb{1}[i = D] \cdot \mathbb{1}[a = a^R_{\mathcal{M}}], \tag{4.33}$$

$$R(s, a) = R_{\text{dense}}(s, a) + R_{\text{sparse}}(s, a). \tag{4.34}$$

The two components of the rewards can be seen to encourage the agent to perform two different behaviours. Intuitively, R_{dense} encourages the agent to leave the chain as soon as possible, while R_{sparse} rewards the agent for staying on the chain until the end. Since UCT balances exploration and exploitation, it will frequently select $a^L_{\mathcal{M}}$ to exploit the immediate reward of $\frac{D-1}{D}$, rather than exploring to find the reward of one.

Compounding this, once a trial of UCT reaches the unit reward for the first time, the value estimates of $\bar{V}_{\text{UCT}}(s^i_{\mathcal{M}})$ still need many more trials to converge and recommend the optimal policy. This phenomenon can be seen in Figure 4.7, where UCT never recommends the optimal policy, while MaxUCT (TODO: add ref to ch2/3) that uses the same search policy with Bellman backups will recommend the optimal policy once one trial reaches the unit reward at the end of the chain.

Coquelin and Munos [15] formally prove that UCT will suffer a regret of $\Omega((\exp \circ \exp \circ \ldots \circ \exp)(1))$, where $\circ$ denotes functional composition, and there are D many composed exponential functions.

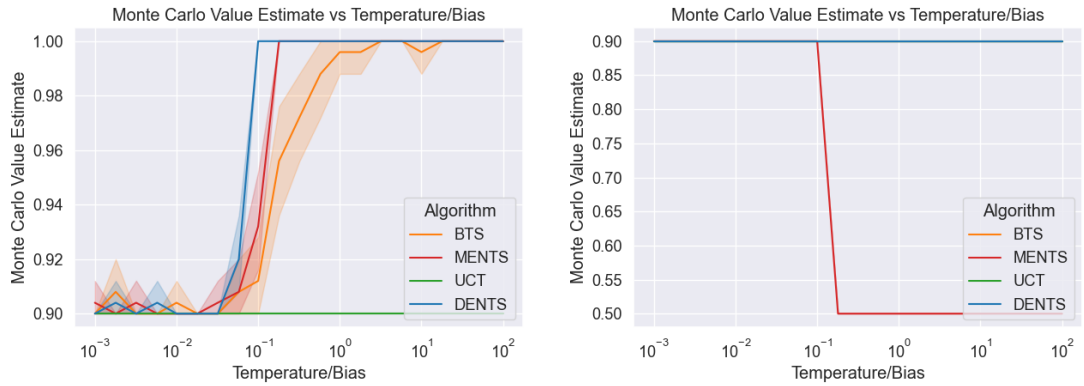


Figure 4.7: Results on the D-chain problem (left), and the modified D-chain problem (right), for a variety of parameters and with $D = 10$. Each algorithm was run for 5000 trials, and results are averaged over 25 repeated experiments. For BTS the temperature function is set to a constant value, and in DENTS the temperature function and entropy weighting function are set to the same constant value. TODO: update figures to have y-axis 0 to 1

From the perspective of practical single-objective reinforcement learning, this issue may appear to be one of poor reward shaping. However, rewards with conflicting objectives similar to this naturally arise in the context of multi-objective reinforcement learning. An example of this will be seen in the *Deep Sea Treasure* problem in Chapter 5, where the scalarised reward combines a dense time cost with a sparse treasure reward, yielding a structure similar to Equation (4.34).

Introducing entropy as an exploration bonus naturally encourages the agent to explore along the chain, because once the agent leaves the chain it can no longer collect a bonus entropy reward. As can be seen in Figure 4.7, the D-chain problem is an example of an MDP where the standard and maximum-entropy objectives are well aligned.

To illustrate that in maximum entropy methods the temperature parameter requires careful per-MDP tuning, consider the *modified D-chain problem*, where the sparse reward at the end of the chain is replaced with $R(s^D_{\mathcal{M}}, a^R_{\mathcal{M}}) = \frac{1}{2}$. In this case the optimal policy takes the immediate reward of $R(s^1_{\mathcal{M}}, a^L_{\mathcal{M}}) = \frac{D-1}{D}$ from the initial state, and as illustrated in Figure 4.7, the maximum entropy objective is misaligned with the standard objective for a temperature $\alpha > \frac{1}{5}$.

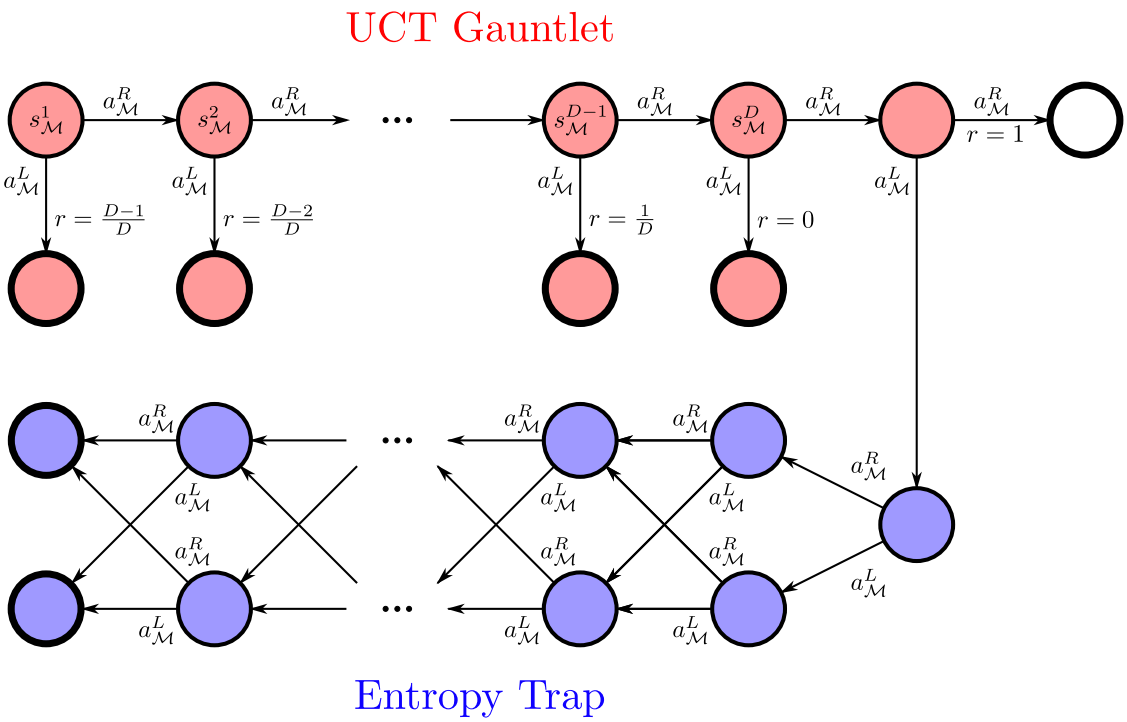


Figure 4.8: An illustration of the *Entropy Trap problem*. In red, the original D-chain problem (Figure 4.6) is labelled as the UCT Gauntlet, and the additional states added for the Entropy Trap are in blue. If a reward is not specified for a transition it is zero. TODO: rephrase labels to D-Chain and Entropy Chain

4.3.2 The Entropy Trap

In the *D-chain with Entropy Trap problem* (or *Entropy Trap problem* for short), an additional chain of states is appended to the end of the D-chain, designed to exploit the misalignment between the maximum entropy and standard objectives. At the end of the D-chain, the agent must choose between collecting the sparse reward of one, or entering the *entropy chain* where no reward is given but the agent can accumulate entropy "reward". The Entropy Trap problem is depicted in Figure 4.8.

At each step of the entropy chain the agent still has the option of the two actions $a^L_{\mathcal{M}}$ and $a^R_{\mathcal{M}}$. The optimal maximum entropy policy is uniform, taking either action with probability $\frac{1}{2}$ with an entropy of $\log(2)$ per step. If there are E steps in the entropy chain and a temperature of α is used in the maximum entropy objective, then the maximum entropy bonus that can be collected from the chain is $\alpha E \log(2)$. Therefore, $\frac{1}{E\log(2)}$ is an upper bound on the misalignment threshold for the entropy

trap, as if $\alpha > \frac{1}{E\log(2)}$, a maximum entropy agent will select the entropy chain over the reward of one at the end of the D-chain.

The entropy trap problem can therefore be made arbitrarily difficult for UCT by increasing D, while the misalignment threshold can be made arbitrarily small by increasing E. Theorem 4.3.1 uses the case when $D = E$ to show that MENTS must either suffer from misalignment or require an exponential number of trials with respect to the size of the state space.

Theorem 4.3.1. *For any value of α_{MENTS}, either the maximum entropy objective is misaligned, or MENTS requires an exponential number of trials to recommend the optimal policy.*

More precisely, after n trials of MENTS: $\Pr(\psi_{\text{MENTS}} \neq \pi^*) > \frac{1}{2}$ *if* $n \leq 2^{D-1}$*; and either* $\mathbb{E}\left[\texttt{sim_regr}(s_0, \psi_{\text{MENTS}})\right] \not\to 0$ *or there exist constants* $c, k > 0$ *such that* $\mathbb{E}\left[\texttt{sim_regr}(s_0, \psi_{\text{MENTS}})\right] \geq c\left(1 - \frac{n}{k^{|\mathcal{S}|}}\right)$.

Results on the Entropy Trap problem are given in Figure 4.9 for $(D, E) = (10, 10)$ and $(D, E) = (15, 25)$. In the $(D, E) = (15, 25)$ case, 500,000 trials were run per parameter setting. The misalignment threshold is bounded above by $\frac{1}{25\log(2)} \approx 0.06$, which is prohibitively small for effective exploration. Below this threshold, no more than 1000 of the 500,000 trials reached the third state of the D-chain, and typically fewer than 100. While BTS and MaxUCT are able to produce the optimal policy given a sufficiently large temperature/bias parameter, increasing D further would eventually prevent them from doing so within a reasonable number of trials. DENTS, by contrast, is able to recommend the optimal policy across a wider range of settings since its Bellman value estimates are unaffected by the entropy chain. For BTS and DENTS, each of α_{BTS}, α_{DNTS} and β_{DNTS} is set to a constant value. While this leads to the majority of trials in DENTS entering the entropy chain, this can be resolved by using a function that tends to zero for β_{DNTS}.

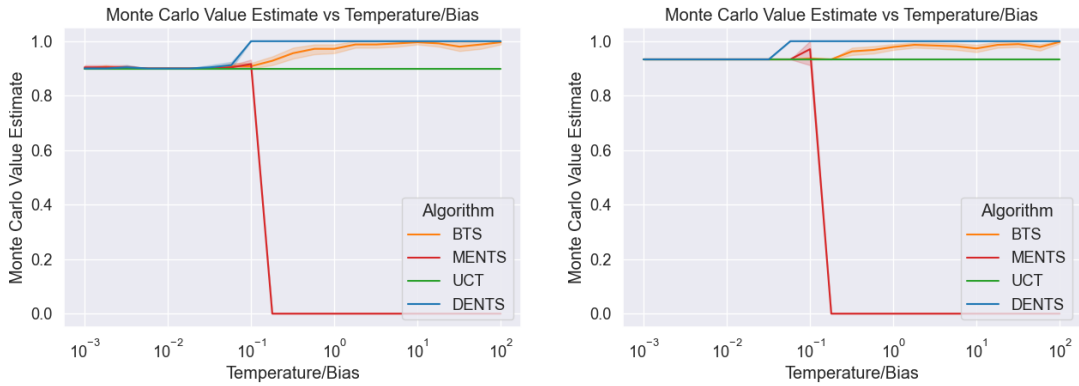


Figure 4.9: Results on the Entropy Trap problem, with $(D, E) = (10, 10)$ (left) and $(D, E) = (15, 25)$ (right). Each algorithm was run with a variety of temperature and bias parameters, averaged over 25 repeated experiments. For BTS the temperature function is set to a constant, and in DENTS the temperature and entropy weighting functions are set to the same constant. All other exploration parameters ϵ and the heuristic value function $\hat{V}_{\text{heuristic}}$ are set to zero. In the $(D, E) = (10, 10)$ case each algorithm was run for 5000 trials: MENTS produces the optimal policy for a small range of temperatures, and BTS and MaxUCT do so for sufficiently large temperature and bias parameters. In the $(D, E) = (15, 25)$ case each algorithm was run for 500,000 trials: at the misalignment threshold MENTS recommendations drop to zero due to the entropy trap, while DENTS recommends the optimal policy using its Bellman value estimates.

4.4 Empirical Results

TODO: Clean up plots, legend placement etc.

TODO: Reference appendix for hyperparameters used. State somewhere that all of the generic edits to the selection function (normalisation, search bias and backup bias), and reference the BTS equations for those. Actually, make the appendix called additional details on Ch4 empirical results, and there give the entire algorithm used give the full details with full backups and biasing terms for UCT, MENTS etc

TODO: also back reference RTDP, and how BTS with single node expansion turned off is essentially that. Difference is that temperature is decayed but not to zero in RTDP and BTS adds the epsilon exploration term

TODO: also make a comment about how just use H-MCTS as a simple regret baseline, but the simplex regret MCTS methods loosly boil down to a mixture of UCT and uniform combinations.

TODO: Wasn't too careful about changing from active voice to passive voice here in this entire section

TODO: Get some better results using hyperparam optimise //// want a dense env where the (not entropy) temp decay fn gets optimised to soemthing that decays a lot /// want a sparse env where the temp decay fn gets optimised to something flat (or basically flat)

This section provides empirical results on gridworld environments and on the game of Go. It begins by describing the environments, the evaluation setup, and is followed by the empirical results and discussion. The main algorithms discussed in this chapter, BTS and DENTS will be evaluated, using UCT, MENTS, RENTS, TENTS and H-MCTS as baselines TODO: ref litrev or cite?.

4.4.1 Environments

4.4.1.1 (Deterministic) Frozen Lake

The *(Deterministic) Frozen Lake* TODO: remove deterministic is a grid world environment with one goal state. The agent can move in any cardinal direction at each time step, and walking into a wall leaves the agent in the same location. TODO: Add is slippery description. Trap states exist where the agent falls into a hole and the trial ends. TODO: remove next sentence and add paragraph below. If the agent arrives at the goal state after t timesteps, then a sparse reward of 0.99^t is received. TODO: Also add dense reward -1, remove the determinsitic bit, and describe the is slippery option too.

TODO: Two rewards considered. Dense uninformative reward of -1 everywhere, and when falling in a hole the agent gets stuck and recieves a total return of -H. Second reward is sparse, where agents gets a reward of one for reaching the goal state. When is slippery is not on, a discount factor is used, so that shorter paths gain more reward than longer ones. This can effectively be implemented by returning gamma to the t in the deterministic case when agent reaches goal after t timesteps.

TODO: Extra credit: would like to have experiments which vary the proportion of the sparse reward. So have FL(lambda), where lambda specifies the ratio between

```
SFFFFFHF
FFFFFFFF
FHFHFFFF
FFFFFFHH
FFFHFFFF
FHHHFFFF
FFFFFHFF
FFFFFFFG
```

(a) 8x8 Frozen Lake.

```
SFHFFFHFFFFF
FFFFFFFHFFFF
HFFFFFHFFFFF
FHFFHFFFFFFF
HHFFFFFFFFFF
FHFFFFHFFFFF
FHFFFHHFHFFF
FFFFFFFFFHHG
```

(b) 8x12 Frozen Lake.

Figure 4.10: Maps used for the Frozen Lake environment. `S` is the starting location for the agent, `F` represents floor that the agent can move too, `H` are holes that end the agents trial and `G` is the goal location. TODO: Add 4x4, 5x5 or 4x6, 6x6 or 4x8 maps for stoch envs TODO: change to images of 4x4 and 8x8 frozen lake from running gym, just take screenshots

the dense and sparse rewards. Then investigate what happens as vary lambda. So this is the grid world experiments run again,

Figure 4.10 gives the specific maps used for Frozen Lake in these experiments. In the maps, `S` denotes the starting location of the agent, `F` denotes spaces the agent can move to, `H` denote holes that end the agents trial and `G` is the goal location.

An 8x8 map (Figure B.1a) is used in Section 4.4.3.1 to demonstrate how the different Boltzmann search and entropy based algorithms perform with a variety of temperature parameters. The 8x12 map (Figure B.1b) is used for an environment that is sufficiently large map to test and compare algorithms. Each of these maps was randomly generated, with each location having a probability of 1/5 of being a hole, and the maps were checked to have a viable path from the starting location to the goal location. TODO: with the exception of maps XXX, YYY, which were not randomly generated for these experiments, and chosen to correspond to the default maps for that size in Gymnasium (TODO: cite)

4.4.1.2 Sailing Problem

The *Sailing Problem* is another grid world environment with one goal state, at the opposite corner to the starting location of the agent. This problem has often been used to evaluate MCTS algorithms [42, 30, 62, 19].

```
FFFFFG
FFFFFF
FFFFFF
FFFFFF
FFFFFF
SFFFFF
```

(a) 6x6 Sailing Problem map. TODO: Replace with blue grid, boat and buoy clip art if time.

$$\begin{pmatrix} 0.4 & 0.3 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.3 \\ 0.4 & 0.3 & 0.3 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.4 & 0.3 & 0.3 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.4 & 0.3 & 0.3 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.4 & 0.2 & 0.4 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.3 & 0.3 & 0.4 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.3 & 0.3 & 0.4 \\ 0.4 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.3 & 0.3 \end{pmatrix}$$

(b) Wind transition probabilities.

Figure 4.11: The map and wind transition probabilities used for the 6x6 Sailing Problem. The same notation is used for the map as Figure 4.10 For the wind transition probabilities, the $(i, j)th$ element of the matrix denotes the probability that the wind changes from direction i to direction j, where 0 denotes North/up, 1 denotes North-East/up-right, and so on. TODO: Update caption.

The agent has 8 different actions, where it can travel in any cardinal or semi-cardinal direction. In each state of the MDP, the wind is blowing in a given direction and will stochastically change after every transition. The agent cannot sail directly into the wind. Actions and wind directions can take values in $\{0, 1, \ldots, 7\}$, with a value of 0 representing North/up, 2 representing East/right, 4 representing South/down and 6 representing West/right. The remaining numbers represent the inter-cardinal directions. The cost of each action depends on the *tack*, the angle between the direction of the agent's travel and the wind. TODO: The agent incurs a (dense) cost of (thetawind - theta)/45 + 1 for traveling at angle theta when wind is in direction thetawind.

Figure **??** gives the map and wind transition probabilities for the Sailing problem, using the same map notation as for Frozen Lake.

4.4.1.3 Go

For a more challenging domain, a round-robin tournament was run on the game of Go, which has widely motivated the development of MCTS methods [**?** 56, 57]. *Area scoring* is used to score the games, with a *komi* (a score handicap for black) of 7.5.

The open-source library KataGo [67] was used to provide a value network $\tilde{V}$ and prior policy network $\tilde{\pi}$ for the algorithms.

PUCT algorithm [4] was used as a baseline algorithm, as described in Alpha Go Zero [57], using prioritised UCB [49] to utilise the policy neural network. Each algorithm was limited to 5 seconds of compute time per move, allowed to use 32 search threads per move, and had access to 80 Intel Xeon E5-2698V4 CPUs clocked at 2.2GHz, and a single Nvidia V100 GPU on a shared compute cluster.

The methods described in this chapter can be adapted for two-player games by using a discount factor in the MDP. The value of a policy π with a discount factor γ is defined as:

$$V^{\pi}(s;t) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{i=t}^{H-1}\gamma^{i-t}r_i \middle| s_t = s\right]. \tag{4.35}$$

TODO: should define this in ch2? It's only really relevant here, and kind of in Frozen lake If the two players are using the policies π_0 and π_1, and π is the policy that interleves π_0 at even timesteps with π_1 at odd timesteps, then using a discount factor of $\gamma = -1$ is sufficient to handle two-player games. TODO: assuming that the reward function always gives rewards from the perspective of the player taking the current action.

In these experiments we used a recommendation policy that recommends the action that was sampled the most, i.e. $\psi(s) = \max_a N(s,a)$, as this tends to be more robust to any noise from neural network outputs.

For our final results we used the same parameters on 19x19 Go with a komi of 7.5, giving each algorithm 5.0 seconds per move.

Finally, we also ran AR-BTS against Kata Go directly limiting each algorithm to 1600 trials. KataGo beat AR-BTS by 31-19, confirming that the aditional exploration is outweighed by the information contained in the neural networks, and in Go the Boltzmann search algorithms gain their advantage via the Alias method and being able to run more trials quickly.

4.4.2 Evaluation Proceedure

Consider an algorithm with search tree $\mathcal{T}$, which provides a partial recommendation policy ψ_{alg}. The recommendation policy is made complete by using a uniformly

random policy for states and actions that are outside of the tree $\mathcal{T}$ as follows:

$$\psi(a|s) = \begin{cases} 1 & \text{if } s \in \mathcal{T} \text{ and } a = \psi_{\text{alg}}(s), \\ 0 & \text{if } s \in \mathcal{T} \text{ and } a \neq \psi_{\text{alg}}(s), \\ \frac{1}{|\mathcal{A}|} & \text{otherwise.} \end{cases} \tag{4.36}$$

Using the completed policy ψ, a monte carlo value estimate of V^{ψ} is computed by sampling a number of trajectories from ψ and averaging the returns. TODO: Kind of want a better argument for this line of eval here. Although this proceedure is evaluating the algorithms in an *offline planning* setting, it still indicates how the algorithms perform in an *online* setting when planning in simulation is interleaved with letting the agent act in the real environment. TODO: either scrap the evaluation question, or do something more to justify this.But should say something like "this evaluates the policy, and the online performance depends on the policies produced online, therefore correlated"

All experiments in this chapter were run with `single_node_expansion` turned off and did not use the Alias method as described in Section 4.2.3, apart from the experiments on Go, which used `single_node_expansion`and the Alias method.

4.4.2.1 Gridworld Hyperparameter Selection

To select hyperparameters for the algorithms in the gridworld environments, the BayesOpt package [35] was used. Full details on the hyperparameter tuning and the selected parameters are given in Appendix **??** TODO: double check this ref.

These experiments were run with `single_node_expansion` turned off and did not use the Alias method (Section 4.2.3).

4.4.2.2 Go Hyperparameter Selection

To tune hyperparemters for algorithms used in the Go experiments, a methodical sequence of round robin tournaments were run on a 9×9 board, where in each tournament one parameter was tuned at a time. Full details of these tournaments and the selected hyperparamters are given in Appendix B.3 TODO: double check this ref.

Of note, both Bellman value estimates and average returns were considered for the Go round robin. In the hyperparamter tuning the two versions of each

algorithm were tuned seperately and then played against each other. In each case the average returns algorithms decisively beat the Bellman versions. This could have been because of average returns helping to average out noisy neural network outputs, and the ability of the AR algorithms to run more trials (as described in Section 4.2.3) in the given search time.

These experiments were run with `single_node_expansion` turned on and made use of the Alias method (Section 4.2.3). Additionally, the value network $\tilde{V}$ was used for the value initialisation function $\hat{V}_{\text{heuristic}}$, and prior policy network $\tilde{\pi}$ was mixed into the search policy as described in Section 4.2.3.

4.4.3 Results and Discussion

TODO: Wanted to show the breadth first search if Qinit incorrectly set then get breadth first search in costs case. If can estimate, for example know Rmin and can get Qinit to H times Rmin then can do that. Another zero knowledge Qinit function can use is Qinit equals min a Q(s,a) where for (s,a) in tree, if no such (s,a) then zero (its uniform policy then anyway).

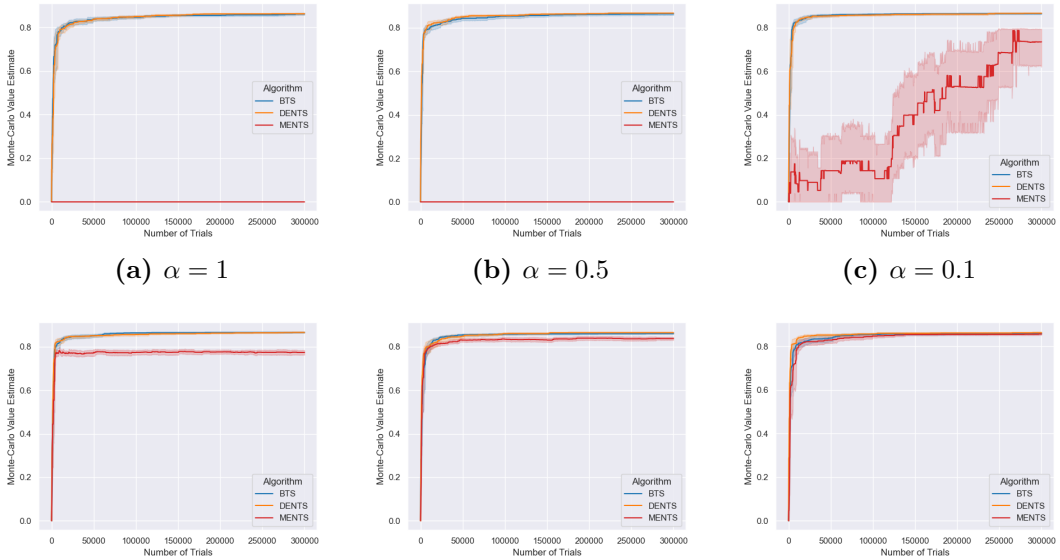
TODO: Talk about how exploitation is useful in two cases in the exploration setting. When have stochastic env, need to exploit to get better value estimates. When env is very large, need exploitation to explore deeper parts of env. Also exploiting helpful when reward is dense and/or informative

TODO: Should also eval AR-BTS etc on these envs? Might make the arguments/results nicer and more complete

This subsection first demonstrates the parameter sensitivity of maximum entropy methods on the Frozen Lake environment. Then results on Frozen Lake and the Sailing problem are given with optimised parameters, and finally results from the Go round robin.

4.4.3.1 Parameter sensitivity in Frozen Lake

TODO: this subsubsection currently just a c and p from neurips



(a) $\alpha = 1$ **(b)** $\alpha = 0.5$ **(c)** $\alpha = 0.1$

(d) $\alpha = 0.05$ **(e)** $\alpha = 0.01$ **(f)** $\alpha = 0.005$

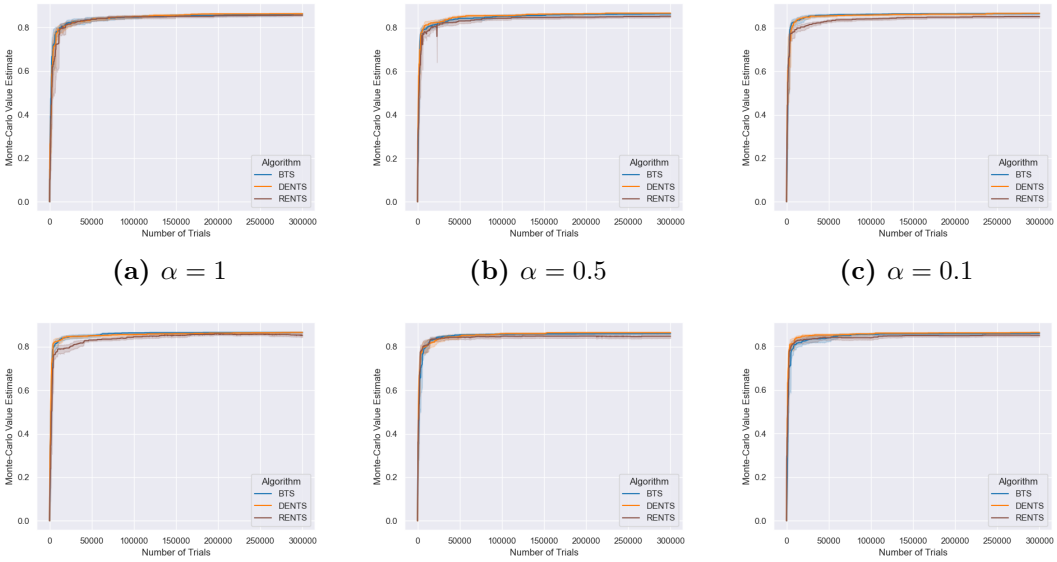
Figure 4.12: MENTS with a variety of temperatures on an 8x8 Frozen Lake environment. BTS and DENTS are included for reference. TODO: Update fig?

Further discussion on sensitivity to the temperature parameter in MENTS [69], RENTS and TENTS [17] is now given. Results on the 8x8 Frozen Lake environment (Figure B.1a) are given using a variety of temperatures to demonstrate how each algorithm performs with different temperatures in that domain.

TODO: clean up hyperparam setting

In Figures 4.12 and 4.14 MENTS and TENTS take the scenic route to the goal state for medium temperatures, where the reward for reaching the goal is still significant, but they can obtain more entropy reward by wondering around the gridworld for a while first. For higher temperatures they completely ignore the goal state, opting to rather maximise policy entropy. Interestingly, RENTS in Figure 4.13 fared better than MENTS and TENTS and never really ignored the goal state for all of the temperatures considered.

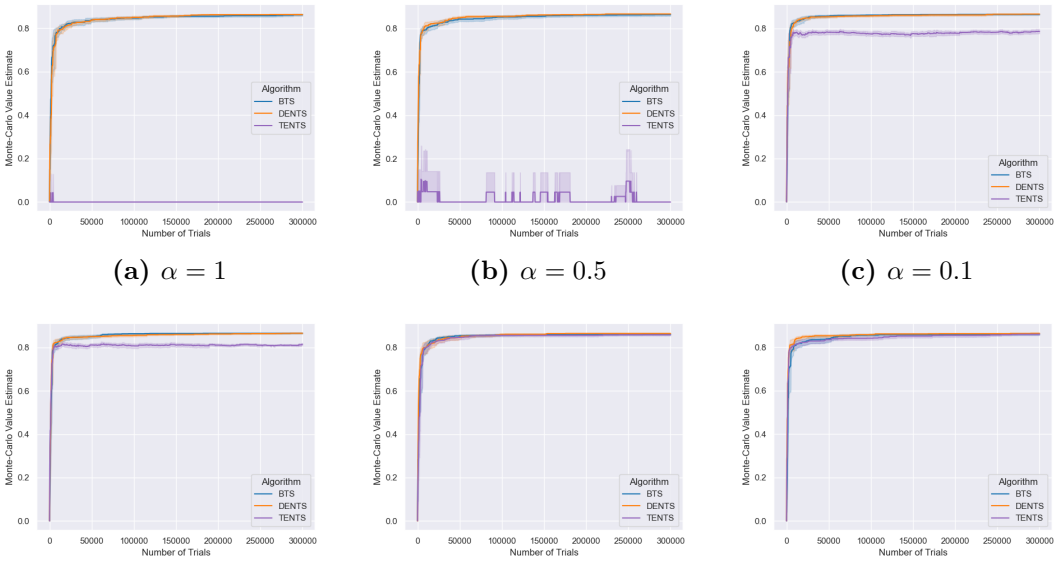
In contrast, both BTS and DENTS were relativly agnostic to the temperature parameter in this environment (with $\epsilon = 1$) and were always able to find the goal state. Curves for BTS and DENTS are included in all of Figures 4.12, 4.14 and 4.13 for reference and as a comparison for MENTS, RENTS and TENTS.



(a) $\alpha = 1$ **(b)** $\alpha = 0.5$ **(c)** $\alpha = 0.1$

(d) $\alpha = 0.05$ **(e)** $\alpha = 0.01$ **(f)** $\alpha = 0.005$

Figure 4.13: RENTS with a variety of temperatures on an 8x8 Frozen Lake environment. BTS and DENTS are included for reference. TODO: Update fig?



(a) $\alpha = 1$ **(b)** $\alpha = 0.5$ **(c)** $\alpha = 0.1$

(d) $\alpha = 0.05$ **(e)** $\alpha = 0.01$ **(f)** $\alpha = 0.005$

Figure 4.14: TENTS with a variety of temperatures on an 8x8 Frozen Lake environment. BTS and DENTS are included for reference. TODO: Update fig?

4.4.3.2 Gridworld Results

TODO: this subsubsection currently just a c and p from neurips

The 8x12 Frozen Lake environment (Figure B.1b) and a 6x6 Sailing Problem (Figure **??**) were used to evaluate the algorithms introduced in this chapter against the baseline algorithms.

Each algorithm is run 25 times on each environment and evaluated every 250 trials using 250 trajectories. A horizon of 100 was used for Frozen Lake and 50 for the Sailing Problem. Curves of these evaluations are given in Figure 4.15.

In Frozen Lake (Figure 4.15a), entropy proved to be a useful exploration bonus for the *sparse reward*. Values in UCT and BTS remain at zero until a trial successfully reaches the goal. However, entropy guides agents to avoid trap states, where the entropy is zero. DENTS was able to perform similarly to MENTS, and BTS was able to improve its policy over time more than UCT.

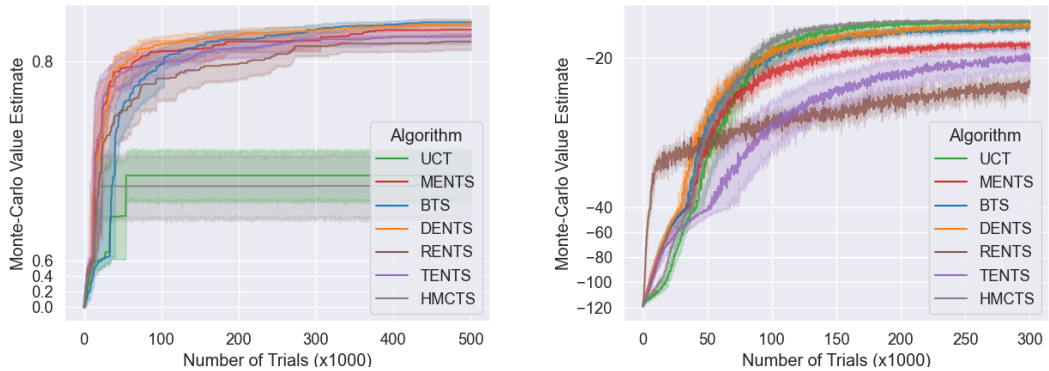
In the Sailing Problem (Figure 4.15b) UCT performs well due to the dense reward. TODO: also mention how exploiting is useful to get better value estimates in stochastic environments. BTS and DENTS also manage to keep up with UCT. MENTS and TENTS appear to be slightly hindered by entropy in this environment. The relative entropy encourages RENTS to pick the same actions over time, so it tends to pick a direction and stick with it regardless of cost.

Finally, BTS and DENTS were able to perform well in both domains with a sparse and dense reward structure, whereas the existing methods performed better on one than the other, hence making BTS and DENTS good candidates for a general purpose MCTS algorithm.

TODO: Say here that the initial wind directions for Sailing we set to North, and South-East.

4.4.3.3 Go Results

Results of the round-robin are summarised in Table 4.2, where each algorithm played 50 games as black and 50 as white. BTS was able to run the most trials per move and beat all of the other algorithms other than DENTS which it drew. The



(a) 8x12 Frozen Lake. **(b)** 6x6 Sailing Problem.

Figure 4.15: Results for gridworld environments. TODO: update fig? TODO: add more caption?

Alias method allowed the Boltzmann search algorithms to run significantly more trials per move than PUCT. BTS and DENTS were able to beat PUCT with results of 57-43 and 58-42 respectively. Using entropy did not seem to have much benefit in these experiments, as can be witnessed by MENTS only beating TENTS, and DENTS drawing 50-50 with BTS. This is likely because the additional exploration provided by entropy is vastly outweighed by utilising the information contained in the neural networks $\tilde{V}$ and $\tilde{\pi}$. Interestingly RENTS had the best performance out of the prior works, losing 43-57 to PUCT, and the use of relative entropy appears to take advantage of a heuristic for Go that the RAVE [22] algorithm used: the value of a move is typically unaffected by other moves on the board.

To validate the strength of the `THTS++` PUCT agent, it was played directly against KataGo [67], with a budget of 1600 trials per move. Interestingly, the `THTS++` PUCT agent won 61-39 in 9x9 Go, and lost 35-65 in 19x19 Go, suggesting that our PUCT agent is strong enough to provide a meaningful comparison for our other general purpose algorithms. Finally, note that the neural networks from KataGo were not fine-tuned, so the Boltzmann search algorithms directly used the networks that were trained for use in PUCT (with average returns).

Black \White	PUCT	AR-M	AR-R	AR-T	AR-B	AR-D	Trials/move
PUCT	-	33-17	27-23	42-8	17-33	15-35	1054
AR-MENTS	12-48	-	13-37	38-12	10-40	12-38	4851
AR-RENTS	20-30	24-26	-	39-11	18-32	14-36	3672
AR-TENTS	8-42	11-39	9-41	-	6-44	10-40	5206
AR-BTS	25-25	35-15	31-19	34-16	-	15-35	5375
AR-DENTS	23-27	36-14	29-21	36-14	15-35	-	4677

Table 4.2: Results for the Go round-robin tournament. The first column gives the agent playing as black. The final column gives the average trials run per move across the entire round-robin. In the top row, we abbreviate the algorithm names for space.

4.5 Theoretical Results

This section provides the theoretical analysis and proofs for this chapter. It is structured as follows:

1. First, Section 4.5.1 defines additional notation so that *MCTS processes* can be reasoned about with respect to the number of trials run;

2. Second, Section 4.5.2 establishes preliminary results, lemmas and definitions used in subsequent proofs;

3. Third, Section 4.5.3 proves the consistency results for (AR-)BTS and (AR-)DENTS from Section 4.2 (Theorems 4.2.1, 4.2.2, 4.2.4, 4.2.5, 4.2.6, 4.2.8);

4. Finally, Section 4.5.4 proves Theorem 4.3.1, showing that the maximum entropy objective is either misaligned or requires an exponential number of trials in the Entropy Trap problem.

Some of the particularly verbose arguments are left to Appendix C TODO: double check this ref so that they do not interrupt the flow of the main arguments. In particular, Theorems 4.2.3 and 4.2.7, which show exponential simple regret bounds on BTS and DENTS, are left to Appendix C.2 TODO: double check this ref.

The following assumptions are made for this section, which do not affect the asymptotic behaviour of the algorithms but simplify the proofs and presentation:

- the unique state assumption (Section 2.4.1) remains, `single_node_expansion` is set to false and `transposition_table` is set to false;

- heuristic values are set to zero, $\hat{V}_{\text{heuristic}}(s) = 0$ and $\hat{Q}_{\text{heuristic}}(s, a) = 0$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}$;

- the length of all trajectories and the planning horizon $H_{\texttt{THTS++}}$ are equal to the horizon H of the MDP;

- the pseudo-visit biasing terms $N_{\text{BTS}}^{\hat{V}_{\text{heuristic}}}$, $N_{\text{BTS}}^{\hat{Q}_{\text{heuristic}}}$, $N_{\text{DNTS}}^{\hat{V}_{\text{heuristic}}}$, $N_{\text{DNTS}}^{\hat{Q}_{\text{heuristic}}}$, $N_{\text{AR-BTS}}^{\hat{V}_{\text{heuristic}}}$, $N_{\text{AR-BTS}}^{\hat{Q}_{\text{heuristic}}}$, $N_{\text{AR-DNTS}}^{\hat{V}_{\text{heuristic}}}$, $N_{\text{AR-DNTS}}^{\hat{Q}_{\text{heuristic}}}$, from Section 4.2 are all set to zero.

Finally, note that mathematically (AR-)BTS is a special case of (AR-)DENTS, with $\beta_{\text{DNTS}}(x) = 0$ and $\beta_{\text{AR-DNTS}}(x) = 0$. Results will be proven about (AR-)DENTS, and the corresponding results for BTS follow as a direct consequence.

4.5.1 Notation

Because of the unique state assumption (Section 2.4.1), any state $s_t \in \tau = (s_0, a_0, r_0, \ldots, s_{H-1}, a_{H-1}, r_{H-1}, s_H)$ may only appear at timestep t in any trajectory. Throughout this chapter, s_t and a_t are used to denote an arbitrary state and action at an arbitrary timestep t.

When making arguments that apply to multiple algorithms, the general policies π and ψ are used. When making arguments about specific algorithms, subscripts are used, e.g. π_{BTS}.

To reason about MCTS processes over multiple trials, the existing notation is extended with superscripts to track both the number of trials run and the number of times each node has been visited.

Most variables are denoted with a superscript n to denote that it is the value used on the nth trial of MCTS. For example:

- If an algorithm uses a search policy π and outputs a recommendation policy ψ, then the search policy followed on the nth trial is denoted π^n, and the recommendation policy after n trials is denoted ψ^n.

- When it is necessary to reason about multiple trajectories, the superscript notation will also be used. The trajectory sampled in the nth trial is $\tau^n =$

$(s_0^n, a_0^n, r_0^n, \ldots, s_{H-1}^n, a_{H-1}^n, r_{H-1}^n, s_H^n)$. If it is sampled from a search policy π^n, then $\tau^n \sim \pi^n$.

- The search tree after n trials is denoted $\mathcal{T}^n$, the initial search tree is $\mathcal{T}^0 = \{s_0\}$, and $\mathcal{T}^n = \mathcal{T}^{n-1} \cup \{s \in \tau^n\} \cup \{(s,a) \in \tau^n\}$.

Let $T^n(s_t)$ and $T^n(s_t, a_t)$ denote the sets of trial indices on which s_t was visited and action a_t was selected respectively, in the first n trials, where t is the timestep at which the node appears in the trajectory:

$$T^n(s_t) = \{i | 1 \leq i \leq n, s_t^i = s_t\} \tag{4.37}$$

$$T^n(s_t, a_t) = \{i | 1 \leq i \leq n, s_t^i = s_t, a_t^i = a_t\}. \tag{4.38}$$

The visit counts after n trials are defined as follows, with $s_t \in \mathcal{S}$ and $a_t \in \mathcal{A}$:

$$N^n(s_t) = \sum_{i=1}^{n} \mathbb{1}[s_t^i = s_t] = |T^n(s_t)|, \tag{4.39}$$

$$N^n(s_t, a_t) = \sum_{i=1}^{n} \mathbb{1}[s_t^i = s_t \cap a_t^i = a_t] = |T^n(s_t, a_t)|. \tag{4.40}$$

Under the unique state assumption, the visit counts can also be expressed in the following equivalent forms, which are more convenient for the proofs:

$$N^n(s_t, a_t) = \sum_{i \in T^n(s_t)} \mathbb{1}[a_t^i = a_t], \tag{4.41}$$

$$N^n(s_{t+1}) = \sum_{i \in T^n(s_t, a_t)} \mathbb{1}[s_{t+1}^i = s_{t+1}] = |T^n(s_t, a_t)|. \tag{4.42}$$

The key observation is that $N^n(s_t, a_t)$ can be computed as a sum over the $N^n(s_t)$ trials on which s_t was visited, and $N^n(s_{t+1})$ as a sum over the $N^n(s_t, a_t)$ trials on which a_t was selected from s_t.

Value estimates are the one exception to the superscript notation to denote the number of trials. Superscripts for value estimates denote the number of times that the value has been backed up. To differentiate this counting with respect to backups, instead of with respect to trials, a superscript with brackets will be used. For example, the backups for DENTS value estimates would be written as follows. Let $\tau^n \sim \pi^n_{\text{DNTS}}$ be a trajectory sampled from the DENTS search

policy on the nth trial. Let $\tau^n = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h)$. The backups for $t = h-1, \ldots, 0$ are then:

$$\hat{Q}_{\text{DNTS}}^{(N^n(s_t,a_t))}(s_t, a_t) = R(s_t, a_t) + \sum_{s' \in \text{Succ}(s_t,a_t)} \left(\frac{N^n(s')}{N^n(s_t, a_t)} \hat{V}_{\text{DNTS}}^{(N^n(s'))}(s') \right), \tag{4.43}$$

$$\hat{V}_{\text{DNTS}}^{(N^n(s_t))}(s_t) = \max_{a \in \mathcal{A}} \hat{Q}_{\text{DNTS}}^{(N^n(s_t,a))}(s_t, a), \tag{4.44}$$

$$\bar{\mathcal{H}}_{Q,\text{DNTS}}^{(N^n(s_t,a_t))}(s_t, a_t) = \sum_{s' \in \text{Succ}(s_t,a_t)} \frac{N^n(s')}{N^n(s_t, a_t)} \bar{\mathcal{H}}_{V,\text{DNTS}}^{(N^n(s'))}(s'), \tag{4.45}$$

$$\bar{\mathcal{H}}_{V,\text{DNTS}}^{(N^n(s_t))}(s_t) = \mathcal{H}(\pi_{\text{DNTS}}^n(\cdot|s_t)) + \sum_{a \in \mathcal{A}} \pi_{\text{DNTS}}^n(a_t|s_t) \bar{\mathcal{H}}_{Q,\text{DNTS}}^{(N^n(s_t,a_t))}(s_t, a_t). \tag{4.46}$$

For reference, all of the algorithms considered in this Section are written out in full with this additional notation in Appendix **??** TODO: double check this ref. TODO: make sure handle initv and initq properly in the appendix

4.5.2 Preliminaries

This subsection introduces definitions and preliminary lemmas that are used in the proofs of Sections 4.5.3 and 4.5.4.

The proofs in this section make use of the *immediate simple regret*, which measures the regret incurred by taking a single action step rather than following the full policy. Working with immediate simple regret simplifies the proofs, since bounds and convergence results transfer between the two notions via Lemma 4.5.1.

Definition 4.5.1. *The* immediate simple regret *is defined as:*

$$\texttt{imm_regr}(s_t, \psi^n) = V^*(s_t) - \mathbb{E}_{a_t \sim \psi^n(s_t)}[Q^*(s_t, a_t)]. \tag{4.47}$$

The following lemma shows that asymptotic bounds and convergence results transfer between immediate and simple regret, so it suffices to prove results about immediate simple regret.

Lemma 4.5.1. $\mathbb{E}[\texttt{sim_regr}(s_t, \psi^n)] = O(f(n))$ *for all* $s_t \in \mathcal{S}_0$ *if and only if* $\mathbb{E}[\texttt{imm_regr}(s_t, \psi^n)] = O(f(n))$ *for all* $s_t \in \mathcal{S}_0$. *Consequently,* $\mathbb{E}[\texttt{sim_regr}(s_t, \psi^n)] \to 0$ *if and only if* $\mathbb{E}[\texttt{imm_regr}(s_t, \psi^n)] \to 0$.

Proof. Firstly, the simple regret can be written recursively in terms of the immediate simple regret:

$$\begin{aligned}
\texttt{sim_regr}(s_t, \psi^n) =& V^*(s_t) - V^{\psi^n}(s_t) && (4.48)\\
=& V^*(s_t) - \mathbb{E}_{a_t \sim \psi^n(s_t)}[Q^{\psi^n}(s_t, a_t)] && (4.49)\\
=& V^*(s_t) - \mathbb{E}_{a_t \sim \psi^n(s_t)}[R(s_t, a_t) + \mathbb{E}_{s_{t+1} \sim \Pr(\cdot|s_t,a_t)}[V^{\psi^n}(s_{t+1})]] && (4.50)\\
=& V^*(s_t) - \mathbb{E}_{a_t \sim \psi^n(s_t)}\Big[Q^*(s_t, a_t) - \mathbb{E}_{s_{t+1} \sim \Pr(\cdot|s_t,a_t)}[V^*(s_{t+1})]\\
&+ \mathbb{E}_{s_{t+1} \sim \Pr(\cdot|s_t,a_t)}[V^{\psi^n}(s_{t+1})]\Big] && (4.51)\\
=& V^*(s_t) - \mathbb{E}_{a_t \sim \psi^n(s_t)}[Q^*(s_t, a_t)]\\
&+ \mathbb{E}_{a_t \sim \psi^n(s_t), s_{t+1} \sim \Pr(\cdot|s_t,a_t)}[V^*(s_{t+1}) - V^{\psi^n}(s_{t+1})] && (4.52)\\
=& \texttt{imm_regr}(s_t, \psi^n)\\
&+ \mathbb{E}_{a_t \sim \psi^n(s_t), s_{t+1} \sim \Pr(\cdot|s_t,a_t)}[\texttt{sim_regr}(s_{t+1}, \psi^n)], && (4.53)
\end{aligned}$$

where line (4.51) uses $R(s_t, a_t) = Q^*(s_t, a_t) - \mathbb{E}_{s_{t+1} \sim \Pr(\cdot|s_t,a_t)}[V^*(s_{t+1})]$, a rearrangement of the Bellman optimality equation for $Q^*(s_t, a_t)$.

Immediately, this shows that $\mathbb{E}[\texttt{imm_regr}(s_t, \psi^n)] \leq \mathbb{E}[\texttt{sim_regr}(s_t, \psi^n)]$ and hence if $\mathbb{E}[\texttt{sim_regr}(s_t, \psi^n)] = O(f(n))$ then $\mathbb{E}[\texttt{imm_regr}(s_t, \psi^n)] = O(f(n))$.

If $\mathbb{E}[\texttt{imm_regr}(s_t, \psi^n)] = O(f(n))$ then by induction over $t = H, \ldots, 0$, using the relation in Equation (4.53):

$$\begin{aligned}
\mathbb{E}[\texttt{sim_regr}(s_t, \psi^n)] &= \mathbb{E}\Big[O(f(n)) + \mathbb{E}_{a_t \sim \psi^n(s_t), s_{t+1} \sim \Pr(\cdot|s_t,a_t)}[O(f(n))]\Big] && (4.54)\\
&= O(f(n)). && (4.55)
\end{aligned}$$

The base case is $t = H$: since s_H is a terminal state $\texttt{sim_regr}(s_H, \psi^n) = \texttt{imm_regr}(s_H, \psi^n) = 0$. □

The DKW inequality (Theorem 2.5.1) is used to show that the empirical transition probabilities converge in probability to the true transition probabilities.

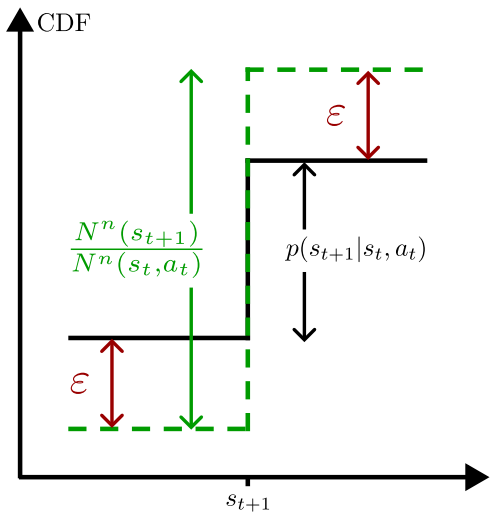


Figure 4.16: Bounding the empirical transition probabilities to the true transition probabilities. The true cdf is shown as a solid black line, the empirical cdf is shown as a dashed green line, and a worst case error of $\varepsilon/2$, using Theorem 2.5.1, is shown in red. The probability mass of $p(s_{t+1}|s_t, a_t)$ and empirical probability mass of $\frac{N^n(s_{t+1})}{N^n(s_t,a_t)}$ is also indicated to demonstrate how the constructed distribution gives Corollary C.2.1.

Corollary 4.5.1. *For all $(s_t, a_t) \in \mathcal{S} \times \mathcal{A}$ and for all $\varepsilon > 0$:*

$$\Pr\left(\max_{s_{t+1}\in \mathrm{Succ}(s_t,a_t)} \left|\frac{N^n(s_{t+1})}{N^n(s_t,a_t)} - p(s_{t+1}|s_t,a_t)\right| > 2\varepsilon\right) \leq 2\exp\left(-2\varepsilon^2 N^n(s_t,a_t)\right), \tag{4.56}$$

as $N^n(s_t, a_t) \to \infty$. Consequently $\frac{N^n(s_{t+1})}{N^n(s_t,a_t)} \xrightarrow{p} p(s_{t+1}|s_t, a_t)$ as $N^n(s_t, a_t) \to \infty$.

Proof outline. Consider the cumulative distribution function of the successor state distribution $p(s_{t+1}|s_t, a_t)$, using some arbitrary ordering over the successor states in $\mathrm{Succ}(s_t, a_t)$. The result follows by applying Theorem 2.5.1 with $m = N^n(s_t, a_t)$.

To see why the bound involves 2ε rather than ε, consider Figure C.1. Applying the DKW inequality bounds the error in the cumulative distribution function by ε at every point. Since the probability mass of any single successor state corresponds to the difference between two CDF values, the errors at either side may accumulate, giving a worst-case error of 2ε in the empirical probability of any individual successor state. □

4.5.3 Consistency Results

This subsection proves the consistency results for (AR-)DENTS. The proof of Theorem 4.2.8 is given in full, as it uses all of the preliminary lemmas. The proofs

of the remaining theorems are given as outlines, highlighting only where they differ.

Theorems 4.2.1, 4.2.2 and 4.2.4 for (AR-)BTS follow directly from Theorems 4.2.5, 4.2.6 and 4.2.8 respectively, by setting $\beta_{\text{DNTS}}(x) = 0$ and $\beta_{\text{AR-DNTS}}(x) = 0$.

Theorem 4.2.8. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{AR-DNTS}} > 0$, $\alpha_{\text{AR-DNTS}}(x) \to 0$ and $\beta_{\text{AR-DNTS}}(x) \to 0$, then $\pi_{\text{AR-DNTS}} \xrightarrow{p} \pi^*$, $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{AR-DNTS}})] \to 0$ and $\mathbb{E}[\texttt{sim_regr}(s_0, \texttt{mv}_{\text{AR-DNTS}})] \to 0$.*

Proof. Firstly by Corollary 4.5.2 (of Lemma 4.5.2), with the assumption that $\epsilon_{\text{AR-DNTS}} > 0$, all visit counts $N^n(s_t)$ and $N^n(s_t, a_t)$ will tend to infinity almost surely as $n \to \infty$.

Then the (Q-)value estimates and search policy will converge almost surely to their optimal counterparts. This follows by induction on the timestep t. For the base case $t = H$ at the planning horizon, $\hat{V}^{(N^n(s_H))}_{\text{AR-DNTS}}(s_H) = V^*(s_H) = 0$ by definition. The inductive steps proceed as follows, for each $s_t \in \mathcal{S}_0$ and $a_t \in \mathcal{A}$:

1. Assume that $\hat{V}^{(N^n(s_{t+1}))}_{\text{AR-DNTS}}(s_{t+1}) \xrightarrow{p} V^*(s_{t+1})$;

2. By Lemma 4.5.3, it follows that $\hat{Q}^{(N^n(s_t,a_t))}_{\text{AR-DNTS}}(s_t, a_t) \xrightarrow{p} Q^*(s_t, a_t)$ and consequently $\psi^n_{\text{AR-DNTS}}(a_t|s_t) \xrightarrow{p} \pi^*(a_t|s_t)$;

3. Letting $\dot{Q}^n(s_t, a_t) = \hat{Q}^{(N^n(s_t,a_t))}_{\text{AR-DNTS}}(s_t, a_t) + \beta_{\text{AR-DNTS}}(N^n(s_t))\bar{\mathcal{H}}^{(N^n(s_t,a_t))}_{Q,\text{AR-DNTS}}(s_t, a_t)$, the assumption that $\beta_{\text{AR-DNTS}}(x) \to 0$ ensures that $\dot{Q}^n(s_t, a_t) \xrightarrow{p} Q^*(s_t, a_t)$. Then using Lemma 4.5.4, with the assumption that $\alpha_{\text{AR-DNTS}}(x) \to 0$, it follows that $\pi^n_{\text{AR-DNTS}}(a_t|s_t) \xrightarrow{p} \pi^*(a_t|s_t)$;

4. Then by Lemma 4.5.5 it follows that $\frac{N^n(s_t,a_t)}{N^n(s_t)} \xrightarrow{p} \pi^*(a_t|s_t)$ and consequently $\texttt{mv}^n_{\text{AR-DNTS}}(a_t|s_t) \xrightarrow{p} \pi^*(a_t|s_t)$;

5. By Lemma 4.5.7, $\hat{V}^{(N^n(s_t))}_{\text{AR-DNTS}}(s_t) \xrightarrow{p} V^*(s_t)$.

The immediate regret (Definition 4.5.1) then satisfies:

$$\mathbb{E}\left[\texttt{imm_regr}(s_t, \psi^n_{\text{AR-DNTS}})\right] = \sum_{a \in \mathcal{A}} \left(V^*(s_t) - Q^*(s_t, a)\right) \psi^n_{\text{AR-DNTS}}(a|s_t) \tag{4.57}$$

$$\xrightarrow{p} \sum_{a \in \mathcal{A}} \left(V^*(s_t) - Q^*(s_t, a)\right) \pi^*(a|s_t) \tag{4.58}$$

$$= \mathbb{E}\left[\texttt{imm_regr}(s_t, \pi^*)\right] \tag{4.59}$$

$$= 0, \tag{4.60}$$

and similarly

$$\mathbb{E}\left[\texttt{imm_regr}(s_t, \texttt{mv}^n_{\text{AR-DNTS}})\right] \to 0. \tag{4.61}$$

The result follows by Lemma 4.5.1 to lift the convergence of the immediate regret to convergence of the simple regret. □

The proof of Theorem 4.2.5 proceeds similarly, but since only the recommendation policy ψ_{DNTS} is required to converge, steps 3 and 4 are unnecessary.

Theorem 4.2.5. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{DNTS}} > 0$, the expected simple regret of ψ_{DNTS} tends to zero: $\mathbb{E}[\texttt{sim_regr}(s_0, \psi_{\text{DNTS}})] \to 0$.*

Proof outline. Proof proceeds similarly to the proof of Theorem 4.2.8, where the induction steps 3 and 4 are unnecessary, and hence the conditions $\alpha_{\text{DNTS}}(x) \to 0$ and $\beta_{\text{DNTS}}(x) \to 0$ are not required. Induction step 5 uses Lemma 4.5.6 instead of Lemma 4.5.7. □

The proof of Theorem 4.2.6 is nearly identical to that of Theorem 4.2.8, the only difference being that Lemma 4.5.6 is used in place of Lemma 4.5.7.

Theorem 4.2.6. *For any MDP $\mathcal{M}$, if $\epsilon_{\text{DNTS}} > 0$, $\alpha_{\text{DNTS}}(x) \to 0$ and $\beta_{\text{DNTS}}(x) \to 0$, then $\pi_{\text{DNTS}} \xrightarrow{p} \pi^*$ and $\mathbb{E}[\texttt{sim_regr}(s_0, \texttt{mv}_{\text{DNTS}})] \to 0$.*

Proof outline. Proof proceeds similarly to the proof of Theorem 4.2.8. □

The following lemma considers a bandit setting at a single decision node, and shows that the logarithmically decaying exploration term ensures every action is selected infinitely often almost surely.

Lemma 4.5.2. *Let ρ^m be an arbitrary policy and consider the following search policy used on the mth round of a bandit problem:*

$$\pi^m(a|s) = (1 - \lambda(m))\rho^m(a) + \lambda(m)\frac{1}{|\mathcal{A}|}, \tag{4.62}$$

$$\lambda(m) = \min(1, \epsilon/\log(e + m)), \tag{4.63}$$

where $\epsilon > 0$, $s \in \mathcal{S}$ and $a \in \mathcal{A}$. Let $a^m \sim \pi^m$ be the mth independently sampled action, and let $M^m(a)$ be the number of times action a was selected in the first m rounds, i.e. $M^m(a) = \sum_{i=1}^m \mathbb{1}[a^i = a]$. Then $M^m(a) \overset{as}{\to} \infty$ as $m \to \infty$ for all $a \in \mathcal{A}$.

Proof. Suppose that the result does not hold and there exists some $b \in \mathcal{A}$ such that:

$$\Pr\left(M^m(b) \to \infty\right) < 1. \tag{4.64}$$

If $M^m(b) \not\to \infty$, then it is logically equivalent to say that there exists some round $\ell \in \mathbb{N}$ from which b is never sampled again (with $\ell \geq \exp(\epsilon + 1)$). That is:

$$\Pr\left(\bigcap_{m=\ell}^{\infty} a^m \neq b\right) = \Pr\left(M^m(b) \not\to \infty\right) > 0. \tag{4.65}$$

However, from Equation (4.62) action b is sampled with probability at least:

$$\Pr\left(a^m = b\right) \geq \frac{\lambda(m)}{|\mathcal{A}|} = \frac{\epsilon}{|\mathcal{A}|\log(e + m)}. \tag{4.66}$$

So the probability of never sampling action b again after the first $\ell - 1$ rounds is at most:

$$\Pr\left(\bigcap_{m=\ell}^{\infty} a^m \neq b\right) = \lim_{k\to\infty} \prod_{m=\ell}^{k} \Pr(a^m \neq b) \tag{4.67}$$

$$\leq \lim_{k\to\infty} \prod_{m=\ell}^{k} \left(1 - \frac{\epsilon}{|\mathcal{A}| \log(e+k)}\right) \tag{4.68}$$

$$\leq \lim_{k\to\infty} \left(1 - \frac{\epsilon}{|\mathcal{A}| \log(e+k)}\right)^{k-\ell} \tag{4.69}$$

$$= 0, \tag{4.70}$$

which is in contradiction to inequality (4.65) that was assumed.

Inequality (4.69) holds because the factors in the product are increasing with respect to m. To see why the final equality (4.70) holds, consider the simpler function $f(x) = \left(1 - \frac{1}{\log(x)}\right)^x$. Taking logarithms and applying L'Hôpital's rule multiple times gives $\lim_{x\to\infty} \log f(x) = -\infty$, so $\lim_{x\to\infty} f(x) = 0$.

□

As a consequence of Lemma 4.5.2, every reachable state (and state-action pair) will be visited infinitely often in any MCTS process using a search policy of the form given in Equation (4.62).

The following corollary lifts Lemma 4.5.2 to the full tree search setting, where the search policy is indexed by the number of trials n, but the lemma is applied at each node with respect to its visit count $m = N^n(s_t)$.

Corollary 4.5.2. *For all $s_t \in \mathcal{S}_0$ and $a_t \in \mathcal{A}$, any MCTS process using a search policy of the following form on the nth trial:*

$$\pi^n(a_t|s_t) = (1 - \lambda(s_t, \epsilon))\rho^n(a_t|s_t) + \lambda(s_t, \epsilon)\frac{1}{|\mathcal{A}|}, \tag{4.71}$$

$$\lambda(s_t, \epsilon) = \min(1, \epsilon/\log(e + N^n(s_t))), \tag{4.72}$$

where ρ^n is an arbitrary policy and $\epsilon > 0$. All reachable states and state-action pairs will be visited infinitely often. That is $N^n(s_t) \xrightarrow{as} \infty$ and $N^n(s_t, a_t) \xrightarrow{as} \infty$ as $n \to \infty$.

Proof outline. The result follows by induction over Lemma 4.5.2. At $t = 0$ the result holds directly from the lemma with $m = n$. For the inductive step, if $N^n(s_t, a_t) \xrightarrow{as} \infty$ then $N^n(s_{t+1}) \xrightarrow{as} \infty$ since s_{t+1} is sampled from $p(\cdot|s_t, a_t)$ with positive probability, then $N^n(s_{t+1}, a_{t+1}) \xrightarrow{as} \infty$ by applying Lemma 4.5.2 with $m = N^n(s_{t+1})$. □

The following lemma shows that if the value estimates converge to the optimal values, then the Q-value estimates computed via a sample-weighted average of child node values also converge to the optimal Q-values.

Lemma 4.5.3. *For all $s_t \in \mathcal{S}_0$, $a_t \in \mathcal{A}$, if $N^n(s_t, a_t) \to \infty$ and $\dot{V}^{(N^n(s'))}(s') \xrightarrow{p} V^*(s')$ for all $s' \in \mathrm{Succ}(s_t, a_t)$, then the Q-values computed by:*

$$\dot{Q}^{(N^n(s_t,a_t))}(s_t, a_t) = R(s_t, a_t) + \sum_{s' \in \mathrm{Succ}(s_t,a_t)} \frac{N^n(s')}{N^n(s_t, a_t)} \dot{V}^{(N^n(s'))}(s'), \tag{4.73}$$

converge to the optimal Q-values: $\dot{Q}^{(N^n(s_t,a_t))}(s_t, a_t) \xrightarrow{p} Q^*(s_t, a_t)$.

Proof outline. By Corollary C.2.1, $\frac{N^n(s')}{N^n(s_t,a_t)} \xrightarrow{p} p(s'|s_t, a_t)$ as $N^n(s_t, a_t) \to \infty$. Then:

$$\dot{Q}^{(N^n(s_t,a_t))}(s_t, a_t) = R(s_t, a_t) + \sum_{s' \in \mathrm{Succ}(s_t,a_t)} \frac{N^n(s')}{N^n(s_t, a_t)} \dot{V}^{(N^n(s'))}(s') \tag{4.74}$$

$$\xrightarrow{p} R(s_t, a_t) + \sum_{s' \in \mathrm{Succ}(s_t,a_t)} p(s'|s_t, a_t) V^*(s') \tag{4.75}$$

$$= Q^*(s_t, a_t). \tag{4.76}$$

□

For Boltzmann search policies, if the Q-value estimates converge to the optimal Q-values and the temperature tends to zero, then the search policy converges to the optimal policy.

Lemma 4.5.4. *If a search policy of the following form is used in MCTS:*

$$\pi^n(a_t|s_t) = (1 - \lambda(s_t, \epsilon))\rho^n(a_t|s_t) + \lambda(s_t, \epsilon)\frac{1}{|\mathcal{A}|}, \tag{4.77}$$

$$\rho^n \propto \exp\left(\frac{\dot{Q}^n(s_t, a_t)}{\alpha(N^n(s_t))}\right), \tag{4.78}$$

$$\lambda(s_t, \epsilon) = \min(1, \epsilon/\log(e + N^n(s_t))), \tag{4.79}$$

where $\epsilon > 0$, $\alpha(x) \to 0$ *and* $\dot{Q}^n(s_t, a_t) \xrightarrow{p} Q^*(s_t, a_t)$ *as* $n \to \infty$ *for all* $s_t \in \mathcal{S}_0$ *and* $a_t \in \mathcal{A}$. *Then* $\pi^n(a_t|s_t) \xrightarrow{p} \pi^*(a_t|s_t)$ *as* $n \to \infty$.

Proof outline. From Corollary 4.5.2, $N^n(s_t) \xrightarrow{as} \infty$ as $n \to \infty$, and hence $\lambda(s_t, \epsilon) \xrightarrow{as} 0$, so $\pi^n(a_t|s_t) - \rho^n(a_t|s_t) \xrightarrow{as} 0$.

It is known that a Boltzmann distribution converges to the one-hot distribution over the argmax as the temperature tends to zero: if $\alpha(N^n(s_t)) \to 0$ and $\dot{Q}^n(s_t, a_t) \xrightarrow{p} Q^*(s_t, a_t)$, then $\rho^n(a_t|s_t) \xrightarrow{p} \mathbb{1}[a_t = \arg\max_{a\in\mathcal{A}} Q^*(s_t, a)] = \pi^*(a_t|s_t)$.

Combining these two results gives $\pi^n(a_t|s_t) \xrightarrow{p} \pi^*(a_t|s_t)$. □

The following lemma shows that if the search policy converges to a limiting policy under which one action is strictly preferred at every state, then the most-visited recommendation policy converges to that uniquely preferred action.

Lemma 4.5.5. *Suppose* $\pi^n \xrightarrow{as} \pi_{\text{lim}}$ *as* $n \to \infty$, *where* π_{lim} *is a policy maximised by some action* a_t^*. *That is,* $\pi_{\text{lim}}(a_t^*|s_t) > \pi_{\text{lim}}(a_t|s_t)$ *for all* $a_t \in \mathcal{A} - \{a_t^*\}$ *and* $s_t \in \mathcal{S}_0$. *Then the empirical action frequencies converge almost surely to the limiting policy:*

$$\frac{N^n(s_t, a_t)}{N^n(s_t)} \xrightarrow{as} \pi_{\text{lim}}(a_t|s_t). \tag{4.80}$$

Consequently, since π_{lim} *has a unique maximising action* $a_t^* = \arg\max_{a\in\mathcal{A}} \pi_{\text{lim}}(a|s_t)$ *at every state, the most-visited recommendation policy converges to always recommending* a_t^*.

Proof.

$$\frac{N^n(s_t, a_t)}{N^n(s_t)} = \sum_{i \in T^n(s_t)} \frac{\mathbb{1}[a_t^i = a_t]}{N^n(s_t)} \tag{4.81}$$

$$\xrightarrow{as} \lim_{n\to\infty} \sum_{i \in T^n(s_t)} \frac{\pi^i(a_t|s_t)}{N^n(s_t)} \tag{4.82}$$

$$= \pi_{\lim}(a_t|s_t), \tag{4.83}$$

where line (4.81) uses Equation (4.41). The almost sure convergence follows from Kolmogorov's criterion[6] [20]. The final equality uses Corollary 4.5.2 and Stolz-Cesàro's theorem: since $\pi^n(a_t|s_t) \xrightarrow{as} \pi_{\lim}(a_t|s_t)$ by assumption, the running average converges to $\pi_{\lim}(a_t|s_t)$.

Since $\frac{N^n(s_t,a_t)}{N^n(s_t)} \xrightarrow{as} \pi_{\lim}(a_t|s_t)$ and $\pi_{\lim}$ has a unique maximising action a_t^* at every state, it follows that $\arg\max_{a\in\mathcal{A}} N^n(s_t, a) \xrightarrow{as} a_t^*$, so the most-visited recommendation policy (e.g. Equation (4.24)) converges to always recommending a_t^*. □

The next two lemmas show that the state value estimates converge to the optimal values, for DENTS and AR-DENTS respectively, given that the Q-value estimates have already converged.

For DENTS, the state value is computed as the maximum over Q-values, so convergence follows directly.

Lemma 4.5.6. *If $N^n(s_t) \xrightarrow{as} \infty$ and $\hat{Q}_{\text{DNTS}}^{(N^n(s_t,a_t))}(s_t, a_t) \xrightarrow{p} Q^*(s_t, a_t)$ for all $s_t \in \mathcal{S}_0$ and $a_t \in \mathcal{A}$, then $\hat{V}_{\text{DNTS}}^{(N^n(s_t))}(s_t) \xrightarrow{p} V^*(s_t)$.*

Proof outline. From the definition of the DENTS state value backup (Equation (C.8)), it follows that:

$$\hat{V}_{\text{DNTS}}^{(N^n(s_t))}(s_t) = \max_{a\in\mathcal{A}} \hat{Q}_{\text{DNTS}}^{(N^n(s_t,a))}(s_t, a) \tag{4.84}$$

$$\xrightarrow{p} \max_{a\in\mathcal{A}} Q^*(s_t, a) \tag{4.85}$$

$$= V^*(s_t). \tag{4.86}$$

[6] A form of the Strong Law of Large Numbers for independent but *not* necessarily identically distributed random variables.

□

For AR-DENTS, the state value is computed as a visit-count weighted average of Q-values, so convergence additionally requires the search policy to converge.

Lemma 4.5.7. *If* $N^n(s_t) \xrightarrow{as} \infty$, ${\pi_{\text{AR-DNTS}}}^n(a_t|s_t) \xrightarrow{p} \pi^*(a_t|s_t)$ *and* $\hat{Q}^{(N^n(s_t,a_t))}_{\text{AR-DNTS}}(s_t, a_t) \xrightarrow{p} Q^*(s_t, a_t)$ *for all* $s_t \in \mathcal{S}_0$ *and* $a_t \in \mathcal{A}$*, then* $\hat{V}^{(N^n(s_t))}_{\text{AR-DNTS}}(s_t) \xrightarrow{p} V^*(s_t)$.

Proof outline. An equivalent equation, computed using child Q-values and visit counts, for the AR-DENTS state value backup is:

$$\hat{V}^{(N^n(s_t))}_{\text{AR-DNTS}}(s_t) = \sum_{a \in \mathcal{A}} \frac{N^n(s_t, a)}{N^n(s_t)} \hat{Q}^{(N^n(s_t,a))}_{\text{AR-DNTS}}(s_t, a). \tag{4.87}$$

By Lemma 4.5.5, $\frac{N^n(s_t,a)}{N^n(s_t)} \xrightarrow{p} \pi^*(a|s_t)$ as $N^n(s_t) \to \infty$. It follows that:

$$\hat{V}^{(N^n(s_t))}_{\text{AR-DNTS}}(s_t) \xrightarrow{p} \sum_{a \in \mathcal{A}} \pi^*(a|s_t) Q^*(s_t, a) \tag{4.88}$$

$$= V^*(s_t). \tag{4.89}$$

□

4.5.4 The Entropy Trap

This subsection provides the proof of Theorem 4.3.1, that informally says that for any temperature either the maximum entropy objective is misaligned, or MENTS requires an exponential number of trials to recommend the optimal policy. The core of the argument reduces to bounding the *hitting time* of the state $s^E_{\mathcal{M}}$ — the number of trials required for MENTS to first reach the end of the D-chain portion of the Entropy Trap MDP. For reference, the Entropy Trap MDP is repeated in Figure 4.17, with additional state labels added for the proof.

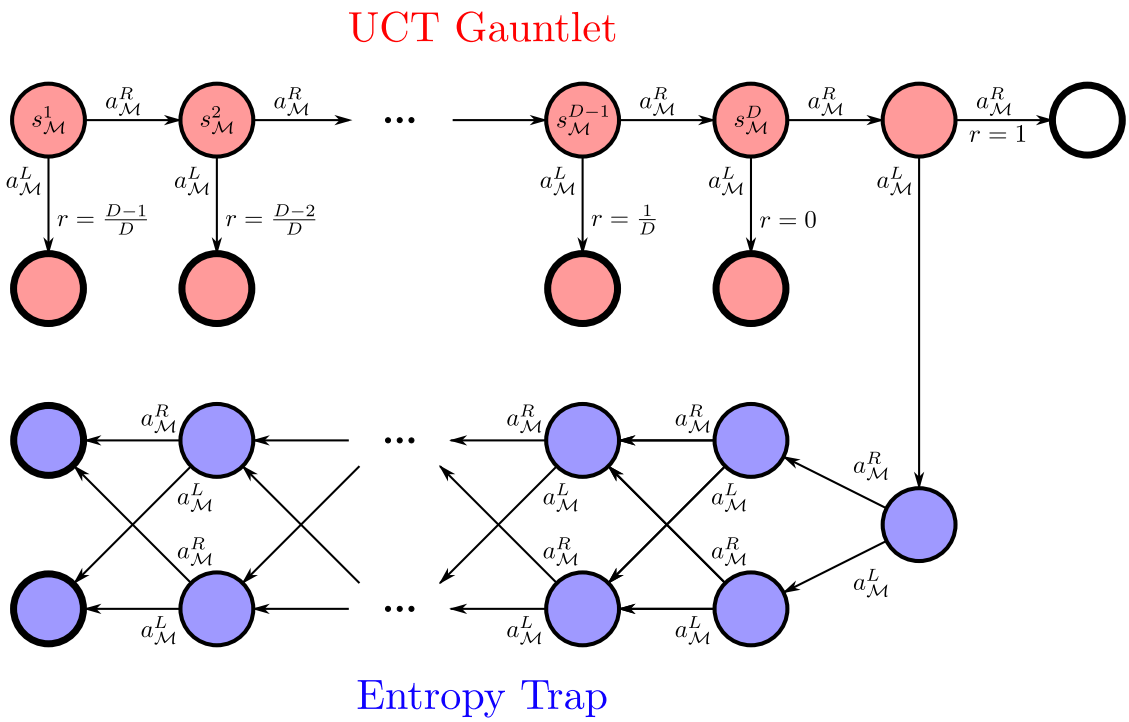


Figure 4.17: (Repeated). An illustration of the *Entropy Trap problem*, with state labels added for reference in the proof below. TODO: Update figure to include an additional state s to the E and s to the F

Theorem 4.3.1. *For any value of α_{MENTS}, either the maximum entropy objective is misaligned, or MENTS requires an exponential number of trials to recommend the optimal policy.*

More precisely, after n trials of MENTS: $\Pr(\psi_{\text{MENTS}} \neq \pi^*) > \frac{1}{2}$ *if* $n \leq 2^{D-1}$*; and either* $\mathbb{E}\left[\texttt{sim_regr}(s_0, \psi_{\text{MENTS}})\right] \not\to 0$ *or there exist constants* $c, k > 0$ *such that* $\mathbb{E}\left[\texttt{sim_regr}(s_0, \psi_{\text{MENTS}})\right] \geq c\left(1 - \frac{n}{k^{|\mathcal{S}|}}\right)$.

Proof outline. Proof is by construction. Consider the Entropy Trap MDP defined in Figure 4.17, which is parameterised by D the length of the D-chain, and K, half the number of states in the entropy trap. In this proof, the case where $K = D$ is considered.

Note that for clarity, the full superscript notation established in Section 4.5.1 is dropped in this proof outline to avoid confusion with the Entropy Trap state notation $s^i_{\mathcal{M}}$.

Two cases need to be considered: when the temperature is sufficiently high for

the maximum entropy objective to be misaligned; and, when the temperature is lower than this threshold.

Case 1: $\alpha_{\text{MENTS}} > \frac{1}{\log(2)D}$ **(Maximum entropy objective is misaligned)**

When $\alpha_{\text{MENTS}} > \frac{1}{\log(2)D}$, the optimal soft value of the state $s^F_{\mathcal{M}}$ is greater than one, and so action $a^L_{\mathcal{M}}$ is preferred over $a^R_{\mathcal{M}}$ at state $s^E_{\mathcal{M}}$. Let ϕ be a uniform policy. Then:

$$
\begin{aligned}
Q^*_{\text{sft}}(s^E_{\mathcal{M}}, a^L_{\mathcal{M}}) &= V^*_{\text{sft}}(s^F_{\mathcal{M}}) && (4.90)\\
&= 0 + \alpha_{\text{MENTS}} \cdot \mathcal{H}(\phi) && (4.91)\\
&= \alpha_{\text{MENTS}} \cdot \log(2)D && (4.92)\\
&> 1 && (4.93)\\
&= Q^*_{\text{sft}}(s^E_{\mathcal{M}}, a^R_{\mathcal{M}}). && (4.94)
\end{aligned}
$$

Hence the maximum entropy objective is misaligned: $\mathbb{E}\left[\texttt{sim_regr}(s^1_{\mathcal{M}}, \psi_{\text{MENTS}})\right] \to 1 > 0$.

Case 2: $\alpha_{\text{MENTS}} \leq \frac{1}{\log(2)D}$

In this case, the temperature is sufficiently low that the MENTS search policy is nearly greedy. Hence MENTS will have a small probability of ever having a trial hitting state $s^E_{\mathcal{M}}$. For now, assume that $\hat{V}_{\text{MENTS}}(s^E_{\mathcal{M}}) = 0$, and $N(s^E_{\mathcal{M}}) = 0$.

First consider the composed and simplified soft backup on the D-chain states for $1 \leq i \leq D$:

$$
\begin{aligned}
\hat{V}_{\text{MENTS}}(s^i_{\mathcal{M}}) &\leftarrow \alpha \log\left(\frac{1}{\alpha}\left(\frac{D-i}{D} + \hat{V}_{\text{MENTS}}(s^{i+1}_{\mathcal{M}})\right)\right) && (4.95)\\
&\leq \max\left(\frac{D-i}{D}, \hat{V}_{\text{MENTS}}(s^{i+1}_{\mathcal{M}})\right) + \alpha_{\text{MENTS}} \log(2) && (4.96)\\
&\leq \max\left(\frac{D-i}{D}, \hat{V}_{\text{MENTS}}(s^{i+1}_{\mathcal{M}})\right) + \frac{1}{D} && (4.97)
\end{aligned}
$$

where inequality (4.96) uses the property of log-sum-exp that $\alpha \log \sum_{j=1}^{\ell} \exp(x_i/\alpha) \leq \max_j(x_j) + \alpha \log(\ell)$.

Then, by induction on $i = D, D-1, \ldots, 1$, it holds that $\hat{V}_{\text{MENTS}}(s^i_{\mathcal{M}}) \leq \frac{D-(i-1)}{D}$:

$$\hat{V}_{\text{MENTS}}(s^i_{\mathcal{M}}) \leq \max\left(\frac{D-i}{D}, \hat{V}_{\text{MENTS}}(s^{i+1}_{\mathcal{M}})\right) + \frac{1}{D} \tag{4.98}$$

$$= \frac{D-i}{D} + \frac{1}{D} \tag{4.99}$$

$$= \frac{D-(i-1)}{D}. \tag{4.100}$$

Hence, it must be the case that $\hat{Q}_{\text{MENTS}}(s^i_{\mathcal{M}}, a^L_{\mathcal{M}}) > \hat{Q}_{\text{MENTS}}(s^i_{\mathcal{M}}, a^R_{\mathcal{M}})$ and $\pi_{\text{MENTS}}(a^R_{\mathcal{M}}|s^i_{\mathcal{M}}) \leq \frac{1}{2}$ for all $1 \leq i \leq D$.

Let $Y(n)$ be the event that the state $s^E_{\mathcal{M}}$ is visited in the nth trial. Additionally, let $Z(n)$ be the event that $s^E_{\mathcal{M}}$ has not been visited on any of the first n trials. That is, $Z(n) = \bigcap_{j=1}^{n} \neg Y(j)$. Additionally note that $Z(n)$ implies that $\hat{V}_{\text{MENTS}}(s^E_{\mathcal{M}}) = 0$ and so:

$$\Pr(Y(n)|Z(n-1)) = \Pr(Y(n)|\hat{V}_{\text{MENTS}}(s^E_{\mathcal{M}}) = 0) \tag{4.101}$$

$$\leq 2^{-D}. \tag{4.102}$$

Hence, the probability that $s^E_{\mathcal{M}}$ is not visited in the first n trials can be bounded below inductively:

$$\Pr(Z(n)) = \Pr\left(\neg Y(n) \cap Z(n-1)\right) \tag{4.103}$$

$$= \Pr\left(\neg Y(n)|Z(n-1)\right) \Pr(Z(n-1)) \tag{4.104}$$

$$\geq (1 - 2^{-D}) \Pr(Z(n-1)) \tag{4.105}$$

$$\geq \ldots \tag{4.106}$$

$$\geq (1 - 2^{-D})^n \Pr(Z(0)) \tag{4.107}$$

$$\geq (1 - n2^{-D}), \tag{4.108}$$

where the last line uses Bernoulli's inequality and $\Pr(Z(0)) = 1$. $Z(0)$ is trivially true, as no trials have been run yet.

From the argument above, $Z(n)$ implies that $\hat{Q}_{\text{MENTS}}(s^1_{\mathcal{M}}, a^R_{\mathcal{M}}) \leq \frac{D-1}{D} = \hat{Q}_{\text{MENTS}}(s^1_{\mathcal{M}}, a^L_{\mathcal{M}})$ and so $\psi_{\text{MENTS}}(s^1_{\mathcal{M}}) = a^L_{\mathcal{M}} \neq a^R_{\mathcal{M}} = \pi^*(s^1_{\mathcal{M}})$. Hence, $\Pr(\psi_{\text{MENTS}} \neq \pi^*) = \Pr(Z(n)) \geq (1 - n2^{-D})$, which is greater than $\frac{1}{2}$ if $n \leq 2^{D-1}$.

For the second part of the result, a simple regret of at least $\frac{D-1}{D}$ is incurred with probability at least $\Pr(Z(n))$. Hence, the expected simple regret can be bounded below as follows:

$$\mathbb{E}\left[\texttt{sim_regr}(s^1_{\mathcal{M}}, \psi_{\text{MENTS}})\right] \geq \left(1 - \frac{D-1}{D}\right) \Pr(Z(n)) \tag{4.109}$$

$$\geq \frac{1}{D}\left(1 - n2^{-D}\right). \tag{4.110}$$

Then $|\mathcal{S}| = 4D + 3 \leq 5D$ for $D \geq 3$ and so substituting $D \geq \frac{|\mathcal{S}|}{5}$ gives the result with $c = \frac{1}{D}$ and $k = 2^{1/5}$:

$$\mathbb{E}\left[\texttt{sim_regr}(s^1_{\mathcal{M}}, \psi_{\text{MENTS}})\right] \geq \frac{1}{D}\left(1 - n2^{-D}\right) \tag{4.111}$$

$$\geq \frac{1}{D}\left(1 - \frac{n}{\left(2^{\frac{1}{5}}\right)^{|\mathcal{S}|}}\right). \tag{4.112}$$

□

4.6 Summary

TODO: This chapter introduced BTS and DENTS, established their consistency properties, and demonstrated empirically that DENTS can solve MDPs where UCT and MENTS fail. Chapter 5 extends these ideas to the multi-objective setting.

- What questions were answered
- Copy the intro and reword
- Investigated and demonstrated that entropy can be misaligned
- Demonstrated that can use it as an auxilary bonus objective
- Entropy is revisited in Chapter 6
- Now the focus shifts to the true multi-objective setting

5

Convex Hull Monte Carlo Tree Search

Contents

5.1 Introduction and Motivation 106
5.1.1 Consistent Action Selection 107
5.1.2 Contextual Tree Search 108
5.2 Contextual Zooming for Trees 109
5.2.1 Algorithm Definition 110
5.2.2 Differences from Contextual Zooming 113
5.3 Convex Hull Monte Carlo Tree Search 114
5.3.1 Common Framework 114
5.3.2 Convex Hull UCT 115
5.3.3 Convex Hull BTS 115
5.3.4 Convex Hull CZT 116
5.3.5 Prior Search Policies 117
5.3.6 Optimisations 119
5.3.7 Entropy and Convex Hulls 122
5.4 Empirical Results 123
5.4.1 Evaluation Proceedure 124
5.4.2 Environments 126
5.4.3 Results and Discussion 126
5.5 Theoretical Discussion 126
5.5.1 Context Independent Policies 126
5.5.2 Convergence of CHMCTS 127
5.5.3 The Fundamental Assumption of MOMCTS 127
5.6 Summary 134

This chapter considers multi-objective Monte Carlo Tree Search (MOMCTS) for the decision support scenario (TODO: ref back to ch2 fig) with linear utilities,

where the agent must plan for an unknown weight vector $\mathbf{w}$ over the objectives. Two approaches are considered in this chapter, the first following the same design paradigm as UCT where an algorithm for CMAB problems (Section 2.1.3) is applied at each decision node; the second is to maintain a convex hull value set at each node in the search tree, representing the set of achievable expected returns across all possible weightings over the objectives which is updated using convex hull backups (Section 2.6.1).

This chapter is organised as follows:

- Section 5.1 motivates the multi-objective decision making framework used in this chapter, and generalises the notions of simple and cumulative regret to *contextual simple regret* and *contextual cumulative regret* for the multi-objective setting;
- Section 5.2 introduces *Contextual Zooming for Trees* (CZT), a MOMCTS algorithm following the same design paradigm as UCT;
- Section 5.3 generalises prior work in MOMCTS to stochastic environments through *convex hull backups*;
- Section 5.4 presents empirical results on Deep Sea Treasure environments and a range of multi-objective environments from the Multi-Objective Gymnasium library TODO: cite?;
- Section 5.5 provides a brief theoretical discussion of the MOMCTS algorithms introduced in this chapter.

5.1 Introduction and Motivation

This section motivates the use of *contextual tree search* for planning in multi-objective settings, and highlights why consistent action selection with respect to a fixed objective is a necessary component of effective MOMCTS. The notions of cumulative and simple regret are then generalised to the multi-objective setting.

Figure 5.1: Tree MOMDP, with two actions at the root. From the first action, rewards of $(1, 0)$, $(0.5, 0.5)$ and $(0, 0.75)$ are available. From the second, rewards of $(0.75, 0)$, $(0.5, 0.5)$ and $(0, 1)$ are available. The convex hull value set is formed of $\{(1, 0), (0, 1)\}$.

5.1.1 Consistent Action Selection

Consider the MOMDP illustrated in Figure 5.1, with two objectives. The convex hull value set of the MOMDP is $\{(1, 0), (0, 1)\}$: the reward $(1, 0)$ is only reachable via the left action, and $(0, 1)$ only via the right action. The optimal action at the root node therefore depends entirely on the weight vector $\mathbf{w}$: if $w_1 > w_2$ then the left action is optimal, and if $w_2 > w_1$ then the right action is optimal.

Once the agent has taken the left action from the root node, it has implicitly committed to optimising towards the first objective (i.e. $w_1 > w_2$) because the rewards available from the right action are no longer reachable. Although all actions at the immediate child nodes are *locally optimal* (i.e. they lead to a value in the local convex hull value set), only the action leading to $(1, 0)$ is consistent with the implicit commitment made at the root node.

The problem of *consistent action selection* is also discussed by TODO: citet vanmoffaert14a. They compute and track Pareto-optimal policies using an adaptation of Q-learning and highlight that even if an action is *locally optimal*, it may not be *globally optimal*. To demonstrate this, they use an example with three optimal paths (Figure 5.2), where the black path alternates between multiple optimal

Figure 5.2: TODO: update caption with

paths, taking only locally optimal actions at each step but fails to follow any single globally optimal path.

TODO: actually add cite to bibtex

5.1.2 Contextual Tree Search

TODO: citet vanmoffaert and double check solve the consistent action selection problem by explicitly tracking the optimal policies across the full MOMDP. The approach taken in this thesis is to consider a *contextual tree search*, where a context weight vector $\mathbf{w}$ is sampled at the start of each trial and used to guide action selection throughout that trial. In the context of MORL with linear utilities, this is sufficient to resolve the consistent action selection problem, ensuring that all actions within a trial are consistent with a single objective.

In the `THTS++` framework, the context weight $\mathbf{w}$ is sampled during the `sample_context` function (Section **??**) and passed to all subsequent steps of the trial, including the search policy and backup functions.

Definition 5.1.1. *A* contextual policy $\pi(a|s;\mathbf{w})$ *is a policy that may depend on the context weight vector* $\mathbf{w} \in \Delta^D$*, where* $\pi(a|s;\mathbf{w})$ *gives the probability of selecting action* a *at state* s *under weight* $\mathbf{w}$*.*

A policy π is context independent *if $\pi(a|s;\mathbf{w}) = \pi(a|s;\mathbf{w}')$ for all $\mathbf{w}, \mathbf{w}' \in \Delta^D$ and all $s \in \mathcal{S}$, $a \in \mathcal{A}$. Otherwise, π is* context dependent.

To reason about the performance of a contextual tree search algorithm, the notions of cumulative and simple regret are generalised to *contextual cumulative regret* and *contextual simple regret*. These definitions reduce to the standard definitions (Definitions 2.1.1 and 2.1.2) when $D = 1$.

Definition 5.1.2. *Let $\{\mathbf{w}^i\}_{i=1}^n$ be a sequence of weight vectors sampled uniformly from Δ^D. The* contextual cumulative regret *of the search policies $\pi^{1:n} = (\pi^1, \ldots, \pi^n)$ after n trials is:*

$$\texttt{ctx_regr}(s, \pi^{1:n}) = \sum_{i=1}^{n} \left(\max_{\pi'} \mathbf{w}^{i\top} \mathbf{V}^{\pi'}(s) - \mathbf{w}^{i\top} \mathbf{V}^{\pi^i}(s) \right). \tag{5.1}$$

Definition 5.1.3. *The* contextual simple regret *of a recommendation policy ψ at state $s \in \mathcal{S}$ is:*

$$\texttt{ctx_sim_regr}(s, \psi) = \mathbb{E}_{\mathbf{w}\sim\Delta^D} \left[\max_{\pi'} \mathbf{w}^\top \mathbf{V}^{\pi'}(s) - \mathbf{w}^\top \mathbf{V}^{\psi(\cdot;\mathbf{w})}(s) \right], \tag{5.2}$$

where $\mathbf{w} \sim \Delta^D$ denotes a weight vector sampled uniformly from the D-dimensional unit simplex.

Several prior MOMCTS works TODO: cite, discussed in Section TODO: ref ch3, use a context-independent search policy. It is shown in Section 5.5 that any context-independent search policy suffers linear contextual cumulative regret, and any context-independent recommendation policy suffers linear contextual simple regret.

TODO: Theorem that context independent search policy suffers linear contextual cumulative regret AND context independent recommendation policy suffers linear contextual simple regret

5.2 Contextual Zooming for Trees

A common design paradigm in MCTS is to adapt a MAB algorithm to run at each decision node of the search tree, treating the node as a (non-stationary) bandit problem. UCT follows this paradigm by running UCB at each node (Section 2.4.3).

Figure 5.3: The design paradigm for CZT. UCB is lifted to UCT by applying it at each node of the search tree. CZ generalises UCB to contextual bandits. CZT follows both directions, applying CZ at each node of the search tree.

The *Contextual Zooming* (CZ) algorithm [29, 59] is a generalisation of the UCB algorithm to contextual bandits (Section 2.1.3). Following the same methodology, *Contextual Zooming for Trees* (CZT) runs the CZ algorithm at each decision node of the search tree. The relationship between these algorithms is illustrated in Figure 5.3.

CZT is presented in this section in the context of MOMCTS, where the context is the weight vector $\mathbf{w}$ sampled at the start of each trial. Although CZ, and by extension CZT, is not motivated by the exploration setting this thesis focuses on, it is introduced as a natural contextual analogue of UCT, and as a contextual search policy baseline for MOMCTS, where no such baseline currently exists.

5.2.1 Algorithm Definition

Contextual Zooming operates over a similarity space (defined in Section 2.1.3), which in the case of CZT is the weight-action space $\Delta^D \times \mathcal{A}$. The following metric is defined over the weight-action space for use in CZT:

$$d_{\text{czt}}((\mathbf{w}, a), (\mathbf{w}', a')) = \|\mathbf{w} - \mathbf{w}'\|_2 + C_{\text{czt}} \mathbb{1}\left[a \neq a'\right], \tag{5.3}$$

where $C_{\text{czt}} > \sqrt{2}$ is a large constant such that the distance between any two actions is always greater than the distance between any two weights.

CZ maintains a set of *active balls* that cover the weight-action space. In CZT, let $A(s, a)$ denote the set of active balls for action a at decision node s, where

a ball with centre $(\mathbf{w}, a)$ and radius r is defined as $\texttt{ball}((\mathbf{w}, a), r) = \{(\mathbf{w}', a') \in \Delta^D \times \mathcal{A} \mid d((\mathbf{w}, a), (\mathbf{w}', a')) < r\}$. Let $A(s)$ be the set of all active balls at decision node s, i.e. $A(s) = \cup_{a \in \mathcal{A}} A(s, a)$.

Each set of active balls is initialised to a single ball, using the centroid of the simplex $\frac{1}{D}\mathbf{1}$, where $\mathbf{1}$ is the vector of all ones. Specifically,

$$A^0(s, a) = \left\{ \texttt{ball}\left(\left(\frac{1}{D}\mathbf{1}, a \right), \sqrt{\frac{D-1}{D}} \right) \right\}. \tag{5.4}$$

Each ball $B \in A(s, a)$ has an associated visit count $M(B)$ and average return value estimate $\bar{\mathbf{Q}}_{\texttt{CZT}}(B)$. The visit count and value estimates are initialised to $M(B) = 0$ and $\bar{\mathbf{Q}}_{\texttt{CZT}}(B) = \mathbf{0}$ respectively.

Two rules are applied to run contextual zooming:

- the *selection rule*, to select a ball (and action) from the set of active balls at the current decision node, as part of the `select_action` function;
- the *activation rule*, which optionally adds a ball with smaller radius to the set of active balls, as part of the `backup_v` function.

Selection Rule Let $\mathbf{w}$ be the context weight sampled for the nth trial, let $r(B)$ denote the radius of a ball $B \in A(s, a)$. The *domain* of B in CZ consists of the portion of the ball not covered by any smaller active ball. Specifically,

$$\texttt{dom}(B) = B - \left(\bigcup_{B' \in A(s,a): r(B') < r(B)} B' \right). \tag{5.5}$$

The *confidence radius* $\texttt{conf}(B)$ of ball B is:

$$\texttt{conf}(B) = b_{\texttt{CZT}} \sqrt{\frac{\log(N(s))}{1 + M(B)}}, \tag{5.6}$$

where $b_{\texttt{CZT}} > 0$ is a UCB style bias parameter.

Let $\texttt{relevant}(s, a; \mathbf{w})$ denote the set of *relevant* balls at a decision node s for action a. A ball is relevant if $(\mathbf{w}, a)$ is in the domain of the ball. Specifically,

$$\texttt{relevant}(s, a; \mathbf{w}) = \{B \in A(s, a) \mid (\mathbf{w}, a) \in \texttt{dom}(B)\}, \tag{5.7}$$

$$\texttt{relevant}(s; \mathbf{w}) = \cup_{a \in \mathcal{A}} \texttt{relevant}(s, a; \mathbf{w}). \tag{5.8}$$

Contextual Zooming defines a confidence interval term, called the *pre-index*, around each ball, similar to UCB's confidence interval term:

$$I^{\texttt{pre}}_{\texttt{CZT}}(B;\mathbf{w}) = \mathbf{w}^\top \bar{\mathbf{Q}}_{\texttt{CZT}}(B) + r(B) + \texttt{conf}(B). \tag{5.9}$$

The full *index* of a ball B allows a newly activated ball with few visits to inherit an optimistic value estimate from nearby well-explored balls, ensuring it remains eligible for selection. Specifically,

$$I_{\texttt{CZT}}(B;\mathbf{w}) = r(B) + \min_{B' \in A(s)} \left(I^{\texttt{pre}}_{\texttt{CZT}}(B';\mathbf{w}) + d(B,B') \right), \tag{5.10}$$

where $d(B,B')$ is the distance between the centres of balls B and B' according to the metric d. In CZT the constant C_{czt} is set to a large enough value such that balls with a different action will never minimise the expression, so it suffices to consider only balls with the same action as B in Equation (5.10).

To select a ball, the `select_ball` function chooses the ball from the set of relevant balls with the highest index:

$$\texttt{select_ball}(s,\mathbf{w}) = \operatorname*{arg\,max}_{B \in \texttt{relevant}(s;\mathbf{w})} I_{\texttt{CZT}}(B). \tag{5.11}$$

If $\texttt{select_ball}(s,\mathbf{w}) = \texttt{ball}((\mathbf{w}_B, a_B), r_B)$, then the CZT selection policy returns $\pi_{\texttt{CZT}}(s;\mathbf{w}) = a_B$.

Backups and Activation Rule Let $\tau_{:h} \sim \pi_{\texttt{CZT}}$ be the trajectory sampled in the selection phase of CZT. Let B_t be the ball selected at the decision node s_t.

The backup functions update the ball selection counts and the multi-objective average returns for $t = h-1, \ldots, 0$:

$$M(B_t) \leftarrow M(B_t) + 1, \tag{5.12}$$

$$\bar{\mathbf{Q}}_{\texttt{CZT}}(B_t) \leftarrow \frac{M(B_t)-1}{M(B_t)} \bar{\mathbf{Q}}_{\texttt{CZT}}(B_t) + \frac{\hat{\mathbf{V}}_{\text{heuristic}}(s_h) + \sum_{i=t}^{h-1} \mathbf{r}_i}{M(B_t)}, \tag{5.13}$$

where $\hat{\mathbf{V}}_{\text{heuristic}}(s_h)$ is the (multi-objective) heuristic value function.

The *activation rule* introduces a new ball with smaller radius when the following conditions are met:

$$M(B_t) > M_{\min} \quad \text{and} \quad \texttt{conf}(B_t) \leq r(B_t), \tag{5.14}$$

where $M_{\min}$ is a minimum selection threshold, and when both conditions are met, a new ball $B_{\text{new}} = \texttt{ball}((\mathbf{w}, a_t), \frac{r(B_t)}{2})$ is added to the set of active balls $A(s, a)$.

Recommendation Policy To recommend an action in CZT, the ball with maximum linear utility from the set of active balls is selected. Specifically,

$$\psi_{\texttt{CZT}}(s; \mathbf{w}) = \arg\max_{a \in \mathcal{A}} \max_{B \in \texttt{relevant}(s,a;\mathbf{w})} \mathbf{w}^\top \bar{\mathbf{Q}}_{\texttt{CZT}}(B). \tag{5.15}$$

5.2.2 Differences from Contextual Zooming

Section 5.2.1 described the CZT algorithm. The main adaptations from CZ are summarised below:

1. In CZT, the set of active balls is specialised to a set for each action. In CZ, a single set of active balls is used across the entire *similarity space* (defined in Section 2.1.3).

2. CZ assumes that the algorithm is run for a known number of rounds T, and uses this value instead of the current number of visits at the decision node $N(s)$ in the confidence radii (Equation **??**).

3. The activation rule of CZ does not have a hard lower bound on the number of ball visits required to add a new child ball. CZT makes this adaptation to account for item 2, which results in smaller confidence radii.

4. In CZT, all balls contain exactly one action, whereas in CZ with a general continuous space of arms, balls cover many arms. After a ball is selected in CZ, an arm is sampled uniformly randomly from the domain of the selected ball.

CZ TODO: cite extends its regret analysis from a fixed number of rounds T to an unknown number of rounds using the *doubling trick* [9]. The doubling trick runs an independent instance of the algorithm for T rounds, then $2T$ rounds, then $4T$ rounds and so on. In practice, however, discarding all accumulated data at each doubling interval is undesirable. Instead, in CZT the number of visits $N(s)$

at the current node is used directly in place of T, which avoids the need for the doubling trick at the cost of losing regret guarantees.

TODO: Make sure to comment about how CZT without the additional activation rule leads to a blowup in the number of balls and is very slow.

5.3 Convex Hull Monte Carlo Tree Search

This section introduces *Convex Hull Monte Carlo Tree Search* (CHMCTS), which generalises prior work in MOMCTS TODO: ref ch3 to stochastic environments by maintaining *convex hull value sets* at each node in the search tree and updating them using convex hull backups (Section 2.6.1). When the MOMDP is deterministic and a search policy from prior work is used, CHMCTS reduces to the prior MOMCTS algorithms discussed in Chapter **??**.

The common `THTS++` framework and backup equations shared by all CHMCTS algorithms are given first. The search policies for each algorithm are then defined. Two practical optimisations are discussed in Section 5.3.6, and the section concludes with a brief discussion on the relationship between entropy and convex hull value sets.

5.3.1 Common Framework

Let ALG denote any of the CHMCTS algorithms defined in this section. At each decision node s, a convex hull value set $\hat{\boldsymbol{\mathcal{V}}}_{\text{ALG}}(s)$ is maintained, and at each chance node (s, a), a convex hull Q-value set $\hat{\boldsymbol{\mathcal{Q}}}_{\text{ALG}}(s, a)$ is maintained. The specific form of the search policy varies between algorithms and is defined in the subsections below.

Given a context weight $\mathbf{w} \in \Delta^D$ and a trajectory $\tau_{:h} = (s_0, a_0, \mathbf{r}_0, \ldots, s_{h-1}, a_{h-1}, \mathbf{r}_{h-1}, s_h) \sim \pi_{\text{ALG}}(\cdot|\cdot; \mathbf{w})$, the value set estimates are updated for $t = h-1, \ldots, 0$:

$$\hat{\boldsymbol{\mathcal{V}}}_{\text{ALG}}(s_t) \leftarrow \texttt{cvx_prune}\left(\bigcup_{a \in \mathcal{A}} \hat{\boldsymbol{\mathcal{Q}}}_{\text{ALG}}(s_t, a)\right), \tag{5.16}$$

$$\hat{\boldsymbol{\mathcal{Q}}}_{\text{ALG}}(s_t, a_t) \leftarrow \mathbf{R}(s_t, a_t) + \sum_{s' \in \text{Succ}(s_t, a_t)} \frac{N(s')}{N(s_t, a_t)} \hat{\boldsymbol{\mathcal{V}}}_{\text{ALG}}(s'). \tag{5.17}$$

The `cvx_prune` operation, defined in Equation (2.73) and repeated here for convenience, retains only those vectors in a set $\boldsymbol{\mathcal{V}}$ that are optimal for at least

one weight vector:

$$\texttt{cvx_prune}(\boldsymbol{\mathcal{V}}) = \{\mathbf{v} \in \boldsymbol{\mathcal{V}} \mid \exists \mathbf{w} \in \Delta^D.\ \forall \mathbf{v}' \in \boldsymbol{\mathcal{V}} - \{\mathbf{v}\}.\ \mathbf{w}^\top \mathbf{v} > \mathbf{w}^\top \mathbf{v}'\}. \tag{5.18}$$

Since the context weight is passed to policies, it is useful to define *contextual value estimates*, which extract a scalar value from a convex hull value set for a given weight $\mathbf{w} \in \Delta^D$:

$$\hat{V}^{\mathbf{w}}_{\text{ALG}}(s) = \max_{\mathbf{v} \in \hat{\boldsymbol{\mathcal{V}}}_{\text{ALG}}(s)} \mathbf{w}^\top \mathbf{v}, \tag{5.19}$$

$$\hat{Q}^{\mathbf{w}}_{\text{ALG}}(s,a) = \max_{\mathbf{q} \in \hat{\boldsymbol{\mathcal{Q}}}_{\text{ALG}}(s,a)} \mathbf{w}^\top \mathbf{q}. \tag{5.20}$$

The recommendation policy selects the action with the highest contextual Q-value under the given weight:

$$\psi_{\text{ALG}}(s; \mathbf{w}) = \arg\max_{a \in \mathcal{A}} \hat{Q}^{\mathbf{w}}_{\text{ALG}}(s,a). \tag{5.21}$$

5.3.2 Convex Hull UCT

Another natural extension of UCT to the contextual multi-objective setting is to replace the scalar Q-value estimates (used in Section 2.4.3) with contextual value estimates from Equations (5.19) and (5.20). The resulting algorithm, *Convex Hull UCT* (CH-UCT), uses the following search policy:

$$\pi_{\text{CH-UCT}}(s; \mathbf{w}) = \arg\max_{a \in \mathcal{A}} \left[\hat{Q}^{\mathbf{w}}_{\text{CH-UCT}}(s,a) + c_{\text{CH-UCT}} \sqrt{\frac{\log(N(s))}{N(s,a)}} \right], \tag{5.22}$$

where $c_{\text{CH-UCT}} > 0$ is a bias parameter controlling exploration. The search policy is context-dependent, since the contextual Q-value $\hat{Q}^{\mathbf{w}}_{\text{CH-UCT}}(s,a)$ depends on $\mathbf{w}$, ensuring that action selection is consistent with the sampled objective throughout each trial.

5.3.3 Convex Hull BTS

Convex Hull BTS (CH-BTS) uses the BTS search policy (Section 4.2.1) within the CHMCTS framework, using contextual value estimates in place of the scalar

Bellman value estimates:

$$\rho_{\text{CH-BTS}}(a|s;\mathbf{w}) \propto \exp\left(\frac{\hat{Q}^{\mathbf{w}}_{\text{CH-BTS}}(s,a)}{\alpha_{\text{CH-BTS}}(N(s))}\right), \tag{5.23}$$

$$\pi_{\text{CH-BTS}}(a|s;\mathbf{w}) = (1-\lambda(s,\epsilon_{\text{CH-BTS}}))\rho_{\text{CH-BTS}}(a|s;\mathbf{w}) + \frac{\lambda(s,\epsilon_{\text{CH-BTS}})}{|\mathcal{A}|}, \tag{5.24}$$

$$\lambda(s,\epsilon) = \min\left(1, \frac{\epsilon}{\log(e+N(s))}\right), \tag{5.25}$$

where $\alpha_{\text{CH-BTS}}(x) > 0$ is a temperature function and $\epsilon_{\text{CH-BTS}} \in [0,\infty)$ is an exploration parameter. As in BTS, the temperature function controls the trade-off between exploration and exploitation, and the exploration parameter ensures that all actions are eventually explored.

5.3.4 Convex Hull CZT

In single-objective MCTS, Bellman backups often outperform average return backups in the exploration setting (Chapter 4), since they propagate value information more directly through the tree by bootstrapping estimates from successor states rather than averaging over full trajectories. Convex hull backups can be viewed as the multi-objective analogue of Bellman backups: rather than maintaining a scalar average return at each node, they maintain a set of achievable value vectors that more accurately represents the range of outcomes reachable from that node. The CZT ball value estimates $\bar{\mathbf{Q}}_{\texttt{CZT}}(B)$ provide a coarser representation, averaging over a region of weight space rather than capturing the full structure of achievable values.

Since the CHMCTS backups (Equations (5.16) and (5.17)) are independent of the search policy, they can be combined with any search policy, including CZT. The resulting algorithm, *Convex Hull CZT* (CH-CZT), runs the CZT algorithm but maintains additional convex hull value sets at each node and uses the recommendation policy from Equation (5.21) in place of the CZT recommendation policy. This combines the contextual exploration of CZT with the richer value representation of CHMCTS.

5.3.5 Prior Search Policies

The MOMCTS algorithms discussed in Chapter **??** can be recovered within the CHMCTS framework by substituting appropriate search policies. In general, prior work uses search policies of the following form:

$$\pi_{\text{ALG}}(a|s;\mathbf{w}) = \arg\max_{a\in\mathcal{A}} \left[\zeta_{\text{ALG}}(\hat{\boldsymbol{\mathcal{Q}}}_{\text{ALG}}(s,a);\mathbf{w}) + c_{\text{ALG}}\sqrt{\frac{\log(N(s))}{N(s,a)}}\right], \tag{5.26}$$

where ζ_{ALG} is a scalarisation function that maps a convex hull Q-value set to a scalar, and $c_{\text{ALG}} > 0$ is a bias parameter. Notably, when ζ_{ALG} does not depend on $\mathbf{w}$, the search policy is context-independent, which as discussed in Section 5.1 leads to linear contextual cumulative regret.

Hypervolume CHMCTS (H-CHMCTS). TODO: move hypervolume defn to ch2

TODO: cite and ref use the *hypervolume indicator* (Definition 5.3.1) as the scalarisation function ζ_{ALG} in the search policy:

Definition 5.3.1. *Let $\boldsymbol{\mathcal{V}} \subset \mathbb{R}^D$ be a finite set of vectors and let $\mathbf{r}_{\text{ref}} \in \mathbb{R}^D$ be a* reference point *such that $(r_{\text{ref}})_d \leq \min_{\mathbf{v}\in\boldsymbol{\mathcal{V}}} v_d$ for all $d = 1,\ldots,D$. The* hypervolume indicator *of $\boldsymbol{\mathcal{V}}$ with respect to $\mathbf{r}_{\text{ref}}$ is the Lebesgue measure of the space dominated by $\boldsymbol{\mathcal{V}}$ and bounded below by $\mathbf{r}_{\text{ref}}$:*

$$\text{HV}(\boldsymbol{\mathcal{V}};\mathbf{r}) = \lambda_{\text{Lebesgue}}\left(\bigcup_{\mathbf{v}\in\boldsymbol{\mathcal{V}}} \{\mathbf{x}\in\mathbb{R}^D \mid \mathbf{r}_{\text{ref}} \preceq \mathbf{x} \preceq \mathbf{v}\}\right), \tag{5.27}$$

where $\lambda_{\text{Lebesgue}}$ denotes the Lebesgue measure and $\preceq$ is Pareto domination (Definition 2.6.12).

That is, $\zeta_{\text{H-CHMCTS}}(\boldsymbol{\mathcal{V}};\mathbf{w}) = \text{HV}(\boldsymbol{\mathcal{V}};\mathbf{r}_{\text{ref}})$, where $\mathbf{r}_{\text{ref}}$ is a parameter of the algorithm.

Chebyshev CHMCTS (C-CHMCTS). TODO: cite and ref use the *Chebyshev utility*:

Definition 5.3.2. *Let* $\mathbf{v}, \mathbf{r}_{\text{uto}} \in \mathbb{R}^D$ *and* $\mathbf{w} \in \Delta^D$*. The* Chebyshev utility *of* $\mathbf{v}$ *with respect to* $\mathbf{w}$ *and* $\mathbf{r}_{\text{uto}}$ *is:*

$$u_{\text{chebychev}}(\mathbf{v}; \mathbf{w}, \mathbf{r}_{\text{uto}}) = -\max_{i=1,\dots,D} w_i |v_i - (r_{\text{uto}})_i|. \tag{5.28}$$

The scalarisation function for C-CHMCTS is:

$$\zeta_{\text{C-CHMCTS}}(\boldsymbol{\mathcal{V}}; \mathbf{w}, \mathbf{r}_{\text{uto}}) = \max_{\mathbf{v} \in \boldsymbol{\mathcal{V}}} u_{\text{chebychev}}(\mathbf{v}; \mathbf{w}, \mathbf{r}_{\text{uto}}), \tag{5.29}$$

where $\mathbf{r}_{\text{uto}}$ is a *utopian point* satisfying $\mathbf{r}_{\text{uto}} \succeq \mathbf{v}$ for all $\mathbf{v} \in \boldsymbol{\mathcal{V}}$ and is a parameter of the algorithm.

Pareto CHMCTS (P-CHMCTS). The notable exception to this form is Pareto MCTS TODO: cite, which uses a Pareto UCB algorithm TODO: cite at each decision node, and does not fit directly into the template of Equation (5.26). *Pareto CHMCTS* (P-CHMCTS) is the algorithm that results from running Pareto UCB at each decision node within the CHMCTS framework.

Pareto UCB adds a UCB-style bonus to each vector in the value set:

$$\hat{\boldsymbol{\mathcal{V}}}^{\texttt{conf}}_{\text{P-CHMCTS}}(s) = \texttt{cvx_prune}\left(\hat{\boldsymbol{\mathcal{V}}}^{U}_{\text{P-CHMCTS}}(s)\right), \tag{5.30}$$

$$\hat{\boldsymbol{\mathcal{V}}}^{U}_{\text{P-CHMCTS}}(s) = \left\{(\mathbf{q} + U_{\text{pucb}}(s,a)\mathbf{1}, a) \mid a \in \mathcal{A}, \mathbf{q} \in \hat{\boldsymbol{\mathcal{Q}}}_{\text{P-CHMCTS}}(s,a)\right\}, \tag{5.31}$$

$$U_{\text{pucb}}(s,a) = b_{\text{P-CHMCTS}} \sqrt{\frac{\log(N(s))}{N(s,a)}}. \tag{5.32}$$

$$\tag{5.33}$$

where $b_{\text{P-CHMCTS}} > 0$ is a bias parameter and where the notation $(\mathbf{q}, a)$ tags each vector with its corresponding action to allow the action to be recovered after the `cvx_prune` operation.

The search policy samples a point uniformly randomly from the *confidence value set*, and selects the corresponding action. That is:

$$\pi_{\text{P-CHMCTS}}(a|s; \mathbf{w}) = \sum_{(\mathbf{v},a') \in \hat{\boldsymbol{\mathcal{V}}}^{\texttt{conf}}_{\text{P-CHMCTS}}(s)} \frac{\mathbb{1}[a = a']}{|\hat{\boldsymbol{\mathcal{V}}}^{\texttt{conf}}_{\text{P-CHMCTS}}(s)|}. \tag{5.34}$$

5.3.6 Optimisations

Two practical optimisations are used in the implementation of CHMCTS: *solved labelling*, which avoids wasting computation on well-explored parts of the search tree; and *size pruning*, which limits the number of points stored in each convex hull value set.

Solved Labelling. In planning, *labelling* TODO: cite lrtdp is used to mark nodes as solved and prune them from further consideration, avoiding wasted computation on parts of the tree that are already solved. A node is marked as solved when its value estimate has converged to the true value, which requires knowledge of the true transition distribution. TODO: double check lrtdp paper In the setting considered here, the true transition distribution $p(\cdot|s,a)$ is unknown, so a probabilistic analogue for labelling is used instead.

Each chance node (s,a) is assigned an *unsolved value* $\texttt{unsolved}(s,a) \in [0,1]$, where $\texttt{unsolved}(s,a) = 0$ indicates that the node is solved and $\texttt{unsolved}(s,a) = 1$ indicates that it is unsolved. The solved value of a node is an upper bound on the probability that the node's value estimate has not yet converged.

The unsolved value of a decision node is computed as the maximum unsolved value of its children:

$$\texttt{unsolved}(s) = \max_{a \in \mathcal{A}} \texttt{unsolved}(s,a). \tag{5.35}$$

The unsolved values are updated during the backpropagation phase of `THTS++` and used during the selection phase to filter out actions that have already been solved, to a tolerance of $\texttt{unsolved}_{\text{tol}}$, with high probability. That is, at a decision node s, any action a with $\texttt{unsolved}(s,a) \leq \texttt{unsolved}_{\text{tol}}$ is filtered out before selecting an action. The remainder of this section describes how the unsolved values are propagated through chance nodes, that is, how the value of $\texttt{unsolved}(s,a)$ is computed.

The solved value of a chance node (s,a) is computed as an upper bound on the expected solved values of its successor states, using the empirical transition distribution $\frac{N(s')}{N(s,a)}$ as a proxy for the true distribution $p(s'|s,a)$. Since the true

distribution is unknown, the bound must account for the estimation error in the empirical distribution. Let $\delta_{\text{unsolved}} > 0$ be a confidence parameter. The unsolved value $\texttt{unsolved}(s, a)$ is defined as an upper bound satisfying:

$$\Pr\left(\mathbb{E}_{s' \sim p(\cdot|s,a)}[\texttt{unsolved}(s')] \leq \texttt{unsolved}(s, a)\right) \geq 1 - \delta_{\text{unsolved}}. \tag{5.36}$$

In words, with probability at least $1 - \delta_{\text{unsolved}}$, the expected solved value of the children of chance node (s, a) is bounded by the upper bound $\texttt{unsolved}(s, a)$.

To derive this bound, the expected unsolved value is decomposed into contributions from observed and unobserved successor states:

$$\mathbb{E}[\texttt{unsolved}(s')] = \sum_{s' \in \text{Succ}_{\text{obs}}(s,a)} p(s'|s, a)\texttt{unsolved}(s') \tag{5.37}$$

$$+ \sum_{s' \notin \text{Succ}_{\text{obs}}(s,a)} p(s'|s, a)\texttt{unsolved}(s'), \tag{5.38}$$

where $\text{Succ}_{\text{obs}}(s, a) = \{s' \in \text{Succ}(s, a) \mid N(s') > 0\}$ is the set of successor states observed so far.

For the observed successors, Corollary C.2.1 of the DKW inequality (Theorem 2.5.1) gives with probability at least $1 - \frac{\delta_{\text{unsolved}}}{2}$ for all $s' \in \text{Succ}_{\text{obs}}(s, a)$:

$$p(s'|s, a) \leq \frac{N(s')}{N(s, a)} + 2\sqrt{\frac{\log(4/\delta_{\text{unsolved}})}{2N(s, a)}}, \tag{5.39}$$

so the contribution of observed successors in (5.37) is bounded by:

$$\sum_{s' \in \text{Succ}_{\text{obs}}(s,a)} p(s'|s, a)\texttt{unsolved}(s') \tag{5.40}$$

$$\leq \sum_{s' \in \text{Succ}_{\text{obs}}(s,a)} \left(\frac{N(s')}{N(s, a)} + 2\sqrt{\frac{\log(4/\delta_{\text{unsolved}})}{2N(s, a)}}\right) \texttt{unsolved}(s'). \tag{5.41}$$

The *Good-Turing estimate* (Section 2.5) for the missing mass is:

$$\texttt{missing_mass}_{\text{GT}}(s, a) = \sum_{s' \in \text{Succ}(s,a)} \frac{\mathbb{1}\left[N(s') = 1\right]}{N(s, a)}. \tag{5.42}$$

For the unobserved successors, since $\texttt{unsolved}(s') \leq 1$ for all s', the contribution of unobserved successors (5.38) is bounded by the missing mass using Theorem 2.5.2 with probability at least $1 - \frac{\delta_{\text{unsolved}}}{2}$:

$$\sum_{s' \notin \text{Succ}_{\text{obs}}(s,a)} p(s'|s, a) \leq \texttt{missing_mass}_{\text{GT}}(s, a) + \sqrt{\frac{\log(2/\delta_{\text{unsolved}})}{N(s, a)}}. \tag{5.43}$$

Combining these bounds via the union bound, the following assignment of $\texttt{unsolved}(s, a)$ satisfies (5.36):

$$\texttt{unsolved}(s, a) \leftarrow \sum_{s' \in \mathrm{Succ}_{\mathrm{obs}}(s,a)} \left(\frac{N(s')}{N(s, a)} + 2\sqrt{\frac{\log(4/\delta_{\mathrm{unsolved}})}{2N(s, a)}} \right) \texttt{unsolved}(s') \tag{5.44}$$

$$+ \texttt{missing_mass}_{\mathrm{GT}}(s, a) + \sqrt{\frac{\log(2/\delta_{\mathrm{unsolved}})}{N(s, a)}}. \tag{5.45}$$

When the `unsolved` labelling is used in CHMCTS, $\texttt{unsolved}_{\mathrm{tol}}$ and $\delta_{\mathrm{unsolved}}$ are hyperparameters of the algorithm. A node is marked as solved when $\texttt{unsolved}(s, a) \leq \texttt{unsolved}_{\mathrm{tol}}$, at which point it is excluded from further trials.

Convex Hull Size Pruning. As the search progresses, the convex hull value sets at each node may grow large, increasing the computational cost of the `cvx_prune` operation and the convex hull backups. To limit this cost, a maximum size $K_{\max}$ is imposed on each convex hull value set. When a value set exceeds this limit after a backup, points are pruned using the *crowding distance* from the NSGA-II algorithm TODO: cite.

The crowding distance of a point $\mathbf{v}$ in a set $\boldsymbol{\mathcal{V}}$ measures how isolated $\mathbf{v}$ is from its neighbours in objective space. For each objective dimension $i = 1, \ldots, D$, the points in $\boldsymbol{\mathcal{V}}$ are sorted by their ith coordinate. The boundary points (those with the smallest and largest values in dimension i) are assigned an infinite crowding distance and are always retained. For interior points, the crowding distance contribution from dimension i is:

$$\mathrm{cd}_i(\mathbf{v}) = \frac{v_i^+ - v_i^-}{\max_{\mathbf{v}' \in \boldsymbol{\mathcal{V}}} v_i' - \min_{\mathbf{v}' \in \boldsymbol{\mathcal{V}}} v_i'}, \tag{5.46}$$

where v_i^+ and v_i^- are the ith coordinates of the nearest neighbours of $\mathbf{v}$ above and below in dimension i respectively. The total crowding distance is $\mathrm{cd}(\mathbf{v}) = \sum_{i=1}^{D} \mathrm{cd}_i(\mathbf{v})$. When the value set exceeds $K_{\max}$ points, the $K_{\max}$ points with the largest crowding distances are retained, preserving the spread of the convex hull.

Figure 5.4: A convex hull value set where a single point is optimal for two different weight vectors $\mathbf{w}$ and $\mathbf{w}'$. Although both weight vectors achieve the same value at the root state (a), they induce different policies at downstream states (b), resulting in different entropy values. TODO: make figs for two convex hulls, one with two weights with same value, and a downstream state with different values for the same two weights

5.3.7 Entropy and Convex Hulls

Convex hull value sets represent the set of achievable expected returns in *objective space*, storing value vectors attainable by different policies. Entropy, by contrast, is a property of a policy rather than a value vector: it measures the uncertainty of the action distribution induced by a policy at each state.

A key observation is that a single point on the convex hull does not correspond to a unique policy, as illustrated in Figure 5.4. Since the contextual policies considered in this chapter depend on the weight vector $\mathbf{w}$, a given value vector $\mathbf{v} \in \hat{\boldsymbol{\mathcal{V}}}_{\text{ALG}}(s)$ may be optimal for many different weight vectors, each inducing a different policy $\pi(\cdot|s;\mathbf{w})$ with a different entropy.

As a result, there is no principled way to associate a single entropy value with a point on the convex hull. This is the fundamental reason why the entropy-based exploration used in DENTS (Chapter 4) does not extend naturally to the multi-objective setting with convex hull value sets, and motivates the simplex map approach introduced in Chapter 6.

5.4 Empirical Results

TODO: Clean up plots, legend placement etc.

New Ch5 plan:

- Empirical results
- Say that when possible (have a cpp port) use CHVI as a baseline and also CHVI with prioritised sweeping

should have a QHull reference when talking about the implementation

say that we prune using that, unless it is a degenerate case where we fall back to using the linear programming method

this was the fastest option available

mention that used a well spaced set of points for the EUM calculation and cite the source for how it was computed.

TODO: part 0: discuss the evaluation prodceedure, and how the results are evaluated.

TODO: part 1: run algorithms on the tree envs that show the context dependent stuff better

TODO: part 1a: run algorithms with and without vectorised visit counts and show this helps with exploration?

TODO: part 2: run on suite of mo-gym environments

TODO: part 3: find environments where the convex hull blows up to motivate ch6. Give the CH fully and plot it out.

TODO: part 3a: also consider showing some of the optimal convex hulls where possible

TODO: should really reference back to the ch4 evaluation proceedure too

TODO: list

- Results from CHMCTS paper
- Get same plots from C++ code, but compare expected utility, rather than the confusing hypervolume ratio stuff

BD: A range of algorithms will be considered in this experimental section. Firstly, for baselines, the convex hull backup versions of algorithms from Ch3 (REF) will be used. Additionally, CZT, CH-CZT, CH-UCT, CH-BTS and CH-DENTS will be considered

This section provides an empirical evalutation of the algorithms developed in this chapter. The baseline algorithms considered will also be CHMCTS algorithms, using the action selection methods outline in Sections TODO: ref litrev and above.

BD: This para is very braindump. It is worth noting that in multi-objective MCTS algorithms that the accuracy and quality of the convex hull at the root node is the primary concern, even when a multi-objective MCTS algorithm is used in an *online* manor, where tree search is interleaved between actions taken by the agent. Once a policy has been selected at the initial state it implicitly defines a weighting over the objectives as detailed in Section TODO: ref sec 2. And following the discussion from TODO: ref above, to follow an optimal policy the actions need to be taken consistently with respect to the same objective weights. As such, a more efficient online algorithm will be to run the multi-objective tree search from the initial state, and run a single-objective MCTS algorithm in a scalarised single-objective MDP for the remaining actions. Where in the scalarised reward is $R(s, a; \mathbf{w}) = \mathbf{w}^\top \mathbf{R}(s, a)$.

5.4.1 Evaluation Proceedure

To evaluate the quality of the convex hull at the root node, the following metrics from the multi-objective literature [25] are considered: *Expected Utility Metric*, *Hypervolume Metric* and a *Sparsity Metric*. TODO: Add EUM and HV and Sparsity here or in ch2?

To evaluate the quality of the convex hull at the root node, where the optimal convex hull is not known a-priori, the *Expected Utility Metric*,

Firstly, given a context $\mathbf{w}$ and CHMCTS search tree $\mathcal{T}$ that defines a partial recommendation poilicy ψ_{alg}, the policy will be made complete similarly to Section

TODO: ref eval proceedure in ch4:

$$\texttt{complete}(\psi_{\text{alg}})(a|s;\mathbf{w}) = \begin{cases} 1 & \text{if } s \in \mathcal{T} \text{ and } a = \psi_{\text{alg}}(s;\mathbf{w}), \\ 0 & \text{if } s \in \mathcal{T} \text{ and } a \neq \psi_{\text{alg}}(s;\mathbf{w}), \\ \frac{1}{|\mathcal{A}|} & \text{otherwise.} \end{cases} \tag{5.47}$$

TODO: should reword to account for CZT and how it does the same thing

The Convex Hull (of policies) can then be written as

$$\Psi_{\text{alg}} = \{\texttt{complete}(\psi_{\text{alg}}) \mid \mathbf{w} \in \Delta^D\}. \tag{5.48}$$

This Convex Hull of policies is then evaluated using the *Expected Utility Metric* (EUM), which is defined as:

$$\text{EUM}(\Psi_{\text{alg}}) = \mathbb{E}_{\mathbf{w}\sim U(\Delta^D)}\left[\max_{\psi\in\Psi_{\text{alg}}} \mathbf{w}^\top \mathbf{V}^\psi(s_0)\right], \tag{5.49}$$

where $U(\Delta^D)$ is the uniform distribution over vectors in Δ^D. A monte-carlo estimate of $\text{EUM}(\Psi_{\text{alg}})$ will be computed, following TODO: cite the deep MORL paper, let $E(\Delta^D)$ be an evenly spaced set of points in Δ^D. Then the EUM can be approximated as:

$$\text{EUM}(\Psi_{\text{alg}}) \approx \sum_{\mathbf{w}\in E(\Delta^D)} \mathbf{w}^\top \mathbf{V}^{\tau(\mathbf{w})}, \tag{5.50}$$

where $\tau(\mathbf{w}) \sim \texttt{complete}(\psi_{\text{alg}})$ is a sampled trajectory for the point $\mathbf{w} \in \Delta^D$, and $\mathbf{V}^{\tau(\mathbf{w})}$ is the cumulative return from the trajectory. That is, if TODO: double check indices are consistent with how been doing it before the trajectory is $\tau(\mathbf{w}) = (s_0, a_0, \mathbf{r}_0, ..., s_{h-1}, a_{h-1}, \mathbf{r}_{h-1}, s_h)$, then $\mathbf{V}^{\tau(\mathbf{w})} = \sum_{i=0}^{h-1} \mathbf{r}_i$.

For the results given in TODO: ref, the number of sampled used to approximate the EUM was XXX. That is $|E(\Delta^D)| = XXX$.

TODO: double check this is consistent with the litrev paper on MORL and also cite it. Also what are the other metrics?

Experiments are run with `single_node_expansion` turned off, as no memory issues were encountered during the evalutation.

Similarly to the gridworld environments in Chapter TODO: ref, the BayesOpt package [35] was used to tune the hyperparameters of all algorithms in this chapter. Full details on the hyperparameter tuning and the selected parameters are given in Appendix TODO: new appendix? join the appendix?.

5.4.2 Environments

The MO-Gymnasium TODO: cite suite of environments was used to evaluate the algorithms.

TODO: Pick a couple envs to describe in more detail, and refer to cite for full details

Additionally, the Deep Sea Treasure is used, and the TODO: extended? (whatever the updated one is) are used. TODO: Also mention that we can generate maps of varying size to perform a scalability analysis

5.4.3 Results and Discussion

TODO: Paste in results and talk about them

5.5 Theoretical Discussion

This section BD: provides some theoretical discussion around CHMCTS and MOMCTS in general. It is structured as follows:

- Section 5.5.1 shows that context independent policies suffer linear contextual regret;
- Section 5.5.2 outlines convergence proofs for CHMCTS;
- Section 5.5.3 discusses a fundimental assumption made in MOMCTS algorithms and BD: shows that MOMCTS algorithms using this assumption can only compute optimal policies for linear utility functions.

5.5.1 Context Independent Policies

1. PREAMBLE 2. Figure 3. THeorem copy

5.5.2 Convergence of CHMCTS

1. Preamble - say not going to actually prove things 2. Quote CHVI theorem 3. Define optimal CHVS (or check that it was defined in Ch2) 4. CHMCTS values converge to optimal CHVS (proof = it would contradict CHVI theorem) 6. General UCB picks all actions inf often 5. Just give an example translation from DENTS results to CHMCTS results 7. CHUCT converges

5.5.3 The Fundamental Assumption of MOMCTS

1. Preamble - define the setup 2. General utility, and the form of the search/recommendation policy 3. Wordy description of the offending MOMDP with chebyshev utility 4. Theorem

Theorem 5.5.1 (Per-step action selection requires linear utility)**.** *Let $\boldsymbol{\mathcal{M}}$ be an MOMDP, Π the set of all policies and the optimal value sets be:*

$$\boldsymbol{\mathcal{V}}^*(s) = \{\mathbf{V}^\pi(s) \mid \pi \in U(\Pi; u)\},$$
$$\boldsymbol{\mathcal{Q}}^*(s,a) = \{\mathbf{Q}^\pi(s,a) \mid \pi \in U(\Pi; u)\},$$

where $U(\Pi; u)$ is the undominated set of policies (Definition 2.6.7) and u is a utility function. Let $\pi_u^{\mathbf{w}} \in \arg\max_{\pi\in\Pi} u(\mathbf{V}^\pi(s_0); \mathbf{w})$ denote a utility-optimal policy. Consider the greedy selection rule

$$\pi_{\text{greedy}}(s; \mathbf{w}) \in \arg\max_{a\in\mathcal{A}} \max_{\mathbf{q}\in\boldsymbol{\mathcal{Q}}^*(s,a)} u(\mathbf{q}; \mathbf{w}).$$

Then $\pi_{\text{greedy}}(\cdot; \mathbf{w})$ is utility-optimal (i.e. $u(\mathbf{V}^{\pi_{\text{greedy}}}(s_0); \mathbf{w}) = u(\mathbf{V}^{\pi_u^{\mathbf{w}}}(s_0); \mathbf{w})$) for every MOMDP $\boldsymbol{\mathcal{M}}$ and every $\mathbf{w} \in \Delta^D$ if and only if $u(\cdot; \mathbf{w})$ is linear in its first argument for every $\mathbf{w} \in \Delta^D$.

Proof outline. BD: If direction. Suppose $u(\cdot; \mathbf{w})$ is linear in its first argument for every $\mathbf{w} \in \Delta^D$, i.e. $u(\mathbf{v}; \mathbf{w}) = \mathbf{w} \cdot \mathbf{v}$ (up to a $\mathbf{w}$-dependent constant, which does not affect the $\arg\max$). Then $U(\Pi; u)$ coincides with the convex coverage set, and the greedy selection rule reduces to standard single-objective action selection under the scalarised reward $\mathbf{w} \cdot \mathbf{r}(s,a)$. Utility-optimality of π_{greedy} then follows from

the correctness of convex hull value iteration (Theorem TODO: ref CHVI), which reduces to standard value iteration for each fixed $\mathbf{w}$.

BD: Only if direction. We prove the contrapositive by exhibiting a single MOMDP and utility function for which π_{greedy} is not utility-optimal. Let u be the Chebyshev utility with reference point $\mathbf{z}^* = (6, 6)$,

$$u(\mathbf{v}; \mathbf{w}) = -\max_i w_i \left| v_i - z_i^* \right|,$$

which is non-linear in its first argument. Consider the MOMDP in Figure TODO: ref fig, which has a single action a_0 at s_0 transitioning with equal probability to successor states s_1 or s_2, at each of which the agent selects between a_0 and a_1 to receive a terminal vector reward. The four deterministic paths yield

$$s_0 \to s_1 \to a_0 : (5, 0), \qquad s_0 \to s_1 \to a_1 : (2, 2),$$
$$s_0 \to s_2 \to a_0 : (2, 2), \qquad s_0 \to s_2 \to a_1 : (0, 5).$$

Fix the preference $\mathbf{w} = (\frac{1}{2}, \frac{1}{2})$. By inspection, the four stationary policies yield expected values at s_0 of $(3.5, 1)$, $(2.5, 2.5)$, $(2, 2)$, and $(1, 3.5)$, with the policy achieving $(2.5, 2.5)$ being utility-optimal:

$$u((3.5, 1); \mathbf{w}) = -2.5, \quad u((2.5, 2.5); \mathbf{w}) = -1.75, \quad u((1, 3.5); \mathbf{w}) = -2.5.$$

(The value $(2, 2)$ is Pareto-dominated and is omitted.) The utility-optimal policy selects a_0 at s_1 (receiving $(5, 0)$) and a_1 at s_2 (receiving $(0, 5)$), giving expected value $(2.5, 2.5)$ and utility -1.75.

Now consider π_{greedy}. The Pareto fronts at the successor states are singletons,

$$\mathcal{Q}^*(s_1, a_0) = \{(5, 0)\}, \qquad \mathcal{Q}^*(s_1, a_1) = \{(2, 2)\},$$
$$\mathcal{Q}^*(s_2, a_0) = \{(2, 2)\}, \qquad \mathcal{Q}^*(s_2, a_1) = \{(0, 5)\},$$

and the Chebyshev utilities are $u((5, 0); \mathbf{w}) = u((0, 5); \mathbf{w}) = -3$ and $u((2, 2); \mathbf{w}) = -2$. Hence π_{greedy} selects a_1 at s_1 and a_0 at s_2, both yielding $(2, 2)$. The expected value at s_0 is

$$\mathbf{V}^{\pi_{\text{greedy}}}(s_0) = \tfrac{1}{2}(2, 2) + \tfrac{1}{2}(2, 2) = (2, 2),$$

with utility $u((2,2);\mathbf{w}) = -2 < -1.75$. Therefore π_{greedy} is not utility-optimal.

The obstruction is structural: under Chebyshev utility the optimal policy requires selecting the *locally* worst continuation ($(5,0)$ at s_1 and $(0,5)$ at s_2), whose expectation across the stochastic transition yields a *globally* preferable balanced outcome. No per-state $\arg\max$ on u can discover this coordination, because u does not commute with the expectation over successor states. Linearity of u is precisely the condition under which this commutation holds, and hence precisely the condition under which per-state action selection is utility-optimal. □

Theory results:

- Non-contextual policies garunteed to have a linear contextual simple regret
- Convex Hull algorithms garunteed to converge (lemma that UCT style policy picks actions inf often and reuse ch4 result for BTS), and mainly reproduce the result from CHVI
- Highlight that one step of the convex hull proof holds iff linear utility, use it to say that MOMCTS with value sets doesn't really work for non-linear utility functions
- Reconcile with prior work by saying that the histories are taken into account, and the pareto fronts stored are globally
- But generally highlight that this is a limitation of using MOMCTS, that optimisation at the level of actions is correct if and only if the utility function is linear. Give an example MOMDP where to get the optimal value doesnt allow for chebychev scalarization over value sets
- Discuss the difference between Contextual Zooming and CZT, cite a doubling trick and say that it can be used to generalise CZ to an algorithm for CMAB problems without a fixed number of rounds. Hypothesise that CZT contextual cumulative regret matches that of CZ, but the proofs of CZ are very complex themselves, and UCT theory is also long and hard, so not going to attempt to prove it.

From Van Moffaert paper:

As a result, the Bellman equation no longer holds and the learning algorithm is not proven to converge to the optimal policy (Perny and Weng, 2010; Roijers et al., 2013). Nevertheless, even without this theoretical guarantee, a Chebyshev scalarized MORL algorithm can obtain high-quality solutions (Van Moffaert et al., 2013b).

New Ch5 plan:

- Theory, contextual regret, non-contextual policies garunteed to have a linear contextual regret
- Theory, reproduce the CHMCTS results, and say that garunteed to eventually produce optimal (lemmas are actions selected inf often)
- No theory for CZT directly, makes a heuristic adaptation from the base work, and the proofs are very complex in themselves (would add like 50 pages of maths)

for theory would just like to have convergence, use ch4 proofs and UCT picks actions inf often as lemma and use CHVI proofs

Allude to that boltzmann probably gives exp contextual simple regret, and PUCT gives similar contextual cumulative regret, and the lower bound from exploring bandits means simple regret of UCT methods has a lower bound

TODO: Update theory appendix to be more general theory appendix. First section is additional results from literature, like hoeffdings. Include proof of UCT policy selecting any action infinitely often for an arbitrary Q value here. second section is exponential consistency proofs, split into three subsections, dents process, preliminary lemmas and the proofs.

also more critical to look into litrev for what results about MORL there are, shimons paper probably helpful

want to cite if someone has done that linear scalarisation iff making local decisions result

TODO: part 1: all the (relevant) search policies will select actions infinitely often.

TODO: part 1a: BTS and DENTS follows from ch4. For UCT can argue that if select one argument a finite number of times. Then use the confidence interval term to arrive at contradiction

TODO: part 2: convex hull backups perform the scalarised backups for all possible weights over the objective.

TODO: write up proofs wrote up on ipad. Used this gpt convo in cleaning up details: https://chatgpt.com/c/67fa9295-0cb8-8007-88fd-5bec1ac5393b

TODO: part 2a: refer back to convergence of backups in ch4, clean up and DKW stuff need to

TODO: part 3: if values converge to optimal, then contextual simple regret tends to zero

TODO: part 4: mention that CZT proofs would be much more complex, and given the other algorithms do better. Mention a lower bound which comes from the CZ regret bound. And that this algorithm could be preferable in not in exploring context.

TODO: part 4a: similar with CH-UCT to CZT

TODO: part 5: any proofs for CH-UCT and/or CZT? Which wouldn't be a disaster trying to proove?

Below is what we initially wrote in this section.

TODO: See if with chatgpt help can crack out a rough proof outline? Because assuming no noise in the reward function, the leaf nodes clearly converge with same rate. Just really need the induction step. Feel like it should be very do-able, by bootstrapping on the CZT proofs.

TODO: Can easily show that CH converges, by using every action sampled infinitly often still, and bootstrap off the convex hull convergence proof.

TODO: One thing need to prove is that our form of backups converges, because we're using the sample averages our backups are slightly different. Should

TODO: Remainder of properties bootstrap off the proofs in Ch4. Every action is sampled inf often, hence the convex hull backups converge. Don't bother showing convergence rates. Just show that they are consistent.

TODO: Also want to show that given any convex hull value estimates, that following the policy extracted from convex hulls does give the same multi-objective value.

basically this would say

$$\pi(s;w) = \arg\max_{a\in\mathcal{A}} \circ\mathbf{Q}(s,a;w) \tag{5.51}$$

$$w^\top \mathbf{V}^\pi(s;w) = w^\top \mathbb{E}_{a\sim\pi}[\mathbf{Q}^\pi(s,a;w)] \tag{5.52}$$

$$= \circ\mathbf{Q}(s,a;w) \tag{5.53}$$

$$w^\top \mathbf{Q}^\pi(s,a;w) = ... \tag{5.54}$$

So say that the value of the policy by taking the maimum from value estimates, will achieve the value estimates. We've been handwavy around empirical distriubtion vs actual transition distribution. But basically the argument is inductive, and holds trivially for leaf nodes.

Proposition 5.5.1. *Let $u(w,x,z) = \min_i w_i|x_i - z_i|$ denote the Chebyshev scalarisation, and consider the vectors*

$$a = (10,0), \quad b = c = (1,1), \quad d = (0,10),$$

together with a reference point $z \in \mathbb{R}^2$ satisfying $z_i < \min\{a_i, b_i, c_i, d_i\}$ componentwise. Let $w = (w_1, w_2)$ be any weight vector with $w_1, w_2 > 0$ and $w_1 + w_2 = 1$. Then

$$u(w,b,z) > u(w,a,z) \quad \text{or} \quad u(w,c,z) > u(w,d,z).$$

Proof. The componentwise inequality $z_i < \min\{a_i, b_i, c_i, d_i\}$ permits the absolute values to be dropped, so that

$$u(w,x,z) = \min_i w_i(x_i - z_i)$$

for each of $x \in \{a,b,c,d\}$. The shifted coordinates

$$\alpha_i = a_i - z_i, \qquad \beta_i = b_i - z_i = c_i - z_i, \qquad \delta_i = d_i - z_i$$

are therefore all strictly positive. From $z_1 < 0$ and $z_2 < 0$ (which follow from $z < d$ and $z < a$ respectively), the following strict inequalities are obtained:

$$\alpha_2 = -z_2 < 1 - z_2 = \beta_2, \qquad \delta_1 = -z_1 < 1 - z_1 = \beta_1. \tag{5.55}$$

Since $b = c$, the equality $u(w, b, z) = u(w, c, z)$ holds throughout.

Suppose, for contradiction, that both disjuncts fail, i.e.

$$\min(w_1\alpha_1,\, w_2\alpha_2) \geq \min(w_1\beta_1,\, w_2\beta_2), \tag{5.56}$$

$$\min(w_1\delta_1,\, w_2\delta_2) \geq \min(w_1\beta_1,\, w_2\beta_2). \tag{5.57}$$

From (5.56), the left-hand side is bounded above by $w_2\alpha_2$, so

$$w_2\alpha_2 \;\geq\; \min(w_1\beta_1,\, w_2\beta_2).$$

By (5.55) we have $w_2\alpha_2 < w_2\beta_2$, whence the minimum on the right cannot equal $w_2\beta_2$. It follows that

$$\min(w_1\beta_1,\, w_2\beta_2) = w_1\beta_1, \qquad \text{so} \qquad w_1\beta_1 \leq w_2\beta_2. \tag{5.58}$$

By the analogous argument applied to (5.57), using $w_1\delta_1 < w_1\beta_1$,

$$\min(w_1\beta_1,\, w_2\beta_2) = w_2\beta_2, \qquad \text{so} \qquad w_2\beta_2 \leq w_1\beta_1. \tag{5.59}$$

Combining (5.58) and (5.59) yields $w_1\beta_1 = w_2\beta_2$. Substituting back into (5.56) gives $w_2\alpha_2 \geq w_1\beta_1 = w_2\beta_2$, i.e. $\alpha_2 \geq \beta_2$, contradicting (5.55).

The assumption that both disjuncts fail is therefore untenable, and at least one of $u(w, b, z) > u(w, a, z)$ or $u(w, c, z) > u(w, d, z)$ must hold. □

Remark 5.5.1. *The argument relies only on the strict inequalities $a_2 < b_2$ and $d_1 < c_1$ (together with $b = c$). Consequently the proposition generalises: whenever a common point $b = c$ strictly dominates a in one coordinate and d in the other, no single Chebyshev weight vector can simultaneously render both a preferable to b and d preferable to c.*

5.6 Summary

- This chapter introduced Contextual Zooming for Trees (CZT), a contextual multi-armed bandit algorithm that is used for decision making in multi-objective settings.
- The chapter also introduced Convex Hull Monte Carlo Tree Search (CHMCTS), which generalises the prior work in multi-objective MCTS to handle stochastic environments by using backups over Convex Hull Value Sets.
- The chapter also discussed the theoretical properties of the CZT and CHMCTS algorithms.

New Ch5 plan:

- Conclusion, wrap up, motivate move to simplex maps (convex hull slow) and weight space (entropy)

future work includes using the biasing terms from Ch4 in these algorithms too, currently result essentially set the psuedo visits to 0.

future work includes using more efficient weight space data structure (value curves) instead of convex hulls

From Van Moffaert paper: Lizotte et al. (2010) reduce the asymptotic space and time complexity of the bootstrapping rule by simultaneously learning several value functions corresponding to different weights and by calculating their piecewise linear spline representation. Recently, they validated their work on clinical trial data for three objectives, although the practical possibilities for higher dimensional spaces are not straightforward (Lizotte et al., 2012).

6

Simplex Maps for Multi-Objective Monte Carlo Tree Search

Contents

6.1 Introduction and Motivation 136
6.1.1 Limitations of CHMCTS and CZT 136
6.1.2 Motivation for Working in Weight Space 137
6.1.3 Adaptive Subdivisions of the Weight Space 138
6.2 Simplex Maps 139
6.2.1 Interface 139
6.2.2 Implementation 141
6.2.3 Complexity Discussion 145
6.3 Simplex Maps for Tree Search 147
6.4 Empirical Results 149
6.5 Summary 150

This chapter introduces *simplex maps*, an adaptive data structure for maintaining value and entropy estimates as a function of the weight vector $\mathbf{w} \in \Delta^D$ over the unit simplex. Simplex maps address two limitations of the algorithms considered in Chapter 5: the computational cost of maintaining convex hull value sets in stochastic environments, and the inability to naturally support entropy-based exploration (Chapter 4). By partitioning the weight space into a hierarchical and adaptive triangulation, simplex maps provide a more scalable representation: values can be looked up faster, the number of stored values grows only where the weight space

requires finer resolution, and per-vertex entropy estimates allow entropy-based exploration to be lifted to the multi-objective setting.

The remainder of this chapter is organised as follows:

- Section 6.1 motivates the simplex map approach, drawing on the empirical results of Chapter 5;
- Section 6.2 defines the simplex map data structure, describing its interface (Section 6.2.1), implementation (Section 6.2.2) and a brief discussion on the computational complexity of the operations (Section 6.2.3);
- Section 6.3 introduces MOMCTS algorithms using simplex maps as a value representation, generalising BTS and DENTS to the multi-objective setting;
- Section 6.4 presents empirical results using the same experimental setup as Chapter 5, with CZT and CHMCTS algorithms as baselines.

6.1 Introduction and Motivation

This section discusses some of the limitations of the algorithms introduced in Chapter 5, and motivates the use of simplex maps as a more scalable alternative that supports entropy-based exploration.

6.1.1 Limitations of CHMCTS and CZT

TODO: update stochastic DST to whatever we call it in results

The empirical results of Chapter 5 TODO: ref results section highlight a failure case for CHMCTS, where it is outperformed by CZT in the stochastic DST environment.

While convex hull value sets provide an accurate representation of achievable returns encountered during the search — which is beneficial in the exploration setting — they become computationally expensive in stochastic environments. The number of points in the intermediate convex hull value sets can grow rapidly, causing

Figure 6.1: A convex hull value set (left) and its corresponding value curve in weight space (right). Each point in objective space corresponds to a line in weight space. In bold the maximum linear utility $\max_{\mathbf{v}\in\mathcal{V}} \mathbf{w}^\top \mathbf{v}$ as a function of the weight vector $\mathbf{w} \in \Delta^D$ is shown, i.e. the *value curve*.

a significant increase in the computational cost of each backup and a corresponding reduction in the number of trials that can be run within a fixed time budget.

In contrast, CZT avoids this issue: by limiting the rate of ball additions via the $M_{\min}$ parameter, it can run a comparable number of trials per unit time in stochastic environments. Despite this advantage, CZT still exhibits the exploration limitations similar to those described in Chapter 4: it tends to exploit aggressively and does not explore sufficiently, in the same way that UCT can get stuck in local optima in the single-objective setting. As shown in Figure TODO: ref ch5 fig, while CZT outperforms CHMCTS algorithms on the stochastic DST environment, it still recommends suboptimal actions compared to CHVI.

CZT also suffers from the undesirable property that it will always add new balls to the cover, regardless of whether further refinement is warranted. This was addressed in practice by introducing a minimum visit threshold $M_{\min}$ before a new ball is created (Section 5.2), but this is an ad hoc fix rather than a principled solution.

6.1.2 Motivation for Working in Weight Space

Convex hull value sets operate in *objective space*, representing the set of achievable expected return vectors. An alternative perspective is to consider the *value curve*

in weight space: the function $\mathbf{w} \mapsto \max_{\mathbf{v} \in \boldsymbol{\mathcal{V}}} \mathbf{w}^\top \mathbf{v}$ that maps each weight vector to the maximum linear utility achievable under that weight. This value curve is a piecewise-linear, concave function over Δ^D, and is illustrated in Figure 6.1. Maintaining an estimate of this function directly over the weight space, rather than maintaining the full convex hull value set, suggests a more scalable approach.

Working in weight space has led to more scalable approaches in the prior literature. For example, TODO: citet lizotte2dim use piecewise linear splines as an alternative to convex hulls, yielding a more efficient value iteration algorithm, although this approach does not extend straightforwardly to higher dimensions. TODO: cite lizotteNdim

Following the argument made in Section **??**, working in objective space also does not naturally support entropy-based exploration, and moving to weight space provides a natural way to support entropy-based exploration.

While CZT does operate in weight space, each ball covers a region of the weight space and maintains a single value estimate for that entire region. As such, they cannot naturally store an entropy estimate associated with a single weight vector.

6.1.3 Adaptive Subdivisions of the Weight Space

This chapter investigates the use of an *adaptive subdivision* of the weight space as an alternative representation. Rather than covering the weight space with balls, the simplex Δ^D is triangulated into a set of simplices, each associated with a vertex at which a value estimate and entropy estimate are maintained. The triangulation is refined adaptively during the search, subdividing regions of the weight space where the value function varies across the region and leaving regions where the value is uniform intact.

Adaptive subdivisions have been used in a range of settings. *Hierarchical Optimistic Optimisation* (HOO) TODO: cite HOO is an algorithm for continuous-armed bandit problems and uses a tree of hierarchically nested sets to adaptively search a continuous domain, refining the cover in regions of high estimated value.

Unlike the approach of this chapter, HOO focuses on identifying a single region of high value rather than estimating the value function across the entire domain.

In numerical analysis, the finite element method [72] is a method for solving partial differential equations by discretising the domain into a mesh and solving the equation on each element. *Adaptive* finite element methods [39] alternate between solving a partial differential equation over a mesh and refining the mesh in regions where the solution is not yet well resolved, a paradigm widely used in fluid dynamics and structural mechanics. The simplex map data structure introduced in this chapter draws on similar principles, adapting the resolution of the weight space cover to the demands of the search.

A substantial body of work has studied adaptive mesh refinement strategies for triangulations. In particular, *longest edge bisection* (LEB) [44, 45] is a popular refinement strategy that splits the longest edge of a simplex into two equal length edges. It has been shown in two and three dimensions that iterative LEB of a unit simplex produces a finite set of similarity classes of simplices [48, 41, 2] TODO: fix cites. For higher dimensions the LEB refinement is conjectured to produce finitely many similarity classes, but a proof remains an open problem [52] TODO: fix cite.

6.2 Simplex Maps

This section defines the *simplex map* data structure. Section 6.2.1 describes the interface that a simplex map provides, Section 6.2.2 describes the implementation, and Section 6.2.3 discusses the complexity of simplex maps operations compared to CZT and CHMCTS.

6.2.1 Interface

A simplex map $\boldsymbol{\mathcal{G}}_{\text{SM}} = (\boldsymbol{\mathcal{V}}_{\text{SM}}, \boldsymbol{\mathcal{E}}_{\text{SM}}, \boldsymbol{\mathcal{T}}_{\text{SM}}, f_{\text{SM}})$ consists of a set of vertices $\boldsymbol{\mathcal{V}}_{\text{SM}} \subseteq \Delta^D$, a set of undirected edges $\boldsymbol{\mathcal{E}}_{\text{SM}} \subseteq \boldsymbol{\mathcal{V}}_{\text{SM}} \times \boldsymbol{\mathcal{V}}_{\text{SM}}$, a binary tree of simplices $\boldsymbol{\mathcal{T}}_{\text{SM}}$ and a function f_{SM} mapping vertices to value estimates. The root node of $\boldsymbol{\mathcal{T}}_{\text{SM}}$ is the unit simplex and the tree is formed using longest edge bisections so that the leaf

Figure 6.2: Illustration of a simplex map $\mathcal{G}_{\text{SM}} = (\mathcal{V}_{\text{SM}}, \mathcal{E}_{\text{SM}}, \mathcal{T}_{\text{SM}}, f_{\text{SM}})$ with $D = 3$. (Left) The graph of vertices $(\mathcal{V}_{\text{SM}}, \mathcal{E}_{\text{SM}})$ is shown. (Right) The corresponding binary tree of simplices $\mathcal{T}_{\text{SM}}$.

nodes of the tree form a triangulation of the unit simplex. Figure 6.2 illustrates a simplex map with $D = 3$.

`find_containing_simplex`$(\mathcal{G}_{\text{SM}}, \mathbf{w})$**:** Given a weight vector $\mathbf{w} \in \Delta^D$, returns the leaf node in the binary tree of simplices $\mathcal{T}_{\text{SM}}$ that contains $\mathbf{w}$.

`find_closest_vertex`$(\mathcal{G}_{\text{SM}}, \mathbf{w})$**:** Given a weight vector $\mathbf{w} \in \Delta^D$, finds the (approximately) closest vertex in the simplex map $\mathbf{w}_{\text{SM}} \in \mathcal{V}_{\text{SM}}$ and returns the corresponding value estimate $f_{\text{SM}}(\mathbf{w}_{\text{SM}})$.

`push`$(\mathcal{G}_{\text{SM}}, \mathbf{w}_{\text{SM}})$**:** Given a vertex $\mathbf{w}_{\text{SM}} \in \mathcal{V}_{\text{SM}}$, attempts to share its value estimate with its neighbours in the vertex graph via breadth-first search. A neighbour $\mathbf{w}'_{\text{SM}} \in \mathcal{V}_{\text{SM}}$ accepts the value from $\mathbf{w}_{\text{SM}}$ if it is an improvement under $\mathbf{w}'_{\text{SM}}$'s own weight vector:

$$f_{\text{SM}}(\mathbf{w}'_{\text{SM}}) \leftarrow f_{\text{SM}}(\mathbf{w}_{\text{SM}}) \quad \text{if } {\mathbf{w}'_{\text{SM}}}^{\top} f_{\text{SM}}(\mathbf{w}_{\text{SM}}) > {\mathbf{w}'_{\text{SM}}}^{\top} f_{\text{SM}}(\mathbf{w}'_{\text{SM}}). \tag{6.1}$$

If the value estimate at $\mathbf{w}'_{\text{SM}}$ is updated, then the breadth-first search is continued from $\mathbf{w}'_{\text{SM}}$, up to a maximum push radius $\texttt{push_radius}_{\text{SM}}$.

`maybe_split`$(\mathcal{G}_{\text{SM}}, \mathcal{S}_{\text{SM}})$**:** Given a leaf simplex $\mathcal{S}_{\text{SM}} \in \mathcal{T}_{\text{SM}}$, performs a longest edge bisection and adds the two new simplices in the tree.

This interface is sufficient to define MOMCTS algorithms using simplex maps. In particular, if the value estimates f_{SM} of a simplex map $\boldsymbol{\mathcal{G}}_{\text{SM}}$ represent multi-objective Q-values at a chance node (s, a), then the following can be used as a contextual value estimate:

$$\hat{Q}^{\mathbf{w}}_{\text{SM}}(s, a) = \mathbf{w}^\top f_{\text{SM}}(\mathbf{w}_{\text{SM}}) \quad \text{where } \mathbf{w}_{\text{SM}} = \texttt{find_closest_vertex}(\boldsymbol{\mathcal{G}}_{\text{SM}}, \mathbf{w}). \tag{6.2}$$

6.2.2 Implementation

Further details on the implementation of simplex maps are given in this section. Some implementation details are simpler in two dimensions than in the general D-dimensional case; $D = 3$ is used for examples throughout.

Conforming triangulations are described first, followed by the initialisation and implementation of each operation.

An additional auxiliary structure, called the *bipartite triangulation graph* (BTG), is used in the implementation but not defined explicitly to avoid unnecessary complexity. The BTG consists of edges of $\boldsymbol{\mathcal{E}}_{\text{SM}}$ on one side and the leaf simplices of $\boldsymbol{\mathcal{T}}_{\text{SM}}$ on the other. The BTG is used to determine adjacency between simplices and to support the conformity checks described below.

Throughout this thesis, Δ^D refers to the D-dimensional unit simplex with D vertices (Definition TODO: ref). In geometry it is more common to refer to this object as the $(D-1)$-*simplex*, since it forms a $(D-1)$-dimensional space in $\mathbb{R}^D$. For example, when $D = 3$ the simplex is a triangle, which is geometrically a 2-simplex. As this section discusses some geometric concepts, the correct geometric terminology is used here.

Conforming Triangulations. Two simplices are *adjacent* if they share at least one point. A pair of simplices are said to be *conforming* if the intersection of the two simplices is exactly a point, edge or face of *both* simplices. A triangulation is *conforming* if every pair of adjacent simplices is conforming. Figure 6.3 illustrates a conforming and non-conforming triangulation.

Figure 6.3: (Left) A conforming triangulation of the unit simplex. (Right) A non-conforming triangulation of the unit simplex, with the non-conforming simplex highlighted in red.

It has been shown that recursive longest edge bisection, in a general D-dimensions, produces a conforming triangulation within a finite number of steps [43, 44, 45].

The implementation of the `find_closest_vertex` operation assumes that the nearest vertex to any point $\mathbf{w}$ lies either in the simplex containing $\mathbf{w}$ or in an adjacent simplex. This assumption is supported by the conjectured result that LEB refinement produces a finite number of similarity classes in higher dimensions [52].

The BTG can be used to determine if a simplex in $\boldsymbol{\mathcal{T}}_{\text{SM}}$ is conforming by checking if the number of connections in the BTG is equal to the number of edges of a simplex. For example, in Figure 6.3, the non-conforming simplex is highlighted in red. It has 3 edges, but is adjacent to 4 edges of $\boldsymbol{\mathcal{E}}_{\text{SM}}$ of the vertex graph.

The simplex map is implemented to be *eventually conforming*, meaning that a non-conforming triangulation is tolerated temporarily, and after every operation described in Section 6.2.1, a non-conforming simplex is identified and the `maybe_split` operation is performed on it.

Initialisation. The simplex map is initialised with the unit simplex Δ^D, whose D vertices are the standard basis vectors $\mathbf{e}_1, \ldots, \mathbf{e}_D \in \mathbb{R}^D$. The edges are initialised

to connect every pair of vertices, and the binary tree is initialised to a single root node containing the unit simplex. The value estimates are initialised to zero.

`find_containing_simplex`($\boldsymbol{\mathcal{G}}_{\text{SM}}, \mathbf{w}$): To find the simplex in the triangulation that contains a weight vector $\mathbf{w} \in \Delta^D$, the triangulation is traversed by the binary tree $\boldsymbol{\mathcal{T}}_{\text{SM}}$ using a sequence of halfspace checks at each internal node.

`find_closest_vertex`($\boldsymbol{\mathcal{G}}_{\text{SM}}, \mathbf{w}$): In the case of $D = 2$, the closest vertex can be found exactly by returning the closest endpoint of the line segment (1-simplex) containing $\mathbf{w}$.

For the general case of $D > 2$, the closest vertex can be found approximately. First the simplex containing $\mathbf{w}$ is found using `find_containing_simplex`. Then the *k-ring* (k-step neighbourhood) of simplices is checked for the closest vertex to $\mathbf{w}$.

k is technically a parameter used in the simplex map implementation, but it is always set to 1 in this thesis. The assumption is made that the nearest vertex to $\mathbf{w}$ is either on the simplex containing $\mathbf{w}$ or an adjacent simplex. This assumption is supported by the conjecture that LEB refinement produces finitely many similarity classes [52], which would enforce that the diameter of adjacent simplices is bounded, and ensure that the nearest vertex cannot lie far from the containing simplex.

Figure 6.4 illustrates that for $D = 3$, setting $k = 0$ is insufficient and that a neighbouring triangle may contain the closest vertex to $\mathbf{w}$. Also illustrated for $D = 3$ is that $k = 1$ is insufficient to find the closest vertex if the triangulation is non-conforming.

`maybe_split`($\boldsymbol{\mathcal{G}}_{\text{SM}}, \boldsymbol{\mathcal{S}}_{\text{SM}}$): The `maybe_split` operation is controlled by three parameters $\texttt{split_min_radius}_{\text{SM}}$,$\texttt{split_min_delta}_{\text{SM}}$ and $\texttt{split_min_visit}_{\text{SM}}$, which determine the conditions under which a simplex is split. Let $\texttt{N}_{\text{SM}}(\boldsymbol{\mathcal{S}}_{\text{SM}})$ be a counter for the number of times that `maybe_split` has been called on the simplex $\boldsymbol{\mathcal{S}}_{\text{SM}}$.

The following steps are performed to determine if the simplex should be split:

- If the simplex $\boldsymbol{\mathcal{S}}_{\text{SM}}$ is non-conforming, then the simplex is split as described below;

Figure 6.4: Closest vertex lookup examples. In all subfigures, the weight vector $\mathbf{w}$ is shown in red, and the closest vertex in the triangulation is shown in blue. (a) Finding the closest vertex in $D = 2$. (b) Example where the closest vertex is in a neighbouring simplex. (c) Example where the triangulation is non-conforming and the closest vertex is not in a neighbouring simplex.

Figure 6.5: Example of the `maybe_split` operation of a simplex map $\mathcal{G}_{\mathrm{SM}}$. The longest edge of the simplex is bisected and the two new simplices are added to the tree. The green edges are added to $\mathcal{E}_{\mathrm{SM}}$ and the red edge is removed. (Left) The vertex graph before the split. (Right) The vertex graph after the split.

- If the radius of the simplex $\boldsymbol{\mathcal{S}}_{\text{SM}}$ is less than $\texttt{split_min_radius}_{\text{SM}}$, then the simplex is not split;
- If $\max_{\mathbf{w},\mathbf{w}' \in \boldsymbol{\mathcal{S}}_{\text{SM}}} \|f_{\text{SM}}(\mathbf{w}) - f_{\text{SM}}(\mathbf{w}')\| < \texttt{split_min_delta}_{\text{SM}}$, then the counter $\texttt{N}_{\text{SM}}(\boldsymbol{\mathcal{S}}_{\text{SM}})$ is reset to 0, and the simplex is not split;
- $\texttt{N}_{\text{SM}}(\boldsymbol{\mathcal{S}}_{\text{SM}})$ is incremented by 1, and if $\texttt{N}_{\text{SM}}(\boldsymbol{\mathcal{S}}_{\text{SM}}) < \texttt{split_min_visit}_{\text{SM}}$, then the simplex is not split;
- Otherwise, the simplex is split as described below.

Let $\boldsymbol{\mathcal{S}}_{\text{SM}} = \{\mathbf{w}^{(1)}_{\text{SM}}, \mathbf{w}^{(2)}_{\text{SM}}, \mathbf{w}^{(3)}_{\text{SM}}, \ldots, \mathbf{w}^{(D)}_{\text{SM}}\}$, where without loss of generality $(\mathbf{w}^{(1)}_{\text{SM}}, \mathbf{w}^{(2)}_{\text{SM}})$ is the longest edge. The new vertex $\mathbf{w}^{(\text{mid})}_{\text{SM}} = \frac{1}{2}(\mathbf{w}^{(1)}_{\text{SM}} + \mathbf{w}^{(2)}_{\text{SM}})$ is added to $\boldsymbol{\mathcal{V}}_{\text{SM}}$, the edge $(\mathbf{w}^{(1)}_{\text{SM}}, \mathbf{w}^{(2)}_{\text{SM}})$ is removed from $\boldsymbol{\mathcal{E}}_{\text{SM}}$ and edges to connect the new vertex are added:

$$\boldsymbol{\mathcal{E}}_{\text{SM}} \leftarrow \left(\boldsymbol{\mathcal{E}}_{\text{SM}} - \{(\mathbf{w}^{(1)}_{\text{SM}}, \mathbf{w}^{(2)}_{\text{SM}})\}\right) \cup \{(\mathbf{w}_{\text{SM}}, \mathbf{w}^{(\text{mid})}_{\text{SM}}) \mid \mathbf{w}_{\text{SM}} \in \boldsymbol{\mathcal{S}}_{\text{SM}}\}. \tag{6.3}$$

Figure 6.5 illustrates this operation. Finally, the new vertex $\mathbf{w}^{(\text{mid})}_{\text{SM}}$ inherits its value estimate from the values of the edge that was split:

$$f_{\text{SM}}(\mathbf{w}^{(\text{mid})}_{\text{SM}}) \leftarrow \begin{cases} f_{\text{SM}}(\mathbf{w}^{(1)}_{\text{SM}}) & \text{if } \mathbf{w}^{(\text{mid})\top}_{\text{SM}} f_{\text{SM}}(\mathbf{w}^{(1)}_{\text{SM}}) > \mathbf{w}^{(\text{mid})\top}_{\text{SM}} f_{\text{SM}}(\mathbf{w}^{(2)}_{\text{SM}}), \\ f_{\text{SM}}(\mathbf{w}^{(2)}_{\text{SM}}) & \text{otherwise.} \end{cases} \tag{6.4}$$

The BTG is updated accordingly; the details are omitted as they do not affect the algorithm semantics.

`push:` The descriptions of `update` and `push` given in Section 6.2.1 are sufficient to implement these operations. `push` is illustrated in Figure 6.6. The function f_{SM} is implemented as a hash map from vertices to value estimates.

6.2.3 Complexity Discussion

The computational cost of simplex map operations is briefly compared to those of CHMCTS and CZT as follows.

For CHMCTS, using K_{CH} to denote the number of points in a convex hull value set, taking the union of two value sets during the $\hat{\boldsymbol{\mathcal{V}}}_{\text{ALG}}$ backup (Equation 5.16) costs

Figure 6.6: Illustration of the `push` operation of a simplex map, where the red vertex is pushing its values estimate, the red arrows indicate the direction of value propagation and the blue vertex does not accept or propagate the value.

$\Omega(K_{\text{CH}})$. However, computing the $\hat{\mathbf{Q}}_{\text{ALG}}$ backup (Equation 5.17) requires adding two value sets, which involves computing a Minkowski sum of $\Omega(K_{\text{CH}}^2)$ candidate points before pruning. This provides intuition for why CHMCTS becomes significantly slower in stochastic environments, where intermediate value sets grow large.

For CZT, the cost of selection scales with the number of active balls via Equation (5.10), which grows unboundedly over time.

For simplex maps, under the assumption that LEB refinement produces finitely many similarity classes, the maximum number of edges incident to any vertex is $O(2^D)$, and the BTG has a similar bound on its degree. Simplex lookups via the binary tree cost roughly $O(D \log(K_{\text{SM}}))$, where K_{SM} is the number of vertices and the factor of D comes from the halfspace checks at each internal node.

Therefore, for an MOMDP where the number of achievable values grows large, i.e. $K_{\text{CH}} \gg 2^D$, the computational complexity of CHMCTS will dominate the complexity of simplex map operations. Conversely, when an MOMDP has a small number of achievable values, simplex maps may introduce additional computational overhead over CHMCTS, as the order of vertices in the simplex map will still be $\Omega(2^D)$.

6.3 Simplex Maps for Tree Search

This section defines the DENTS with Simplex Maps (SM-DENTS) algorithm. SM-DENTS is a variant of DENTS that uses simplex maps to maintain the value and entropy estimates. The algorithm uses parameters $\alpha_{\text{SM-DNTS}}$, $\beta_{\text{SM-DNTS}}$, and corresponding to the DENTS parameters. Throughout this section $\mathbf{w}$ is the randomly sampled context weight for a trial.

Each decision node s and chance node (s, a) keeps an associated simplex map

$$\boldsymbol{\mathcal{G}}_{\text{SM}}(s) = (\boldsymbol{\mathcal{V}}_{\text{SM}}(s), \boldsymbol{\mathcal{E}}_{\text{SM}}(s), \boldsymbol{\mathcal{T}}_{\text{SM}}(s), \hat{\mathbf{V}}_{\text{SM-DNTS}}(s;\cdot), \bar{\mathcal{H}}_{V,\text{SM-DNTS}}(s;\cdot)) \tag{6.5}$$

$$\boldsymbol{\mathcal{G}}_{\text{SM}}(s, a) = (\boldsymbol{\mathcal{V}}_{\text{SM}}(s, a), \boldsymbol{\mathcal{E}}_{\text{SM}}(s, a), \boldsymbol{\mathcal{T}}_{\text{SM}}(s, a), \hat{\mathbf{Q}}_{\text{SM-DNTS}}(s, a;\cdot), \bar{\mathcal{H}}_{Q,\text{SM-DNTS}}(s, a;\cdot)), \tag{6.6}$$

noting that $\hat{\mathbf{V}}_{\text{SM-DNTS}}$ and $\hat{\mathbf{Q}}_{\text{SM-DNTS}}$ are the value and Q-value estimate functions, and $\bar{\mathcal{H}}_{V,\text{SM-DNTS}}(s)$ and $\bar{\mathcal{H}}_{Q,\text{SM-DNTS}}(s, a)$ are additional entropy value estimate functions, maintained in each simplex map. Contextual value estimates are computed using the simplex maps, as outlined in Equation (6.2):

$$\mathbf{w}_V = \texttt{find_closest_vertex}(\boldsymbol{\mathcal{G}}_{\text{SM}}(s), \mathbf{w}), \tag{6.7}$$

$$\mathbf{w}_Q = \texttt{find_closest_vertex}(\boldsymbol{\mathcal{G}}_{\text{SM}}(s, a), \mathbf{w}), \tag{6.8}$$

$$\hat{V}^{\mathbf{w}}_{\text{SM-DNTS}}(s) = \mathbf{w}^\top \hat{\mathbf{V}}_{\text{SM-DNTS}}(s; \mathbf{w}_V), \tag{6.9}$$

$$\hat{Q}^{\mathbf{w}}_{\text{SM-DNTS}}(s, a) = \mathbf{w}^\top \hat{\mathbf{Q}}_{\text{SM-DNTS}}(s, a; \mathbf{w}_Q), \tag{6.10}$$

Search Policy The search policy is defined similarly to DENTS (Section TODO: ref), using the contextual value and entropy estimates:

$$\hat{Q}^{\mathbf{w},\text{sft}}_{\text{SM-DNTS}}(s, a) = \hat{Q}^{\mathbf{w}}_{\text{SM-DNTS}}(s, a) + \beta_{\text{SM-DNTS}}(N(s))\bar{\mathcal{H}}_{Q,\text{SM-DNTS}}(s, a; \mathbf{w}), \tag{6.11}$$

$$\pi_{\text{SM-DNTS}}(a|s; \mathbf{w}) = (1 - \lambda(s, \epsilon_{\text{SM-DNTS}}))\rho_{\text{SM-DNTS}}(a|s; \mathbf{w}) + \frac{\lambda(s, \epsilon_{\text{SM-DNTS}})}{|\mathcal{A}|}, \tag{6.12}$$

$$\rho_{\text{SM-DNTS}}(a|s; \mathbf{w}) \propto \exp\left(\frac{\hat{Q}^{\mathbf{w},\text{sft}}_{\text{SM-DNTS}}(s, a)}{\alpha_{\text{SM-DNTS}}(N(s))}\right), \tag{6.13}$$

$$\lambda(s, \epsilon) = \min\left(1, \frac{\epsilon}{\log(e + N(s))}\right). \tag{6.14}$$

Backups Given a trajectory $\tau = (s_0, a_0, \mathbf{r}_0, \ldots, s_{h-1}, a_{h-1}, \mathbf{r}_{h-1}, s_h) \sim \pi_{\text{SM-DNTS}}$, the `backup_q` and `backup_v` functions update the multi-objective and entropy value estimates for $t = h-1, \ldots, 0$:

$$\mathbf{w}_Q = \texttt{find_closest_vertex}(\boldsymbol{\mathcal{G}}_{\text{SM}}(s,a), \mathbf{w}), \tag{6.15}$$

$$\mathbf{w}_V = \texttt{find_closest_vertex}(\boldsymbol{\mathcal{G}}_{\text{SM}}(s), \mathbf{w}), \tag{6.16}$$

$$\bar{\mathcal{H}}_{Q,\text{SM-DNTS}}(s_t, a_t; \mathbf{w}_Q) \leftarrow \sum_{s' \in \text{Succ}(s_t,a_t)} \frac{N(s')}{N(s_t,a_t)} \bar{\mathcal{H}}_{V,\text{SM-DNTS}}(s'; \mathbf{w}_Q), \tag{6.17}$$

$$\begin{aligned} \bar{\mathcal{H}}_{V,\text{SM-DNTS}}(s_t; \mathbf{w}_V) &\leftarrow \mathcal{H}(\pi_{\text{SM-DNTS}}(\cdot|s_t; \mathbf{w}_V)) \\ &\quad + \sum_{a \in \mathcal{A}} \pi_{\text{SM-DNTS}}(a|s_t; \mathbf{w}_V) \bar{\mathcal{H}}_{Q,\text{SM-DNTS}}(s_t, a; \mathbf{w}_Q), \end{aligned} \tag{6.18}$$

$$\begin{aligned} \hat{\mathbf{Q}}_{\text{SM-DNTS}}(s_t, a_t; \mathbf{w}_Q) &\leftarrow \mathbf{R}(s_t, a_t) \\ &\quad + \sum_{s' \in \text{Succ}(s_t,a_t)} \left(\frac{N(s')}{N(s_t,a_t)} \hat{\mathbf{V}}_{\text{SM-DNTS}}(s'; \mathbf{w}_Q) \right), \end{aligned} \tag{6.19}$$

$$\hat{\mathbf{V}}_{\text{SM-DNTS}}(s_t; \mathbf{w}_V) \leftarrow \hat{\mathbf{Q}}_{\text{SM-DNTS}} \left(s_t, \arg\max_{a \in \mathcal{A}} \hat{Q}^{(\mathbf{w}_V)}_{\text{SM-DNTS}}(s_t, a); \mathbf{w}_V \right). \tag{6.20}$$

There are two significant differences of note in the backups of SM-DENTS compared to its single-objective counterpart DENTS and multi-objective counterpart in CH-BTS:

- The value estimates are updated with respect to the closest vertices $\mathbf{w}_Q$ and $\mathbf{w}_V$ looked up from the simplex maps of $\boldsymbol{\mathcal{G}}_{\text{SM}}(s,a)$ and $\boldsymbol{\mathcal{G}}_{\text{SM}}(s)$ from Equations 6.15 and 6.16.

- Since a maximum is not well-defined over vectors, Equation (6.20) computes the action that leads to the maximum contextual Q-value for $\mathbf{w}_V$, which is then used to update the value estimate.

After the Q-values are updated from Equations (6.17) and (6.19), a call is made to the `maybe_split` operation to split a simplex in the simplex map if necessary. Specifically,

$$\texttt{maybe_split}(\boldsymbol{\mathcal{G}}_{\text{SM}}(s_t, a_t), \texttt{find_containing_simplex}(\boldsymbol{\mathcal{G}}_{\text{SM}}(s_t, a_t), \mathbf{w}_Q)). \tag{6.21}$$

Similarly, after the value estimates are updated from Equations (6.18) and (6.20), the following call is made:

$$\texttt{maybe_split}(\boldsymbol{\mathcal{G}}_{\text{SM}}(s_t), \texttt{find_containing_simplex}(\boldsymbol{\mathcal{G}}_{\text{SM}}(s_t), \mathbf{w}_V)). \tag{6.22}$$

Recommendation Policy The recommendation policy is defined to follow the maximal contextual Q-values:

$$\psi_{\text{SM-DNTS}}(s; \mathbf{w}) = \arg\max_{a \in \mathcal{A}} \hat{Q}^{\mathbf{w}}_{\text{SM-DNTS}}(s, a). \tag{6.23}$$

BTS with Simplex Maps BTS with Simplex Maps (SM-BTS) is defined by setting the entropy coefficient function $\beta_{\text{SM-DNTS}}(x) = 0$.

6.4 Empirical Results

TODO: Clean up plots, legend placement etc.

TODO: Refer to ch5 results for evaluation procedure and environments. State that picking CZT, CHBTS (and maybe CHUCT) as baselines for this ch. And then give results for SMBTS and SMDENTS

New Ch6 plan:

- Empirical results
- Theory, contextual regret, non-contextual policies garunteed to have a linear contextual regret
- Maybe some theory, probably bootstrap of the ch4 + ch5 results
- Scalability on DST
- Scalability test with respect to dimension?

TODO: list

- Results from MO-Gymnasium
- Compare algorithms using expected utility

6.5 Summary

TODO: Summary of contributions

BD: For future work, some further approximations, such as subsampling the neighbours in the message passing could be considered.

BD: For future work, include bias terms similar to the ones in ch4. Say that we didn't include them in Ch6, and they were not added in ch5 so that ch5 and ch6 results are comparible.

7
Conclusion

Contents

7.1 Summary of Contributions **151**
7.2 Future Work **151**

TODO: Something about we'll conclude by looking back at contributions and possible future work.

7.1 Summary of Contributions

TODO: go through each of the research questions and contributions, and write about how the work answers the research questions

7.2 Future Work

TODO: outline some avenues of potential future work

- Decaying TENTS, the idea can be applied directly to Tsallis entropy
- Applying DENTS logic to deep learning
- Applying BTS/DENTS in alpha zero style loops

- value curves instead of convex hulls for where convex hull is better than simplex map
- adding entropy as an additional objective in simplex maps work
- multi-objective alpha zero (simplex map/convex hull), cite some of the work about catastrophic forgetting where learned policy $pi(a|s;w)$ degrades to the same policy for many weights w, maybe ensuring that the policy is updated for many weights w in the loss could help with this

TODO: Also mention the people that have used/cited DENTS work

Appendices

A

Additional Algorithm Details For Chapter 4

This appendix gives the full equations for AR-BTS and AR-DENTS for reference. Each algorithm is specified minimally using the `THTS++` schema given in Section 2.4.2.

A.1 AR-BTS

Value Estimates and Parameters.

- $\bar{V}_{\text{AR-BTS}}(s)$ - The value estimate for state s;
- $\bar{Q}_{\text{AR-BTS}}(s, a)$ - The Q-value estimate for state s and action a;
- $N(s)$ - The visit count for state s;
- $N(s, a)$ - The visit count for state s and action a;
- $\alpha_{\text{AR-BTS}}(x) > 0$ - The search temperature function;
- $\epsilon_{\text{AR-BTS}} \in [0, \infty)$ - The uniform exploration parameter;
- `normalised_Q` - A flag indicating whether the Q-values should be normalised when used in the search policy;
- $N_{\text{AR-BTS}}^{\hat{V}_{\text{heuristic}}}$ - The number of pseudo-visits to use for biasing the backups;

- $N_{\text{AR-BTS}}^{\hat{Q}_{\text{heuristic}}}$ - The number of pseudo-visits to use for biasing the Q-values;
- $\hat{V}_{\text{heuristic}}(s)$ - The heuristic value estimate for state s;
- $\hat{Q}_{\text{heuristic}}(s,a)$ - The heuristic Q-value estimate for state s and action a;

Initialisation.

$$N(s) \leftarrow 0, \tag{A.1}$$

$$N(s,a) \leftarrow 0, \tag{A.2}$$

$$\bar{V}_{\text{AR-BTS}}(s) \leftarrow 0, \tag{A.3}$$

$$\bar{Q}_{\text{AR-BTS}}(s,a) \leftarrow 0. \tag{A.4}$$

Search Policy. Intermediate Q-values:

$$\bar{Q}_{\text{AR-BTS}}^{\text{bias}}(s,a) = \frac{N(s,a)\bar{Q}_{\text{AR-BTS}}(s,a) + N_{\text{AR-BTS}}^{\hat{Q}_{\text{heuristic}}}\hat{Q}_{\text{heuristic}}(s,a)}{N(s,a) + N_{\text{AR-BTS}}^{\hat{Q}_{\text{heuristic}}}}, \tag{A.5}$$

$$\hat{Q}_{\text{BTS}}^{\text{norm}}(s,a) = \begin{cases} \frac{\bar{Q}_{\text{AR-BTS}}^{\text{bias}}(s,a) - \min_{a\in\mathcal{A}} \bar{Q}_{\text{AR-BTS}}^{\text{bias}}(s,a)}{\max_{a\in\mathcal{A}} \bar{Q}_{\text{AR-BTS}}^{\text{bias}}(s,a) - \min_{a\in\mathcal{A}} \bar{Q}_{\text{AR-BTS}}^{\text{bias}}(s,a)} & \text{if } \texttt{normalised_Q}, \\ \bar{Q}_{\text{AR-BTS}}^{\text{bias}}(s,a) & \text{otherwise.} \end{cases} \tag{A.6}$$

Search policy:

$$\pi_{\text{AR-BTS}}(a|s) = (1 - \lambda(s, \epsilon_{\text{AR-BTS}}))\rho_{\text{AR-BTS}}(a|s) + \frac{\lambda(s, \epsilon_{\text{AR-BTS}})}{|\mathcal{A}|}, \tag{A.7}$$

$$\rho_{\text{AR-BTS}}(a|s) \propto \exp\left(\frac{\bar{Q}_{\text{AR-BTS}}^{\text{norm}}(s,a)}{\alpha_{\text{AR-BTS}}(N(s))}\right), \tag{A.8}$$

$$\lambda(s, \epsilon_{\text{AR-BTS}}) = \min\left(1, \frac{\epsilon_{\text{AR-BTS}}}{\log(e + N(s))}\right). \tag{A.9}$$

Backups. Let $\tau_{:h} = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h) \sim \pi_{\text{AR-BTS}}$:

$$\bar{Q}_{\text{AR-BTS}}(s_t, a_t) \leftarrow R(s_t, a_t) + \sum_{s' \in \text{Succ}(s_t, a_t)} \left(\frac{N(s')}{N(s_t, a_t)} \bar{V}_{\text{AR-BTS}}(s')\right), \tag{A.10}$$

$$\bar{V}_{\text{AR-BTS}}(s_t) \leftarrow \frac{N_{\text{AR-BTS}}^{\hat{V}_{\text{heuristic}}}\hat{V}_{\text{heuristic}}(s_t)}{N(s_t) + N_{\text{AR-BTS}}^{\hat{V}_{\text{heuristic}}}} + \sum_{a\in\mathcal{A}} \frac{N(s_t, a)}{N(s_t) + N_{\text{AR-BTS}}^{\hat{V}_{\text{heuristic}}}} \bar{Q}_{\text{AR-BTS}}(s_t, a). \tag{A.11}$$

Recommendation Policy.

$$\psi_{\text{AR-BTS}}(s) = \arg\max_{a \in \mathcal{A}} \bar{Q}_{\text{AR-BTS}}(s, a), \tag{A.12}$$

$$\texttt{mv}_{\text{AR-BTS}}(s) = \arg\max_{a \in \mathcal{A}} N(s, a). \tag{A.13}$$

A.2 AR-DENTS

Value Estimates and Parameters.

- $\bar{V}_{\text{AR-DNTS}}(s)$ - The value estimate for state s;
- $\bar{Q}_{\text{AR-DNTS}}(s, a)$ - The Q-value estimate for state s and action a;
- $N(s)$ - The visit count for state s;
- $N(s, a)$ - The visit count for state s and action a;
- $\alpha_{\text{AR-DNTS}}(x) > 0$ - The search temperature function;
- $\epsilon_{\text{AR-DNTS}} \in [0, \infty)$ - The uniform exploration parameter;
- `normalised_Q` - A flag indicating whether the Q-values should be normalised when used in the search policy;
- $N_{\text{AR-DNTS}}^{\hat{V}_{\text{heuristic}}}$ - The number of pseudo-visits to use for biasing the backups;
- $N_{\text{AR-DNTS}}^{\hat{Q}_{\text{heuristic}}}$ - The number of pseudo-visits to use for biasing the Q-values;
- $\hat{V}_{\text{heuristic}}(s)$ - The heuristic value estimate for state s;
- $\hat{Q}_{\text{heuristic}}(s, a)$ - The heuristic Q-value estimate for state s and action a;

Initialisation.

$$N(s) \leftarrow 0, \tag{A.14}$$

$$N(s, a) \leftarrow 0, \tag{A.15}$$

$$\bar{V}_{\text{AR-DNTS}}(s) \leftarrow 0, \tag{A.16}$$

$$\bar{Q}_{\text{AR-DNTS}}(s, a) \leftarrow 0. \tag{A.17}$$

Search Policy. Intermediate Q-values:

$$\hat{Q}^{\text{sft}}_{\text{AR-DNTS}}(s,a) = \bar{Q}_{\text{AR-DNTS}}(s,a) + \beta_{\text{AR-DNTS}}(N(s))\bar{\mathcal{H}}_{Q,\text{AR-DNTS}}(s,a), \tag{A.18}$$

$$\hat{Q}^{\text{sft,bias}}_{\text{AR-DNTS}}(s,a) = \frac{N(s,a)\hat{Q}^{\text{sft}}_{\text{AR-DNTS}}(s,a) + N^{\hat{Q}_{\text{heuristic}}}_{\text{AR-DNTS}}\hat{Q}_{\text{heuristic}}(s,a)}{N(s,a) + N^{\hat{Q}_{\text{heuristic}}}_{\text{AR-DNTS}}}, \tag{A.19}$$

$$\hat{Q}^{\text{sft,norm}}_{\text{AR-DNTS}}(s,a) = \begin{cases} \frac{\hat{Q}^{\text{sft,bias}}_{\text{AR-DNTS}}(s,a) - \min_{a\in\mathcal{A}} \hat{Q}^{\text{sft,bias}}_{\text{AR-DNTS}}(s,a)}{\max_{a\in\mathcal{A}} \hat{Q}^{\text{sft,bias}}_{\text{AR-DNTS}}(s,a) - \min_{a\in\mathcal{A}} \hat{Q}^{\text{sft,bias}}_{\text{AR-DNTS}}(s,a)} & \text{if } \texttt{normalised_Q}, \\ \hat{Q}^{\text{sft,bias}}_{\text{AR-DNTS}}(s,a) & \text{otherwise.} \end{cases} \tag{A.20}$$

Search policy:

$$\pi_{\text{AR-DNTS}}(a|s) = (1 - \lambda(s, \epsilon_{\text{AR-DNTS}}))\rho_{\text{AR-DNTS}}(a|s) + \frac{\lambda(s, \epsilon_{\text{AR-DNTS}})}{|\mathcal{A}|}, \tag{A.21}$$

$$\rho_{\text{AR-DNTS}}(a|s) \propto \exp\left(\frac{\hat{Q}^{\text{sft,norm}}_{\text{AR-DNTS}}(s,a)}{\alpha_{\text{AR-DNTS}}(N(s))}\right), \tag{A.22}$$

$$\lambda(s, \epsilon_{\text{AR-DNTS}}) = \min\left(1, \frac{\epsilon_{\text{AR-DNTS}}}{\log(e + N(s))}\right). \tag{A.23}$$

Backups. Let $\tau_{:h} = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h) \sim \pi_{\text{AR-DNTS}}$:

$$\bar{\mathcal{H}}_{Q,\text{AR-DNTS}}(s_t, a_t) \leftarrow \sum_{s'\in\text{Succ}(s_t,a_t)} \frac{N(s')}{N(s_t,a_t)} \bar{\mathcal{H}}_{V,\text{AR-DNTS}}(s'), \tag{A.24}$$

$$\bar{\mathcal{H}}_{V,\text{AR-DNTS}}(s_t) \leftarrow \mathcal{H}(\pi_{\text{AR-DNTS}}(\cdot|s_t)) + \sum_{a\in\mathcal{A}} \pi_{\text{AR-DNTS}}(a|s_t)\bar{\mathcal{H}}_{Q,\text{AR-DNTS}}(s_t,a), \tag{A.25}$$

$$\bar{Q}_{\text{AR-DNTS}}(s_t, a_t) \leftarrow R(s_t, a_t) + \sum_{s'\in\text{Succ}(s_t,a_t)} \left(\frac{N(s')}{N(s_t,a_t)} \bar{V}_{\text{AR-DNTS}}(s')\right), \tag{A.26}$$

$$\bar{V}_{\text{AR-DNTS}}(s_t) \leftarrow \frac{N^{\hat{V}_{\text{heuristic}}}_{\text{AR-DNTS}}\hat{V}_{\text{heuristic}}(s_t)}{N(s_t) + N^{\hat{V}_{\text{heuristic}}}_{\text{AR-DNTS}}} + \sum_{a\in\mathcal{A}} \frac{N(s_t,a)}{N(s_t) + N^{\hat{V}_{\text{heuristic}}}_{\text{AR-DNTS}}} \bar{Q}_{\text{AR-DNTS}}(s_t,a). \tag{A.27}$$

Recommendation Policy.

$$\psi_{\text{AR-DNTS}}(s) = \arg\max_{a\in\mathcal{A}} \bar{Q}_{\text{AR-DNTS}}(s,a), \tag{A.28}$$

$$\texttt{mv}_{\text{AR-DNTS}}(s) = \arg\max_{a\in\mathcal{A}} N(s,a). \tag{A.29}$$

B

Additional Experimental Details for Chapter 4

This appendix gives additional details for the experiments conducted in Chapter 4. It is structured as follows:

- Section B.1 gives additional details for the environemnts used in TODO: ref FL and Sailing results;
- Section B.2 gives details on the hyperparameters used for the experiments in TODO: ref FL and Sailing results and the optimisation process used to select them;
- Section **??** gives additional details on the Go experiments, including hyperparameter selection;

B.1 Additional DeterministicGridWorld, FrozenLake details

Figure B.1 shows the maps used in the DeterministicGridWorld and FrozenLake environments. The 4x4 and 8x8 maps are the standard maps from the Gymnasium library TODO: cite. The remaining maps are randomly generated, where the starting position is always the top left corner, the goal position is always the bottom

```
SFFFFFHF
FFFFFFFF
FHFHFFFF
FFFFFFHH
FFFHFFFF
FHHHFFFF
FFFFFHFF
FFFFFFFG
```

(a) 8x8 FrozenLake.

```
SFHFFFHFFFFF
FFFFFFFHFFFF
HFFFFFHFFFFF
FHFFHFFFFFFF
HHFFFFFFFFFF
FHFFFFHFFFFF
FHFFFHHFHFFF
FFFFFFFFFHHG
```

(b) 8x12 FrozenLake.

Figure B.1: Maps used for the DeterministicGridWorld and FrozenLake environments. `S` is the starting location for the agent, `F` represents floor that the agent can move too, `H` are holes that end the agents trial and `G` is the goal location. TODO: Add 4x4, 5x5 or 4x6, 6x6 or 4x8 maps for stoch envs TODO: Add 8x8, 8x12 or 8x16, 12x12 or 16x16 maps TODO: update these maps to the final maps used and update caption.

right corner and the remaining positions are randomly sampled with a $\frac{1}{5}$ probability of being a hole. Rejection sampling is used to ensure that there is always a path from the starting position to the goal.

B.2 Hyperparameter Details for DeterministicGridWorld, FrozenLake and Sailing Experiments

This section is organised as follows:

- Section B.2.1 gives a list of shared and BD: specific to a subset of algorithms hyperparameters used for the experiments;
- Section B.2.2 describes how the common hyperparameters are integrated into prior algorithms, using UCT as an example;
- Section B.2.3 gives details on the hyperparameter optimisation and the final hyperparameters that were selected.

B.2.1 Common Hyperparameters Reference

The following parameters were used by all algorithms considered in section TODO: ref gridworlds results:

- `single_node_expansion` - a `THTS++` parameter that specifies whether the selection phase should end when a new decision node is added to the tree or not;
- `transposition_table` - a `THTS++` parameter that specifies whether a transposition table should be used to store and re-use the decision nodes of the search tree when they are reached again via another path;
- `normalised_Q` - if Q-values should be normalised before being used in the search policy;
- `init_v` - the heuristic value function;
- `init_q` - the heuristic Q-value function;
- $N^{\hat{V}_{\text{heuristic}}}$ - the number of pseudo-visits assigned to `init_v` in backups;
- $N^{\hat{Q}_{\text{heuristic}}}$ - the number of pseudo-visits assigned to `init_q` to the Q-value estimate;

The following parameters are used by specific algorithms:

- ϵ - the (uniform) exploration parameter used by algorithms using a Boltzmann search policy, e.g. MENTS, BTS;
- α - the temperature parameter for algorithms using a maximum entropy objective, e.g. MENTS, RENTS, TENTS;
- $\alpha(x)$ - the temperature schedule for Boltzmann distributions in BTS and DENTS;
- $\beta(x)$ - the entropy weighting function for DENTS;
- b - the bias term used in UCB style policies, e.g. UCT, H-MCTS;
- `uct_budget` - TODO: check if its seq halving into uct or uct into seq halving, the budget for running UCT before swtiching to Sequential Halving in H-MCTS;

BD: To be compatible with hyperparameter optimisation, the functional form of α and β BD: need to be specified in a parameterised form. For the experiments in Section TODO: ref gridworlds results, the following forms were used:

$$\alpha(x) = \frac{\alpha_0}{\sqrt{1 + \alpha_{\text{decay}} x}} \tag{B.1}$$

$$\beta(x) = \begin{cases} \beta_c - \frac{\beta_c}{\beta_{\text{thresh}}} x & \text{if } x \leq \beta_{\text{thresh}}, \\ 0 & \text{otherwise.} \end{cases} \tag{B.2}$$

These functions are chosen so that the entropy weighting becomes negligible, and more specifically so that $\frac{\beta(x)}{\alpha(x)}$ tends to zero as x tends to infinity.

B.2.2 Integration of Common Hyperparameters into Prior Algorithms

Consider an algorithm with value estimates $\bar{V}(s)$ and $\bar{Q}(s,a)$. To use the $N^{\hat{Q}_{\text{heuristic}}}$ and `normalised_Q` parameters, the biased Q-values $\bar{Q}^{\text{norm}}(s,a)$ used in the search policy are given by:

$$\bar{Q}^{\text{bias}}(s,a) = \frac{N(s,a)\bar{Q}(s,a) + N^{\hat{Q}_{\text{heuristic}}}\hat{Q}_{\text{heuristic}}(s,a)}{N(s,a) + N^{\hat{Q}_{\text{heuristic}}}}, \tag{B.3}$$

$$\bar{Q}^{\text{norm}}(s,a) = \begin{cases} \frac{\bar{Q}^{\text{bias}}(s,a) - \min_{a\in\mathcal{A}} \bar{Q}^{\text{bias}}(s,a)}{\max_{a\in\mathcal{A}} \bar{Q}^{\text{bias}}(s,a) - \min_{a\in\mathcal{A}} \bar{Q}^{\text{bias}}(s,a)} & \text{if } \texttt{normalised_Q}, \\ \bar{Q}^{\text{bias}}(s,a) & \text{otherwise.} \end{cases} \tag{B.4}$$

All visit counts and value estimates are initialised to zero: $N(s) = 0$, $N(s,a) = 0$, $\bar{V}(s) = 0$ and $\bar{Q}(s,a) = 0$. While MCTS algorithms are often presented as initialising new leaf nodes directly with a visit and the heuristic value TODO: ref UCT init, this biasing term can instead be encorporated into the backups instead, allowing the algorithm to be run with an without the `single_node_expansion` parameter.

Given a trajectory $\tau = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h)$, the *full backups* for the average returns $\bar{V}$ and $\bar{Q}$ including the biasing term for $t = h-1, \ldots, 0$ are given by:

$$\bar{Q}(s_t, a_t) \leftarrow R(s_t, a_t) + \sum_{s' \in \text{Succ}(s_t, a_t)} \left(\frac{N(s'|s_t, a_t)}{N(s_t, a_t)} \bar{V}(s') \right), \tag{B.5}$$

$$\bar{V}(s_t) \leftarrow \frac{N^{\hat{V}_{\text{heuristic}}}\hat{V}_{\text{heuristic}}(s_t)}{N(s_t) + N^{\hat{V}_{\text{heuristic}}}} + \sum_{a\in\mathcal{A}} \frac{N(s_t, a)}{N(s_t) + N^{\hat{V}_{\text{heuristic}}}} \bar{Q}(s_t, a). \tag{B.6}$$

BD: In the main thesis the visit counts $N(s'|s,a)$ BD: are breifly mentioned in Section XXX, but not used for the general presentation to follow prior works presentation. When using `transposition_table`, all backups need to be of the *full backups*, as discussed in Section TODO: ref and argued by TODO: cite kata go.

BD: This is of particular importance for algorithms that used average returns that are typically computed using incremental backups. Note that the benefits of using `transposition_table` BD: come when the unique state assumption does not hold. The correctness of using full backups when the unique state assumption does not hold is beyond the scope of this thesis TODO: cite katago.

B.2.2.1 UCT Implementation with Full Backups

As a worked example, the version of UCT used in experiments that utilises all of the parameters from Section B.2.1 is given. The algorithm is specified minimally using the `THTS++` schema given in Section 2.4.2.

Value Estimates and Parameters.

- $\bar{V}_{\text{UCT}}(s)$ - The value estimate for state s;
- $\bar{Q}_{\text{UCT}}(s,a)$ - The Q-value estimate for state s and action a;
- $N(s)$ - The visit count for state s;
- $N(s,a)$ - The visit count for state s and action a;
- `normalised_Q` - A flag indicating whether the Q-values should be normalised when used in the search policy;
- $N_{\text{UCT}}^{\hat{V}_{\text{heuristic}}}$ - The number of pseudo-visits to use for biasing the backups;
- $N_{\text{UCT}}^{\hat{Q}_{\text{heuristic}}}$ - The number of pseudo-visits to use for biasing the Q-values;
- $\hat{V}_{\text{heuristic}}(s)$ - The heuristic value estimate for state s;
- $\hat{Q}_{\text{heuristic}}(s,a)$ - The heuristic Q-value estimate for state s and action a;
- b_{UCT} - the bias parameter used in the UCT search policy;

Initialisation.

$$N(s) \leftarrow 0, \tag{B.7}$$

$$N(s,a) \leftarrow 0, \tag{B.8}$$

$$\bar{V}_{\text{UCT}}(s) \leftarrow 0, \tag{B.9}$$

$$\bar{Q}_{\text{UCT}}(s,a) \leftarrow 0. \tag{B.10}$$

Search Policy. Intermediate Q-values:

$$\bar{Q}^{\text{bias}}_{\text{UCT}}(s,a) = \frac{N(s,a)\bar{Q}_{\text{UCT}}(s,a) + N^{\hat{Q}_{\text{heuristic}}}_{\text{UCT}}\hat{Q}_{\text{heuristic}}(s,a)}{N(s,a) + N^{\hat{Q}_{\text{heuristic}}}_{\text{UCT}}}, \tag{B.11}$$

$$\bar{Q}^{\text{norm}}_{\text{UCT}}(s,a) = \begin{cases} \frac{\bar{Q}^{\text{bias}}_{\text{UCT}}(s,a) - \min_{a\in\mathcal{A}} \bar{Q}^{\text{bias}}_{\text{UCT}}(s,a)}{\max_{a\in\mathcal{A}} \bar{Q}^{\text{bias}}_{\text{UCT}}(s,a) - \min_{a\in\mathcal{A}} \bar{Q}^{\text{bias}}_{\text{UCT}}(s,a)} & \text{if } \texttt{normalised_Q}, \\ \bar{Q}^{\text{bias}}_{\text{UCT}}(s,a) & \text{otherwise.} \end{cases} \tag{B.12}$$

Search policy:

$$\pi_{\text{UCT}}(s) = \arg\max_{a\in\mathcal{A}} \bar{Q}^{\text{norm}}_{\text{UCT}}(s,a) + b_{\text{UCT}}\sqrt{\frac{\log(N(s))}{N(s,a)}}, \tag{B.13}$$

Backups. Let $\tau_{:h} = (s_0, a_0, r_0, \ldots, s_{h-1}, a_{h-1}, r_{h-1}, s_h) \sim \pi_{\text{UCT}}$:

$$\bar{Q}_{\text{UCT}}(s_t,a_t) \leftarrow R(s_t,a_t) + \sum_{s'\in\text{Succ}(s_t,a_t)} \left(\frac{N(s'|s_t,a_t)}{N(s_t,a_t)} \bar{V}_{\text{UCT}}(s') \right), \tag{B.14}$$

$$\bar{V}_{\text{UCT}}(s_t) \leftarrow \frac{N^{\hat{V}_{\text{heuristic}}}\hat{V}_{\text{heuristic}}(s_t)}{N(s_t) + N^{\hat{V}_{\text{heuristic}}}} + \sum_{a\in\mathcal{A}} \frac{N(s_t,a)}{N(s_t) + N^{\hat{V}_{\text{heuristic}}}} \bar{Q}_{\text{UCT}}(s_t,a). \tag{B.15}$$

Recommendation Policy.

$$\psi_{\text{UCT}}(s) = \arg\max_{a\in\mathcal{A}} \bar{Q}_{\text{UCT}}(s,a), \tag{B.16}$$

$$\texttt{mv}_{\text{UCT}}(s) = \arg\max_{a\in\mathcal{A}} N(s,a). \tag{B.17}$$

B.2.3 Hyperparameter Optimisation

The parameters shared between all algorithms were set manually to the following values for all environments:

- `single_node_expansion` = False - the selection phase continues until the planning horizon or a terminal state is reached;

Parameter	Minimum	Maximum	Scaling
ϵ	10^{-6}	1	Logarithmic
α	10^{-5}	10^{2}	Logarithmic
α_0	10^{-5}	10^{2}	Logarithmic
α_{decay}	10^{-6}	10^{2}	Logarithmic
β_c	10^{-5}	10^{2}	Logarithmic
β_{thresh}	10^{3}	10^{12}	Logarithmic
b	10^{-5}	10^{2}	Logarithmic
`uct_budget`	1	10^{5}	Linear

Table B.1: The hyperparameter search space for gridworld experiments (Section TODO: ref). Recall that α is the temperature used for maximum entropy algorithms (e.g. MENTS), and α_0, α_{decay} β_c and β_{thresh} are parameters for $\alpha(x)$ and $\beta(x)$ used in BTS and DENTS. The search space for α_{decay} and β_{thresh} are selected such that the function spaces for $\alpha(x)$ and $\beta(x)$ include functions that are nearly constant for the duration of the search.

- `transposition_table` = True - transposition tables are used to store and re-use decision nodes when they are reached again via another path;
- `normalised_Q` = True - Q-values are linearly normalised before being used in the search policy;
- $N^{\hat{V}_{\text{heuristic}}} = 0$ - the number of pseudo-visits assigned to `init_v` in backups;
- $N^{\hat{Q}_{\text{heuristic}}} = 1$ - the number of pseudo-visits assigned to `init_q` to the Q-value estimate.

Using the principle of optimism in the face of uncertainty, the heuristic value functions were set to a trivial upper bound on the values that could be achieved in each environment. For the DeterministicGridWorld and FrozenLake environments, this meant the value functions were set to $\hat{V}_{\text{heuristic}}(s) = \hat{Q}_{\text{heuristic}}(s, a) = 1.0$ for all states s and actions a. For the Sailing environments, because all rewards are negative, the value functions were set to $\hat{V}_{\text{heuristic}}(s) = \hat{Q}_{\text{heuristic}}(s, a) = 0.0$ for all states s and actions a.

The remaining hyperparameters were selected using Bayesian optimisation via the BayesOpt library TODO: cite. Table B.1 gives the space searched over for each parameter, and if BD: parameters were sampled using a linear or logarithmic scalarised.

Algorithm	b	`uct_budget`
UCT	7.00	
MaxUCT	6.86	
H-MCTS	1.50	9.95×10^4

(a) UCT-based search algorithms.

Algorithm	ϵ	α	α_0	α_{decay}	β_c	β_{thresh}
MENTS	1.00	6.81×10^{-4}				
RENTS	1.00	1.34×10^{-4}				
TENTS	1.00	9.20×10^{-5}				
BTS	1.00		1.00×10^{-5}	3.16		
DENTS	0.744		4.04×10^{-3}	0.977	3.36×10^{-4}	10^{12}

(b) Boltzmann-based search algorithms.

Table B.2: Hyperparameters used for DeterministicGridWorld experiments. Empty cells indicate the parameter is not used by that algorithm.

Parameters were optimised using the TODO: ref map for DeterministicGridWorld, the TODO: ref map for FrozenLake and the TODO: ref map for Sailing.

To evaluate the performance of a given set of parameter values, the algorithm was run for 50,000 trials, and 1024 trials used to evaluate the recommendation policy. At least 10 runs were performed for each set of parameters, and runs were repeated until the standard deviation across the runs was less than 0.025 for DeterministicGridWorld and FrozenLake, and less than 1.0 for Sailing.

The selected hyperparameters for DeterministicGridWorld, FrozenLake, Sailing(North) and Sailing(South-East) are given in Tables B.2, B.3, B.4 and B.5 respectively.

B.3 Additional Details for Go Experiments

Because running Bayesian optimisation at the scale of the Go experiments would be prohibitively expensive, a round robin tournament was used to select the hyperparameters. This section gives the results of the round robin tournaments.

For PUCT, the parameters used were copied from the reported values for Kata Go and Alpha Go Zero TODO: fix cites

This section is organised as follows:

Algorithm	b	`uct_budget`
UCT	7.07	
MaxUCT	2.58	
H-MCTS	6.81	28

(a) UCT-based search algorithms.

Algorithm	ϵ	α	α_0	α_{decay}	β_c	β_{thresh}
MENTS	0.248	1.02×10^{-2}				
RENTS	0.432	2.89×10^{-3}				
TENTS	0.288	1.01×10^{-2}				
BTS	0.460		0.134	4.82×10^{-2}		
DENTS	0.444		0.264	0.200	9.17×10^{-4}	1.38×10^{4}

(b) Boltzmann-based search algorithms.

Table B.3: Hyperparameters used for FrozenLake experiments. Empty cells indicate the parameter is not used by that algorithm.

Algorithm	b	`uct_budget`
UCT	9.48×10^{-2}	
MaxUCT	9.05×10^{-2}	
H-MCTS	8.10×10^{-2}	32

(a) UCT-based search algorithms.

Algorithm	ϵ	α	α_0	α_{decay}	β_c	β_{thresh}
MENTS	1.00	5.40×10^{-4}				
RENTS	0.319	1.57×10^{-4}				
TENTS	1.00	7.61×10^{-4}				
BTS	1.00		2.32×10^{-3}	1.16×10^{-3}		
DENTS	0.971		1.02×10^{-5}	0.813	1.14×10^{-3}	2.18×10^{4}

(b) Boltzmann-based search algorithms.

Table B.4: Hyperparameters used for Sailing(North) experiments. Empty cells indicate the parameter is not used by that algorithm.

- Section B.3.1 outlines the implementation of the AR-MENTS, AR-RENTS and AR-TENTS algorithms;
- Section B.3.2 gives details on the round robin tournament used to select the hyperparameters for Go experiments;
- Section B.3.3 gives results for matches between the Bellman and average return versions of each algorithm.

Algorithm	b	`uct_budget`
UCT	0.181	
MaxUCT	0.226	
H-MCTS	0.145	100

(a) UCT-based search algorithms.

Algorithm	ϵ	α	α_0	α_{decay}	β_c	β_{thresh}
MENTS	1.00	6.05×10^{-4}				
RENTS	1.00	1.00×10^{-5}				
TENTS	1.00	3.24×10^{-3}				
BTS	1.00		2.32×10^{-3}	1.17×10^{-3}		
DENTS	1.00		1.01×10^{-5}	9.91×10^{2}	1.01×10^{-5}	3.91×10^{4}

(b) Boltzmann-based search algorithms.

Table B.5: Hyperparameters used for Sailing(South-East) experiments. Empty cells indicate the parameter is not used by that algorithm.

B.3.1 AR-MENTS, AR-RENTS and AR-TENTS

BD: TO implement AR-MENTS the AR-DENTS (Section **??**) BD: algorithm was used, but with $\beta(x) = \alpha(x) = \alpha_{\text{fix}}$. Implementing AR-RENTS and AR-TENTS is similar, replacing the used of Shannon entropy $\mathcal{H}$ in the backups by the corresponding relative entropy and Tsallis entropy functions. For AR-TENTS, the search policy uses the *sparsemax* function used in TENTS instead of the softmax function.

B.3.2 Go Round Robin Hyperparameter Selection

The following parameters were tuned for the BTS, DENTS, MENTS, RENTS and TENTS algorithms:

- ϵ - the exploration parameter used by algorithms using a Boltzmann search policy, e.g. MENTS, BTS;
- $\epsilon_{\tilde{\lambda}}$ - the weighting of the prior policy used by algorithms using a Boltzmann search policy, e.g. BTS;
- α - the temperature parameter for algorithms using a maximum entropy objective, e.g. MENTS, RENTS, TENTS;
- α_{init} - the initial temperature parameter for BTS and DENTS algorithms;

Black \White	3	1	0.3	0.1	0.03
3	-	7-8	9-6	12-3	12-3
1	15-0	-	12-3	15-0	15-0
0.3	13-2	11-4	-	14-1	15-0
0.1	14-1	10-5	11-4	-	15-0
0.03	13-2	5-10	8-7	13-2	-

Table B.6: Round robin hyperparameter selection for α_{init} for BTS. The value of 1.0 won all four of its matches so was selected.

Black \White	3	1	0.3	0.1	0.03
3	-	3-12	4-11	3-12	1-14
1	14-1	-	8-7	9-6	7-8
0.3	14-1	8-7	-	7-8	11-4
0.1	15-0	11-4	9-6	-	9-6
0.03	14-1	8-7	4-11	8-7	-

Table B.7: Round robin hyperparameter selection for α_{init} for BTS. The value of 0.1 won all four of its matches so was selected.

- β_{init} - the initial entropy coefficient for DENTS algorithms.

The temperature and entropy coefficient functions used in BTS and DENTS for Go experiments were of the form $\alpha(x) = \alpha_{\text{init}}/\sqrt{x}$ and $\beta(x) = \beta_{\text{init}}/\sqrt{x}$ respectively. For DENTS, only the entropy coefficient was tuned independently and used the parameters tuned for BTS.

Parameters are tuned one at a time, with a round robin tournament used to select the best value for each parameter, keeping the remaining parameters fixed. The parameter that won the most matches is selected, with ties broken by the agent that won the most games. If the winning agent had the largest or smallest value, then another tournament is run adjusting the values accordingly.

These tuning tournaments were run using the game of 9x9 Go with a komi of 6.5, and giving each algorithm 2.5 seconds per move.

Initially the values of ϵ and $\epsilon_{\tilde{\lambda}}$ were set to 0.03 and 1.0 respectively and the search temperatures were tuned first. Tables B.6 and B.7 give results of tuning the initial search temperature α_{init} for BTS and AR-BTS respectively.

$\epsilon_{\tilde{\lambda}}$ was tuned next, using the selected values of α_{init} for BTS and AR-BTS. Tables B.8 and B.9 give results for the round robin tournament to select the prior

Black \White	10	5	3	2	1
10	-	5-10	3-12	1-14	3-12
5	14-1	-	12-3	11-4	11-4
3	15-0	15-0	-	12-3	12-3
2	15-0	12-3	12-3	-	10-5
1	15-0	14-1	12-3	9-6	-

Table B.8: Round robin hyperparameter selection for $\epsilon_{\tilde{\lambda}}$ for BTS. The value of 2.0 won the most matches so was selected.

Black \White	3	2	1	0.75	0.5
3	-	14-1	9-6	13-2	14-1
2	12-3	-	12-3	15-0	10-5
1	12-3	13-2	-	11-4	13-2
0.75	11-4	10-5	12-3	-	14-1
0.5	11-4	13-2	9-6	7-8	-

Table B.9: Round robin hyperparameter selection for $\epsilon_{\tilde{\lambda}}$ for AR-BTS. The value of 1.0 won the most matches so was selected.

policy weight for BTS and AR-BTS.

Next the exploration parameter ϵ was tuned, using the selected values of α_{init} and $\epsilon_{\tilde{\lambda}}$ for BTS and AR-BTS. Tables B.10 and B.11 give results for the round robin tournament to select the exploration parameter for BTS and AR-BTS. Although the lowest value of 0.001 was selected for AR-BTS, it is close enough to zero such that very few actions are randomly sampled. A round robin with values between 10^{-3} and 10^{-5} was tried, but all matches resulted in draws.

To tune the MENTS, RENTS and TENTS algorithms, the values of ϵ, $\epsilon_{\tilde{\lambda}}$ selected for BTS were used, and the value of α varied. Similarly, to tune the AR-MENTS, AR-RENTS and AR-TENTS algorithms, the values of ϵ, $\epsilon_{\tilde{\lambda}}$ selected for AR-BTS were used, and the value of α varied. Results of the round robin tournaments to

Black \White	0.1	0.03	0.01	0.003	0.001
0.1	-	12-3	10-5	8-7	7-8
0.03	9-6	-	8-7	13-2	12-3
0.01	11-4	9-6	-	7-8	12-3
0.003	13-2	9-6	11-4	-	11-4
0.001	12-3	11-4	8-7	9-6	-

Table B.10: Round robin hyperparameter selection for ϵ for BTS. The value of 0.003 won the most matches so was selected.

Black \White	0.1	0.03	0.01	0.003	0.001
0.1	-	12-3	14-1	12-3	13-2
0.03	15-0	-	11-4	14-1	14-1
0.01	15-0	11-4	-	13-2	13-2
0.003	15-0	14-0	14-1	-	13-2
0.001	14-1	14-1	14-1	15-0	-

Table B.11: Round robin hyperparameter selection for ϵ for AR-BTS. The value of 0.001 won the most matches so was selected.

Black \White	3	1	0.3	0.1	0.03
3	-	9-6	9-6	11-4	10-5
1	11-4	-	11-4	9-6	10-5
0.3	9-6	7-8	-	11-4	6-9
0.1	4-11	4-11	0-15	-	4-11
0.03	2-13	2-13	0-15	0-15	-

Table B.12: Round robin hyperparameter selection for α for MENTS. The value of 1.0 won the most matches so was selected.

select the temperature parameter α are given in the following tables:

- MENTS - Table B.12;
- AR-MENTS - Table B.13;
- RENTS - Table B.14;
- AR-RENTS - Table B.15;
- TENTS - Table B.16;
- AR-TENTS - Table B.17.

Black \White	3	1	0.3	0.1	0.03
3	-	2-13	1-14	3-12	5-10
1	14-1	-	11-4	13-2	15-0
0.3	15-0	12-3	-	12-3	14-1
0.1	15-0	9-6	9-6	-	15-0
0.03	15-0	8-7	9-6	14-1	-

Table B.13: Round robin hyperparameter selection for α for AR-MENTS. The value of 0.3 won the most matches so was selected.

Black \White	3	1	0.3	0.1	0.03
3	-	7-8	9-6	10-5	9-6
1	13-2	-	12-3	11-4	12-3
0.3	10-5	11-4	-	10-5	5-10
0.1	3-12	2-13	1-14	-	3-12
0.03	3-12	0-15	0-15	0-15	-

Table B.14: Round robin hyperparameter selection for α for RENTS. The value of 1.0 won the most matches so was selected.

Black \White	3	1	0.3	0.1	0.03
3	-	4-11	2-13	9-6	3-12
1	15-0	-	12-3	13-2	13-2
0.3	15-0	13-2	-	14-1	15-0
0.1	15-0	10-5	13-2	-	15-0
0.03	13-2	13-2	12-3	14-1	-

Table B.15: Round robin hyperparameter selection for α for AR-RENTS. The value of 0.3 won the most matches so was selected.

Black \White	300	100	30	10	3
300	-	3-12	5-10	7-8	9-6
100	6-9	-	9-6	12-3	12-3
30	8-7	7-8	-	9-6	11-4
10	0-15	2-13	2-13	-	6-9
3	1-14	1-14	2-13	5-10	-

Table B.16: Round robin hyperparameter selection for α for TENTS. The value of 30.0 won the most matches so was selected.

Black \White	30	10	3	1	0.3
30	-	14-1	15-0	14-1	14-1
10	15-0	-	13-2	15-0	15-0
3	15-0	14-1	-	14-1	15-0
1	15-0	15-0	14-1	-	15-0
0.3	15-0	15-0	14-1	15-0	-

Table B.17: Round robin hyperparameter selection for α for AR-TENTS. The value of 3.0 won the most matches so was selected.

Black \White	0.1	0.03	0.01	0.003	0.001
0.1	-	10-5	12-3	8-7	11-4
0.03	13-2	-	11-4	13-2	10-5
0.01	12-3	11-4	-	10-5	11-4
0.003	7-8	13-2	13-2	-	10-5
0.001	13-2	13-2	11-4	11-4	-

Table B.18: Round robin hyperparameter selection for β_{init} for DENTS. The value of 0.3 won the most matches so was selected.

Black \White	3	1	0.3	0.1	0.03
3	-	11-4	12-3	12-3	14-1
1	13-2	-	11-4	13-2	13-2
0.3	14-1	13-2	-	13-2	14-1
0.1	12-3	12-3	12-3	-	11-4
0.03	12-3	13-2	13-2	14-1	-

Table B.19: Round robin hyperparameter selection for β_{init} for AR-DENTS. The value of 0.3 won the most matches so was selected.

Finaly, for (AR-)DENTS, all parameters were copied from (AR-)BTS, and the entropy temperature β was tuned. Tables B.18 and B.19 give results for the round robin tournament to select the initial entropy temperature β_{init} for DENTS and AR-DENTS respectively.

B.3.3 Average Returns vs Bellman Updates for Go

After tuning all of the algorithms, each algorithm using Bellman updates played a match against its average returns counterpart, with results given in Table B.20. Across all matches, the average returns algorithms won each match by a significant margin.

BD: My hypothesis for this is that average returns are less sensitive to any noisy outputs from the neural networks, as taking a maximum over random variables is more sensitive to extreme values than averages. Additionally, the neural networks are trained using PUCT and so are trained to work with average returns. It could be the case that if the neural networks were fine-tuned or trained using the Bellman versions of the algorithms that the performance would be closer or better from the Bellman versions.

Black \White	BTS	AR-BTS
BTS	-	16-9
AR-BTS	22-3	-
Black \White	DENTS	AR-DENTS
DENTS	-	21-4
AR-DENTS	22-3	-
Black \White	MENTS	AR-MENTS
MENTS	-	0-25
AR-MENTS	25-0	-
Black \White	RENTS	AR-RENTS
RENTS	-	2-23
AR-RENTS	24-1	-
Black \White	TENTS	AR-TENTS
TENTS	-	8-17
AR-TENTS	23-2	-

Table B.20: Results for matches between the average return and Bellman versions of BTS, DENTS, MENTS, RENTS and TENTS.

C

Additional Theoretical Results For Chapter 4

This appendix provides the full definition of the DENTS stochastic process in Section C.1 and the proofs for exponential convergence results for BTS and DENTS are given in Section TODO: ref.

C.1 The DENTS Stochastic Process

The DENTS stochastic process formalises the DENTS algorithm as a stochastic process, which is defined as follows. This section uses the notation and assumptions established in TODO: ref to ch4.

All value estimates are initialised to zero for all $s \in \mathcal{S}$ and $a \in \mathcal{A}$:

$$\hat{V}^{(0)}_{\text{DNTS}}(s) = 0, \quad \hat{Q}^{(0)}_{\text{DNTS}}(s,a) = 0, \quad \bar{\mathcal{H}}^{(0)}_{V,\text{DNTS}}(s) = 0, \quad \bar{\mathcal{H}}^{(0)}_{Q,\text{DNTS}}(s,a) = 0. \tag{C.1}$$

On the nth trial, a trajectory is sampled using the search policy $\tau^n \sim \pi^n_{\text{DNTS}}$.

Where for all $s, a \in \mathcal{S} \times \mathcal{A}$, the search policy is defined as:

$$\hat{Q}^{(N^n(s,a))}_{\text{DNTS,sft}}(s,a) = \hat{Q}^{(N^n(s,a))}_{\text{DNTS}}(s,a) + \beta_{\text{DNTS}}(N^n(s))\bar{\mathcal{H}}^{(N^n(s,a))}_{Q,\text{DNTS}}(s,a), \tag{C.2}$$

$$\rho^m_{\text{DNTS}}(a|s) = \frac{\exp\left(\frac{\hat{Q}^{(N^n(s,a))}_{\text{DNTS,sft}}(s,a)}{\alpha_{\text{DNTS}}(N(s,m))}\right)}{\sum_{a' \in \mathcal{A}} \exp\left(\frac{\hat{Q}^{(N^n(s,a'))}_{\text{DNTS,sft}}(s,a')}{\alpha_{\text{DNTS}}(N^n(s))}\right)}. \tag{C.3}$$

$$\lambda^m(s, \epsilon_{\text{DNTS}}) = \min\left(1, \frac{\epsilon_{\text{DNTS}}}{\log(e + N^{n-1}(s))}\right), \tag{C.4}$$

$$\pi^n_{\text{DNTS}}(a|s) = (1 - \lambda^n(s, \epsilon_{\text{DNTS}}))\rho^n_{\text{DNTS}}(a|s) + \frac{\lambda^n(s, \epsilon_{\text{DNTS}})}{|\mathcal{A}|}. \tag{C.5}$$

$$\tag{C.6}$$

Letting $\tau^n = (s^n_0, a^n_0, r^n_0, \ldots, s^n_{H-1}, a^n_{H-1}, r^n_{H-1}, s^n_H)$, the DENTS value and entropy estimates are computed as follows for $t = H-1, \ldots, 0$:

$$\hat{Q}^{(N(s_t,a_t,m))}_{\text{DNTS}}(s_t,a_t) = R(s_t,a_t) + \sum_{s' \in \text{Succ}(s_t,a_t)} \left(\frac{N(s',m)}{N(s_t,a_t,m)} \hat{V}^{(N(s',m))}_{\text{DNTS}}(s')\right), \tag{C.7}$$

$$\hat{V}^{(N(s_t,m))}_{\text{DNTS}}(s_t) = \max_{a \in \mathcal{A}} \hat{Q}^{(N(s_t,a,m))}_{\text{DNTS}}(s_t,a), \tag{C.8}$$

$$\bar{\mathcal{H}}^{(N(s_t,a_t,m))}_{Q,\text{DNTS}}(s_t,a_t) = \sum_{s' \in \text{Succ}(s_t,a_t)} \frac{N(s',m)}{N(s_t,a_t,m)} \bar{\mathcal{H}}^{(N(s',m))}_{V,\text{DNTS}}(s'), \tag{C.9}$$

$$\bar{\mathcal{H}}^{(N(s_t))}_{V,\text{DNTS}}(s_t) = \mathcal{H}(\pi^m_{\text{DNTS}}(\cdot|s_t)) + \sum_{a \in \mathcal{A}} \pi^m_{\text{DNTS}}(a_t|s_t)\bar{\mathcal{H}}^{(N(s_t,a_t,m))}_{Q,\text{DNTS}}(s_t,a_t). \tag{C.10}$$

Recall that the visit counts are *defined* as a sum of indicator random variables using the sampled trajectories:

$$N^n(s_t) = \sum_{i=1}^n \mathbb{1}[s^i_t = s_t], \tag{C.11}$$

$$N^n(s_t,a_t) = \sum_{i=1}^n \mathbb{1}[s^i_t = s_t \cap a^i_t = a_t]. \tag{C.12}$$

And note that if $s \notin \tau^n$, then $N^n(s) = N^{n-1}(s)$, and similarly if $(s,a) \notin \tau^n$, then $N^n(s,a) = N^{n-1}(s,a)$. More specifically, this implies, for example, that if $s \notin \tau^n$ then:

$$\hat{V}^{(N^n(s))}_{\text{DNTS}}(s) = \hat{V}^{(N^{n-1}(s))}_{\text{DNTS}}(s). \tag{C.13}$$

Finally, the maximum value and most visited recommendation policies after n trials are respectively defined as:

$$\psi^n_{\text{DNTS}}(s) = \arg\max_{a\in\mathcal{A}} \hat{Q}^{(N^n(s,a))}_{\text{DNTS}}(s,a), \tag{C.14}$$

$$\texttt{mv}^n_{\text{DNTS}}(s) = \arg\max_{a\in\mathcal{A}} N^n(s,a). \tag{C.15}$$

C.1.1 The AR-DENTS Process

TODO: Copy DENTS after its written. Swap out notation and backups.

C.1.2 The (AR-)BTS process.

The (AR-)BTS process is a special case of the (AR-)DENTS process when $\beta_{\text{DNTS}}(x) = 0$.

TODO: Move to preliminaries. Something like: these intermediate values are useful in the proofs for exponential convergence results.

TODO: From the neurips, also wrote the following (everything left in this subsubsection is copied and cleaned up, but probably needs a proof read) Let $\hat{V}^{(N(s))}_{\rho,\text{DNTS}}(s)$ be defined as the value:

$$\begin{aligned}\hat{V}^{(N(s))}_{\rho,\text{DNTS}}(s) =& \alpha_{\text{DNTS}}(N(s,m)) \log\left[\sum_{a'\in\mathcal{A}} \exp\left(\frac{1}{\alpha_{\text{DNTS}}(N(s,m))}\left(\hat{Q}^{(N(s,a'))}_{\text{DNTS}}(s,a')\right.\right.\right.\\ &\left.\left.\left.+ \beta_{\text{DNTS}}(N(s,m))\bar{\mathcal{H}}^{(N(s,a'))}_{Q,\text{DNTS}}(s,a')\right)\right)\right],\end{aligned} \tag{C.16}$$

and notice that the value of $\exp(\hat{V}^{(N(s,m))}_{\rho,\text{DNTS}}(s)/\alpha_{\text{DNTS}}(N(s,m)))$ is equal to the denominator in Equation (C.3), and so by rearranging we can write ρ^m_{DNTS} as

$$\rho^m_{\text{DNTS}}(a|s) = \exp\left(\frac{1}{\alpha_{\text{DNTS}}(N(s,m))}\left(\hat{Q}^{(N(s,a,m))}_{\text{DNTS}}(s,a) + \beta_{\text{DNTS}}(N(s,m))\bar{\mathcal{H}}^{(N(s,a,m))}_{Q,\text{DNTS}}(s,a) - \hat{V}^{(N(s,m}_{\rho,\text{DNTS}}\right.\right. \tag{C.17}$$

and subsequently, the full DENTS policy can be written as:

$$\pi^m_{\text{DNTS}}(a|s) = (1-\lambda(s,m))\exp\left(\frac{1}{\alpha_{\text{DNTS}}(N(s,m))}\left(\hat{Q}^{(N(s,a,m))}_{\text{DNTS}}(s,a) + \beta_{\text{DNTS}}(N(s,m))\bar{\mathcal{H}}^{(N(s,a))}_{Q,\text{DNTS}}(s,a\right.\right. \tag{C.18}$$

C.2 Exponential Convergence Results

This section provides the proofs of exponential convergence results for the DENTS, AR-DENTS, BTS and AR-BTS stochastic processes. It is structured similarly to Section 4.5 as follows:

1. TODO: todo

TODO: This section give proofs for exponential convergence results for DENTS and MENTS, it is structured as follows:

1. First, Appendix C.2.1 provides a set of Lemmas that are used later in the proofs for exponential convergence bounds;

2. Second, Appendix C.2.2 gives Lemmas that provide sufficient conditions for the maximum entropy objective and standard objective to be aligned;

3. Third, in Appendix C.2.3 it is shown in a general way that is Q-values are computed as a sample average of child value functions, and those child value functions admit a concentration inequality, then the Q-value also admits a concentration inequality;

4. Fourth, in Appendix C.2.4, exponential convergence bounds (requiring restrictive conditions on the temperature parameter) are shown for MENTS;

5. Finally, in Appendix C.2.5, Theorem 4.2.3 and 4.2.7 are

TODO: Make sure that

TODO: Make sure assumptions and conditions stated. Anything that depends on the min prob needs to have the lower/upper bounds on the decay functions for example.

C.2.1 Preliminaries

This subsection contains preliminary lemmas that are used for the building blocks to prove.

TODO: <beg> dump

TODO: This section contains things related to exponential bounds, which might move to appendix This subsection contains lemmas that will be useful in multiple times and are used to avoid repeating the same argument multiple times.

TODO: change m to n, and use mth when need to reason about prev trials (whole subsection)

Firstly it will be that the union of exponentially unlikely events is also exponentially unlikely, and that the intersection of exponentially likely events is exponentially likely. Although this is a special case of the union bound TODO: ref? TODO: it is stated here so that it can be referenced, because this fact is used regularly TODO: Some comment about how this is just a special case of the union bound (and ref that?) and also

Lemma C.2.1. *Let $A_1, ..., A_\ell$ be some events that satisfy for $1 \leq i \leq \ell$ the inequality $\Pr(\neg A_i) \leq C_i \exp(-k_i)$ then:*

$$\Pr\left(\bigcup_{i=1}^{\ell} \neg A_i\right) \leq C \exp(-k), \tag{C.19}$$

$$\Pr\left(\bigcap_{i=1}^{\ell} A_i\right) = 1 - \Pr\left(\bigcup_{i=1}^{\ell} \neg A_i\right) \geq 1 - C \exp(-k), \tag{C.20}$$

where $C = \sum_{i=1}^{\ell} C_i$ and $k = \min_i k_i$.

Proof outline. Lemma C.2.1 is a consequence of the union bound, using $\exp(k_i) \leq \exp(k)$ and simplifying. Inequality (C.20) is just a negation of Inequality (C.19). □

The following two lemma's show that the MENTS and DENTS processes always have a minimum probability of picking any action. TODO: not in good headspace writing up these two lemmas. Do a clean up, double checking maths and that have eliminated using 'we' and so on

Lemma C.2.2. *For any MENTS process there exists some $\pi^{\min} > 0$, for any state $s_t \in \mathcal{S}$, for all $a_t \in \mathcal{A}$ and any number of trials $m \in \mathbb{N}$, such that $\pi^m_{\text{MENTS}}(a_t|s_t) \geq \pi^{\min}$.*

Proof outline. TODO: Fix N having to be N(s,m) not N(s) etc Define the $Q^{\min}$ function as follows:

$$Q^{\min}(s_t, a_t) = \min_{s_{t+1}, a_{t+1}, \ldots, s_H, a_H} \sum_{i=t}^{H} \min(0, Q^{\text{init}}_{\text{sft}}(s_i), R(s_i, a_i)). \tag{C.21}$$

TODO: clean up with respect to init function

And define the $V^{\max}$ function as:

$$V^{\max}(s_t) = \alpha_{\text{MENTS}} \log \sum_{a \in \mathcal{A}} \exp(Q^{\max}(s_t, a)/\alpha_{\text{MENTS}}), \tag{C.22}$$

$$Q^{\max}(s_t, a_t) = R(s_t, a_t) + \max_{s_{t+1} \in \text{Succ}(s_t, a_t)} V^{\max}(s_{t+1}). \tag{C.23}$$

Via induction, it is possible to show that $Q^{\min}(s_t, a_t) \leq \hat{Q}^{(N(s_t,a_t,m))}_{\text{MENTS}}(s_t, a_t)$ and $V^{\max}(s_t) \geq \hat{V}^{(N(s_t,m))}_{\text{MENTS}}(s_t)$ for any arbitrary s_t, a_t and m. Now, define $\pi^{\min}$ as:

$$\pi^{\min} = \inf_{\lambda \in [0,1]} \min_{(s,a) \in \mathcal{S} \times \mathcal{A}} (1 - \lambda) \exp\left(\left(Q^{\min}(s, a) - V^{\max}(s)\right)/\alpha\right) + \frac{\lambda}{|\mathcal{A}|}. \tag{C.24}$$

Because the value of an exponential is positive, as is $1/|\mathcal{A}|$, it follows that $\pi^{\min} > 0$. Recall the MENTS policy (Equation (**??**)). By the monotonicity of the exponential function, it follows that for any $s_t \in \mathcal{S}, a_t \in \mathcal{A}, n \in \mathbb{N}$:

$$\pi^m_{\text{MENTS}}(a_t|s_t) = (1 - \lambda(s_t, m)) \exp\left(\left(\frac{1}{\alpha_{\text{MENTS}}}\left(\hat{Q}^{(N(s_t,a_t,m))}_{\text{MENTS}}(s_t, a_t) - \hat{V}^{(N(s_t,m))}_{\text{MENTS}}(s_t)\right)\right)\right) + \frac{\lambda(s_t, m)}{|\mathcal{A}|} \tag{C.25}$$

$$\geq (1 - \lambda(s_t, m)) \exp\left(\left(Q^{\min}(s_t, a_t) - V^{\max}(s_t)\right)/\alpha\right) + \frac{\lambda(s_t, m)}{|\mathcal{A}|} \tag{C.26}$$

$$\geq \pi^{\min}. \tag{C.27}$$

□

TODO: this whole lemma just needs cleaning up really

TODO: this lemma requires $\beta_{\text{DNTS}}(x) < U$ for all x, or β_{DNTS} is bounded. AND it requires $\alpha_{\text{DNTS}}(x) < U'$ so α_{DNTS} to be bounded. TODO: THERE IS

CURRENTLY ERROR IN THIS. EXP BOUND REQUIRES A MINIMUM TEMP. Might need to reason around logsumexp's around here. But, as alpha tends to infinity, the probability distribution will tend to a uniform distribution, and min prob is fine, if alpha tends to zero, then softmax tends to max, and then min prob can be zero, so dont have a min prob. Might need to use a different value of alpha for the computation of V, so might need to have both a lower and upper bound on alpha.

Lemma C.2.3. *For any DENTS process there exists some $\pi^{\min} > 0$, for any state $s_t \in \mathcal{S}$, for all $a_t \in \mathcal{A}$ and any number of trials $m \in \mathbb{N}$, such that $\pi^m_{\text{DNTS}}(a_t|s_t) \geq \pi^{\min}$.*

Proof outline. This proof outline follows similar reasoning to Lemma C.2.2. The same definition for $Q^{\min}$ can be used:

$$Q^{\min}(s_t, a_t) = \min_{s_{t+1}, a_{t+1}, \ldots, s_H, a_H} \sum_{i=t}^{H} \min(0, Q^{\text{init}}_{\text{sft}}(s_i), R(s_i, a_i)). \tag{C.28}$$

TODO: clean up with respect to init function

However, the definition of $V^{\max}$ from Equation (C.22) needs to altered to:

$$V^{\max}(s_t) = \max_{a \in \mathcal{A}} Q^{\max}(s_t, a) \tag{C.29}$$

$$Q^{\max}(s_t, a_t) = R(s_t, a_t) + \max_{s' \in \text{Succ}(s_t, a_t)} V^{\max}(s') \tag{C.30}$$

$$V^{\max}_{\rho}(s_t) = \alpha \log \sum_{a \in \mathcal{A}} \exp\left(\left(Q^{\max}(s_t, a) + \beta_{\max} H \log |\mathcal{A}| \right) / \alpha \right), \tag{C.31}$$

TODO: clean up the use of alpha in this one. Should be alpha max (and also define alpha max), and need to work out if should be using a sup or inf or whatever. Maybe just put a sup outside it. TODO: H was the horizon, make that consistent with what we've defined before (above and in below paragraph) where $\beta_{\max} = \sup_{x \in \mathbb{R}} \beta(x)$. Similarly to the MENTS process case, it can be shown by induction that $Q^{\min}(s_t, a_t) \leq \hat{Q}^{(N(s_t, a_t, m))}_{\text{DNTS}}(s_t, a_t)$ and $V^{\max}_{\rho}(s_t) \geq V^{N(s_t, m)}_{\rho}(s_t)$, where the latter implicitly uses that $0 \leq \mathcal{H}^{N(s_t, a_t)}_{Q}(s_t, a_t) \leq (H - t) \log |\mathcal{A}| \leq H \log |\mathcal{A}|$ TODO: using well-known properties of entropy (left was what was originally written. Consider rewording this entire paragraph tho).

Define $\pi^{\min}$ similarly to Equation (C.24), with the updated definition of $V_\rho^{\max}$:

$$\pi^{\min} = \inf_{\lambda\in[0,1]} \min_{(s,a)\in\mathcal{S}\times\mathcal{A}} (1-\lambda)\exp\left(\left(Q^{\min}(s,a) - V_\rho^{\max}(s)\right)/\alpha_{\max}\right) + \frac{\lambda}{|\mathcal{A}|}. \tag{C.32}$$

where $\alpha_{\max} = \inf_x \alpha_{\text{DNTS}}(x)$ TODO: double check this is right. Its not. Although exp(1/x) is decreasing in x, which is why we reasoned that it should be max, it ignores that V rho max depends on alpha too. Also V rho max should be using the . Recalling the DENTS policy (Equation (C.18)), and using similar reasoning to before, as well as $\beta_{\text{DNTS}}(N(s,m))\bar{\mathcal{H}}_{Q,\text{DNTS}}^{(N(s,a))}(s,a) \geq 0$, the result follows:

$$\begin{aligned}
\pi_{\text{DNTS}}^m(a_t|s_t) =& (1-\lambda(s,m))\exp\left(\frac{1}{\alpha_{\text{DNTS}}(N(s,m))}\left(\hat{Q}_{\text{DNTS}}^{(N(s,a,m))}(s,a) + \beta_{\text{DNTS}}(N(s,m))\bar{\mathcal{H}}_{Q,\text{DNTS}}^{(N(s,a))}(s,a) - \right.\right. && \text{(C.33)}\\
\geq& (1-\lambda(s,m))\exp\left(\frac{1}{\alpha_{\text{DNTS}}(N(s,m))}\left(\hat{Q}_{\text{DNTS}}^{(N(s,a))}(s,a) - \hat{V}_{\rho,\text{DNTS}}^{(N(s))}(s)\right)\right) + \frac{\lambda(s,m)}{|\mathcal{A}|} && \text{(C.34)}\\
\geq& (1-\lambda(s,m))\exp\left(\left(Q^{\min}(s_t,a_t) - V_\rho^{\max}(s_t)\right)/\alpha_{\max}\right) + \frac{\lambda(s,m)}{|\mathcal{A}|} && \text{(C.35)}\\
\geq& \pi^{\min}. && \text{(C.36)}
\end{aligned}$$

□

Additionally, Hoeffding's inequality will be useful to bound the difference between a sum of indicator random variables and its expectation.

Theorem C.2.1. *Let $\{X_i\}_{i=1}^k$ be indicator random variables (i.e. $X_i \in \{0,1\}$), and $S_k = \sum_{i=1}^k X_i$. Then Hoeffding's inequality for indicator random variables states for any $\varepsilon > 0$ that:*

$$\Pr(|S_k - \mathbb{E}S_k| > \varepsilon) \leq 2\exp\left(-\frac{2\varepsilon^2}{k}\right). \tag{C.37}$$

Proof. This is a specific case of Hoeffding's inequality. See TODO: add cite for proof. □

It will also be convenient to be able to 'translate' bounds that depend on some $N(s,a,m)$ to a corresponding bound on $N(s,m)$:

Lemma C.2.4. *Consider any Boltzmann MCTS process. Suppose that every action a_t has some minimum probability η of being chosen from some state s_t (irrespective of the number of trials), i.e.* $\Pr(a_t^i = a_t | s_t^i = s_t) \geq \eta$. *And suppose for some $C', k' > 0$ that some event E admits a bound:*

$$\Pr(E) \leq C' \exp(-k' N(s_t, a_t, m)). \tag{C.38}$$

Then, there exists $C, k > 0$ such that:

$$\Pr(E) \leq C \exp(-k N(s_t, m)). \tag{C.39}$$

Proof. TODO: Define m again somewhere? Recall from Equation (4.41) that $N(s_t, a_t, m) = \sum_{i \in T(s_t,m)} \mathbb{1}[a_t^i = a_t] = \sum_{i \in T(s_t,m)} \mathbb{1}[a_t^i = a_t | s_t^i = s_t]$ TODO: this should be the other sum, from i equal to one to N(st,m). By taking expectations TODO: with respect to at and using the assumed $\Pr(a_t^i = a_t | s_t^i = s_t) \geq \eta$ it follows that $\mathbb{E}N(s_t, a_t, m) \geq \eta N(s_t, m)$ (and more specifically as a consequence $\mathbb{E}N(s_t, a_t, m) - \eta N(s_t, m)/2 \geq \eta N(s_t, m)/2$). The probability of $N(s_t, a_t, m)$ being below a multiplicative ratio of $N(s_t, m)$ is bounded as follows:

$$\Pr\left(N(s_t, a_t, m) < \frac{1}{2}\eta N(s_t, m)\right) \tag{C.40}$$

$$\leq \Pr\left(N(s_t, a_t, m) < \mathbb{E}N(s_t, a_t, m) - \frac{1}{2}\eta N(s_t, m)\right) \tag{C.41}$$

$$= \Pr\left(\mathbb{E}N(s_t, a_t, m) - N(s_t, a_t, m) > \frac{1}{2}\eta N(s_t, m)\right) \tag{C.42}$$

$$\leq \Pr\left(|\mathbb{E}N(s_t, a_t, m) - N(s_t, a_t, m)| > \frac{1}{2}\eta N(s_t, m)\right) \tag{C.43}$$

$$\leq 2 \exp\left(-\frac{1}{2}\eta^2 N(s_t, m)\right). \tag{C.44}$$

The first inequality follows from $\mathbb{E}N(s_t, a_t, m) - \eta N(s_t, m)/2 \geq \eta N(s_t, m)/2$, the second line is a rearrangement, the third line comes from the inequality in (C.42) implying the inequality in (C.43), and the final line uses Theorem C.2.1, a Hoeffding bound for the sum of indicator random variables.

Finally, the bound using $N(s_t, a_t, m)$ can be converted into one depending on $N(s_t, m)$ using the law of total probability as follows:

$$\begin{aligned}
\Pr(E) = &\Pr\left(E \middle| N(s_t, a_t, m) \geq \frac{1}{2}\eta N(s_t, m)\right) \Pr\left(N(s_t, a_t, m) \geq \frac{1}{2}\eta N(s_t, m)\right) \\
&+ \Pr\left(E \middle| N(s_t, a_t, m) < \frac{1}{2}\eta N(s_t, m)\right) \Pr\left(N(s_t, a_t, m) < \frac{1}{2}\eta N(s_t, m)\right) \qquad \text{(C.45)} \\
\leq &\Pr\left(E \middle| N(s_t, a_t, m) \geq \frac{1}{2}\eta N(s_t, m)\right) \cdot 1 \\
&+ 1 \cdot \Pr\left(N(s_t, a_t, m) < \frac{1}{2}\eta N(s_t, m)\right) \qquad \text{(C.46)} \\
\leq &C' \exp\left(-\frac{1}{2}k'\eta N(s_t, m)\right) + 2\exp\left(-\frac{1}{2}\eta^2 N(s_t, m)\right) \qquad \text{(C.47)} \\
\leq &C \exp(-kN(s_t, m)), \qquad \text{(C.48)}
\end{aligned}$$

where (C.47) uses the assumed Inequality (C.38) with the condition $N(s_t, a_t) \geq \eta N(s_t)/2$, and also uses the bound from (C.44). The final inequality (C.48) follows from Lemma C.2.1, with $C = C' + 2$ and $k = \min(-k'\eta/2, -\eta^2/2)$. □

TODO: consider just defining a logsumexp function, and define the temp to have at zero it equal to max. Also finish writing up the preliminaries (commented out below)

Similar to Lemma C.2.4, it will also be convenient to be able to translate bounds that depend on $N(s')$, for some $s' \in \text{Succ}(s, a)$, into bounds that depend on $N(s, a)$:

Lemma C.2.5. *Consider any Boltzmann MCTS process. For some state action pair (s_t, a_t), and for some $s'_{t+1} \in \text{Succ}(s_t, a_t)$, suppose for some $C', k' > 0$ that some event E admits a bound:*

$$\Pr(E) \leq C' \exp(-k' N(s'_t)). \qquad \text{(C.49)}$$

Then, there exists $C, k > 0$ such that:

$$\Pr(E) \leq C \exp(-kN(s_t, a_t)). \qquad \text{(C.50)}$$

Proof outline. Proof is similar to Lemma C.2.4. Instead of having a minimum probability of selecting an action η, replace it with the corresponding probability from the transition distribution $p(s_{t+1}|s_t, a_t)$. Then swapping any $N(s_t)$ with $N(s_t, a_t)$, any $N(s_t, a_t)$ with $N(s'_t)$, and using $N(s_{t+1}) = \sum_{i \in T(s_t, a_t)} \mathbb{1}[s^i_{t+1} = s_{t+1}]$ (Equation (4.42)) as the sum of indicator random variables will give the result. □

TODO: <end> dump

C.2.2 Maximum Entropy Reinforcement Learning Results

TODO: change m to n, and use mth when need to reason about prev trials (whole subsection)

TODO: DEFINITELY APPENDIX MATERIAL

This subsection shows some basic results that relate the maximum entropy and standard objectives, which will be used to show results about MENTS. This subsection temporarily reintroduces the time-step parameter into value functions to simplify other notation. Two results about soft Q-values are given: first, that the optimal standard Q-value is less than the optimal soft Q-value, and secondly, that given a sufficiently small temperature, the optimal soft Q-values will preserve any *strict* ordering over actions given by the optimal standard Q-values.

TODO: recall the max entropy objective, and ref that below

For some policy π, the definition of V^π_{sft} (Equation (TODO: ref)) can be rearranged, to give a relation between the soft Q-value, the standard Q-value and the entropy of the policy:

$$Q^\pi_{\text{sft}}(s, a; t) = Q^\pi(s, a; t) + \alpha \mathbb{E}_\pi \left[\sum_{i=t+1}^{H} \mathcal{H}\left(\pi(\cdot|s_i)\right) \middle| s_t = s, a_t = a \right], \tag{C.51}$$

$$= Q^\pi(s, a; t) + \alpha \mathcal{H}_{t+1}(\pi | s_t = s, a_t = a), \tag{C.52}$$

where $\mathcal{H}_{t+1}$ is used as a shorthand for the entropy term. By using this relation, it can be shown that the optimal soft Q-value will always be at least as large as the optimal standard Q-value:

Lemma C.2.6. $Q^*(s, a; t) \leq Q^*_{\text{sft}}(s, a; t)$.

Proof. Taking a maximum over policies in Equation (C.52), and considering that π^*, the optimal standard policy, is one of the possible policies considered in the maximisation, gives the result:

$$Q^*_{\text{sft}}(s,a;t) = \max_{\pi} \left(Q^{\pi}(s,a;t) + \alpha \mathcal{H}_{t+1}(\pi|s_t = s, a_t = a)\right) \tag{C.53}$$

$$\geq Q^{\pi^*}(s,a;t) + \alpha \mathcal{H}_{t+1}(\pi^*|s_t = s, a_t = a) \tag{C.54}$$

$$\geq Q^*(s,a;t). \tag{C.55}$$

Noting that the entropy function is non-negative function. □

TODO: make below a defn? ALSO THIS DEFN IS NOT APPENDIX MATERIAL

The optimal soft and standard values can be 'tied together' by picking a very low temperature. Let $\delta(s,t)$ be the set of actions that have different optimal standard Q-values, that is $\delta(s,t) = \{(a,a')|Q^*(s,a;t) \neq Q^*(s,a';t)\}$. Now define $\Delta_{\mathcal{M}}$ as follows:

$$\Delta_{s,t} = \min_{(a,a')\in\delta(s,t)} |Q^*(s,a;t) - Q^*(s,a';t)|, \tag{C.56}$$

$$\Delta_{\mathcal{M}} = \min_{s,t} \Delta_{s,t}. \tag{C.57}$$

Note in particular, for some $(a,a') \in \delta(s,t)$ that the definition of $\Delta_{\mathcal{M}}$ implies that if $Q^*(s,a;t) < Q^*(s,a';t)$ then

$$Q^*(s,a;t) + \Delta_{\mathcal{M}} \leq Q^*(s,a';t). \tag{C.58}$$

Using this problem dependent constant, $\alpha < \Delta_{\mathcal{M}}/H \log|\mathcal{A}|$ is a sufficient condition for the optimal standard and optimal soft policies to coincide, a consequence of the following Lemma.

Lemma C.2.7. *If $\alpha < \Delta_{\mathcal{M}}/H \log|\mathcal{A}|$, then for all $t = 1, ..., H$, for all $s \in \mathcal{S}$ and for all $(a,a') \in \delta(s,t)$ we have $Q^*_{\text{sft}}(s,a;t) < Q^*_{\text{sft}}(s,a';t)$ iff $Q^*(s,a;t) < Q^*(s,a';t)$.*

Proof. ($\Leftarrow$) First consider that the optimal soft Q-value is less than or equal to the optimal standard Q-value and maximum possible entropy:

$$Q^*_{\text{sft}}(s,a;t) = \max_\pi \left(Q^\pi(s,a;t) + \alpha\mathcal{H}_{t+1}(\pi)\right) \tag{C.59}$$

$$\leq \max_\pi Q^\pi(s,a;t) + \max_\pi \alpha\mathcal{H}_{t+1}(\pi) \tag{C.60}$$

$$= Q^*(s,a;t) + \alpha(H-t)\log|\mathcal{A}| \tag{C.61}$$

$$\leq Q^*(s,a;t) + \alpha H \log|\mathcal{A}|. \tag{C.62}$$

Then, using $\alpha < \Delta_{\mathcal{M}}/H\log|\mathcal{A}|$, $Q^*(s,a;t)+\Delta_{\mathcal{M}} \leq Q^*(s,a';t)$ and $Q^*(s,a';t) \leq Q_{\text{sft}}(s,a;t)$ from Lemma C.2.6 gives the desired inequality:

$$Q^*_{\text{sft}}(s,a;t) \leq Q^*(s,a;t) + \alpha H\log|\mathcal{A}| \tag{C.63}$$

$$< Q^*(s,a;t) + \Delta_{\mathcal{M}} \tag{C.64}$$

$$\leq Q^*(s,a';t) \tag{C.65}$$

$$\leq Q^*_{\text{sft}}(s,a';t). \tag{C.66}$$

($\Rightarrow$) To show that $Q^*_{\text{sft}}(s,a;t) < Q^*_{\text{sft}}(s,a';t) \Rightarrow Q^*(s,a;t) < Q^*(s,a';t)$ it is easier to show the contrapostive instead, which is $Q^*(s,a;t) \geq Q^*(s,a';t) \Rightarrow Q^*_{\text{sft}}(s,a;t) \geq Q^*_{\text{sft}}(s,a';t)$. Given that it is assumed that $(a,a') \in \delta(s,t)$, the following implications hold:

$$Q^*(s,a;t) \geq Q^*(s,a';t) \tag{C.67}$$

$$\Rightarrow Q^*(s,a;t) > Q^*(s,a';t) \tag{C.68}$$

$$\Rightarrow Q^*_{\text{sft}}(s,a;t) > Q^*_{\text{sft}}(s,a';t) \tag{C.69}$$

$$\Rightarrow Q^*_{\text{sft}}(s,a;t) \geq Q^*_{\text{sft}}(s,a';t), \tag{C.70}$$

where the first implication uses that $(a,a') \in \delta(s)$, the second reuses the ($\Leftarrow$) proof. TODO: Try to respect reader more, dont need to explain the last line... (e.g. cut after this), and the final implication holds generally. □

C.2.3 General Q-value Convergence Result

TODO: This section contains things related to exponential bounds, which might move to appendix

Recall the (soft) Q-value backups used by Boltzmann MCTS processes (Equations (**??**) and (C.7)). Considering that these backups are of identical form, a reward TODO: plus an empirical averaging of child nodes / MC estimate of expected child value. TODO: clean up old writing: Considering that the backups for MENTS and DENTS processes are of similar form, we will show that generally, backups of that form converge (exponentially), given that the values at any child nodes also converge (exponentially). However, towards showing this, we first need to consider the concentration of the empirical transition distribution around the true transition distribution.

TODO: <beg> dump (stuff put in main text)

Theorem C.2.2. *Let $\{X_i\}_{i=1}^m$ be random variables drawn from a probability distribution with a cumulative distribution function of F. Let the empirical cumulative distribution function be $F_m(x) = \frac{1}{m}\sum_{i=1}^m \mathbb{1}[X_i < x]$. Then the Dvoretzky-Kiefer-Wolfowitz inequality is:*

$$\Pr\left(\sup_x |F_m(x) - F(x)| > \varepsilon\right) \leq 2\exp\left(-2m\varepsilon^2\right). \tag{C.71}$$

Proof. See TODO: fix cite □

The Dvoretzky-Kiefer-Wolfowitz inequality is of interest because it allows the empirical transition probability $N(s_{t+1}, n)/N(s_t, a_t, n)$ to be tightly bounded with the true transition probability $p(s_{t+1}|s_t, a_t)$.

Corollary C.2.1. *Consider any Boltzmann MCTS process. For all $(s_t, a_t) \in \mathcal{S} \times \mathcal{A}$ and for all $\varepsilon > 0$ we have:*

$$\Pr\left(\max_{s_{t+1}\in \mathrm{Succ}(s_t,a_t)} \left|\frac{N(s_{t+1}, n)}{N(s_t, a_t, n)} - p(s_{t+1}|s_t, a_t)\right| > \varepsilon\right) \leq 2\exp\left(-\frac{1}{2}\varepsilon^2 N(s_t, a_t)\right). \tag{C.72}$$

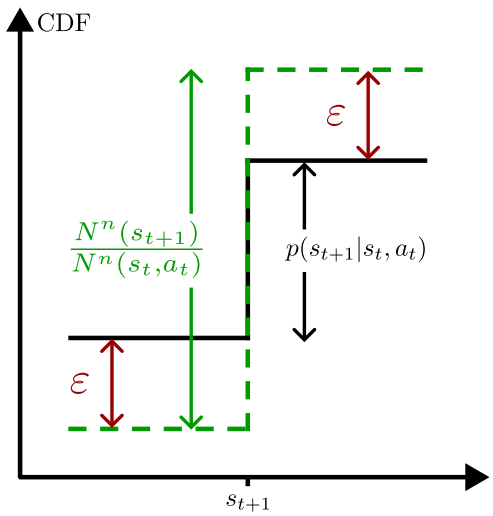


Figure C.1: Bounding the empirical transition probabilities to the true transition probabilities. The true cdf is shown as a solid black line, the empirical cdf is shown as a dashed green line, and a worst case error of $\varepsilon/2$, using Theorem C.2.2, is shown in red. The probability mass of $p(s_{t+1}|s_t, a_t)$ and empirical probability mass of $\frac{N(s_{t+1},n)}{N(s_t,a_t,n)}$ is also indicated to demonstrate how the constructed distribution gives Corollary C.2.1.

Proof outline. By considering some arbitrary ordering over the successor states in $\text{Succ}(s_t, a_t)$ and applying Theorem C.2.2, replacing ε by $\varepsilon/2$, the result follows.

To see why the factor of 1/2 is needed, consider Figure C.1. Because the distribution is discrete, the cumulative distribution function is a (piecewise constant) step function. As Theorem C.2.2 bounds the maximum difference between the empirical and true cumulative distribution functions, the factor of 1/2 is needed to account for the error before and after each s_{t+1} in the worst case. □

TODO: <end> dump (stuff put in main text)

Now that Corollary C.2.1 can be used to bound the empirical transition distribution to the true transition distribution, a general purpose concentration inequality for Q-values can be proved for use in later proofs.

Lemma C.2.8. *Consider any Boltzmann MCTS process, and some state action pair* $(s_t, a_t) \in \mathcal{S} \times \mathcal{A}$. *Let* $\dot{V}^{N(s,n)}(s) : \mathcal{S} \to \mathbb{R}$, $\dot{V}^*(s) : \mathcal{S} \to \mathbb{R}$ *be some estimated and optimal value functions respectively and suppose that for all* $s_{t+1} \in \text{Succ}(s_t, a_t)$ *that there is some* $C_{s_{t+1}}, k_{s_{t+1}} > 0$ *such that for all* $\varepsilon_0 > 0$*:*

$$\Pr\left(\left|\dot{V}^{N(s_{t+1})}(s_{t+1}) - \dot{V}^*(s_{t+1})\right| > \varepsilon_0\right) \leq C_{s_{t+1}} \exp(-k_{s_{t+1}} \varepsilon_0^2 N(s')). \tag{C.73}$$

If the optimal and estimated Q-values are defined as follows:

$$\dot{Q}^*(s,a) = R(s,a) + \mathbb{E}_{s'\sim p(\cdot|s,a)}\left[\dot{V}^*(s')\right], \tag{C.74}$$

$$\dot{Q}^{N(s,a,n)}(s,a) = R(s,a) + \sum_{s'\in \mathrm{Succ}(s,a)} \left[\frac{N(s',n)}{N(s,a,n)}\dot{V}^{N(s',n)}(s')\right]. \tag{C.75}$$

Then there exists some $C, k > 0$, for all $\varepsilon > 0$ such that:

$$\Pr\left(\left|\dot{Q}^{N(s,a,n)}(s,a) - \dot{Q}^*(s,a)\right| > \varepsilon\right) \leq C\exp(-k\varepsilon^2 N(s,a,n)). \tag{C.76}$$

Proof. By the assumed bounds, Lemma C.2.1 and Lemma C.2.5, there is some $C_1, k_1 > 0$, such that for any $\varepsilon_1 > 0$:

$$\Pr\left(\forall s_{t+1}. \left|\dot{V}^{N(s_{t+1})}(s_{t+1}) - \dot{V}^*(s_{t+1})\right| \leq \varepsilon_1\right) > 1 - C_1\exp(-k_1\varepsilon_1^2 N(s_t,a_t,n)). \tag{C.77}$$

And recall that for any $p(s_{t+1}|s_t,a_t) > \varepsilon_2 > 0$, using Corollary C.2.1 that:

$$\Pr\left(\max_{s_{t+1}\in\mathrm{Succ}(s_t,a_t)}\left|\frac{N(s_{t+1},n)}{N(s_t,a_t,n)} - p(s_{t+1}|s_t,a_t)\right| \leq \varepsilon_2\right) > 1 - 2\exp\left(-\frac{1}{2}\varepsilon_2^2 N(s_t,a_t,n)\right). \tag{C.78}$$

If the events in Inequalities (C.77) and (C.78) hold, then the following inequalities must also hold:

$$\dot{V}^*(s_{t+1}) - \varepsilon_1 \leq \dot{V}^{N_n(s_{t+1},n)}(s_{t+1}) \leq \dot{V}^*(s_{t+1}) + \varepsilon_1 \tag{C.79}$$

$$p(s_{t+1}|s_t,a_t) - \varepsilon_2 \leq \frac{N(s_{t+1},n)}{N(s_t,a_t,n)} \leq p(s_{t+1}|s_t,a_t) + \varepsilon_2. \tag{C.80}$$

The upper bounds on $\dot{V}^{N_n(s_{t+1},n)}(s_{t+1})$ and $N(s_{t+1},n)/N(s_t,a_t,n)$ can be used to obtain an upper bound on $\dot{Q}^{N(s_t,a_t,n)}(s_t,a_t)$:

$$\dot{Q}^{N(s_t,a_t,n)}(s_t,a_t) \tag{C.81}$$

$$= R(s_t,a_t) + \sum_{s_{t+1}\in\mathrm{Succ}(s_t,a_t)} \frac{N(s_{t+1},n)}{N(s_t,a_t,n)}\dot{V}^{N(s_{t+1},n)}(s_{t+1}) \tag{C.82}$$

$$\leq R(s_t,a_t) + \sum_{s_{t+1}\in\mathrm{Succ}(s_t,a_t)} (p(s_{t+1}|s_t,a_t) + \varepsilon_2)(\dot{V}^*(s_{t+1}) + \varepsilon_1) \tag{C.83}$$

$$= R(s_t,a_t) + \mathbb{E}_{s_{t+1}\sim p(\cdot|s_t,a_t)}[\dot{V}^*(s_{t+1})] + \varepsilon_1 + \varepsilon_2 \sum_{s_{t+1}\in\mathrm{Succ}(s_t,a_t)} \dot{V}^*(s_{t+1}) + \varepsilon_1\varepsilon_2 \tag{C.84}$$

$$\leq R(s_t,a_t) + \mathbb{E}_{s_{t+1}\sim p(\cdot|s_t,a_t)}[\dot{V}^*(s_{t+1})] + \varepsilon_2\left|\sum_{s_{t+1}\in\mathrm{Succ}(s_t,a_t)} \dot{V}^*(s_{t+1})\right| + \varepsilon_1 + \varepsilon_1\varepsilon_2 \tag{C.85}$$

$$= \dot{Q}^*(s_t,a_t) + \varepsilon_2\left|\sum_{s_{t+1}\in\mathrm{Succ}(s_t,a_t)} \dot{V}^*(s_{t+1})\right| + \varepsilon_1 + \varepsilon_1\varepsilon_2. \tag{C.86}$$

Following the similar reasoning but using the lower bounds on $\dot{V}^{N_n(s_{t+1},n)}(s_{t+1})$ and $N(s_{t+1},n)/N(s_t,a_t,n)$ gives: TODO: There is some special cases when V is negative, having to use the other bound, but the upper and lower bound have a bit of leway which handles all four cases of using plus minus epsilon one and two.

$$\dot{Q}^{N(s_t,a_t,n)}(s_t,a_t) \geq \dot{Q}^*(s_t,a_t) - \varepsilon_2 \left| \sum_{s_{t+1}\in \mathrm{Succ}(s_t,a_t)} \dot{V}^*(s_{t+1}) \right| - \varepsilon_1 - \varepsilon_1\varepsilon_2. \tag{C.87}$$

Now given an arbitrary $\varepsilon > 0$, recalling that $\varepsilon_1, \varepsilon_2 > 0$ were arbitrary, set $\varepsilon_1 = \varepsilon/3$ and $\varepsilon_2 = \min(\varepsilon/3, \varepsilon/3|\sum_{s_{t+1}} \dot{V}^*(s_{t+1})|)$ to give:

$$\dot{Q}^*(s_t,a_t) - \varepsilon \leq \dot{Q}^{N(s_t,a_t,n)}(s_t,a_t) \leq \dot{Q}^*(s_t,a_t) + \varepsilon. \tag{C.88}$$

Using Lemma C.2.1 liberally, there is some $C_2, C_3, k_2, k_3 > 0$ such that:

$$\Pr\left(\left|\dot{Q}^{N(s_t,a_t,n)}(s_t,a_t) - \dot{Q}^*(s_t,a_t)\right| \leq \varepsilon\right) \tag{C.89}$$

$$> \left(1 - C_1 \exp(-k_1\varepsilon_1^2 N(s_t,a_t,n))\right)\left(1 - 2\exp\left(-\frac{1}{2}\varepsilon_2^2 N(s_t,a_t,n)\right)\right) \tag{C.90}$$

$$= \left(1 - C_2 \exp(-k_2\varepsilon^2 N(s_t,a_t,n)\right) \cdot \left(1 - C_3 \exp(-k_3\varepsilon^2 N(s_t,a_t,n)\right) \tag{C.91}$$

$$> 1 - C_2 \exp(-k_2\varepsilon^2 N(s_t,a_t,n)) - C_3 \exp(-k_3\varepsilon^2 N(s_t,a_t,n)). \tag{C.92}$$

Finally, by negating and setting $C = C_2 + C_3$ and $k = \min(k_2, k_3)$ in Lemma TODO: ref the result follows:

$$\Pr\left(\left|\dot{Q}^{N(s_t,a_t,n)}(s_t,a_t) - \dot{Q}^*(s_t,a_t)\right| > \varepsilon\right) \leq C\exp(-k\varepsilon^2 N(s_t,a_t,n)). \tag{C.93}$$

□

C.2.4 MENTS Results

This subsection provides results related to MENTS in the setting of that standard reinforcement learning objective. Concentration inequalities are given around the optimal soft values to start with, although this result is similar to soem of the results provided in [69] TODO: this is still provided to keep this thesis' proofs self contained. (although its not as we use lots of things like hoeffdings, and also use UCT style proofs prsumably later on.) We could say "to keep the proofs compatible with the

other results provided. Or we could just omit it and reference the MENTS paper..." Afterwards, the concentration inequalities are combined with results about maximum entropy reinforcement learning TODO: ref the section where we did that to provide bounds on the simple regret of MENTS, given constraints on the temperature.

To prove the concentration inequality around the optimal soft values, start by showing an inductive step.

Lemma C.2.9. *Consider a MENTS process. Let $s_t \in \mathcal{S}$, with $1 \leq t \leq H$ TODO: update for thtspp notation. If for all $s_{t+1} \in \bigcup_{a \in \mathcal{A}} \mathrm{Succ}(s_t, a)$ there is some $C_{s_{t+1}}, k_{s_{t+1}} > 0$ for any $\varepsilon_{s_{t+1}} > 0$:*

$$\Pr\left(\left|\hat{V}_{\mathrm{MENTS}}^{(N(s_{t+1},n))}(s_{t+1}) - V_{\mathrm{sft}}^{*}(s_{t+1})\right| > \varepsilon_{s_{t+1}}\right) \leq C_{s_{t+1}} \exp\left(-k_{s_{t+1}} \varepsilon_{s_{t+1}}^2 N(s_{t+1}, n)\right), \tag{C.94}$$

then there is some $C, k > 0$, for any $\varepsilon > 0$:

$$\Pr\left(\left|\hat{V}_{\mathrm{MENTS}}^{(N(s_t,n))}(s_t) - V_{\mathrm{sft}}^{*}(s_t)\right| > \varepsilon\right) \leq C \exp\left(-k\varepsilon^2 N(s_t, n)\right). \tag{C.95}$$

Proof. Given the assumptions and by Lemmas C.2.8, C.2.4 and C.2.1, there is some $C, k > 0$ such that for any $\varepsilon > 0$:

$$\Pr\left(\forall a_t \in \mathcal{A}. \left|\hat{Q}_{\mathrm{MENTS}}^{(N(s_t,a_t,n))}(s_t, a_t) - Q_{\mathrm{sft}}^{*}(s_t, a_t)\right| \leq \varepsilon\right) > 1 - C\exp(-k\varepsilon^2 N(s_t, n)). \tag{C.96}$$

So with probability at least $1 - C\exp(-k\varepsilon^2 N(s_t, n))$, for any a_t, the following holds:

$$Q_{\mathrm{sft}}^{*}(s_t, a_t) - \varepsilon \leq \hat{Q}_{\mathrm{MENTS}}^{(N(s_t,a_t,n))}(s_t, a_t) \leq Q_{\mathrm{sft}}^{*}(s_t, a_t) + \varepsilon. \tag{C.97}$$

Using the upper bound on $\hat{Q}_{\mathrm{MENTS}}^{(N(s_t,a_t,n))}(s_t, a_t)$ in the soft backup equation for $\hat{V}_{\mathrm{MENTS}}^{(N(s_t,n))}(s_t)$ (Equation (**??**) gives:

$$\begin{aligned}
\hat{V}_{\mathrm{MENTS}}^{(N(s_t,n))}(s_t) &= \alpha \log \sum_{a \in \mathcal{A}} \exp\left(\frac{\hat{Q}_{\mathrm{MENTS}}^{(N(s_t,a_t,n))}(s_t, a_t)}{\alpha}\right) && \text{(C.98)}\\
&\leq \alpha \log \sum_{a \in \mathcal{A}} \exp\left(\frac{Q_{\mathrm{sft}}^{*}(s_t, a_t) + \varepsilon}{\alpha}\right) && \text{(C.99)}\\
&= \left[\alpha \log \sum_{a \in \mathcal{A}} \exp\left(\frac{Q_{\mathrm{sft}}^{*}(s_t, a_t)}{\alpha}\right)\right] + \varepsilon && \text{(C.100)}\\
&= V_{\mathrm{sft}}^{*}(s_t) + \varepsilon, && \text{(C.101)}
\end{aligned}$$

noting that the *softmax* function monotonically increases in its arguments. Then with similar reasoning using the lower bound on $\hat{Q}_{\text{MENTS}}^{(N(s_t,a_t,n))}(s_t, a_t)$ gives:

$$\hat{V}_{\text{MENTS}}^{(N(s_t,n))}(s_t) \geq V_{\text{sft}}^*(s_t) - \varepsilon, \tag{C.102}$$

and hence:

$$|\hat{V}_{\text{MENTS}}^{(N(s_t,n))}(s_t) - V_{\text{sft}}^*(s_t)| \leq \varepsilon. \tag{C.103}$$

This therefore shows: TODO: some comment about how if A implies B then pr(B) more than pr(A)

$$\Pr\left(\left|\hat{V}_{\text{MENTS}}^{(N(s_t,n))}(s_t) - V_{\text{sft}}^*(s_t)\right| \leq \varepsilon_1\right) > 1 - C\exp(-k\varepsilon^2 N(s_t, n)), \tag{C.104}$$

and negating probabilities gives the result. □

Then completing the induction gives the concentration inequalities desired for any state that MENTS might visit.

Theorem C.2.3. *Consider a MENTS process, let $s_t \in \mathcal{S}$ then there is some $C, k > 0$ for any $\varepsilon > 0$:*

$$\Pr\left(\left|\hat{V}_{\text{MENTS}}^{(N(s_t,n))}(s_t) - V_{\text{sft}}^*(s_t)\right| > \varepsilon\right) \leq C\exp\left(-k\varepsilon^2 N(s_t, n)\right). \tag{C.105}$$

Moreover, at the root node s_0:

$$\Pr\left(\left|\hat{V}_{\text{MENTS}}^{(N(s_0,n))}(s_0) - V_{\text{sft}}^*(s_0)\right| > \varepsilon\right) \leq C\exp\left(-k\varepsilon^2 n\right). \tag{C.106}$$

And hence $\hat{V}_{\text{MENTS}}^{(N(s_t,n))}(s_t) \xrightarrow{p} V_{\text{sft}}^*(s_t)$.

Proof. Consider that the result holds for $t = H + 1$, because $\hat{V}_{\text{MENTS}}^{(N(s_{H+1},n))}(s_{H+1}) = V_{\text{sft}}^*(s_{H+1}) = 0$. Therefore the result holds for any $t = 0, ..., H+1$ by induction using Lemma C.2.9. Noting that $N(s_0) = n$ gives (C.106). TODO: To see the convergence in probability, let $n \to \infty$. TODO: Double check the H stuf is consistent with the thtspp and MDP defns □

Provided MENTS's temperature parameter is set small enough, such that the optimal standard and soft values lead to identical recommendation policies TODO: ref the max entropy proof of this, then MENTS is consistent and the expected simple regret tends to zero exponentially. This is shown in the following lemma.

Lemma C.2.10. *Consider a MENTS process with* $\alpha_{\text{MENTS}} < \Delta_{\mathcal{M}}/3H \log|\mathcal{A}|$. *TODO: Make sure replaced alpha with alphaments in ments theorems Let* $s_t \in \mathcal{S}$, *with* $1 \leq t \leq H$ *then there is some* $C', k' > 0$ *such that:*

$$\mathbb{E}\texttt{sim_regr}(s_t, \psi^n_{\text{MENTS}}) \leq C' \exp\left(-k' N(s_t, n)\right). \tag{C.107}$$

Proof. TODO: Why cant we just replace this with, the values converge in probability from the previous result, and then use that it means the recommendation policy converges in probability, and then use that to say simple regret go zero? The case where it doesnt work is if there is suboptimal soft value which equals the optimal soft value (which has no entropy), in this case, becase we need the value ESTIMATES to converge, not the optimal values, we need the temperature to be a bit stricter. I think

Let a^* be the optimal action with respect to the soft values, so $a^* = \arg\max_{a \in \mathcal{A}} Q^*_{\text{sft}}(s_t, a)$. Then by Lemma C.2.7 a^* must also be the optimal action for the standard Q-value function $a^* = \arg\max_{a \in \mathcal{A}} Q^*(s_t, a)$. By Theorem C.2.3 and Lemmas C.2.8 and C.2.4 there exists $C_1, k_1 > 0$ such that for all $\varepsilon_1 > 0$: TODO: reword this paragraph, its a bit meh

$$\Pr\left(\forall a_t \in \mathcal{A} \left|\hat{Q}^{(N(s_t,a_t,n))}_{\text{MENTS}}(s_t, a_t) - Q^*_{\text{sft}}(s_t, a_t)\right| \leq \varepsilon_1\right) > 1 - C_1 \exp(-k_1 \varepsilon_1^2 N(s_t, n)). \tag{C.108}$$

Setting $\varepsilon_1 = \Delta_{\mathcal{M}}/3H \log|\mathcal{A}|$ then gives with probability at least $1 - C_1 \exp(-k_2 N(s_t, n))$ (where $k_2 = k_1 \left(\frac{\Delta_{\mathcal{M}}}{3H \log|\mathcal{A}|}\right)^2$) that for all actions $a_t \in \mathcal{A}$:

$$Q^*_{\text{sft}}(s_t, a_t) - \Delta_{\mathcal{M}}/3 \leq \hat{Q}^{(N(s_t,a_t,n))}_{\text{MENTS}}(s_t, a_t) \leq Q^*_{\text{sft}}(s_t, a_t) + \Delta_{\mathcal{M}}/3. \tag{C.109}$$

And hence, with probability at least $1-C_1\exp(-k_2N(s_t,n))$, for all $a\in\mathcal{A}-\{a^*\}$ we have:

$$\begin{aligned}
\hat{Q}^{(N(s_t,a,n))}_{\text{MENTS}}(s_t,a) \leq& Q^*_{\text{sft}}(s_t,a)+\Delta_{\mathcal{M}}/3 && \text{(C.110)}\\
\leq& Q^*(s_t,a)+\alpha H\log|\mathcal{A}|+\Delta_{\mathcal{M}}/3 && \text{(C.111)}\\
\leq& Q^*(s_t,a)+2\Delta_{\mathcal{M}}/3 && \text{(C.112)}\\
\leq& Q^*(s_t,a^*)-\Delta_{\mathcal{M}}/3 && \text{(C.113)}\\
\leq& Q^*_{\text{sft}}(s_t,a^*)-\Delta_{\mathcal{M}}/3 && \text{(C.114)}\\
\leq& \hat{Q}^{(N(s_t,a^*,n))}_{\text{MENTS}}(s_t,a^*). && \text{(C.115)}
\end{aligned}$$

Where in the above, the first line holds from the upper bound on $\hat{Q}^{(N(s_t,a_t,n))}_{\text{MENTS}}(s_t,a_t)$; The second holds from maximising the standard return and entropy portions of the soft value separately (recall Inequality (C.60) in Lemma C.2.7); The third holds from the assumption on α; The fourth holds from the definition of $\Delta_{\mathcal{M}}$ (also see Inequality (C.58)); The fifth holds from the optimal soft value being greater than the optimal standard value (Lemma C.2.6); And the final line holds by using the lower bound on $\hat{Q}^{(N(s_t,a_t,n))}_{\text{MENTS}}(s_t,a_t)$ given above with $a_t=a^*$.

Negating the probability that (C.115) holds gives:

$$\Pr\left(\exists a_t\in\mathcal{A}-\{a^*\}.\left(\hat{Q}^{(N(s_t,a))}_{\text{MENTS}}(s_t,a)>\hat{Q}^{(N(s_t,a^*,n))}_{\text{MENTS}}(s_t,a^*)\right)\right)\leq C_1\exp(-k_2N(s_t)). \tag{C.116}$$

The expected immediate regret can be bounded as follows:

$$\begin{aligned}
&\mathbb{E}\texttt{inst_regr}(s_t,\psi^n_{\text{MENTS}}) && \text{(C.117)}\\
=&\sum_{a\in\mathcal{A}-\{a^*\}}(V^*(s_t)-Q^*(s_t,a))\Pr\left(\psi^n_{\text{MENTS}}(s_t)=a\right) && \text{(C.118)}\\
=&\sum_{a\in\mathcal{A}-\{a^*\}}(V^*(s_t)-Q^*(s_t,a))\Pr\left(a=\arg\max_{a'}\hat{Q}^{(N(s_t,a',n))}_{\text{MENTS}}(s_t,a')\right) && \text{(C.119)}\\
\leq&\sum_{a\in\mathcal{A}-\{a^*\}}(V^*(s_t)-Q^*(s_t,a))\Pr\left(\hat{Q}^{(N(s_t,a,n))}_{\text{MENTS}}(s_t,a)>\hat{Q}^{(N(s_t,a^*,n))}_{\text{MENTS}}(s_t,a^*)\right) && \text{(C.120)}\\
\leq&\sum_{a\in\mathcal{A}-\{a^*\}}(V^*(s_t)-Q^*(s_t,a))\,C_1\exp(-k_2N(s_t,n)), && \text{(C.121)}
\end{aligned}$$

where $k'=k_2$ and $C'=C_1\sum_{a\in\mathcal{A}-\{a^*\}}(V^*(s_t)-Q^*(s_t,a))$. Finally, using TODO: ref lemma gives the result. □

In consequence to Lemma C.2.10, provided the sufficient conditions are met, MENTS is consistent and will converge to a simple regret of zero.

Theorem C.2.4. *Consider a MENTS process with* $\alpha_{\text{MENTS}} < \Delta_{\mathcal{M}}/3H \log |\mathcal{A}|$. *TODO: H consistent with thtspp Let* $s_t \in \mathcal{S}$, *with* $1 \leq t \leq H$ *TODO: H consistent with thtspp then there is some* $C', k' > 0$ *such that:*

$$\mathbb{E}\texttt{sim_regr}(s_t, \psi^n_{\text{MENTS}}) \leq C' \exp\left(-k' N(s_t, n)\right). \tag{C.122}$$

And specifically, at the root node s_0*:*

$$\mathbb{E}\texttt{sim_regr}(s_0, \psi^n_{\text{MENTS}}) \leq C' \exp\left(-k' n)\right). \tag{C.123}$$

Proof. This theorem holds as a consequence of Corollary **??** and Lemma C.2.10, and noting that at the root node $N(s_0, n) = n$. □

TODO: this was a theorem in the main neurips paper. Need to decide how writing this up, so this may go TODO: We have now shown Theorem **??**: TODO: theorem restate

Proof. This is part of Theorem C.2.4. □

C.2.5 DENTS (and BTS) Results

TODO: This is mostly the exponential bound stuff, so can also probably be appendix stuff

This subsection provides theoretical results that give exponential convergence of DENTS value estimates and simple regret. TODO: Some note on no constraints necessary on the parameters of DENTS?

Lemma C.2.11. *TODO: maybe should do directly from the Q values? Also same for the ments one? Consider a DENTS process. Let* $s_t \in \mathcal{S}$, *with* $1 \leq t \leq H$. *TODO: make H consistent with thtspp If for all* $s_{t+1} \in \bigcup_{a \in \mathcal{A}} \text{Succ}(s_t, a)$ *there is some* $C_{s_{t+1}}, k_{s_{t+1}} > 0$ *such that for any* $\varepsilon_{s_{t+1}} > 0$*:*

$$\Pr\left(\left|\hat{V}^{(N(s_{t+1},n))}_{\text{DNTS}}(s_{t+1}) - V^*(s_{t+1})\right| > \varepsilon_{s_{t+1}}\right) \leq C_{s_{t+1}} \exp\left(-k_{s_{t+1}} \varepsilon^2_{s_{t+1}} N(s_{t+1}, n)\right), \tag{C.124}$$

then there is some $C, k > 0$, for any $\varepsilon > 0$:

$$\Pr\left(\left|\hat{V}_{\text{DNTS}}^{(N(s_t,n))}(s_t) - V^*(s_t)\right| > \varepsilon\right) \leq C\exp\left(-k\varepsilon^2 N(s_t,n)\right). \tag{C.125}$$

Proof. Given the assumptions and by Lemmas C.2.8, C.2.1 and C.2.4, for some $C, k > 0$ and for any $\varepsilon_1 > 0$:

$$\Pr\left(\forall a_t \in \mathcal{A}. \left|\hat{Q}_{\text{DNTS}}^{(N(s_t,a_t,n))}(s_t,a_t) - Q^*(s_t,a_t)\right| \leq \varepsilon_1\right) > 1 - C\exp(-k\varepsilon_1^2 N(s_t)). \tag{C.126}$$

Let $\varepsilon > 0$, and set $\varepsilon_1 = \min(\varepsilon, \Delta_{\mathcal{M}}/2)$. So with probability at least $1 - C_1\exp(-k_1\varepsilon_1^2 N(s_t,n))$ for any a_t it must be that:

$$Q^*(s_t,a_t) - \varepsilon_1 \leq \hat{Q}_{\text{DNTS}}^{(N(s_t,a_t,n))}(s_t,a_t) \leq Q^*(s_t,a_t) + \varepsilon_1. \tag{C.127}$$

Let $a^* = \max_{a\in\mathcal{A}} Q^*(s_t,a)$. Using $\varepsilon_1 \leq \Delta_{\mathcal{M}}/2$ in (C.127), then for any $a \in \mathcal{A} - \{a^*\}$:

$$\begin{aligned}
\hat{Q}_{\text{DNTS}}^{(N(s_t,a,n))}(s_t,a) \leq& Q^*(s_t,a) + \Delta_{\mathcal{M}}/2 && \text{(C.128)}\\
\leq& Q^*(s_t,a^*) - \Delta_{\mathcal{M}}/2 && \text{(C.129)}\\
\leq& \hat{Q}_{\text{DNTS}}^{(N(s_t,a^*,n))}(s_t,a^*), && \text{(C.130)}
\end{aligned}$$

and hence $\arg\max_{a\in\mathcal{A}} \hat{Q}_{\text{DNTS}}^{(N(s_t,a,n))}(s_t,a) = a^*$. As a consequence:

$$\hat{V}_{\text{DNTS}}^{(N(s_t,n))}(s_t) = \max_a \hat{Q}_{\text{DNTS}}^{(N(s_t,a,n))}(s_t,a) = \hat{Q}_{\text{DNTS}}^{(N(s_t,a^*,n))}(s_t,a^*). \tag{C.131}$$

Then using (C.127) with $a_t = a^*$ (noting $V^*(s_t) = Q^*(s_t,a^*)$), using (C.131) and using $\varepsilon_1 \leq \varepsilon$ gives:

$$\begin{aligned}
V^*(s_t) - \varepsilon \leq& V^*(s_t) - \varepsilon_1 && \text{(C.132)}\\
\leq& \hat{V}_{\text{DNTS}}^{(N(s_t,n))}(s_t) && \text{(C.133)}\\
\leq& V^*(s_t) + \varepsilon_1 && \text{(C.134)}\\
\leq& V^*(s_t) + \varepsilon. && \text{(C.135)}
\end{aligned}$$

Hence:

$$\begin{aligned}
\Pr\left(\left|\hat{V}_{\text{DNTS}}^{(N(s_t,n))}(s_t) - V^*(s_t)\right| > \varepsilon\right) &\leq C\exp(-k\varepsilon_1^2 N(s_t,n)) && \text{(C.136)}\\
&\leq C\exp(-k\varepsilon^2 N(s_t,n)), && \text{(C.137)}
\end{aligned}$$

which is the result. □

Similarly to the MENTS section, Lemma C.2.11 provides an inductive step, which is used in Theorem C.2.5 to show concentration inequalities at all states that DENTS visits.

Theorem C.2.5. *Consider a DENTS process, let $s_t \in \mathcal{S}$ then there is some $C, k > 0$ for any $\varepsilon > 0$:*

$$\Pr\left(\left|\hat{V}_{\text{DNTS}}^{(N(s_t,n))}(s_t) - V^*(s_t)\right| > \varepsilon\right) \leq C \exp\left(-k\varepsilon^2 N(s_t, n)\right). \tag{C.138}$$

Moreover, at the root node s_0:

$$\Pr\left(\left|\hat{V}_{\text{DNTS}}^{(N(s_0,n))}(s_0) - V^*(s_0)\right| > \varepsilon\right) \leq C \exp\left(-k\varepsilon^2 n\right). \tag{C.139}$$

Proof. The result holds for $t = H + 1$ TODO: consistent with thtspp. SHOULD JUST CTRL-F THIS and sort all of these out at once. because $\hat{V}_{\text{DNTS}}^{(N(s_{H+1}))}(s_{H+1}) = V^*(s_{H+1}) = 0$. TODO: might be an off by one error here dep on how defined H, check over this whole paragraph. Hence the result holds for all $t = 1, ..., H + 1$ by induction using Lemma C.2.11. Noting that $N(s_0) = n$ gives (C.139). □

TODO: is this just repeated? Can we merge two of the proofs?

Again, the concentration inequalities are used to show that the simple regret tends to zero exponentially, and therefore that DENTS will be exponentially likely in the number of visits to recommend the optimal standard action at every node.

Lemma C.2.12. *Consider a DENTS process. Let $s_t \in \mathcal{S}$, with $1 \leq t \leq H$ TODO: defn with thtspp for H agen then there is some $C', k' > 0$ such that:*

$$\mathbb{E}\texttt{inst_regr}(s_t, \psi_{\text{DNTS}}^n) \leq C' \exp\left(-k' N(s_t, n)\right). \tag{C.140}$$

Proof. Let a^* be the locally optimal standard action, so $a^* = \arg\max_{a \in \mathcal{A}} Q^*(s_t, a)$. By Theorem C.2.5 and Lemmas C.2.8 and C.2.4 there exists $C_1, k_1 > 0$ such that for all $\varepsilon_1 > 0$:

$$\Pr\left(\forall a_t \in \mathcal{A}. \left|\hat{Q}_{\text{DNTS}}^{(N(s_t,a_t,n))}(s_t, a_t) - Q^*(s_t, a_t)\right| \leq \varepsilon_1\right) > 1 - C_1 \exp(-k_1 \varepsilon_1^2 N(s_t, n)). \tag{C.141}$$

Setting $\varepsilon_1 = \Delta_{\mathcal{M}}/2$ then gives with probability $1 - C_1 \exp(-k_2 N(s_t, n))$ (where $k_2 = k_1 \Delta_{\mathcal{M}}/2$) for all actions $a \in \mathcal{A}$ that:

$$Q^*(s_t, a) - \Delta_{\mathcal{M}}/2 \leq \hat{Q}_{\text{DNTS}}^{(N(s_t,a,n))}(s_t, a) \leq Q^*(s_t, a) + \Delta_{\mathcal{M}}/2. \tag{C.142}$$

And hence, with probability $1 - C_1 \exp(-k_2 N(s_t))$, for all $a_t \in \mathcal{A} - \{a^*\}$ it must be that:

$$\begin{aligned}
\hat{Q}_{\text{DNTS}}^{(N(s_t,a,n))}(s_t, a_t) \leq & Q^*(s_t, a_t) + \Delta_{\mathcal{M}}/2 && \text{(C.143)}\\
\leq & Q^*(s_t, a^*) - \Delta_{\mathcal{M}}/2 && \text{(C.144)}\\
\leq & \hat{Q}_{\text{DNTS}}^{(N(s_t,a^*,n))}(s_t, a^*). && \text{(C.145)}
\end{aligned}$$

Negating the probability of (C.145) then gives:

$$\Pr\left(\hat{Q}_{\text{DNTS}}^{(N(s_t,a,n))}(s_t, a) > \hat{Q}_{\text{DNTS}}^{(N(s_t,a^*,n))}(s_t, a^*)\right) \leq C_1 \exp(-k_2 N(s_t, n)) \tag{C.146}$$

Finally, the bound on the expected immediate regret follows:

$$\begin{aligned}
& \mathbb{E}\texttt{inst_regr}(s_t, \psi_{\text{DNTS}}^n) && \text{(C.147)}\\
= & \sum_{a \in \mathcal{A} - \{a^*\}} (V^*(s_t) - Q^*(s_t, a)) \Pr\left(\psi_{\text{DNTS}}^n(s_t) = a\right) && \text{(C.148)}\\
= & \sum_{a \in \mathcal{A} - \{a^*\}} (V^*(s_t) - Q^*(s_t, a)) \Pr\left(a = \arg\max_{a'} \hat{Q}_{\text{DNTS}}^{(N(s_t,a,n))}(s_t, a)\right) && \text{(C.149)}\\
\leq & \sum_{a \in \mathcal{A} - \{a^*\}} (V^*(s_t) - Q^*(s_t, a)) \Pr\left(\hat{Q}_{\text{DNTS}}^{(N(s_t,a,n))}(s_t, a) > \hat{Q}_{\text{DNTS}}^{(N(s_t,a^*,n))}(s_t, a^*)\right) && \text{(C.150)}\\
\leq & \sum_{a \in \mathcal{A} - \{a^*\}} (V^*(s_t) - Q^*(s_t, a)) C_1 \exp(-k_2 N(s_t, n)), && \text{(C.151)}
\end{aligned}$$

and setting $k' = k_2$ and $C' = C_1 \sum_{a \in \mathcal{A} - \{a^*\}} (V^*(s_t) - Q^*(s_t, a))$ gives the result. □

Again similarly to before, by using Lemma C.2.12, the bound on the immediate simple regret can be converted to a bound on the simple regret.

Theorem C.2.6. *Consider a DENTS process. Let $s_t \in \mathcal{S}$, with $1 \leq t \leq H$ TODO: thtspp consistent agen then there is some $C', k' > 0$ such that:*

$$\mathbb{E}\texttt{sim_regr}(s_t, \psi_{\text{DNTS}}^n) \leq C' \exp\left(-k' N(s_t, n)\right). \tag{C.152}$$

Moreover, at the root node s_0:

$$\mathbb{E}\texttt{sim_regr}(s_0, \psi_{\text{DNTS}}^n) \leq C' \exp\left(-k' n\right). \tag{C.153}$$

Proof. Theorem holds as a consequence of Corollary **??** and Lemma C.2.12. To arrive at (C.153) note that $N(s_0, n) = n$. □

Theorem **??** follows TODO: this was the neurips main paper theorem, changing how doing the main results bit from the results that we have just shown.

Finally, Theorem **??** hows as it is a subset of what has already been shown. TODO: theorem restate

Proof. The simple regret bound follows immediately from Theorem C.2.6. Using Theorem C.2.5 it holds that $\Pr\left(\left|\hat{V}_{\text{DNTS}}^{(N(s_0,n))}(s_0) - V^*(s_0)\right| > \varepsilon\right) \leq C \exp\left(-k\varepsilon^2 n\right) \to 0$ as $n \to \infty$, and hence $\hat{V}_{\text{DNTS}}^{(N(s_0,n))}(s_0)$ converges in probability to $V^*(s_0)$. □

Finally, the BTS exponential convergence bound follows from it being a special case of DENTS. TODO: need to convert to new latex notation for thesis, so commented out below

Bibliography

[1] Bruce Abramson. Expected-outcome: A general model of static evaluation. *IEEE transactions on pattern analysis and machine intelligence*, 12(2):182–193, 1990.

[2] Guillermo Aparicio, Leocadio G Casado, Eligius MT Hendrix, Boglárka G.-Tóth, and Inmaculada Garcia. On the minimum number of simplex shapes in longest edge bisection refinement of a regular n-simplex. *Informatica*, 26(1): 17–32, 2015.

[3] P Auer. Finite-time analysis of the multiarmed bandit problem, 2002.

[4] David Auger, Adrien Couetoux, and Olivier Teytaud. Continuous upper confidence trees with polynomial exploration–consistency. In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2013, Prague, Czech Republic, September 23-27, 2013, Proceedings, Part I 13*, pages 194–209. Springer, 2013.

[5] Leon Barrett and Srini Narayanan. Learning all optimal policies with multiple criteria. In *Proceedings of the 25th international conference on Machine learning*, pages 41–47, 2008.

[6] Andrew G Barto, Steven J Bradtke, and Satinder P Singh. Learning to act using real-time dynamic programming. *Artificial intelligence*, 72(1-2):81–138, 1995.

[7] Richard Bellman. *Dynamic Programming*. Dover Publications, 1957. ISBN 9780486428093.

[8] Vincent Berthier, Hassen Doghmen, and Olivier Teytaud. Consistency modifications for automatically tuned monte-carlo tree search. In *International Conference on Learning and Intelligent Optimization*, pages 111–124. Springer, 2010.

[9] Lilian Besson and Emilie Kaufmann. What doubling tricks can and can't do for multi-armed bandits. *arXiv preprint arXiv:1803.06971*, 2018.

[10] Patrick Billingsley. *Probability and measure*. John Wiley & Sons, 2017.

[11] Cameron B Browne, Edward Powley, Daniel Whitehouse, Simon M Lucas, Peter I Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in games*, 4(1):1–43, 2012.

[12] Sébastien Bubeck, Rémi Munos, and Gilles Stoltz. Pure exploration in multi-armed bandits problems. In *Algorithmic Learning Theory: 20th International Conference, ALT 2009, Porto, Portugal, October 3-5, 2009. Proceedings 20*, pages 23–37. Springer, 2009.

[13] Tristan Cazenave, Flavien Balbo, Suzanne Pinson, et al. Monte-carlo bus regulation. In *12th international IEEE conference on intelligent transportation systems*, volume 340345, 2009.

[14] Benjamin E Childs, James H Brodeur, and Levente Kocsis. Transpositions and move groups in monte carlo tree search. In *2008 IEEE Symposium On Computational Intelligence and Games*, pages 389–395. IEEE, 2008.

[15] Pierre-Arnaud Coquelin and Rémi Munos. Bandit algorithms for tree search. *arXiv preprint cs/0703062*, 2007.

[16] Adrien Couëtoux, Jean-Baptiste Hoock, Nataliya Sokolovska, Olivier Teytaud, and Nicolas Bonnard. Continuous upper confidence trees. In *International*

conference on learning and intelligent optimization, pages 433–445. Springer, 2011.

[17] Tuan Q Dam, Carlo D'Eramo, Jan Peters, and Joni Pajarinen. Convex regularization in monte-carlo tree search. In *International Conference on Machine Learning*, pages 2365–2375. PMLR, 2021.

[18] Aryeh Dvoretzky, Jack Kiefer, and Jacob Wolfowitz. Asymptotic minimax character of the sample distribution function and of the classical multinomial estimator. *The Annals of Mathematical Statistics*, pages 642–669, 1956.

[19] Zohar Feldman and Carmel Domshlak. Monte-carlo planning: Theoretically fast convergence meets practical efficiency. *arXiv preprint arXiv:1309.6828*, 2013.

[20] William Feller et al. *An introduction to probability theory and its applications*. Wiley New York, 1971.

[21] Sylvain Gelly and David Silver. Combining online and offline knowledge in uct. In *Proceedings of the 24th international conference on Machine learning*, pages 273–280, 2007.

[22] Sylvain Gelly and David Silver. Monte-carlo tree search and rapid action value estimation in computer go. *Artificial Intelligence*, 175(11):1856–1875, 2011.

[23] Tuomas Haarnoja, Haoran Tang, Pieter Abbeel, and Sergey Levine. Reinforcement learning with deep energy-based policies. In *International conference on machine learning*, pages 1352–1361. PMLR, 2017.

[24] Eric A Hansen and Shlomo Zilberstein. Lao*: A heuristic search algorithm that finds solutions with loops. *Artificial Intelligence*, 129(1-2):35–62, 2001.

[25] Conor F Hayes, Roxana Rădulescu, Eugenio Bargiacchi, Johan Källström, Matthew Macfarlane, Mathieu Reymond, Timothy Verstraeten, Luisa M

Zintgraf, Richard Dazeley, Fredrik Heintz, et al. A practical guide to multi-objective reinforcement learning and planning. *Autonomous Agents and Multi-Agent Systems*, 36(1):26, 2022.

[26] Edwin T Jaynes. Information theory and statistical mechanics. *Physical review*, 106(4):620, 1957.

[27] Thomas Keller and Patrick Eyerich. Prost: Probabilistic planning based on uct. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 22, pages 119–127, 2012.

[28] Thomas Keller and Malte Helmert. Trial-based heuristic tree search for finite horizon mdps. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 23, pages 135–143, 2013.

[29] Robert Kleinberg, Aleksandrs Slivkins, and Eli Upfal. Multi-armed bandits in metric spaces. In *Proceedings of the fortieth annual ACM symposium on Theory of computing*, pages 681–690, 2008.

[30] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *European conference on machine learning*, pages 282–293. Springer, 2006.

[31] Levente Kocsis, Csaba Szepesvári, and Jan Willemson. Improved monte-carlo search. *Univ. Tartu, Estonia, Tech. Rep*, 1:1–22, 2006.

[32] Tze Leung Lai and Herbert Robbins. Asymptotically efficient adaptive allocation rules. *Advances in applied mathematics*, 6(1):4–22, 1985.

[33] Tor Lattimore and Csaba Szepesvári. *Bandit algorithms*. Cambridge University Press, 2020.

[34] Tyler Lu, Dávid Pál, and Martin Pál. Contextual multi-armed bandits. In *Proceedings of the Thirteenth international conference on Artificial Intelligence and Statistics*, pages 485–492. JMLR Workshop and Conference Proceedings, 2010.

[35] Ruben Martinez-Cantin. Bayesopt: A bayesian optimization library for nonlinear optimization, experimental design and bandits. Technical report, 2014.

[36] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*, pages 1928–1937. PmLR, 2016.

[37] Thomas M Moerland, Joost Broekens, Aske Plaat, and Catholijn M Jonker. A0c: Alpha zero in continuous action space. *arXiv preprint arXiv:1805.09613*, 2018.

[38] Mausam Natarajan and Andrey Kolobov. *Planning with Markov decision processes: An AI perspective*. Springer Nature, 2022.

[39] Ricardo H Nochetto, Kunibert G Siebert, and Andreas Veeser. Theory of adaptive finite element methods: an introduction. In *Multiscale, nonlinear and adaptive approximation: Dedicated to Wolfgang Dahmen on the occasion of his 60th birthday*, pages 409–542. Springer, 2009.

[40] Michael Painter. THTS++, 2025. URL `https://github.com/MWPainter/thts-plus-plus`. Available at `https://github.com/MWPainter/thts-plus-plus`.

[41] Francisco Perdomo and Ángel Plaza. Properties of triangulations obtained by the longest-edge bisection. *Central European Journal of Mathematics*, 12(12): 1796–1810, 2014.

[42] Laurent Péret and Frédérick Garcia. On-line search for solving markov decision processes via heuristic sampling. *learning*, 16:2, 2004.

[43] Werner C Rheinboldt. On a theory of mesh-refinement processes. *SIAM Journal on Numerical Analysis*, 17(6):766–778, 1980.

[44] M Cecilia Rivara. Algorithms for refining triangular grids suitable for adaptive and multigrid techniques. *International journal for numerical methods in Engineering*, 20(4):745–756, 1984.

[45] Maria-Cecilia Rivara. Mesh refinement processes based on the generalized bisection of simplices. *SIAM Journal on Numerical Analysis*, 21(3):604–613, 1984.

[46] Herbert Robbins. Some aspects of the sequential design of experiments. 1952.

[47] Diederik Marijn Roijers, Shimon Whiteson, and Frans A Oliehoek. Computing convex coverage sets for faster multi-objective coordination. *Journal of Artificial Intelligence Research*, 52:399–443, 2015.

[48] Ivo G Rosenberg and Frank Stenger. A lower bound on the angles of triangles constructed by bisecting the longest side. *Mathematics of Computation*, 29 (130):390–395, 1975.

[49] Christopher D Rosin. Multi-armed bandits with episode context. *Annals of Mathematics and Artificial Intelligence*, 61(3):203–230, 2011.

[50] Christopher D Rosin. Multi-armed bandits with episode context. *Annals of Mathematics and Artificial Intelligence*, 61(3):203–230, 2011.

[51] Abdallah Saffidine, Tristan Cazenave, and Jean Méhat. Ucd: Upper confidence bound for rooted directed acyclic graphs. *Knowledge-Based Systems*, 34:26–33, 2012.

[52] Jose MG Salmerón, Guillermo Aparicio, Leocadio G Casado, Inmaculada García, Eligius MT Hendrix, and B G.-Tóth. Generating a smallest binary tree by proper selection of the longest edges to bisect in a unit simplex refinement. *Journal of Combinatorial Optimization*, 33(2):389–402, 2017.

[53] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[54] Claude Elwood Shannon. A mathematical theory of communication. *The Bell system technical journal*, 27(3):379–423, 1948.

[55] David Silver and Joel Veness. Monte-carlo planning in large pomdps. *Advances in neural information processing systems*, 23, 2010.

[56] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.

[57] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of go without human knowledge. *nature*, 550(7676):354–359, 2017.

[58] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.

[59] Aleksandrs Slivkins. Contextual bandits with similarity information. In *Proceedings of the 24th annual Conference On Learning Theory*, pages 679–702. JMLR Workshop and Conference Proceedings, 2011.

[60] Dennis JNJ Soemers, Chiara F Sironi, Torsten Schuster, and Mark HM Winands. Enhancements for real-time monte-carlo tree search in general video game playing. In *2016 IEEE Conference on Computational Intelligence and Games (CIG)*, pages 1–8. IEEE, 2016.

[61] Richard S Sutton. Reinforcement learning: An introduction. *A Bradford Book*, 2018.

[62] David Tolpin and Solomon Shimony. Mcts based on simple regret. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 26, pages 570–576, 2012.

[63] Marko Tot, Michelangelo Conserva, Diego Perez Liebana, and Sam Devlin. Turning zeroes into non-zeroes: Sample efficient exploration with monte carlo graph search. In *2022 IEEE Conference on Games (CoG)*, pages 300–306. IEEE, 2022.

[64] Michael D Vose. A linear algorithm for generating random numbers with a given distribution. *IEEE Transactions on software engineering*, 17(9):972–975, 1991.

[65] Alastair J Walker. An efficient method for generating discrete random variables with general distributions. *ACM Transactions on Mathematical Software (TOMS)*, 3(3):253–256, 1977.

[66] Ronald J Williams and Jing Peng. Function optimization using connectionist reinforcement learning algorithms. *Connection Science*, 3(3):241–268, 1991.

[67] David J Wu. Accelerating self-play learning in go. *arXiv preprint arXiv:1902.10565*, 2019.

[68] David J. Wu. Monte-carlo graph search from first principles. `https://github.com/lightvector/KataGo/blob/master/docs/GraphSearch.md`, 2023. Accessed: 2026-03-19.

[69] Chenjun Xiao, Ruitong Huang, Jincheng Mei, Dale Schuurmans, and Martin Müller. Maximum entropy monte-carlo planning. *Advances in Neural Information Processing Systems*, 32, 2019.

[70] Brian D Ziebart. *Modeling purposeful adaptive behavior with the principle of maximum causal entropy.* Carnegie Mellon University, 2010.

[71] Brian D Ziebart, Andrew L Maas, J Andrew Bagnell, Anind K Dey, et al. Maximum entropy inverse reinforcement learning. In *Aaai*, volume 8, pages 1433–1438. Chicago, IL, USA, 2008.

[72] Olgierd Cecil Zienkiewicz, Robert Leroy Taylor, Perumal Nithiarasu, and JZ Zhu. *The finite element method*, volume 3. Elsevier, 1977.